

■■■■■■■■■: 2025-12-panoptic

[1/12] contracts/PanopticPool.sol

```
// SPDX-License-Identifier: BUSL-1.1
pragma solidity ^0.8.24;
// Interfaces
import {CollateralTracker} from "@contracts/CollateralTracker.sol";
import {ISemiFungiblePositionManager} from "@contracts/interfaces/ISemiFungiblePositionManager.sol";
import {IRiskEngine} from "@contracts/interfaces/IRiskEngine.sol";
// Inherited implementations
import {Clone} from "clones-with-immutable-args/Clone.sol";
import {Multicall} from "@base/Multicall.sol";
// Libraries
import {Constants} from "@libraries/Constants.sol";
import {EfficientHash} from "@libraries/EfficientHash.sol";
import {Errors} from "@libraries/Errors.sol";
import {InteractionHelper} from "@libraries/InteractionHelper.sol";
import {Math} from "@libraries/Math.sol";
import {PanopticMath} from "@libraries/PanopticMath.sol";
// Custom types
import {LeftRightUnsigned, LeftRightSigned} from "@types/LeftRight.sol";
import {LiquidityChunk} from "@types/LiquidityChunk.sol";
import {PositionBalance, PositionBalanceLibrary} from "@types/PositionBalance.sol";
import {RiskParameters} from "@types/RiskParameters.sol";
import {TokenId} from "@types/TokenId.sol";
import {OraclePack, OraclePackLibrary} from "@types/OraclePack.sol";
/// @title The Panoptic Pool: Create permissionless options on a CLAMM.
/// @author Axicon Labs Limited
/// @notice Manages positions, collateral, liquidations and forced exercises.
contract PanopticPool is Clone, Multicall {
    /*//////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
        EVENTS
    //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////*/
    /// @notice Emitted when an account is liquidated.
    /// @param liquidator Address of the caller liquidating the distressed account
    /// @param liquidatee Address of the distressed/liquidatable account
    /// @param bonusAmounts LeftRight encoding for the the bonus paid for token 0 (right slot) and 1 (left slot) to the ...
    event AccountLiquidated(
        address indexed liquidator,
        address indexed liquidatee,
        LeftRightSigned bonusAmounts
    );
    /// @notice Emitted when a position is force exercised.
    /// @param exercisor Address of the account that forces the exercise of the position
    /// @param user Address of the owner of the liquidated position
    /// @param tokenId TokenId of the liquidated position
    /// @param exerciseFee LeftRight encoding for the cost paid by the exercisor to force the exercise of the token;
    /// the cost for token 0 (right slot) and 1 (left slot) is represented as negative
    event ForcedExercised(
        address indexed exercisor,
        address indexed user,
        TokenId indexed tokenId,
        LeftRightSigned exerciseFee
    );
    /// @notice Emitted when premium is settled independent of a mint/burn (e.g. during `settlePremium`).
    /// @param user Address of the owner of the settled position
    /// @param tokenId TokenId of the settled position
    /// @param legIndex The leg index of `tokenId` that the premium was settled for
    /// @param settledAmounts LeftRight encoding for the amount of premium settled for token0 (right slot) and token1 (l...
    event PremiumSettled(
        address indexed user,
        TokenId indexed tokenId,
        uint256 legIndex,
        LeftRightSigned settledAmounts
    );
    /// @notice Emitted when an option is burned.
    /// @param recipient User that burnt the option
    /// @param positionSize The number of contracts burnt, expressed in terms of the asset
    /// @param tokenId TokenId of the burnt option
    /// @param premiaByLeg LeftRight packing for the amount of premia settled for token0 (right) and token1 (left) for e...
    event OptionBurnt(
        address indexed recipient,
        uint128 positionSize,
        TokenId indexed tokenId,
        LeftRightSigned[4] premiaByLeg
    );
    /// @notice Emitted when an option is minted.
```

```

/// @param recipient User that minted the option
/// @param tokenId TokenId of the created option
/// @param balanceData The `PositionBalance` data for `tokenId` containing the number of contracts, pool utilization...
event OptionMinted(
    address indexed recipient,
    TokenId indexed tokenId,
    PositionBalance balanceData
);
//*****
//IMMUTABLES & CONSTANTS
//*****
/// @notice Lower price bound used when no slippage check is required.
int24 internal constant MIN_SWAP_TICK = Constants.MIN_POOL_TICK - 1;
/// @notice Upper price bound used when no slippage check is required.
int24 internal constant MAX_SWAP_TICK = Constants.MAX_POOL_TICK + 1;
/// @notice Flag that signals to compute premia for both the short and long legs of a position.
bool internal constant COMPUTE_PREMIA_AS_COLLATERAL = true;
/// @notice Flag that indicates only to include the share of (settled) premium that is available to collect when cal...
bool internal constant ONLY_AVAILABLE_PREMIUM = false;
/// @notice Flag that signals to commit both collected Uniswap fees and settled long premium to `s_settledTokens`.
bool internal constant COMMIT_LONG_SETTLED = true;
/// @notice Flag that signals to only commit collected Uniswap fees to `s_settledTokens`.
bool internal constant DONOT_COMMIT_LONG_SETTLED = false;
/// @notice Flag for `_checkSolvency` to indicate that an account should be solvent at all input ticks.
bool internal constant ASSERT_SOLVENCY = true;
/// @notice Flag for `_checkSolvency` to indicate that an account should be insolvent at all input ticks.
bool internal constant ASSERT_INSOLVENCY = false;
/// @notice Flag that signals to add a new position to the user's positions hash (as opposed to removing an existing...
bool internal constant ADD = true;
/// @notice The maximum allowed number of legs across all open positions for a user.
uint64 internal constant MAX_OPEN_LEGS = 25;
/// @notice Multiplier for the collateral requirement in the general case.
uint24 internal constant NO_BUFFER = 10_000_000;
/// @notice Decimals for computation (1 bps (1 basis point) precision: 0.01%).
/// @dev uint type for composability with unsigned integer based mathematical operations.
uint256 internal constant DECIMALS = 10_000;
/// @notice Transient storage slot for the tick price
bytes32 internal constant PRICE_TRANSIENT_SLOT = keccak256("panoptic.price.snapshot");
/// @notice The "engine" of Panoptic - manages AMM liquidity and executes all mints/burns/exercises.
ISemiFungiblePositionManager internal immutable SFPM;
//*****
//STORAGE
//*****
/// @notice Stores a sorted set of 8 price observations used to compute the internal median oracle price.
// The data for the last 8 interactions is stored as such:
//
// timestamp      orderMap      spotEMA      fastEMA      slowEMA      eonseMA      reference      r7      ...
// |<- 24 bits ->|<- 24 bits ->|<- 22 bits ->|<- 22 bits ->|<- 22 bits ->|<- 22 bits ->|<- 22 bits ->|<- 22 bits ->|
//
// LAST UPDATED BLOCK TIMESTAMP (22 bits) -> 22 bits (use 28 bits for the timestamp and truncate the lower 6 bits...
// [BLOCK.TIMESTAMP]
// (000000000000000000000000) // dynamic
//
// ORDERING of tick indices least --> greatest (24 bits)
// The value of the bit codon ([#]) is a pointer to a tick index in the tick array.
// The position of the bit codon from most to least significant is the ordering of the
// tick index it points to from least to greatest.
//
// rank: 0 1 2 3 4 5 6 7
// slot: [7] [5] [3] [1] [0] [2] [4] [6]
// 111 101 011 001 000 010 100 110
//
// [-512] [7]
// 111000000000
//
// [512] [0]
// 001000000000
//
// [-512] [6]
// 111000000000
//
// [512] [1]
// 001000000000
//
// [-512] [5]
// 111000000000
//
// [512] [2]
// 001000000000
//
// [0 = CURRENT TICK] [4]
// (000000000000) // dynamic
//
// [0 = CURRENT TICK] [3]

```

[illegible]

```

ACCESS CONTROL
////////////////////////////////////*/
/// @notice Reverts if the associated Risk Engine is not the caller.
modifier onlyRiskEngine() {
    _onlyRiskEngine();
    -;
}
function _onlyRiskEngine() internal view {
    if (msg.sender != address(riskEngine())) revert Errors.NotGuardian();
}
/// @notice Force safe mode lock: effective safe mode must be treated as level 3.
function lockSafeMode() external onlyRiskEngine {
    s_oraclePack = s_oraclePack.lock();
}
/// @notice Remove forced safe mode lock.
function unlockSafeMode() external onlyRiskEngine {
    s_oraclePack = s_oraclePack.unlock();
}
/*//////////////////////////////////////////
INITIALIZATION
//////////////////////////////////////////*/
/// @notice Store the address of the canonical SemiFungiblePositionManager (SFPm) contract.
/// @param _sfpm The address of the SFPm
constructor(ISemiFungiblePositionManager _sfpm) {
    SFPm = _sfpm;
}
/// @notice Initializes the median oracle of a new `PanopticPool` instance with median oracle state and performs ini...
/// @dev Must be called first (by the factory contract) before any transaction can occur.
function initialize() external {
    // reverts if this contract has already been initialized (assuming block.timestamp > 0)
    if (OraclePack.unwrap(s_oraclePack) != 0) revert Errors.PoolAlreadyInitialized();
    int24 currentTick = getCurrentTick();
    // Store the median data
    uint96 EMAs = OraclePackLibrary.packEMAs(
        currentTick,
        currentTick,
        currentTick,
        currentTick
    );
    s_oraclePack = OraclePackLibrary.storeOraclePack(
        block.timestamp >>> 6,
        0xf590a6, // orderMap
        EMAs,
        currentTick,
        0xe00200e00200e00200e00000, // current residuals
        0,
        0
    );
    /*
        (uint256((block.timestamp >>> 6) % 2 ** 24) <&&& 232) +
        // magic number which adds (7,5,3,1,0,2,4,6) order and minTick in positions 7, 5, 3 and maxTick in 6, 4, 2
        // see comment on s_oraclePack initialization for format of this magic number
        (uint256(0xf590a6000000000000000000000000800e00200e00200e00000)) +
        // eonsEMA at bits 207-186
        (uint256(uint24(currentTick) & 0x3FFFFFF) <&&& 186) +
        // slowEMA at bits 185-164
        (uint256(uint24(currentTick) & 0x3FFFFFF) <&&& 164) +
        // fastEMA at bits 163-142
        (uint256(uint24(currentTick) & 0x3FFFFFF) <&&& 142) +
        // spotEMA at bits 141-120
        (uint256(uint24(currentTick) & 0x3FFFFFF) <&&& 120) +
        // store currentTick as the reference tick at bits 119-96
        (uint256(uint24(currentTick)) <&&& 96);
    */
    // consolidate all 4 approval calls to one library delegatecall in order to reduce bytecode size
    // approves:
    // SFPm: token0, token1
    // CollateralTracker0 - token0
    // CollateralTracker1 - token1
    InteractionHelper.doApprovals(
        SFPm,
        collateralToken0(),
        collateralToken1(),
        collateralToken0().token0(),
        collateralToken0().token1(),
        poolManager()
    );
}
/*//////////////////////////////////////////
EIP SUPPORT
//////////////////////////////////////////*/
// note: this contract does not need to accept batch ERC1155 transfers from the SFPm or supply ERC-165 calls
// thus, `supportsInterface` and `onERC1155BatchReceived` are left unimplemented to reduce contract size
/// @notice Returns magic value when called by the `SemiFungiblePositionManager` contract to indicate that this cont...

```

```

function onERC1155Received(
    address,
    address,
    uint256,
    uint256,
    bytes memory
) external pure returns (bytes4) {
    return this.onERC1155Received.selector;
}
}
/*/////////////////////////////////////////////////////////////////////////
                        QUERY HELPERS
////////////////////////////////////////////////////////////////////////*/
/// @notice Reverts if the caller has a lower collateral balance than required to meet the provided `minValue0` and ...
/// @dev Can be used for composable slippage checks with `multicall` (such as for a force exercise or liquidation).
/// @param minValue0 The minimum acceptable `token0` value of collateral
/// @param minValue1 The minimum acceptable `token1` value of collateral
function assertMinCollateralValues(uint256 minValue0, uint256 minValue1) external view {
    CollateralTracker ct0 = collateralToken0();
    CollateralTracker ct1 = collateralToken1();
    if (ct0.assetsOf(msg.sender) < minValue0 || ct1.assetsOf(msg.sender) < minValue1)
        revert Errors.AccountInsolvent(0, 0);
}
/// @notice Determines if account is eligible to withdraw or transfer collateral.
/// @dev Checks whether account is solvent with `BP_DECREASE_BUFFER` according to `_validateSolvency`.
/// @dev Prevents insolvent and near-insolvent accounts from withdrawing collateral before they are liquidated.
/// @dev Reverts if account is not solvent with `BP_DECREASE_BUFFER`.
/// @param user The account to check for collateral withdrawal eligibility
/// @param positionIdList The list of all option positions held by `user`
/// @param usePremiaAsCollateral Whether to compute accumulated premia for all legs held by the user for collateral ...
function validateCollateralWithdrawable(
    address user,
    TokenId[] calldata positionIdList,
    bool usePremiaAsCollateral
) external view {
    (RiskParameters riskParameters, ) = getRiskParameters(0);
    _validateSolvency(
        user,
        positionIdList,
        riskParameters.bpDecreaseBuffer(),
        usePremiaAsCollateral,
        0
    );
}
/// @notice Returns the total amount of premium accumulated for a list of positions and a list containing the corres...
/// @param user Address of the user that owns the positions
/// @param positionIdList List of positions. Written as `[tokenId1, tokenId2, ...]`
/// @param includePendingPremium If true, include premium that is owed to the user but has not yet settled; if false...
/// @return The total amount of premium owed (which may `includePendingPremium`) to the short legs in `positionIdList`...
/// @return The total amount of premium owed by the long legs in `positionIdList` (token0: right slot, token1: left ...
/// @return A list of `PositionBalance` data (balance and pool utilization/oracle ticks at last mint) for each posit...
function getAccumulatedFeesAndPositionsData(
    address user,
    bool includePendingPremium,
    TokenId[] calldata positionIdList
) external view returns (LeftRightUnsigned, LeftRightUnsigned, PositionBalance[] memory) {
    // Get the current tick of the Uniswap pool
    int24 currentTick = getCurrentTick();
    // Compute the accumulated premia for all tokenId in positionIdList (includes short+long premium)
    return
        _calculateAccumulatedPremia(
            user,
            positionIdList,
            COMPUTE_PREMIA_AS_COLLATERAL,
            includePendingPremium,
            currentTick
        );
}
/// @notice Calculate the accumulated premia owed from the option buyer to the option seller.
/// @param user The holder of options
/// @param positionIdList The list of all option positions held by user
/// @param usePremiaAsCollateral Whether to compute accumulated premia for all legs held by the user for collateral ...
/// @param includePendingPremium If true, include premium that is owed to the user but has not yet settled; if false...
/// @param atTick The current tick of the Uniswap pool
/// @return shortPremium The total amount of premium owed (which may `includePendingPremium`) to the short legs in `...
/// @return longPremium The total amount of premium owed by the long legs in `positionIdList` (token0: right slot, t...
/// @return balances A list of balances and pool utilization for each position, of the form `[[tokenId0, balances0],...`
function _calculateAccumulatedPremia(
    address user,
    TokenId[] calldata positionIdList,
    bool usePremiaAsCollateral,
    bool includePendingPremium,
    int24 atTick
)
    internal

```

```

view
returns (
    LeftRightUnsigned shortPremium,
    LeftRightUnsigned longPremium,
    PositionBalance[] memory balances
)
{
    uint256 pLength = positionIdList.length;
    balances = new PositionBalance[](pLength);
    address c_user = user;
    // loop through each option position/tokenId
    for (uint256 k = 0; k < pLength; ) {
        TokenId tokenId = positionIdList[k];
        {
            PositionBalance positionBalanceData = s_positionBalance[c_user][tokenId];
            if (positionBalanceData.positionSize() == 0) revert Errors.PositionNotOwned();
            balances[k] = positionBalanceData;
        }
        (
            LeftRightSigned[4] memory premiaByLeg,
            uint256[2][4] memory premiumAccumulatorsByLeg
        ) = _getPremia(
            tokenId,
            balances[k].positionSize(),
            c_user,
            usePremiaAsCollateral,
            atTick
        );
        uint256 numLegs = tokenId.countLegs();
        for (uint256 leg = 0; leg < numLegs; ) {
            if (tokenId.width(leg) != 0) {
                if (tokenId.isLong(leg) == 0) {
                    if (!includePendingPremium) {
                        bytes32 chunkKey = PanopticMath.getChunkKey(tokenId, leg);
                        (uint256 totalLiquidity, , ) = _getLiquidities(tokenId, leg);
                        shortPremium = shortPremium.add(
                            _getAvailablePremium(
                                totalLiquidity,
                                s_settledTokens[chunkKey],
                                s_grossPremiumLast[chunkKey],
                                LeftRightUnsigned.wrap(
                                    uint256(LeftRightSigned.unwrap(premiaByLeg[leg]))
                                ),
                                premiumAccumulatorsByLeg[leg]
                            )
                        );
                    } else {
                        shortPremium = shortPremium.add(
                            LeftRightUnsigned.wrap(
                                uint256(LeftRightSigned.unwrap(premiaByLeg[leg]))
                            )
                        );
                    }
                } else {
                    longPremium = LeftRightUnsigned.wrap(
                        uint256(
                            LeftRightSigned.unwrap(
                                LeftRightSigned
                                    .wrap(int256(LeftRightUnsigned.unwrap(longPremium)))
                                    .sub(premiaByLeg[leg])
                                )
                            )
                    );
                }
            }
            unchecked {
                ++leg;
            }
        }
        unchecked {
            ++k;
        }
    }
}

/*
ONBOARD MEDIAN TWAP
*/
/// @notice Updates the internal oracle.
function pokeOracle() external {
    int24 currentTick = getCurrentTick();
    (, OraclePack oraclePack) = riskEngine().computeInternalMedian(s_oraclePack, currentTick);
    if (OraclePack.unwrap(oraclePack) != 0) s_oraclePack = oraclePack;
}
/*

```

```

MINT/BURN INTERFACE
////////////////////////////////////////////////////////////*/
/// @notice Mints or burns each `tokenId` in `positionIdList`.
/// @param positionIdList The list of tokenIds for the option positions to be minted or burnt
/// @param finalPositionIdList The final positionIdList after all the tokens have been minted/burnt
/// @param positionSizes The list of positionSize for the position to be minted (0 for burns)
/// @param tickAndSpreadLimits A Nx3 array containing: the lower [0] and upper [1] bounds of an acceptable open inte...
/// denominated as X10_000 = (`ratioLimit * 10_000`)
/// @param usePremiaAsCollateral Whether to compute accumulated premia for all legs held by the user for collateral ...
/// @param builderCode The builder code for fee distribution
function dispatch(
    TokenId[] calldata positionIdList,
    TokenId[] calldata finalPositionIdList,
    uint128[] calldata positionSizes,
    int24[3][] calldata tickAndSpreadLimits,
    bool usePremiaAsCollateral,
    uint256 builderCode
) external {
    // if safeMode, enforce covered at mint and exercise at burn
    RiskParameters riskParameters;
    LeftRightSigned cumulativeTickDeltas;
    {
        //assembly tload
        bytes32 slot = PRICE_TRANSIENT_SLOT;
        assembly {
            cumulativeTickDeltas := tload(slot)
        }
    }
    {
        int24 startTick;
        (riskParameters, startTick) = getRiskParameters(builderCode);
        if (cumulativeTickDeltas.rightSlot() == 0) {
            // initializes +1 sentinel
            cumulativeTickDeltas = LeftRightSigned.wrap(0).addToLeftSlot(1).addToLeftSlot(
                startTick
            );
        } else {
            cumulativeTickDeltas = LeftRightSigned
                .wrap(0)
                .addToLeftSlot(
                    cumulativeTickDeltas.rightSlot() +
                    int128(Math.abs(int24(cumulativeTickDeltas.leftSlot()) - startTick))
                )
                .addToLeftSlot(startTick);
        }
    }
    for (uint256 i = 0; i < positionIdList.length; ) {
        TokenId tokenId = positionIdList[i];
        // make sure the tokenId is for this Panoptic pool
        if (tokenId.poolId() != poolId()) revert Errors.WrongPoolId();
        PositionBalance positionBalanceData = s_positionBalance[msg.sender][tokenId];
        int24[2] memory _tickLimits;
        _tickLimits[0] = tickAndSpreadLimits[i][0];
        _tickLimits[1] = tickAndSpreadLimits[i][1];
        // if safe mode is larger than 1, mandate all positions to be minted/burnt as covered
        if (riskParameters.safeMode() > 1) {
            if (_tickLimits[0] > _tickLimits[1]) {
                (_tickLimits[0], _tickLimits[1]) = (_tickLimits[1], _tickLimits[0]);
            }
        }
        int24 finalTick;
        if (PositionBalance.unwrap(positionBalanceData) == 0) {
            // revert if more than 2 conditions are triggered to prevent the minting of any positions
            if (riskParameters.safeMode() > 2) revert Errors.StaleOracle();
            uint24 effectiveLiquidityLimit = uint24(tickAndSpreadLimits[i][2]);
            (, finalTick) = _mintOptions(
                tokenId,
                positionSizes[i],
                effectiveLiquidityLimit,
                msg.sender,
                _tickLimits,
                riskParameters
            );
        } else {
            uint128 positionSize = positionBalanceData.positionSize();
            if (positionSize == 0) revert Errors.PositionNotOwned();
            // if input positionSize matches the size stored, this is a settlePremium. Otherwise, this is a burn.
            if (positionSize == positionSizes[i]) {
                finalTick = getCurrentTick();
                _settleOptions(msg.sender, tokenId, positionSize, riskParameters, finalTick);
            } else {
                (, , finalTick) = _burnOptions(
                    tokenId,
                    positionSize,

```

```

        _tickLimits,
        msg.sender,
        COMMIT_LONG_SETTLED,
        riskParameters
    );
}
}
unchecked {
    // update starting tick in leftSlot() and add the cumulative delta to the rightSlot()
    // can never miscast because ticks are int24
    cumulativeTickDeltas = LeftRightSigned
        .wrap(0)
        .addToRightSlot(
            cumulativeTickDeltas.rightSlot() +
            int128(Math.abs(int24(cumulativeTickDeltas.leftSlot()) - finalTick))
        )
        .addToLeftSlot(finalTick);
    ++i;
}
}
unchecked {
    // can never overflow as tickDeltaLiquidation is a int24
    /// @dev revert if the total deviation is more than twice the tickDeltaLiquidation (ie. roundtrips more than...
    if (
        cumulativeTickDeltas.rightSlot() >
        int256(uint256(2 * riskParameters.tickDeltaLiquidation()))
    ) revert Errors.PriceImpactTooLarge();
    {
        //assembly tstore
        bytes32 slot = PRICE_TRANSIENT_SLOT;
        assembly {
            tstore(slot, cumulativeTickDeltas)
        }
    }
}
// Perform solvency check on user's account to ensure they had enough buying power to mint the option
// Add an initial buffer to the collateral requirement to prevent users from minting their account close to inso...
OraclePack oraclePack = _validateSolvency(
    msg.sender,
    finalPositionIdList,
    riskParameters.bpDecreaseBuffer(),
    usePremiaAsCollateral,
    riskParameters.safeMode()
);
// Update `s_oraclePack` with a new observation if the last observation is old enough (returned oraclePack is no...
if (OraclePack.unwrap(oraclePack) != 0) s_oraclePack = oraclePack;
}
/*//////////////////////////////////////////////////////////////
                        POSITION MINTING LOGIC
//////////////////////////////////////////////////////////////*/
/// @notice Validates the current options of the user, and mints a new position.
/// @param tokenId The tokenId of the newly minted position
/// @param positionSize The size of the position to be minted, expressed in terms of the asset
/// @param effectiveLiquidityLimit Maximum amount of "spread" defined as `removedLiquidity/netLiquidity` for a new p...
/// denominated as X32 = (`ratioLimit * 2^32`)
/// @param owner The owner of the option position to be minted
/// @param tickLimits The lower and upper bound of an acceptable open interval for the ending price
/// @param riskParameters The RiskEngine's core parameters
function _mintOptions(
    TokenId tokenId,
    uint128 positionSize,
    uint24 effectiveLiquidityLimit,
    address owner,
    int24[2] memory tickLimits,
    RiskParameters riskParameters
) internal returns (LeftRightSigned paidAmounts, int24 finalTick) {
    // Mint in the SFPM and update state of collateral
    LeftRightUnsigned[4] memory collectedByLeg;
    LeftRightSigned netAmmDelta;
    (collectedByLeg, netAmmDelta, finalTick) = SFPM.mintTokenizedPosition(
        poolKey(),
        tokenId,
        positionSize,
        tickLimits[0],
        tickLimits[1]
    );
    _updateSettlementPostMint(
        riskParameters,
        tokenId,
        collectedByLeg,
        positionSize,
        effectiveLiquidityLimit,
        owner
    );
};

```



```

uint32 poolUtilizations;
(poolUtilizations, paidAmounts) = _payCommissionAndWriteData(
    tokenId,
    positionSize,
    owner,
    netAmmDelta,
    riskParameters
);
{
    // update the users options balance of position `tokenId`
    // NOTE: user can't mint same position multiple times, so set the positionSize instead of adding
    PositionBalance balanceData = PositionBalanceLibrary.storeBalanceData(
        positionSize,
        poolUtilizations,
        0
    );
    s_positionBalance[owner][tokenId] = balanceData;
    emit OptionMinted(owner, tokenId, balanceData);
}
}
/// @notice Take the commission fees for minting `tokenId` and settle any other required collateral deltas.
/// @param tokenId The option position
/// @param positionSize The size of the position, expressed in terms of the asset
/// @param owner The owner of the option position to be minted
/// @param netAmmDelta The amount of tokens moved during creation of the option position
/// @param riskParameters The RiskEngine's core parameters
/// @return utilizations Packing of the pool utilization (how much funds are in the Panoptic pool versus the AMM poo...
/// right 64bits for token0 and left 64bits for token1, defined as `(inAMM * 10_000) / totalAssets()`
/// where totalAssets is the total tracked assets in the AMM and PanopticPool minus fees and donations to the Panopt...
/// @return paidAmounts The amount of tokens paid when creating that option for token0 (right) and token1 (left)
function _payCommissionAndWriteData(
    TokenId tokenId,
    uint128 positionSize,
    address owner,
    LeftRightSigned netAmmDelta,
    RiskParameters riskParameters
) internal returns (uint32 utilizations, LeftRightSigned paidAmounts) {
    // compute how much of tokenId is long and short positions
    (LeftRightSigned longAmounts, LeftRightSigned shortAmounts) = PanopticMath
        .computeExercisedAmounts(tokenId, positionSize, true);
    {
        (uint32 utilization0, int128 paid0) = collateralToken0().settleMint(
            owner,
            longAmounts.rightSlot(),
            shortAmounts.rightSlot(),
            netAmmDelta.rightSlot(),
            riskParameters
        );
        utilizations = utilization0;
        paidAmounts = paidAmounts.addToRightSlot(paid0);
    }
    {
        (uint32 utilization1, int128 paid1) = collateralToken1().settleMint(
            owner,
            longAmounts.leftSlot(),
            shortAmounts.leftSlot(),
            netAmmDelta.leftSlot(),
            riskParameters
        );
        unchecked {
            // no miscast because utilization is <=10_000
            utilizations += uint32(utilization1 <&& 16);
        }
        paidAmounts = paidAmounts.addToLeftSlot(paid1);
    }
    // return pool utilizations as two uint16 (pool Utilization is always <= 10_000)
    return (utilizations, paidAmounts);
}
}
/*//////////////////////////////////////
    POSITION BURNING LOGIC
    //////////////////////////////////////*/
/// @notice Close all options in `positionIdList`.
/// @param owner The owner of the option position to be closed
/// @param tickLimitLow The lower bound of an acceptable open interval for the ending price on each option close
/// @param tickLimitHigh The upper bound of an acceptable open interval for the ending price on each option close
/// @param commitLongSettled Whether to commit the long premium that will be settled to storage (disabled during liq...
/// @param positionIdList The list of option positions to close
/// @return netPaid The net amount of tokens paid after closing the positions
/// @return premiasByLeg The amount of premia settled by the user for each leg of the position
function _burnAllOptionsFrom(
    address owner,
    int24 tickLimitLow,
    int24 tickLimitHigh,
    bool commitLongSettled,

```

```

    tokenId[] callData positionIdList
) internal returns (LeftRightSigned netPaid, LeftRightSigned[4][] memory premiasByLeg) {
    premiasByLeg = new LeftRightSigned[4][](positionIdList.length);
    (RiskParameters riskParameters, ) = getRiskParameters(0);
    for (uint256 i = 0; i < positionIdList.length; ) {
        uint128 positionSize = s_positionBalance[owner][positionIdList[i]].positionSize();
        if (positionSize == 0) revert Errors.PositionNotOwned();
        int24[2] memory tickLimits;
        tickLimits[0] = tickLimitLow;
        tickLimits[1] = tickLimitHigh;
        LeftRightSigned paidAmounts;
        address _owner = owner;
        (paidAmounts, premiasByLeg[i], ) = _burnOptions(
            positionIdList[i],
            positionSize,
            tickLimits,
            _owner,
            commitLongSettled,
            riskParameters
        );
        netPaid = netPaid.add(paidAmounts);
        unchecked {
            ++i;
        }
    }
}

/// @notice Close a single option position.
/// @param tokenId The option position to burn
/// @param positionSize The size of the position to burn
/// @param tickLimits The lower and upper bound of an acceptable open interval for the ending price on each option c...
/// @param owner The owner of the option position to be burned
/// @param commitLongSettled Whether to commit the long premium that will be settled to storage (disabled during liq...
/// @param riskParameters The RiskEngine's core risk parameters
/// @return paidAmounts The net amount of tokens paid after closing the position
/// @return premiaByLeg The amount of premia settled by the user for each leg of the position
/// @return finalTick The final tick after burning the options
function _burnOptions(
    tokenId tokenId,
    uint128 positionSize,
    int24[2] memory tickLimits,
    address owner,
    bool commitLongSettled,
    RiskParameters riskParameters
)
internal
returns (
    LeftRightSigned paidAmounts,
    LeftRightSigned[4] memory premiaByLeg,
    int24 finalTick
)
{
    LeftRightUnsigned[4] memory collectedByLeg;
    LeftRightSigned netAmmDelta;
    (collectedByLeg, netAmmDelta, finalTick) = SFPM.burnTokenizedPosition(
        poolKey(),
        tokenId,
        positionSize,
        tickLimits[0],
        tickLimits[1]
    );
    LeftRightSigned realizedPremia;
    (realizedPremia, premiaByLeg) = _updateSettlementPostBurn(
        owner,
        tokenId,
        collectedByLeg,
        positionSize,
        riskParameters,
        LeftRightSigned.wrap(commitLongSettled ? int128(1) : int128(0))
    );
    (LeftRightSigned longAmounts, LeftRightSigned shortAmounts) = PanopticMath
        .computeExercisedAmounts(tokenId, positionSize, false);
    emit OptionBurnt(owner, positionSize, tokenId, premiaByLeg);
    RiskParameters _rp = riskParameters;
    {
        int128 paid0 = collateralToken0().settleBurn(
            owner,
            longAmounts.rightSlot(),
            shortAmounts.rightSlot(),
            netAmmDelta.rightSlot(),
            realizedPremia.rightSlot(),
            _rp
        );
        paidAmounts = paidAmounts.addToRightSlot(paid0);
    }
}

```

```

    {
        int128 paid1 = collateralToken1().settleBurn(
            owner,
            longAmounts.leftSlot(),
            shortAmounts.leftSlot(),
            netAmmDelta.leftSlot(),
            realizedPremia.leftSlot(),
            _rp
        );
        paidAmounts = paidAmounts.addToLeftSlot(paid1);
    }
}

/// @notice Validates the solvency of `user`.
/// @dev Falls back to the most conservative (least solvent) oracle tick if the sum of the squares of the deltas bet...
/// @dev Effectively, this means that the users must be solvent at all oracle ticks if the at least one of the ticks...
/// @param user The account to validate
/// @param positionIdList The list of positions to validate solvency for
/// @param buffer The buffer to apply to the collateral requirement for `user`
/// @param usePremiaAsCollateral Whether to compute accumulated premia for all legs held by the user for collateral ...
/// @return If nonzero (enough time has passed since last observation), the updated value for `s_oraclePack` with a ...
function _validateSolvency(
    address user,
    TokenId[] calldata positionIdList,
    uint32 buffer,
    bool usePremiaAsCollateral,
    uint8 safeMode
) internal view returns (OraclePack) {
    // check that the provided positionIdList matches the positions in memory
    _validatePositionList(user, positionIdList);
    int24 currentTick = getCurrentTick();
    OraclePack oraclePack;
    int24[] memory atTicks;
    (atTicks, oraclePack) = riskEngine().getSolvencyTicks(currentTick, s_oraclePack);
    if (positionIdList.length != 0) {
        uint256 solvent = _checkSolvencyAtTicks(
            user,
            safeMode,
            positionIdList,
            currentTick,
            atTicks,
            usePremiaAsCollateral,
            uint256(buffer)
        );
        uint256 numberOfTicks = atTicks.length;
        if (solvent != numberOfTicks) revert Errors.AccountInsolvent(solvent, numberOfTicks);
    }
    return oraclePack;
}

}

/*//////////////////////////////////////////////////////////////
                        SETTLEMENTS
//////////////////////////////////////////////////////////////*/

function _settleOptions(
    address owner,
    TokenId tokenId,
    uint128 positionSize,
    RiskParameters riskParameters,
    int24 currentTick
) internal {
    // call _updateSettlementPostBurn to settle the long premia or the short premia (only for self calling)
    LeftRightUnsigned[4] memory emptyCollectedByLegs;
    LeftRightSigned realizedPremia;
    unchecked {
        // cannot be miscast because currentTick is a int24
        (realizedPremia, ) = _updateSettlementPostBurn(
            owner,
            tokenId,
            emptyCollectedByLegs,
            positionSize,
            riskParameters,
            LeftRightSigned.wrap(1).addToLeftSlot(1 + (int128(currentTick) << 2))
        );
    }
    // deduct the paid premium tokens from the owner's balance
    collateralToken0().settleBurn(owner, 0, 0, 0, realizedPremia.rightSlot(), riskParameters);
    collateralToken1().settleBurn(owner, 0, 0, 0, realizedPremia.leftSlot(), riskParameters);
}

/// @notice Adds collected tokens to `s_settledTokens` and adjusts `s_grossPremiumLast` for any liquidity added.
/// @dev Always called after `mintTokenizedPosition`.
/// @param tokenId The option position that was minted
/// @param collectedByLeg The amount of tokens collected in the corresponding chunk for each leg of the position
/// @param positionSize The size of the position, expressed in terms of the asset
/// @param effectiveLiquidityLimit Maximum amount of "spread" defined as `removedLiquidity/netLiquidity`
/// @param owner The owner of the option position to be minted
function _updateSettlementPostMint(

```

[illegible]

```

        )
    }
    .addToLeftSlot(
        uint128(
            (grossCurrent1 *
             positionLiquidity +
             grossPremiumLast.leftSlot() *
             totalLiquidityBefore) / totalLiquidity
        )
    );
}
}
unchecked {
    ++leg;
}
}
}

// @notice Updates settled tokens and grossPremiumLast for a chunk after a burn and returns premium info.
// @param owner The owner of the option position that was burnt
// @param tokenId The option position that was burnt
// @param collectedByLeg The amount of tokens collected in the corresponding chunk for each leg of the position
// @param positionSize The size of the position, expressed in terms of the asset
// @param commitLongSettledAndKeepOpen Whether to commit the long premium that will be settled to storage (rightSlo...
// @return realizedPremia The amount of premia settled by the user
// @return premiaByLeg The amount of premia settled by the user for each leg of the position
function _updateSettlementPostBurn(
    address owner,
    TokenId tokenId,
    LeftRightUnsigned[4] memory collectedByLeg,
    uint128 positionSize,
    RiskParameters riskParameters,
    LeftRightSigned commitLongSettledAndKeepOpen
) internal returns (LeftRightSigned realizedPremia, LeftRightSigned[4] memory premiaByLeg) {
    uint256[2][4] memory premiumAccumulatorsByLeg;
    // compute accumulated fees
    (premiaByLeg, premiumAccumulatorsByLeg) = _getPremia(
        tokenId,
        positionSize,
        owner,
        COMPUTE_PREMIA_AS_COLLATERAL,
        commitLongSettledAndKeepOpen.leftSlot() == 0
        ? type(int24).max
        : int24(commitLongSettledAndKeepOpen.leftSlot() >> 2);
    );
    for (uint256 leg = 0; leg < tokenId.countLegs(); ) {
        if (tokenId.width(leg) != 0) {
            LeftRightSigned legPremia = premiaByLeg[leg];
            bytes32 chunkKey = PanopticMath.getChunkKey(tokenId, leg);
            // collected from Uniswap
            LeftRightUnsigned settledTokens = s_settledTokens[chunkKey].add(
                collectedByLeg[leg]
            );
            // (will be) paid by long legs
            if (tokenId.isLong(leg) == 1) {
                if (commitLongSettledAndKeepOpen.rightSlot() != 0)
                    settledTokens = LeftRightUnsigned.wrap(
                        uint256(
                            LeftRightSigned.unwrap(
                                LeftRightSigned
                                    .wrap(uint256(LeftRightUnsigned.unwrap(settledTokens)))
                                    .sub(legPremia)
                                )
                            )
                    );
            };
            realizedPremia = realizedPremia.add(legPremia);
        } else {
            if (commitLongSettledAndKeepOpen.leftSlot() == 0 || msg.sender == owner) {
                uint256 positionLiquidity;
                uint256 totalLiquidity;
                {
                    LiquidityChunk liquidityChunk = PanopticMath.getLiquidityChunk(
                        tokenId,
                        leg,
                        positionSize
                    );
                    positionLiquidity = liquidityChunk.liquidity();
                    // if position is short, ensure that removed liquidity does not deplete strike beyond MAX_SP...
                    // new totalLiquidity (total sold) = removedLiquidity + netLiquidity (T - R)
                    totalLiquidity = _checkLiquiditySpread(
                        tokenId,
                        leg,
                        riskParameters.maxSpread()
                    );

```

```

    );
}
// T (totalLiquidity is (T - R) after burning)
uint256 totalLiquidityBefore;
unchecked {
    // cannot overflow because total liquidity is less than uint128
    totalLiquidityBefore = commitLongSettledAndKeepOpen.leftSlot() == 0
        ? totalLiquidity + positionLiquidity
        : totalLiquidity;
}
LeftRightUnsigned grossPremiumLast = s_grossPremiumLast[chunkKey];
LeftRightUnsigned availablePremium = _getAvailablePremium(
    totalLiquidityBefore,
    settledTokens,
    grossPremiumLast,
    LeftRightUnsigned.wrap(uint256(LeftRightSigned.unwrap(legPremia))),
    premiumAccumulatorsByLeg[leg]
);
// subtract settled tokens sent to seller
settledTokens = settledTokens.sub(availablePremium);
// add available premium to amount that should be settled
realizedPremia = realizedPremia.add(
    LeftRightSigned.wrap(int256(LeftRightUnsigned.unwrap(availablePremium)))
);
// update the base `premiaByLeg` value to reflect the amount of premium that will actually be se...
premiaByLeg[leg] = LeftRightSigned.wrap(
    int256(LeftRightUnsigned.unwrap(availablePremium))
);
// We need to adjust the grossPremiumLast value such that the result of
// (grossPremium - adjustedGrossPremiumLast) * updatedTotalLiquidityPostBurn / 2**64 is equal to
// (grossPremium - grossPremiumLast) * totalLiquidityBeforeBurn / 2**64 - premiumOwedToPosition
// G: total gross premium (- premiumOwedToPosition)
// T: totalLiquidityBeforeMint
// R: positionLiquidity
// C: current grossPremium value
// L: current grossPremiumLast value
// Ln: updated grossPremiumLast value
// T * (C - L) = G
// (T - R) * (C - Ln) = G - P
//
// T * (C - L) = (T - R) * (C - Ln) + P
// (TC - TL - P) / (T - R) = C - Ln
// Ln = C - (TC - TL - P) / (T - R)
// Ln = (TC - CR - TC + LT + P) / (T-R)
// Ln = (LT - CR + P) / (T-R)
unchecked {
    uint256[2][4]
        memory _premiumAccumulatorsByLeg = premiumAccumulatorsByLeg;
    uint256 _leg = leg;
    // if there's still liquidity, compute the new grossPremiumLast
    // otherwise, we just reset grossPremiumLast to the current grossPremium
    s_grossPremiumLast[chunkKey] = totalLiquidity != 0
        ? LeftRightUnsigned
            .wrap(
                uint128(
                    uint256(
                        Math.max(
                            (int256(
                                grossPremiumLast.rightSlot() *
                                totalLiquidityBefore
                            ) -
                                int256(
                                    _premiumAccumulatorsByLeg[_leg][0] *
                                    positionLiquidity
                                ) +
                                int256(legPremia.rightSlot()) *
                                2 ** 64,
                                0
                            )
                        ) / totalLiquidity
                    )
                )
            )
        .addToLeftSlot(
            uint128(
                uint256(
                    Math.max(
                        (int256(
                            grossPremiumLast.leftSlot() *
                            totalLiquidityBefore
                        ) -
                            int256(
                                _premiumAccumulatorsByLeg[_leg][1] *
                                positionLiquidity
                            ) +
                            int256(legPremia.leftSlot()) * 2 ** 64,

```

```

        0
    )
    ) / totalLiquidity
)
: LeftRightUnsigned
    .wrap(uint128(premiumAccumulatorsByLeg[_leg][0]))
    .addToLeftSlot(uint128(premiumAccumulatorsByLeg[_leg][1]));
}
}
// update settled tokens in storage with all local deltas
s_settledTokens[chunkKey] = settledTokens;
if (commitLongSettledAndKeepOpen.leftSlot() == 0) {
    // erase the s_options entry for that leg
    s_options[owner][tokenId][leg] = LeftRightUnsigned.wrap(0);
} else {
    // update the premium accumulator to the latest value: only if it is a long leg (settleLongPremium) ...
    if (tokenId.isLong(leg) != 0 || msg.sender == owner) {
        s_options[owner][tokenId][leg] = LeftRightUnsigned
            .wrap(0)
            .addToRightSlot(uint128(premiumAccumulatorsByLeg[leg][0]))
            .addToLeftSlot(uint128(premiumAccumulatorsByLeg[leg][1]));
        emit PremiumSettled(owner, tokenId, leg, premiaByLeg[leg]);
    }
}
}
unchecked {
    ++leg;
}
}
if (commitLongSettledAndKeepOpen.leftSlot() == 0) {
    // reset balances and delete stored option data
    s_positionBalance[owner][tokenId] = PositionBalance.wrap(0);
    // REMOVE the current tokenId from the position list hash (hash = XOR of all keccak256(tokenId), remove by X...
    // and decrease the number of positions counter by 1.
    _updatePositionsHash(owner, tokenId, !ADD, riskParameters.maxLegs());
}
}
}
/*//////////////////////
LIQUIDATIONS & FORCED EXERCISES
//////////////////////
/// @notice Dispatches liquidations, forced exercises, or long premium settlements based on account solvency
/// @dev This function determines the appropriate action based on solvency checks at multiple price points:
/// - If insolvent at all ticks: Execute liquidation (burns all positions)
/// - If solvent at all ticks: Execute force exercise or settle long premium based on list lengths
/// - Otherwise: Revert as account is not fully margin called
/// @dev The function uses position list lengths to determine the specific operation:
/// - Same length lists between positionIdListTo and positionIdListToFinal: Settle long premium
/// - Final list one shorter: Force exercise
/// - Final list empty: Liquidation
/// @param positionIdListFrom List of positions held by the caller (msg.sender)
/// @param account The account being acted upon (liquidated, exercised, or settled)
/// @param positionIdListTo Current positions of the target account
/// @param positionIdListToFinal Expected positions after the operation completes
/// @param usePremiaAsCollateral Packed value indicating whether to use premia as collateral:
/// - leftSlot: For the caller (msg.sender)
/// - rightSlot: For the target account
function dispatchFrom(
    TokenId[] calldata positionIdListFrom,
    address account,
    TokenId[] calldata positionIdListTo,
    TokenId[] calldata positionIdListToFinal,
    LeftRightUnsigned usePremiaAsCollateral
) external payable {
    // Assert the account we are liquidating is actually insolvent
    int24 twapTick = getTWAP();
    int24 currentTick = getCurrentTick();
    TokenId tokenId;
    uint256 solvent;
    uint256 numberOfTicks;
    {
        _validatePositionList(account, positionIdListTo);
        // Enforce maximum delta between TWAP and currentTick to prevent extreme price manipulation
        int24 spotTick;
        int24 latestTick;
        (spotTick, , latestTick, ) = riskEngine().getOracleTicks(currentTick, s_oraclePack);
        unchecked {
            (RiskParameters riskParameters, ) = getRiskParameters(0);
            int256 MAX_TWAP_DELTA_LIQUIDATION = int256(
                uint256(riskParameters.tickDeltaLiquidation())
            );
            if (Math.abs(currentTick - twapTick) > MAX_TWAP_DELTA_LIQUIDATION)
                revert Errors.StaleOracle();
        }
    }
}

```

```

    }
    // Ensure the account is insolvent at twapTick (in place of medianTick), currentTick, spotTick, and latestTi...
    int24[] memory atTicks = new int24[](4);
    atTicks[0] = spotTick;
    atTicks[1] = twapTick;
    atTicks[2] = latestTick;
    atTicks[3] = currentTick;
    solvent = _checkSolvencyAtTicks(
        account,
        0,
        positionIdListTo,
        currentTick,
        atTicks,
        COMPUTE_PREMIA_AS_COLLATERAL,
        NO_BUFFER
    );
    numberOfTicks = atTicks.length;
}
{
    uint256 toLength = positionIdListTo.length;
    uint256 finalLength = positionIdListToFinal.length;
    // if account is solvent at all ticks, this is a force exercise or a settlePremium.
    if (solvent == numberOfTicks) {
        unchecked {
            tokenId = positionIdListTo[toLength - 1];
            if (toLength == finalLength) {
                // same length, that's a settle
                {
                    bytes32 toHash = EfficientHash.efficientKeccak256(
                        abi.encodePacked(positionIdListTo)
                    );
                    bytes32 finalHash = EfficientHash.efficientKeccak256(
                        abi.encodePacked(positionIdListToFinal)
                    );
                    if (toHash != finalHash) {
                        revert Errors.InputListFail();
                    }
                }
                _settlePremium(account, tokenId, twapTick, currentTick);
            } else if (toLength == (finalLength + 1)) {
                // final is one element shorter, that's a force exercise
                if (tokenId.countLongs() == 0 || tokenId.validateIsExercisable() == 0)
                    revert Errors.NoLegsExercisable();
                _forceExercise(account, tokenId, twapTick, currentTick);
            } else if (finalLength == 0) {
                // if final length was zero, this was intended to be liquidation, but revert because not margin ...
                revert Errors.NotMarginCalled();
            } else {
                // otherwise, wrong input lists
                revert Errors.InputListFail();
            }
        }
        // ensure the callee is still solvent after the operation
        bool premiaAsCollateral = usePremiaAsCollateral.rightSlot() > 0;
        _validateSolvency(
            account,
            positionIdListToFinal,
            NO_BUFFER,
            premiaAsCollateral,
            0
        );
    }
    } else if (solvent == 0) {
        // if account is insolvent at all ticks, this is a liquidation
        // if the positions lengths are the same, this was intended as a settlePremium, but revert because accoun...
        if (toLength == finalLength) revert Errors.AccountInsolvent(solvent, 4);
        if (positionIdListToFinal.length != 0) revert Errors.InputListFail();
        // if the final position list has a non-zero length, this can't be a complete liquidation, revert
        _liquidate(account, positionIdListTo, twapTick, currentTick);
    } else {
        // otherwise, revert because the account is not fully margin called
        revert Errors.NotMarginCalled();
    }
}
// ensure the caller is still solvent after the operation
_validateSolvency(
    msg.sender,
    positionIdListFrom,
    NO_BUFFER,
    usePremiaAsCollateral.leftSlot() > 0,
    0
);
}
/// @notice Liquidates a distressed account. Will burn all positions and issue a bonus to the liquidator.
/// @dev Will revert if liquidated account is solvent at one of the oracle ticks or if TWAP tick is too far away fro...

```



```

/// @param liquidatee Address of the distressed account
/// @param positionIdList List of positions owned by the user. Written as `[tokenId1, tokenId2, ...]`
function _liquidate(
    address liquidatee,
    TokenId[] calldata positionIdList,
    int24 twapTick,
    int24 currentTick
) internal {
    LeftRightUnsigned tokenData0;
    LeftRightUnsigned tokenData1;
    LeftRightUnsigned shortPremium;
    {
        PositionBalance[] memory positionBalanceArray = new PositionBalance[](
            positionIdList.length
        );
        LeftRightUnsigned longPremium;
        (shortPremium, longPremium, positionBalanceArray) = _calculateAccumulatedPremia(
            liquidatee,
            positionIdList,
            COMPUTE_PREMIA_AS_COLLATERAL,
            ONLY_AVAILABLE_PREMIUM,
            currentTick
        );
        (tokenData0, tokenData1, ) = riskEngine().getMargin(
            positionBalanceArray,
            twapTick,
            liquidatee,
            positionIdList,
            shortPremium,
            longPremium,
            collateralToken0(),
            collateralToken1()
        );
    }
    // The protocol delegates some virtual shares to ensure the burn can be settled.
    collateralToken0().delegate(liquidatee);
    collateralToken1().delegate(liquidatee);
    LeftRightSigned bonusAmounts;
    LeftRightUnsigned haircutTotal;
    {
        LeftRightSigned netPaid;
        LeftRightSigned[4][] memory premiasByLeg;
        // burn all options from the liquidatee
        // Do not commit any settled long premium to storage - we will do this after we determine if any long premiu...
        // This is to prevent any short positions the liquidatee has being settled with tokens that will later be re...
        // NOTE: tick limits are not applied here since it is not the liquidator's position being liquidated
        (netPaid, premiasByLeg) = _burnAllOptionsFrom(
            liquidatee,
            MIN_SWAP_TICK,
            MAX_SWAP_TICK,
            DONOT_COMMIT_LONG_SETTLED,
            positionIdList
        );
        LeftRightSigned collateralRemaining;
        // compute bonus amounts using latest tick data
        (bonusAmounts, collateralRemaining) = riskEngine().getLiquidationBonus(
            tokenData0,
            tokenData1,
            Math.getSqrtRatioAtTick(twapTick),
            netPaid,
            shortPremium
        );
        // premia cannot be paid if there is protocol loss associated with the liquidatee
        // otherwise, an economic exploit could occur if the liquidator and liquidatee collude to
        // manipulate the fees in a liquidity area they control past the protocol loss threshold
        // such that the PLPs are forced to pay out premia to the liquidator
        // thus, we haircut any premium paid by the liquidatee (converting tokens as necessary) until the protocol l...
        // note that the haircutPremia function also commits the settled amounts (adjusted for the haircut) to stora...
        // if premium is haircut from a token that is not in protocol loss, some of the liquidation bonus will be co...
        address _liquidatee = liquidatee;
        int24 _twapTick = twapTick;
        TokenId[] memory _positionIdList = positionIdList;
        LeftRightSigned bonusDeltas;
        LeftRightSigned[4][] memory haircutPerLeg;
        (bonusDeltas, haircutTotal, haircutPerLeg) = riskEngine().haircutPremia(
            _liquidatee,
            _positionIdList,
            premiasByLeg,
            collateralRemaining,
            Math.getSqrtRatioAtTick(_twapTick)
        );
        bonusAmounts = bonusAmounts.add(bonusDeltas);
        InteractionHelper.settleAmounts(
            _liquidatee,

```

```

        _positionIdList,
        haircutTotal,
        haircutPerLeg,
        premiasByLeg,
        collateralToken0(),
        collateralToken1(),
        s_settledTokens
    );
}
// revoke delegated virtual shares and settle any bonus deltas with the liquidator
// native currency is represented as address(0), so it will always be currency0 alphanumerically
collateralToken0().settleLiquidation{value: msg.value}(
    msg.sender,
    liquidatee,
    bonusAmounts.rightSlot()
);
collateralToken1().settleLiquidation(msg.sender, liquidatee, bonusAmounts.leftSlot());
emit AccountLiquidated(msg.sender, liquidatee, bonusAmounts);
}
/// @notice Force the exercise of a single position. Exercisor will have to pay a fee to the force exercisee.
/// @param account Address of the distressed account
/// @param tokenId The position to be force exercised; this position must contain at least one out-of-range long leg
function _forceExercise(
    address account,
    TokenId tokenId,
    int24 twapTick,
    int24 currentTick
) internal {
    CollateralTracker ct0 = collateralToken0();
    CollateralTracker ct1 = collateralToken1();
    uint128 positionSize;
    LeftRightSigned exerciseFees;
    {
        PositionBalance positionBalance = s_positionBalance[account][tokenId];
        positionSize = positionBalance.positionSize();
        if (positionSize == 0) revert Errors.PositionNotOwned();
        // Compute the exerciseFee, this will decrease the further away the price is from the exercised position
        // Include any deltas in long legs between the current and oracle tick in the exercise fee
        exerciseFees = riskEngine().exerciseCost(
            currentTick,
            twapTick,
            tokenId,
            positionBalance
        );
    }
    // The protocol delegates some virtual shares to ensure the burn can be settled.
    ct0.delegate(account);
    ct1.delegate(account);
    {
        int24[2] memory tickLimits;
        tickLimits[0] = MIN_SWAP_TICK;
        tickLimits[1] = MAX_SWAP_TICK;
        (RiskParameters riskParameters, ) = getRiskParameters(0);
        // Exercise the option
        // Turn off ITM swapping to prevent swap at potentially unfavorable price
        _burnOptions(
            tokenId,
            positionSize,
            tickLimits,
            account,
            COMMIT_LONG_SETTLED,
            riskParameters
        );
    }
    // redistribute token composition of refund amounts if user doesn't have enough of one token to pay
    LeftRightSigned refundAmounts = riskEngine().getRefundAmounts(
        account,
        exerciseFees,
        twapTick,
        ct0,
        ct1
    );
    // settle difference between delegated amounts (from the protocol) and exercise fees/substituted tokens
    ct0.refund(account, msg.sender, refundAmounts.rightSlot());
    ct1.refund(account, msg.sender, refundAmounts.leftSlot());
    // revoke the virtual shares that were delegated after settling the difference with the exercisor
    ct0.revoke(account);
    ct1.revoke(account);
    emit ForcedExercised(msg.sender, account, tokenId, exerciseFees);
}
/// @notice Settle unpaid premium for one `legIndex` on a position owned by `owner`.
/// @dev Called by sellers on buyers of their chunk to increase the available premium for withdrawal (before closing...
/// @dev This feature is only available when `owner` is solvent and has the requisite tokens to settle the premium.
/// @param owner The owner of the option position to make premium payments on

```

```

/// @param tokenId The position to be force exercised; this position must contain at least one out-of-range long leg
function _settlePremium(
    address owner,
    TokenId tokenId,
    int24 twapTick,
    int24 currentTick
) internal {
    CollateralTracker ct0 = collateralToken0();
    CollateralTracker ct1 = collateralToken1();
    // The protocol delegates some virtual shares to ensure the premia can be settled.
    ct0.delegate(owner);
    ct1.delegate(owner);
    (RiskParameters riskParameters, ) = getRiskParameters(0);
    uint128 positionSize = s_positionBalance[owner][tokenId].positionSize();
    if (positionSize == 0) revert Errors.PositionNotOwned();
    _settleOptions(owner, tokenId, positionSize, riskParameters, currentTick);
    LeftRightSigned refundAmounts = riskEngine().getRefundAmounts(
        owner,
        LeftRightSigned.wrap(0),
        twapTick,
        ct0,
        ct1
    );
    // allow the caller to settle tokens owed to the protocol by the settlee in exchange for the surplus token
    ct0.refund(owner, msg.sender, refundAmounts.rightSlot());
    ct1.refund(owner, msg.sender, refundAmounts.leftSlot());
    ct0.revoke(owner);
    ct1.revoke(owner);
}

/*//////////////////////
                        SOLVENCY CHECKS
//////////////////////*/
/// @notice Check whether an account is solvent at a given `atTick` with a collateral requirement of `buffer/10_000`...
/// @dev Reverts if `account` is not solvent at all provided ticks and `expectedSolvent == true`, or if `account` is...
/// @param account The account to check solvency for
/// @param safeMode The current safe mode status
/// @param positionIdList The list of positions to check solvency for
/// @param currentTick The current tick of the Uniswap pool (needed for fee calculations)
/// @param atTicks An array of ticks to check solvency at
/// @param buffer The buffer to apply to the collateral requirement
/// @param usePremiaAsCollateral whether to compute accumulated premia for all legs held by the user for collateral ...
/// @return boolean flag that determines if account is solvent
function _checkSolvencyAtTicks(
    address account,
    uint8 safeMode,
    TokenId[] calldata positionIdList,
    int24 currentTick,
    int24[] memory atTicks,
    bool usePremiaAsCollateral,
    uint256 buffer
) internal view returns (uint256) {
    (
        LeftRightUnsigned shortPremium,
        LeftRightUnsigned longPremium,
        PositionBalance[] memory positionBalanceArray
    ) = _calculateAccumulatedPremia(
        account,
        positionIdList,
        usePremiaAsCollateral,
        ONLY_AVAILABLE_PREMIUM,
        currentTick
    );
    // if safeMode is ON, make the collateral requirements for 100% utilizations: no cross-margining, fully covered ...
    if (safeMode &gt; 0) {
        unchecked {
            // cannot miscast because DECIMAL = 10_000
            uint32 maxUtilizations = uint32(DECIMALS + (DECIMALS &lt;&lt; 16));
            positionBalanceArray[0] = PositionBalanceLibrary.storeBalanceData(
                positionBalanceArray[0].positionSize(),
                maxUtilizations,
                0
            );
        }
    }
    uint256 solvent;
    for (uint256 i; i &lt; atTicks.length; ) {
        unchecked {
            if (
                _isAccountSolvent(
                    account,
                    atTicks[i],
                    positionIdList,
                    positionBalanceArray,
                    shortPremium,

```

```

        longPremium,
        buffer
    )
    ) ++solvent;
    ++i;
}
}
return solvent;
}
/// @notice Check whether an account is solvent at a given `atTick` with a collateral requirement of `buffer/10_000`...
/// @param account The account to check solvency for
/// @param atTick The tick to check solvency at
/// @param positionIdList The list of all option positions held by the user
/// @param positionBalanceArray A list of balances and pool utilization for each position, of the form `[[tokenId0, ...
/// @param shortPremium The total amount of premium (prorated by available settled tokens) owed to the short legs of...
/// @param longPremium The total amount of premium owed by the long legs of `account`
/// @param buffer The buffer to apply to the collateral requirement
/// @return Whether the account is solvent at the given tick
function _isAccountSolvent(
    address account,
    int24 atTick,
    TokenId[] calldata positionIdList,
    PositionBalance[] memory positionBalanceArray,
    LeftRightUnsigned shortPremium,
    LeftRightUnsigned longPremium,
    uint256 buffer
) internal view returns (bool) {
    return
        riskEngine().isAccountSolvent(
            positionBalanceArray,
            positionIdList,
            atTick,
            account,
            shortPremium,
            longPremium,
            collateralToken0(),
            collateralToken1(),
            buffer
        );
}
/// @notice Get risk parameters from the risk engine.
/// @dev Also checks whether the current tick has deviated too much from the previously stored ticks. Computed in th...
function getRiskParameters(
    uint256 builderCode
) public view returns (RiskParameters riskParameters, int24 currentTick) {
    currentTick = getCurrentTick();
    riskParameters = riskEngine().getRiskParameters(currentTick, s_oraclePack, builderCode);
}
/// @notice Checks whether the current tick has deviated too much from the previously stored ticks. Computed in the ...
/// @return Whether the current tick has deviated too much to warrant putting the protocol in safe mode
function isSafeMode() external view returns (uint8) {
    (RiskParameters riskParameters, ) = getRiskParameters(0);
    return riskParameters.safeMode();
}
}
/*//////////////////////////////////////
        POSITIONS HASH GENERATION & VALIDATION
//////////////////////////////////////*/
/// @notice Makes sure that the positions in the incoming user's list match the existing active option positions.
/// @param account The owner of the incoming list of positions
/// @param positionIdList The existing list of active options for the owner
function _validatePositionList(
    address account,
    TokenId[] calldata positionIdList
) internal view {
    uint256 pLength = positionIdList.length;
    uint256 fingerprintIncomingList;
    // verify it has no duplicated elements
    if (!PanopticMath.hasNoDuplicateTokenIds(positionIdList)) {
        revert Errors.DuplicateTokenId();
    }
    uint64 _poolId = poolId();
    for (uint256 i = 0; i < pLength; ) {
        TokenId tokenId = positionIdList[i];
        // make sure the tokenId is for this Panoptic pool
        if (tokenId.poolId() != _poolId) revert Errors.WrongPoolId();
        fingerprintIncomingList = PanopticMath.updatePositionsHash(
            fingerprintIncomingList,
            tokenId,
            ADD
        );
    }
    unchecked {
        ++i;
    }
}
}

```

```

    // revert if fingerprint for provided `_positionIdList` does not match the one stored for the `_account`
    if (fingerprintIncomingList != s_positionsHash[account]) revert Errors.InputListFail();
}
/// @notice Updates the hash for all positions owned by an account. This fingerprints the list of all incoming optio...
/// @dev The outcome of this function will be to update the hash of positions.
/// This is done as a duplicate/validation check of the incoming list O(N).
/// @dev The positions hash is stored as the XOR of the keccak256 of each tokenId. Updating will XOR the existing ha...
/// The same update can either add a new tokenId (when minting an option), or remove an existing one (when burning i...
/// @param account The owner of `tokenId`
/// @param tokenId The option position
/// @param addFlag Whether to add `tokenId` to the hash (true) or remove it (false)
function _updatePositionsHash(
    address account,
    TokenId tokenId,
    bool addFlag,
    uint8 maxLegs
) internal {
    // Get the current position hash value (fingerprint of all pre-existing positions created by `_account`)
    // Add the current tokenId to the positionsHash as XOR'd
    // since  $0 \wedge x = x$ , no problem on first mint
    // Store values back into the user option details with the updated hash (leaves the other parameters unchanged)
    uint256 newHash = PanopticMath.updatePositionsHash(
        s_positionsHash[account],
        tokenId,
        addFlag
    );
    if ((newHash >> 248) > maxLegs) revert Errors.TooManyLegsOpen();
    s_positionsHash[account] = newHash;
}
/*//////////////////////
                        QUERIES
//////////////////////*/
/// @notice Computes and returns all oracle ticks.
/// @return currentTick The current tick in the Uniswap pool
/// @return spotTick The fast oracle tick, sourced from the internal 10-minute EMA.
/// @return medianTick The slow oracle tick, calculated as the median of the 8 stored price points in the internal o...
/// @return latestTick The reconstructed absolute tick of the latest observation stored in the internal oracle.
/// @return oraclePack The current value of the 8-slot internal observation queue (`s_oraclePack`)
function getOracleTicks()
    external
    view
    returns (
        int24 currentTick,
        int24 spotTick,
        int24 medianTick,
        int24 latestTick,
        OraclePack oraclePack
    )
{
    currentTick = getCurrentTick();
    (spotTick, medianTick, latestTick, ) = riskEngine().getOracleTicks(
        currentTick,
        s_oraclePack
    );
    oraclePack = s_oraclePack;
}
/// @notice Get the current number of legs across all open positions for an account.
/// @param user The account to query
/// @return Number of legs across the open positions of `user`
function numberOfLegs(address user) external view returns (uint256) {
    return s_positionsHash[user] >> 248;
}
/// @notice Get the `tokenId` position data for `user`.
/// @param user The account that owns `tokenId`
/// @param tokenId The position to query
/// @return `currentTick` at mint
/// @return Fast oracle tick at mint
/// @return Slow oracle tick at mint
/// @return Last observed tick at mint
/// @return Utilization of token0 at mint
/// @return Utilization of token1 at mint
/// @return Size of the position
function positionData(
    address user,
    TokenId tokenId
) external view returns (int24, int24, int24, int24, int256, int256, uint128) {
    return s_positionBalance[user][tokenId].unpackAll();
}
/// @notice Get the oracle price used to check solvency in liquidations.
/// @return twapTick The current oracle price used to check solvency in liquidations
function getTWAP() public view returns (int24 twapTick) {
    twapTick = riskEngine().twapEMA(s_oraclePack);
}
/// @notice Get the current tick of the underlying pool.

```

```

function getCurrentTick() public view returns (int24 currentTick) {
    currentTick = SFPM.getCurrentTick(poolKey());
}
}
/*/////////////////////////////////////////////////////////////////////////
    PREMIA & PREMIA SPREAD CALCULATIONS
////////////////////////////////////////////////////////////////////////*/
/// @notice Ensure the effective liquidity in a given chunk is above a certain threshold.
/// @param tokenId An option position
/// @param leg A leg index of `tokenId` corresponding to a tickLower-tickUpper chunk
/// @param effectiveLiquidityLimit Maximum amount of "spread" defined as removedLiquidity/netLiquidity for a new pos...
/// denominated as X10_000 = (`ratioLimit * 10_000`)
/// @return totalLiquidity The total liquidity deposited in that chunk: `totalLiquidity = netLiquidity + removedLiqu...
function _checkLiquiditySpread(
    TokenId tokenId,
    uint256 leg,
    uint256 effectiveLiquidityLimit
) internal view returns (uint256 totalLiquidity) {
    uint256 netLiquidity;
    uint256 removedLiquidity;
    (totalLiquidity, netLiquidity, removedLiquidity) = _getLiquidities(tokenId, leg);
    // compute and return effective liquidity. Return if short=net=0, which is closing short position
    if (netLiquidity == 0 && removedLiquidity == 0) return totalLiquidity;
    if (netLiquidity == 0) revert Errors.NetLiquidityZero();
    uint256 effectiveLiquidityFactor;
    unchecked {
        // cannot overflow because liquidities are uint128
        effectiveLiquidityFactor = (removedLiquidity * DECIMALS) / netLiquidity;
    }
    // put a limit on how much new liquidity in one transaction can be deployed into this leg
    // the effective liquidity measures how many times more the newly added liquidity is compared to the existing/ba...
    if (effectiveLiquidityFactor > effectiveLiquidityLimit)
        revert Errors.EffectiveLiquidityAboveThreshold();
}
/// @notice Compute the premia collected for a single option position `tokenId`.
/// @param tokenId The option position
/// @param positionSize The number of contracts (size) of the option position
/// @param owner The holder of the tokenId option
/// @param usePremiaAsCollateral Whether to compute accumulated premia for all legs held by the user for collateral ...
/// @param atTick The tick at which the premia is calculated -> use (`atTick < type(int24).max`) to compute it
/// up to current block. `atTick = type(int24).max` will only consider fees as of the last on-chain transaction
/// @return premiaByLeg The amount of premia owed to the user for each leg of the position
/// @return premiumAccumulatorsByLeg The amount of premia accumulated for each leg of the position
function _getPremia(
    TokenId tokenId,
    uint128 positionSize,
    address owner,
    bool usePremiaAsCollateral,
    int24 atTick
) internal
view
returns (
    LeftRightSigned[4] memory premiaByLeg,
    uint256[2][4] memory premiumAccumulatorsByLeg
)
{
    uint256 numLegs = tokenId.countLegs();
    for (uint256 leg = 0; leg < numLegs; ) {
        uint256 isLong = tokenId.isLong(leg);
        if (tokenId.width(leg) != 0 && (isLong == 1 || usePremiaAsCollateral)) {
            LiquidityChunk liquidityChunk = PanopticMath.getLiquidityChunk(
                tokenId,
                leg,
                positionSize
            );
            {
                uint256 vegoid = tokenId.vegoid();
                uint256 tokenType = tokenId.tokenType(leg);
                int24 _atTick = atTick;
                (premiumAccumulatorsByLeg[leg][0], premiumAccumulatorsByLeg[leg][1]) = SFPM
                    .getAccountPremia(
                        poolKey(),
                        address(this),
                        tokenType,
                        liquidityChunk.tickLower(),
                        liquidityChunk.tickUpper(),
                        _atTick,
                        isLong,
                        vegoid
                    );
            }
            unchecked {
                LeftRightUnsigned premiumAccumulatorLast = s_options[owner][tokenId][leg];
                premiaByLeg[leg] = LeftRightSigned

```

```

        .wrap(0)
        .addToRightSlot(
            int128(
                int256(
                    ((premiumAccumulatorsByLeg[leg][0] -
                     premiumAccumulatorLast.rightSlot()) *
                     (liquidityChunk.liquidity())) / 2 ** 64
                )
            )
        )
        .addToLeftSlot(
            int128(
                int256(
                    ((premiumAccumulatorsByLeg[leg][1] -
                     premiumAccumulatorLast.leftSlot()) *
                     (liquidityChunk.liquidity())) / 2 ** 64
                )
            )
        );
    }
    if (isLong == 1) {
        premiaByLeg[leg] = LeftRightSigned.wrap(0).sub(premiaByLeg[leg]);
    }
}
unchecked {
    ++leg;
}
}

}
}

/*////////////////////
AVAILABLE PREMIUM LOGIC
////////////////////*/
/// @notice Query the amount of premium available for withdrawal given a certain `premiumOwed` for a chunk.
/// @dev Based on the ratio between `settledTokens` and the total premium owed to sellers in a chunk.
/// @dev The ratio is capped at 1 (as the base ratio can be greater than one if some seller forfeits enough premium)...
/// @param totalLiquidity The updated total liquidity amount for the chunk
/// @param settledTokens LeftRight accumulator for the amount of tokens that have been settled (collected or paid)
/// @param grossPremiumLast The `last` values used with `premiumAccumulators` to compute the total premium owed to s...
/// @param premiumOwed The amount of premium owed to sellers in the chunk
/// @param premiumAccumulators The current values of the premium accumulators for the chunk
/// @return The amount of token0/token1 premium available for withdrawal
function _getAvailablePremium(
    uint256 totalLiquidity,
    LeftRightUnsigned settledTokens,
    LeftRightUnsigned grossPremiumLast,
    LeftRightUnsigned premiumOwed,
    uint256[2] memory premiumAccumulators
) internal pure returns (LeftRightUnsigned) {
    unchecked {
        // long premium only accumulates as it is settled, so compute the ratio
        // of total settled tokens in a chunk to total premium owed to sellers and multiply
        // cap the ratio at 1 (it can be greater than one if some seller forfeits enough premium)
        uint256 accumulated0 = ((premiumAccumulators[0] - grossPremiumLast.rightSlot()) *
            totalLiquidity) / 2 ** 64;
        uint256 accumulated1 = ((premiumAccumulators[1] - grossPremiumLast.leftSlot()) *
            totalLiquidity) / 2 ** 64;
        return (
            LeftRightUnsigned
                .wrap(
                    uint128(
                        Math.min(
                            (uint256(premiumOwed.rightSlot()) * settledTokens.rightSlot()) /
                                (accumulated0 == 0 ? type(uint256).max : accumulated0),
                            premiumOwed.rightSlot()
                        )
                    )
                )
                .addToLeftSlot(
                    uint128(
                        Math.min(
                            (uint256(premiumOwed.leftSlot()) * settledTokens.leftSlot()) /
                                (accumulated1 == 0 ? type(uint256).max : accumulated1),
                            premiumOwed.leftSlot()
                        )
                    )
                )
            );
    }
}

}
}

/// @notice Query the total amount of liquidity sold in the corresponding chunk for a position leg.
/// @dev totalLiquidity (total sold) = removedLiquidity + netLiquidity (in AMM).
/// @param tokenId The option position
/// @param leg The leg of the option position to get `totalLiquidity` for
/// @return totalLiquidity The total amount of liquidity sold in the corresponding chunk for a position leg

```

```

    /// @return netLiquidity The amount of liquidity available in the corresponding chunk for a position leg
    /// @return removedLiquidity The amount of liquidity removed through buying in the corresponding chunk for a positio...
    function _getLiquidities(
        TokenId tokenId,
        uint256 leg
    )
    internal
    view
    returns (uint256 totalLiquidity, uint128 netLiquidity, uint128 removedLiquidity)
    {
        (int24 tickLower, int24 tickUpper) = tokenId.asTicks(leg);
        LeftRightUnsigned accountLiquidities = SFPM.getAccountLiquidity(
            poolKey(),
            address(this),
            tokenId.tokenType(leg),
            tickLower,
            tickUpper
        );
        netLiquidity = accountLiquidities.rightSlot();
        removedLiquidity = accountLiquidities.leftSlot();
        unchecked {
            totalLiquidity = netLiquidity + removedLiquidity;
        }
    }
}

```

[2/12] contracts/RiskEngine.sol

```

// SPDX-License-Identifier: BUSL-1.1
pragma solidity ^0.8.24;
// Interfaces
import {CollateralTracker} from "./CollateralTracker.sol";
import {PanopticPool} from "./PanopticPool.sol";
// Libraries
import {Constants} from "@libraries/Constants.sol";
import {Errors} from "@libraries/Errors.sol";
import {Math} from "@libraries/Math.sol";
import {PanopticMath} from "@libraries/PanopticMath.sol";
import {SafeTransferLib} from "@libraries/SafeTransferLib.sol";
// Custom types
import {LeftRightUnsigned, LeftRightSigned} from "@types/LeftRight.sol";
import {LiquidityChunk} from "@types/LiquidityChunk.sol";
import {PositionBalance, PositionBalanceLibrary} from "@types/PositionBalance.sol";
import {RiskParameters, RiskParametersLibrary} from "@types/RiskParameters.sol";
import {TokenId} from "@types/TokenId.sol";
import {OraclePack} from "@types/OraclePack.sol";
import {MarketState} from "@types/MarketState.sol";
/// @title Panoptic Risk Engine: The central risk assessment and solvency calculator for the Panoptic Protocol.
/// @author Axicon Labs Limited
/// @notice This contract serves as the central logic hub for calculating collateral requirements, account solvency, and...
/// @dev This contract does not hold funds or state regarding user balances. Instead, it provides the mathematical frame...
/// 1. Calculate collateral requirements for complex option strategies (Spreads, Strangles, Synthetic positions).
/// 2. Manage the internal pricing Oracle, utilizing volatility safeguards, EMAs, and median filters to prevent manipula...
/// 3. Compute the Adaptive Interest Rate based on pool utilization (PID controller logic).
///
/// Key responsibilities:
/// - Verifying if an account is solvent (`isAccountSolvent`).
/// - Calculating the cost to force-exercise a position (`exerciseCost`).
/// - Determining liquidation bonuses (`getLiquidationBonus`).
/// - Calculating dynamic collateral ratios based on pool utilization.
contract RiskEngine {
    using Math for uint256;
    /// @notice Emitted when a borrow rate is updated.
    event BorrowRateUpdated(
        address indexed collateralToken,
        uint256 avgBorrowRate,
        uint256 rateAtTarget
    );
    /// @notice Emitted when tokens are collected from the contract
    /// @param token The address of the token collected
    /// @param recipient The address receiving the tokens
    /// @param amount The amount of tokens collected
    event TokensCollected(address indexed token, address indexed recipient, uint256 amount);
    /*//////////////////////////////////////////////////////////////
                                CONSTANTS
    /*//////////////////////////////////////////////////////////////
    /// @notice Decimals for computation (1 millitick (1/1000th of a basis point) precision: 1e-7 = 0.00001%).
    /// @dev uint type for composability with unsigned integer based mathematical operations.
    uint256 internal constant DECIMALS = 10_000_000; //e ██████████
    int16 internal constant MAX_UTILIZATION = 10_000; //e ██████████ 100%

```



```

uint256 internal constant LND_SCALAR = 6931472; //e ln(2)
uint256 internal constant ONE_BPS = 1000; //e 1 0.01%
uint256 internal constant TEN_BPS = 10000; //e 10 0.1%
//int256 constant EMA_PERIOD_SPOT = 120; // 2 minutes
//int256 constant EMA_PERIOD_FAST = 240; // 4 minutes
//int256 constant EMA_PERIOD_SLOW = 600; // 10 minutes
//int256 constant EMA_PERIOD_EONS = 1800; // 30 minutes
//e EMA uint96
uint96 constant EMA_PERIODS = uint96(120 + (240 &lt;&lt; 24) + (600 &lt;&lt; 48) + (1800 &lt;&lt; 72));
/// @notice The maximum allowed cumulative delta between the fast & slow oracle tick, the current & slow ora...
/// @dev Falls back on the more conservative (less solvent) tick during times of extreme volatility, where the price...
int256 internal constant MAX_TICKS_DELTA = 953;
/// @notice The maximum allowed delta between the currentTick and the Uniswap TWAP tick during a liquidation (~5% do...
/// @dev Mitigates manipulation of the currentTick that causes positions to be liquidated at a less favorable price.
uint16 internal constant MAX_TWAP_DELTA_LIQUIDATION = 513; //e currentTick Uniswap TWAP tick -5%
/// @notice The maximum allowed ratio for a single chunk, defined as `removedLiquidity / netLiquidity`.
/// @dev The long premium spread multiplier that corresponds with the MAX_SPREAD value depends on VEGOID,
/// which can be explored in this calculator: [https://www.desmos.com/calculator/mdeqob2m04](https://www.desmos.com/...
uint24 internal constant MAX_SPREAD = 90_000; //e chunk
/// @notice Multiplier in basis points for the collateral requirement in the event of a buying power decrease, such ...
/// @dev must fit inside a uint26
uint32 internal constant BP_DECREASE_BUFFER = 13_333_333; //e
/// @notice Decimals for WAD calculations.
int256 internal constant WAD = 1e18;
/// @notice Constant, in seconds, used to determine the max elapsed time between adaptive interest rate updates.
/// @dev the time elapsed will be capped at IRM_MAX_ELAPSED_TIME
int256 public constant IRM_MAX_ELAPSED_TIME = 4096; //e
bytes32 internal constant BUILDER_SALT = keccak256("panoptic.builder"); //CREATE2
/// @notice The maximum amount of change, in ticks, permitted between internal median updates.
int24 internal constant MAX_CLAMP_DELTA = 149; //e tick
/// @notice Parameter used to modify the [equation](https://www.desmos.com/calculator/mdeqob2m04) of the utilization...
// v = 1/VEGOID = multiplicative factor for long premium (Eqns 1-5) //e v = 1/VEGOID = 0.25 long premium ...
// Similar to vega in options because the liquidity utilization is somewhat reflective of the implied volatility (IV...
// and vegoid modifies the sensitivity of the streamia to changes in that utilization, streamia
// much like vega measures the sensitivity of traditional option prices to IV.
// The effect of vegoid on the long premium multiplier can be explored here: https://www.desmos.com/calculator/mdeqo...
uint8 internal constant VEGOID = 4;
/////////////////////////////////////////////////////////////
RISK PARAMETERS
/////////////////////////////////////////////////////////////
/// @notice The notional fee, in basis points, collected from PLPs at option mint.
/// @dev can never exceed 10000, so this value must fit inside a uint14 due to RiskParameters packing
uint16 constant NOTIONAL_FEE = 10;
/// @notice The premium fee, in basis points, collected from the premium paid/received.
/// @dev can never exceed 10000, so this value must fit inside a uint14 due to RiskParameters packing
uint16 constant PREMIUM_FEE = 0;
/// @notice The protocol split, in basis points, when a builder code is present.
/// @dev can never exceed 10000, so this value must fit inside a uint14 due to RiskParameters packing
uint16 constant PROTOCOL_SPLIT = 6_500;
/// @notice The builder split, in basis points, when a builder code is present
/// @dev can never exceed 10000, so this value must fit inside a uint14 due to RiskParameters packing
uint16 constant BUILDER_SPLIT = 2_500;
/// @notice Required collateral ratios for selling options, fraction of 1, scaled by 10_000_000.
/// @dev i.e 20% -> 0.2 * 10_000_000 = 2_000_000.
uint256 constant SELLER_COLLATERAL_RATIO = 2_000_000;
/// @notice Required collateral ratios for buying options, fraction of 1, scaled by 10_000_000.
/// @dev i.e 10% -> 0.1 * 10_000_000 = 1_000_000.
uint256 constant BUYER_COLLATERAL_RATIO = 1_000_000;
/// @notice Required collateral margin for loans in excess of notional, fraction of 1, scaled by 10_000_000.
uint256 constant MAINT_MARGIN_RATE = 2_000_000;
/// @notice Basal cost (in bps of notional) to force exercise an out-of-range position.
uint256 constant FORCE_EXERCISE_COST = 102_400;
// Targets a pool utilization (balance between buying and selling)
/// @notice Target pool utilization below which buying+selling is optimal, fraction of 1, scaled by 10_000_000.
/// @dev i.e 50% -> 0.5 * 10_000_000 = 5_000_000.
uint256 constant TARGET_POOL_UTIL = 5_000_000;
/// @notice Pool utilization above which selling is 100% collateral backed, fraction of 1, scaled by 10_000_000.
/// @dev i.e 90% -> 0.9 * 10_000_000 = 9_000_000.
uint256 constant SATURATED_POOL_UTIL = 9_000_000;
uint256 immutable CROSS_BUFFER_0;
uint256 immutable CROSS_BUFFER_1;
address immutable BUILDER_FACTORY;
bytes32 immutable BUILDER_INIT_CODE_HASH;
uint256 constant MAX_OPEN_LEGS = 33;
/////////////////////////////////////////////////////////////
IRM PARAMETERS
/////////////////////////////////////////////////////////////
/// @notice Curve steepness (scaled by WAD).
/// @dev Curve steepness = 4.
int256 public constant CURVE_STEEPNESS = 4 ether;
/// @notice Minimum rate at target per second (scaled by WAD).
/// @dev Minimum rate at target = 0.1% (minimum rate = 0.025%).
int256 public constant MIN_RATE_AT_TARGET = 0.001 ether / int256(365 days);
/// @notice Maximum rate at target per second (scaled by WAD).

```

```

/// @dev Maximum rate at target = 200% (maximum rate = 800%).
int256 public constant MAX_RATE_AT_TARGET = 2.0 ether / int256(365 days);
/// @notice Target utilization (scaled by WAD).
/// @dev Target utilization = 90%.
int256 public constant TARGET_UTILIZATION = 2 ether / int256(3);
/// @notice Initial rate at target per second (scaled by WAD).
/// @dev Initial rate at target = 4% (rate between 1% and 16%).
int256 public constant INITIAL_RATE_AT_TARGET = 0.04 ether / int256(365 days);
/// @notice Adjustment speed per second (scaled by WAD).
/// @dev The speed is per second, so the rate moves at a speed of ADJUSTMENT_SPEED * err each second (while being
/// continuously compounded).
/// @dev Adjustment speed = 50/year.
int256 public constant ADJUSTMENT_SPEED = 50 ether / int256(365 days);
/*/////////////////////////////////////////////////////////////////////////
    INITIALIZATION & PARAMETER SETTINGS
////////////////////////////////////////////////////////////////////////*/
/// @notice Set immutable parameters for the Collateral Tracker.
constructor(
    uint256 _crossBuffer0,
    uint256 _crossBuffer1,
    address _guardian,
    address _builderFactory
) {
    CROSS_BUFFER_0 = _crossBuffer0;
    CROSS_BUFFER_1 = _crossBuffer1;
    GUARDIAN = _guardian;
    BUILDER_FACTORY = _builderFactory;
    BUILDER_INIT_CODE_HASH = keccak256(
        abi.encodePacked(type(BuilderWallet).creationCode, abi.encode(BUILDER_FACTORY))
    );
}
/*/////////////////////////////////////////////////////////////////////////
    GUARDIAN
////////////////////////////////////////////////////////////////////////*/
/// @notice Address allowed to override the automatically computed safe mode.
/// @dev Guardian can only increase the effective safe mode, never relax it.
address public immutable GUARDIAN;
/// @notice Emitted when the guardian updates the enforced safe mode.
/// @param lockMode True when safe mode is forcibly locked, false when the lock is lifted.
event GuardianSafeModeUpdated(bool lockMode);
/// @notice Restricts a function to be callable only by the guardian address.
modifier onlyGuardian() {
    _onlyGuardian();
    _;
}
/// @dev Reverts unless the caller is the guardian.
function _onlyGuardian() internal view {
    if (msg.sender != address(GUARDIAN)) revert Errors.NotGuardian();
}
/// @notice Forces a PanopticPool into locked safe mode.
/// @dev Sets the pool's internal oracle pack into permanent safe-mode override
/// until explicitly unlocked by the guardian.
/// @param pool The PanopticPool to lock.
function lockPool(PanopticPool pool) external onlyGuardian {
    emit GuardianSafeModeUpdated(true);
    pool.lockSafeMode();
}
/// @notice Removes the forced safe-mode lock on a PanopticPool.
/// @dev Restores the pool to using only the automatically computed safe-mode level.
/// @param pool The PanopticPool to unlock.
function unlockPool(PanopticPool pool) external onlyGuardian {
    emit GuardianSafeModeUpdated(true); //q ██████████ false
    pool.unlockSafeMode();
}
/// @notice Returns the address of the guardian
/// @return The guardian address that can override safe mode
function guardian() external returns (address) {
    return GUARDIAN;
}
function _computeBuilderWallet(uint256 builderCode) internal view returns (address wallet) {
    if (builderCode == 0) return address(0);
    bytes32 salt = bytes32(builderCode);
    bytes32 h = keccak256(
        abi.encodePacked(bytes1(0xff), BUILDER_FACTORY, salt, BUILDER_INIT_CODE_HASH)
    );
    wallet = address(uint160(uint256(h)));
}
/*/////////////////////////////////////////////////////////////////////////
    TRANSFERS
////////////////////////////////////////////////////////////////////////*/
/// @notice Collects a specific amount of tokens from this contract
/// @param token The address of the ERC20 token to collect
/// @param recipient The address to send the tokens to
/// @param amount The amount of tokens to collect

```

```

function collect(address token, address recipient, uint256 amount) public onlyGuardian {
    if (amount == 0) revert Errors.BelowMinimumRedemption();
    SafeTransferLib.safeTransfer(token, recipient, amount);
    emit TokensCollected(token, recipient, amount);
}
/// @notice Collects all available tokens of a specific type from this contract
/// @param token The address of the ERC20 token to collect
/// @param recipient The address to send the tokens to
function collect(address token, address recipient) external onlyGuardian {
    // Get the full balance
    uint256 balance = SafeTransferLib.balanceOfOrZero(token, address(this));
    collect(token, recipient, balance);
}
/*//////////////////////
LIQUIDATION/FORCE EXERCISE CALCULATIONS
//////////////////////*/
/// @notice Substitutes surplus tokens to a caller in exchange for any potential token shortages prior to revoking v...
/// @param payor The address of the user being exercised/settled
/// @param fees If applicable, fees to debit from caller (rightSlot = currency0 left = currency1), 0 for `settleLong...
/// @param atTick The tick at which to convert between currency0/currency1 when redistributing the surplus tokens
/// @param ct0 The collateral tracker for currency0
/// @param ct1 The collateral tracker for currency1
/// @return The LeftRight-packed deltas for currency0/currency1 to move from the caller to the payor
function getRefundAmounts( // q test ████████ token0/token1 ████████chatGPT█
    address payor,
    LeftRightSigned fees,
    int24 atTick,
    CollateralTracker ct0,
    CollateralTracker ct1
) external view returns (LeftRightSigned) {
    uint160 sqrtPriceX96 = Math.getSqrtRatioAtTick(atTick);
    // keep everything checked to catch any under/overflow or miscastings
    {
        // if the refunder lacks sufficient currency0 to pay back the virtual shares, have the caller cover the diff...
        int128 fees0 = fees.rightSlot();
        uint256 feeShares0 = ct0.convertToShares(fees0 &lt; 0 ? uint128(-fees0) : uint128(fees0));
        // Liability (&gt;0) adds to shortage; Asset (&lt;0) subtracts from shortage
        int256 balanceShortage = int256(uint256(type(uint248).max)) -
            int256(ct0.balanceOf(payor)) +
            (fees0 &gt; 0 ? int256(feeShares0) : -int256(feeShares0));
        if (balanceShortage &gt; 0) {
            return
                LeftRightSigned
                    .wrap(0)
                    .addToRightSlot(
                        int128(
                            fees.rightSlot() -
                            int256(
                                Math.mulDivRoundingUp(
                                    uint256(balanceShortage),
                                    ct0.totalAssets(),
                                    ct0.totalSupply()
                                )
                            )
                        )
                    )
                )
                .addToLeftSlot(
                    int128(
                        int256(
                            PanopticMath.convert0to1RoundingUp(
                                ct0.convertToAssets(uint256(balanceShortage)),
                                sqrtPriceX96
                            )
                        ) + fees.leftSlot()
                    )
                )
        );
    }
    int128 fees1 = fees.leftSlot();
    uint256 feeShares1 = ct1.convertToShares(fees1 &lt; 0 ? uint128(-fees1) : uint128(fees1));
    // Liability (&gt;0) adds to shortage; Asset (&lt;0) subtracts from shortage
    balanceShortage =
        int256(uint256(type(uint248).max)) -
        int256(ct1.balanceOf(payor)) +
        (fees1 &gt; 0 ? int256(feeShares1) : -int256(feeShares1));
    if (balanceShortage &gt; 0) {
        return
            LeftRightSigned
                .wrap(0)
                .addToRightSlot(
                    int128(
                        int256(
                            PanopticMath.convert1to0RoundingUp(
                                ct1.convertToAssets(uint256(balanceShortage)),
                                sqrtPriceX96

```

```

        )
        ) + fees.rightSlot()
    )
    .addToLeftSlot(
        int128(
            fees.leftSlot() -
            int256(
                Math.mulDivRoundingUp(
                    uint256(balanceShortage),
                    ct1.totalAssets(),
                    ct1.totalSupply()
                )
            )
        )
    );
}

}
// otherwise, no need to deviate from the original deltas
return fees;
}

/// @notice Get the cost of exercising an option. Used during a forced exercise.
/// @notice This one computes the cost of calling the forceExercise function on a position:
/// - The forceExercisor will have to *pay* the exercisee because their position will be closed "against their will"
/// - The cost must be larger when the position is in-range, and should be minimal when it is out of range
/// @param currentTick The current price tick
/// @param oracleTick The price oracle tick
/// @param tokenId The position to be exercised
/// @param positionBalance The position data of the position to be exercised
/// @return exerciseFees The fees for exercising the option position
function exerciseCost(
    int24 currentTick,
    int24 oracleTick,
    TokenId tokenId,
    PositionBalance positionBalance
) external view returns (LeftRightSigned exerciseFees) {
    // keep everything checked to catch any under/overflow or miscastings
    LeftRightSigned longAmounts;
    // we find whether the price is within any leg; any in-range leg will have a cost. Otherwise, the force-exercise...
    bool hasLegsInRange;
    for (uint256 leg = 0; leg < tokenId.countLegs(); ++leg) {
        // short legs are not counted - exercise is intended to be based on long legs
        if (tokenId.isLong(leg) == 0) continue;
        // credit/loans are not counted
        if (tokenId.width(leg) == 0) continue;
        // compute notional moved, add to tally.
        (LeftRightSigned longs, ) = PanopticMath.calculateIOAmounts(
            tokenId,
            positionBalance.positionSize(),
            leg,
            true
        );
        longAmounts = longAmounts.add(longs);
    }
    {
        (int24 rangeDown, int24 rangeUp) = PanopticMath.getRangesFromStrike(
            tokenId.width(leg),
            tokenId.tickSpacing()
        );
        int24 _strike = tokenId.strike(leg);
        if ((currentTick < _strike + rangeUp) && (currentTick >= _strike - rangeDown)) {
            hasLegsInRange = true;
        }
    }
}
uint256 currentValue0;
uint256 currentValue1;
uint256 oracleValue0;
uint256 oracleValue1;
{
    LiquidityChunk liquidityChunk = PanopticMath.getLiquidityChunk(
        tokenId,
        leg,
        positionBalance.positionSize()
    );
    (currentValue0, currentValue1) = Math.getAmountsForLiquidity(
        currentTick,
        liquidityChunk
    );
    (oracleValue0, oracleValue1) = Math.getAmountsForLiquidity(
        oracleTick,
        liquidityChunk
    );
}
// reverse any token deltas between the current and oracle prices for the chunk the exercisee had to mint in...
// the outcome of current price crossing a long chunk will always be less favorable than the status quo, i.e...

```

```

        // if the current price is moved downward such that some part of the chunk is between the current and market...
        // the chunk composition will swap token1 for token0 at a price (token0/token1) more favorable than market (...
        // forcing the exercisee to provide more value in token0 than they would have provided in token1 at market, ...
        // (the excess value provided by the exercisee could then be captured in a return swap across their newly ad...
        exerciseFees = exerciseFees.sub(
            LeftRightSigned
                .wrap(0)
                .addToRightSlot(int128(uint128(currentValue0)) - int128(uint128(oracleValue0)))
                .addToLeftSlot(int128(uint128(currentValue1)) - int128(uint128(oracleValue1)))
        );
    }
    // NOTE: we HAVE to start with a negative number as the base exercise cost because when shifting a negative numb...
    // the result is rounded DOWN and NOT toward zero
    // this divergence is observed when n (the number of half ranges) is > 10 (ensuring the floor is not zero, bu...
    // subtract 1 from max half ranges from strike so fee starts at FORCE_EXERCISE_COST when moving OTM
    int256 fee = hasLegsInRange ? -int256(FORCE_EXERCISE_COST) : -int256(ONE_BPS);
    // store the exercise fees in the exerciseFees variable
    exerciseFees = exerciseFees
        .addToRightSlot(int128((longAmounts.rightSlot() * fee) / int256(DECIMALS)))
        .addToLeftSlot(int128((longAmounts.leftSlot() * fee) / int256(DECIMALS)));
}
/// @notice Compute the pre-haircut liquidation bonuses to be paid to the liquidator and the protocol loss caused by...
/// @param tokenData0 LeftRight encoded word with balance of token0 in the right slot, and required balance in left ...
/// @param tokenData1 LeftRight encoded word with balance of token1 in the right slot, and required balance in left ...
/// @param atSqrtPriceX96 The oracle price used to swap tokens between the liquidator/liquidatee and determine solve...
/// @param netPaid The net amount of tokens paid/received by the liquidatee to close their portfolio of positions
/// @param shortPremium Total owed premium (prorated by available settled tokens) across all short legs being liquid...
/// @return The LeftRight-packed bonus amounts to be paid to the liquidator for both tokens (may be negative)
/// @return The LeftRight-packed protocol loss (pre-haircut) for both tokens, i.e., the delta between the user's sta...
function getLiquidationBonus(
    LeftRightUnsigned tokenData0,
    LeftRightUnsigned tokenData1,
    uint160 atSqrtPriceX96,
    LeftRightSigned netPaid,
    LeftRightUnsigned shortPremium
) external pure returns (LeftRightSigned, LeftRightSigned) {
    int256 bonus0;
    int256 bonus1;
    // keep everything checked to catch any under/overflow or miscastings
    {
        // compute bonus as min(collateralBalance/2, required-collateralBalance)
        {
            // compute the ratio of token0 to total collateral requirements
            // evaluate at TWAP price to maintain consistency with solvency calculations
            (uint256 balanceCross, uint256 thresholdCross) = PanopticMath.getCrossBalances(
                tokenData0,
                tokenData1,
                atSqrtPriceX96
            );
            uint256 bonusCross = Math.min(balanceCross / 2, thresholdCross - balanceCross);
            // `bonusCross` and `thresholdCross` are returned in terms of the lowest-priced token
            if (atSqrtPriceX96 < Constants.FP96) {
                // required0 / (required0 + token0(required1))
                uint256 requiredRatioX128 = Math.mulDiv(
                    tokenData0.leftSlot(),
                    2 ** 128,
                    thresholdCross
                );
                uint256 bonus0U = Math.mulDiv128(bonusCross, requiredRatioX128);
                bonus0 = int256(bonus0U);
                bonus1 = int256(PanopticMath.convert0to1(bonusCross - bonus0U, atSqrtPriceX96));
            } else {
                // required1 / (token1(required0) + required1)
                uint256 requiredRatioX128 = Math.mulDiv(
                    tokenData1.leftSlot(),
                    2 ** 128,
                    thresholdCross
                );
                uint256 bonus1U = Math.mulDiv128(bonusCross, requiredRatioX128);
                bonus1 = int256(bonus1U);
                bonus0 = int256(PanopticMath.convert1to0(bonusCross - bonus1U, atSqrtPriceX96));
            }
        }
        // negative premium (owed to the liquidatee) is credited to the collateral balance
        // this is already present in the netPaid amount, so to avoid double-counting we remove it from the balance
        int256 balance0 = int256(uint256(tokenData0.rightSlot())) -
            int256(uint256(shortPremium.rightSlot()));
        int256 balance1 = int256(uint256(tokenData1.rightSlot())) -
            int256(uint256(shortPremium.leftSlot()));
        int256 paid0 = bonus0 + int256(netPaid.rightSlot());
        int256 paid1 = bonus1 + int256(netPaid.leftSlot());
        // note that "balance0" and "balance1" are the liquidatee's original balances before token delegation by a l...
        // their actual balances at the time of computation may be higher, but these are a buffer representing the a...
        // have to work with before cutting into the liquidator's funds
    }
}

```

```

    if (!(paid0 > balance0 && paid1 > balance1)) {
        // liquidatee cannot pay back the liquidator fully in either token, so no protocol loss can be avoided
        if ((paid0 > balance0)) {
            // liquidatee has insufficient token0 but some token1 left over, so we use what they have left to mi...
            // we do this by substituting an equivalent value of token1 in our refund to the liquidator, plus a ...
            // we want to convert the minimum amount of tokens required to achieve the lowest possible protocol ...
            // the maximum level of protocol loss mitigation that can be achieved is the liquidatee's excess tok...
            // and paid0 - balance0 is the amount of token0 that the liquidatee is missing, i.e the protocol los...
            // if the protocol loss is lower than the excess token1 balance, then we can fully mitigate the loss...
            // if the protocol loss is higher than the excess token1 balance, we can only mitigate part of the l...
            // thus, the value converted should be min(balance1 - paid1, paid0 - balance0)
            bonus1 += Math.min(
                balance1 - paid1,
                PanopticMath.convert0to1(paid0 - balance0, atSqrtPriceX96)
            );
            bonus0 -= Math.min(
                PanopticMath.convert1to0RoundingUp(balance1 - paid1, atSqrtPriceX96),
                paid0 - balance0
            );
        }
        if ((paid1 > balance1)) {
            // liquidatee has insufficient token1 but some token0 left over, so we use what they have left to mi...
            // we do this by substituting an equivalent value of token0 in our refund to the liquidator, plus a ...
            // we want to convert the minimum amount of tokens required to achieve the lowest possible protocol ...
            // the maximum level of protocol loss mitigation that can be achieved is the liquidatee's excess tok...
            // and paid1 - balance1 is the amount of token1 that the liquidatee is missing, i.e the protocol los...
            // if the protocol loss is lower than the excess token0 balance, then we can fully mitigate the loss...
            // if the protocol loss is higher than the excess token0 balance, we can only mitigate part of the l...
            // thus, the value converted should be min(balance0 - paid0, paid1 - balance1)
            bonus0 += Math.min(
                balance0 - paid0,
                PanopticMath.convert1to0(paid1 - balance1, atSqrtPriceX96)
            );
            bonus1 -= Math.min(
                PanopticMath.convert0to1RoundingUp(balance0 - paid0, atSqrtPriceX96),
                paid1 - balance1
            );
        }
        // recompute netPaid based on new bonus amounts
        paid0 = bonus0 + int256(netPaid.rightSlot());
        paid1 = bonus1 + int256(netPaid.leftSlot());
    }
    return (
        LeftRightSigned.wrap(0).addToRightSlot(int128(bonus0)).addToLeftSlot(
            int128(bonus1)
        ),
        LeftRightSigned.wrap(0).addToRightSlot(int128(balance0 - paid0)).addToLeftSlot(
            int128(balance1 - paid1)
        )
    );
}

}

/// @notice Haircut/clawback any premium paid by `liquidatee` on `positionIdList` over the protocol loss threshold d...
/// @param liquidatee The address of the user being liquidated
/// @param positionIdList The list of position ids being liquidated
/// @param premiasByLeg The premium paid (or received) by the liquidatee for each leg of each position
/// @param collateralRemaining The remaining collateral after the liquidation (negative if protocol loss)
/// @param atSqrtPriceX96 The oracle price used to swap tokens between the liquidator/liquidatee and determine solve...
/// @return bonusDeltas The delta, if any, to apply to the existing liquidation bonus
/// @return haircutTotal Total premium clawed back from the liquidatee
/// @return haircutPerLeg Per-position/per-leg haircut amounts
function haircutPremia(
    address liquidatee,
    TokenId[] memory positionIdList,
    LeftRightSigned[4][] memory premiasByLeg,
    LeftRightSigned collateralRemaining,
    uint160 atSqrtPriceX96
)
    external
    returns (
        LeftRightSigned bonusDeltas,
        LeftRightUnsigned haircutTotal,
        LeftRightSigned[4][] memory haircutPerLeg
    )
{
    unchecked {
        LeftRightSigned haircutBase;
        LeftRightSigned longPremium;
        /// Get haircutBase, longPremium, bonusDelta
        /// Ignore any surplus collateral - the liquidatee is either solvent or it converts to <1 unit of the othe...
        {
            int256 collateralDelta0 = -Math.min(collateralRemaining.rightSlot(), 0);
            int256 collateralDelta1 = -Math.min(collateralRemaining.leftSlot(), 0);
            // get the amount of premium paid by the liquidatee

```

```

for (uint256 i = 0; i < positionIdList.length; ++i) {
    tokenId = positionIdList[i];
    uint256 numLegs = tokenId.countLegs();
    for (uint256 leg = 0; leg < numLegs; ++leg) {
        if (tokenId.isLong(leg) == 1) {
            longPremium = longPremium.sub(premiasByLeg[i][leg]);
        }
    }
}
// if the premium in the same token is not enough to cover the loss and there is a surplus of the other ...
// the liquidator will provide the tokens (reflected in the bonus amount) & receive compensation in ...
if (
    longPremium.rightSlot() < collateralDelta0 &&
    longPremium.leftSlot() > collateralDelta1
) {
    int256 protocolLoss1 = collateralDelta1;
    (collateralDelta0, collateralDelta1) = (
        -Math.min(
            collateralDelta0 - longPremium.rightSlot(),
            PanopticMath.convert1to0(
                longPremium.leftSlot() - collateralDelta1,
                atSqrtPriceX96
            )
        ),
        Math.min(
            longPremium.leftSlot() - collateralDelta1,
            PanopticMath.convert0to1(
                collateralDelta0 - longPremium.rightSlot(),
                atSqrtPriceX96
            )
        )
    );
    // It is assumed the sum of `protocolLoss1` and `collateralDelta1` does not exceed `2^127 - 1` given...
    // on token supplies and deposit limits
    haircutBase = LeftRightSigned.wrap(longPremium.rightSlot()).addToLeftSlot(
        int128(protocolLoss1 + collateralDelta1)
    );
} else if (
    longPremium.leftSlot() < collateralDelta1 &&
    longPremium.rightSlot() > collateralDelta0
) {
    int256 protocolLoss0 = collateralDelta0;
    (collateralDelta0, collateralDelta1) = (
        Math.min(
            longPremium.rightSlot() - collateralDelta0,
            PanopticMath.convert1to0(
                collateralDelta1 - longPremium.leftSlot(),
                atSqrtPriceX96
            )
        ),
        -Math.min(
            collateralDelta1 - longPremium.leftSlot(),
            PanopticMath.convert0to1(
                longPremium.rightSlot() - collateralDelta0,
                atSqrtPriceX96
            )
        )
    );
    // It is assumed the sum of `protocolLoss0` and `collateralDelta0` does not exceed `2^127 - 1` given...
    // on token supplies and deposit limits
    haircutBase = LeftRightSigned
        .wrap(int128(protocolLoss0 + collateralDelta0))
        .addToLeftSlot(longPremium.leftSlot());
} else {
    // for each token, haircut until the protocol loss is mitigated or the premium paid is exhausted
    // the size of `collateralDelta0/1` and `longPremium.rightSlot()/leftSlot()` is limited to `2^127 - ...
    haircutBase = LeftRightSigned
        .wrap(int128(Math.min(collateralDelta0, longPremium.rightSlot())))
        .addToLeftSlot(int128(Math.min(collateralDelta1, longPremium.leftSlot())));
    collateralDelta0 = 0;
    collateralDelta1 = 0;
}
bonusDeltas = LeftRightSigned
    .wrap(0)
    .addToRightSlot(Math.toInt128(collateralDelta0))
    .addToLeftSlot(Math.toInt128(collateralDelta1));
}
// liquidatee
// positionIdList
// premiaByLeg
// haircutBase
// longPremium
// settledTokens
{

```

[illegible]


```

}
/// @notice Calculates a slow-moving, weighted average price from the on-chain EMAs.
/// @dev Extracts the fast, slow, and eons EMA tick values from the packed `oraclePack`
/// structure. It then computes and returns a blended average with a 60/30/10 weighting
/// respectively. This heavily smoothed value is designed to be highly resistant to
/// manipulation and serves as a robust price feed for critical system functions like solvency checks.
/// @param oraclePack The packed `s_oraclePack` storage slot containing the oracle's state,
/// including the on-chain EMAs.
/// @return The blended time-weighted average price, represented as an int24 tick.
function twapEMA(OraclePack oraclePack) external pure returns (int24) {
    // Extract current EMAs from oraclePack
    (int256 eonsEMA, int256 slowEMA, int256 fastEMA, , ) = oraclePack.getEMAs();
    return int24((6 * fastEMA + 3 * slowEMA + eonsEMA) / 10);
}
/// @notice Takes a packed structure representing a sorted 8-slot queue of ticks and returns the median of those val...
/// @dev Also inserts the latest Uniswap observation into the buffer, resorts, and returns if the last entry is at l...
/// @param oraclePack The packed structure representing the sorted 8-slot queue of ticks
/// @param currentTick The current tick as return from slot0
/// @return medianTick The median of the provided 8-slot queue of ticks in `oraclePack`
/// @return updatedOraclePack The updated 8-slot queue of ticks with the latest observation inserted if the last ent...
function computeInternalMedian(
    OraclePack oraclePack,
    int24 currentTick
) external view returns (int24 medianTick, OraclePack updatedOraclePack) {
    return oraclePack.computeInternalMedian(currentTick, EMA_PERIODS, MAX_CLAMP_DELTA);
}
}
/*//////////////////////
                        HEALTH AND COLLATERAL TRACKING
//////////////////////*/
/// @notice Computes and returns the risk parameters for the pool
/// @param currentTick The current tick of the pool
/// @param oraclePack The oracle pack containing historical price data
/// @param builderCode The builder code for determining fee recipient
/// @return The computed risk parameters including safe mode status and fee configuration
function getRiskParameters(
    int24 currentTick,
    OraclePack oraclePack,
    uint256 builderCode
) external view returns (RiskParameters) {
    uint8 safeMode = isSafeMode(currentTick, oraclePack);
    uint128 feeRecipient = uint256(uint160(_computeBuilderWallet(builderCode))).toUint128();
    return
        RiskParametersLibrary.storeRiskParameters(
            safeMode,
            NOTIONAL_FEE,
            PREMIUM_FEE,
            PROTOCOL_SPLIT,
            BUILDER_SPLIT,
            MAX_TWAP_DELTA_LIQUIDATION,
            MAX_SPREAD,
            BP_DECREASE_BUFFER,
            MAX_OPEN_LEGS,
            feeRecipient
        );
}
/// @notice Computes the fee recipient address from a builder code
/// @param builderCode The builder code to compute the fee recipient from
/// @return feeRecipient The computed fee recipient address
function getFeeRecipient(uint256 builderCode) external view returns (address feeRecipient) {
    feeRecipient = _computeBuilderWallet(builderCode);
    // Optional: enforce whitelist by checking that the contract actually exists
    if (builderCode != 0) {
        if (feeRecipient.code.length == 0) revert Errors.InvalidBuilderCode();
    }
}
/// @notice Checks for significant oracle deviation to determine if Safe Mode should be active.
/// @param currentTick The current tick of the pool
/// @param oraclePack The oracle pack containing historical price data
/// @dev Safe Mode is triggered if ANY of three conditions are met:
/// 1. "External Shock": The live spot price deviates too far from the responsive spot EMA
/// 2. "Internal Disagreement": The fast EMA deviates too far from the more stable slow EMA, indicating high vo...
/// 3. "High Divergence": The EMAs show significant divergence from each other
/// @return safeMode A number representing whether the protocol is in Safe Mode.
function isSafeMode(
    int24 currentTick,
    OraclePack oraclePack
) public pure returns (uint8 safeMode) {
    // Extract the relevant EMAs from oraclePack
    (int24 spotEMA, int24 fastEMA, int24 slowEMA, , int24 medianTick) = oraclePack.getEMAs();
    unchecked {
        // can never miscart because all math is int24 or below
        // Condition 1: Check for a sudden deviation of the spot price from the spot EMA.
        // This is your primary defense against a flash crash or single-block manipulation.
        bool externalShock = Math.abs(currentTick - spotEMA) > MAX_TICKS_DELTA;
    }
}

```

```

    // Condition 2: Check for high internal volatility by comparing the spot and fast EMAs.
    // If the spot EMA is moving much faster than the fast EMA, it signals an unstable market.
    // We use a smaller threshold here (e.g., half of the main delta) to be more sensitive to internal stress.
    bool internalDisagreement = Math.abs(spotEMA - fastEMA) > (MAX_TICKS_DELTA / 2);
    // Condition 3: Check for high internal divergence due to staleness by comparing the median and slow EMAs.
    // If the median tick is deviating too much from the slow EMA, it signals an unstable market.
    // We use a larger threshold here (e.g., twice of the main delta) to be less sensitive to lag.
    bool highDivergence = Math.abs(medianTick - slowEMA) > (MAX_TICKS_DELTA * 2);
    // check lock mode, add value = 3 to returned safeMode.
    uint8 lockMode = oraclePack.lockMode();
    safeMode =
        uint8(externalShock ? 1 : 0) +
        uint8(internalDisagreement ? 1 : 0) +
        uint8(highDivergence ? 1 : 0) +
        lockMode;
}
}
/// @notice Determines which ticks to check for solvency based on market volatility
/// @param currentTick The current tick of the pool
/// @param _oraclePack The oracle pack containing historical price data
/// @return atTicks Array of ticks at which to check solvency
/// @return oraclePack The oracle pack (potentially updated)
function getSolvencyTicks(
    int24 currentTick,
    OraclePack _oraclePack
) external view returns (int24[] memory, OraclePack) {
    (int24 spotTick, int24 medianTick, int24 latestTick, OraclePack oraclePack) = _oraclePack
        .getOracleTicks(currentTick, EMA_PERIODS, MAX_CLAMP_DELTA);
    int24[] memory atTicks;
    // Fall back to a conservative approach if there's high deviation between internal ticks:
    // Check solvency at the medianTick, currentTick, and latestTick instead of just the spotTick.
    // Deviation is measured as the magnitude of a 3D vector:
    // (spotTick - medianTick, latestTick - medianTick, currentTick - medianTick)
    // This approach is more conservative than checking each tick difference individually,
    // as the Euclidean norm is always greater than or equal to the maximum of the individual differences.
    if (
        int256(spotTick - medianTick) ** 2 +
        int256(latestTick - medianTick) ** 2 +
        int256(currentTick - medianTick) ** 2 >
        MAX_TICKS_DELTA ** 2
    ) {
        // High deviation detected; check against all four ticks.
        atTicks = new int24[](4);
        atTicks[0] = spotTick;
        atTicks[1] = medianTick;
        atTicks[2] = latestTick;
        atTicks[3] = currentTick;
    } else {
        // Normal operation; check against the spot tick = 10 mins EMA.
        atTicks = new int24[](1);
        atTicks[0] = spotTick;
    }
    return (atTicks, oraclePack);
}
/// @notice Get the collateral status/margin details of an account/user.
/// @dev NOTE: It's up to the caller to confirm from the returned result that the account has enough collateral.
/// @dev This can be used to check the health: how many tokens a user has compared to the margin threshold.
/// @param user The account to check collateral/margin health for
/// @param positionBalanceArray The list of all open positions held by the `optionOwner`, stored as `[balance/poolUt...`
/// @param atTick The tick at which to evaluate the account's positions
/// @param positionIdList The list of all option positions held by `user`
/// @param shortPremia The total amount of premium (prorated by available settled tokens) owed to the short legs of ...
/// @param longPremia The total amount of premium owed by the long legs of `user`
/// @param ct0 The Address of the CollateralTracker for token0
/// @param ct1 The Address of the CollateralTracker for token1
/// @param buffer The buffer to apply to the collateral requirement
/// @return Whether the account is solvent at the given tick
function isAccountSolvent(
    PositionBalance[] calldata positionBalanceArray,
    TokenId[] calldata positionIdList,
    int24 atTick,
    address user,
    LeftRightUnsigned shortPremia,
    LeftRightUnsigned longPremia,
    CollateralTracker ct0,
    CollateralTracker ct1,
    uint256 buffer
) external view returns (bool) {
    (
        LeftRightUnsigned tokenData0,
        LeftRightUnsigned tokenData1,
        PositionBalance globalUtilizations
    ) = _getMargin(
        positionBalanceArray,

```

```

        positionIdList,
        atTick,
        user,
        shortPremia,
        longPremia,
        ct0,
        ct1
    );
    uint160 sqrtPriceX96 = Math.getSqrtRatioAtTick(atTick);
    uint256 maintReq0 = Math.mulDivRoundingUp(tokenData0.leftSlot(), buffer, DECIMALS);
    uint256 maintReq1 = Math.mulDivRoundingUp(tokenData1.leftSlot(), buffer, DECIMALS);
    uint256 bal0 = tokenData0.rightSlot();
    uint256 bal1 = tokenData1.rightSlot();
    uint256 scaledSurplusToken0 = Math.mulDiv(
        bal0 > maintReq0 ? bal0 - maintReq0 : 0,
        _crossBufferRatio(globalUtilizations.utilization0(), CROSS_BUFFER_0),
        DECIMALS
    );
    uint256 scaledSurplusToken1 = Math.mulDiv(
        bal1 > maintReq1 ? bal1 - maintReq1 : 0,
        _crossBufferRatio(globalUtilizations.utilization1(), CROSS_BUFFER_1),
        DECIMALS
    );
    if (sqrtPriceX96 < Constants.FP96) {
        bool isSolvent0 = bal0 + PanopticMath.convert1to0(scaledSurplusToken1, sqrtPriceX96) >=
            maintReq0;
        bool isSolvent1 = PanopticMath.convert1to0(bal1, sqrtPriceX96) + scaledSurplusToken0 >=
            PanopticMath.convert1to0RoundingUp(maintReq1, sqrtPriceX96);
        return isSolvent0 & & isSolvent1;
    } else {
        bool isSolvent0 = PanopticMath.convert0to1(bal0, sqrtPriceX96) + scaledSurplusToken1 >=
            PanopticMath.convert0to1RoundingUp(maintReq0, sqrtPriceX96);
        bool isSolvent1 = bal1 + PanopticMath.convert0to1(scaledSurplusToken0, sqrtPriceX96) >=
            maintReq1;
        return isSolvent0 & & isSolvent1;
    }
}

/// @notice Compute margin inputs for a user at a given tick.
/// @dev Purely informational: does not make a solvency decision.
/// Returns per-asset maintenance requirement (left slot) and available balance including settled premia (right...)
/// Units:
/// - Requirements are in raw token units
/// - Balances are in raw token units
/// - Ratios elsewhere in the engine use DECIMALS = 10_000_000
/// @param user Account to evaluate
/// @param positionBalanceArray Array of [balanceOrUtilAtMint] for all open positions of `user`
/// @param atTick Tick at which exposures are valued
/// @param positionIdList The list of all option positions held by `user`
/// @param shortPremia Total short premia owed to `user` (right slot = token0 credit, left slot = token1 credit)
/// @param longPremia Total long premia owed by `user` (right slot = token0 debit, left slot = token1 debit)
/// @param ct0 CollateralTracker for token0
/// @param ct1 CollateralTracker for token1
/// @return tokenData0 LeftRightUnsigned for token0 with left = maintenance requirement, right = available balance
/// @return tokenData1 LeftRightUnsigned for token1 with left = maintenance requirement, right = available balance
function getMargin(
    PositionBalance[] calldata positionBalanceArray,
    int24 atTick,
    address user,
    TokenId[] calldata positionIdList,
    LeftRightUnsigned shortPremia,
    LeftRightUnsigned longPremia,
    CollateralTracker ct0,
    CollateralTracker ct1
)
    external
    view
    returns (
        LeftRightUnsigned tokenData0,
        LeftRightUnsigned tokenData1,
        PositionBalance globalUtilizations
    )
{
    if (positionIdList.length != positionBalanceArray.length) revert Errors.LengthMismatch();
    return
        _getMargin(
            positionBalanceArray,
            positionIdList,
            atTick,
            user,
            shortPremia,
            longPremia,
            ct0,
            ct1
        );
}

```

```

}
/// @notice Internal workhorse for margin computation.
/// @dev Aggregates balances, accrued interest, and per-position requirements to produce
///      LeftRightUnsigned pairs for token0 and token1 where:
///      - left slot = total maintenance requirement in that token
///      - right slot = total available balance in that token including settled short premia
///      Caller is responsible for any cross-asset conversion, haircuts, and final solvency logic.
/// @param user Account to evaluate
/// @param positionBalanceArray Array of [balanceOrUtilAtMint] for all open positions of `user`
/// @param atTick Tick at which exposures are valued
/// @param positionIdList The list of all option positions held by `user`
/// @param shortPremia Total short premia owed to `user` (right slot = token0 credit, left slot = token1 credit)
/// @param longPremia Total long premia owed by `user` (right slot = token0 debit, left slot = token1 debit)
/// @param ct0 CollateralTracker for token0
/// @param ct1 CollateralTracker for token1
/// @return tokenData0 LeftRightUnsigned for token0 with left = maintenance requirement, right = available balance
/// @return tokenData1 LeftRightUnsigned for token1 with left = maintenance requirement, right = available balance
function _getMargin(
    PositionBalance[] calldata positionBalanceArray,
    TokenId[] calldata positionIdList,
    int24 atTick,
    address user,
    LeftRightUnsigned shortPremia,
    LeftRightUnsigned longPremia,
    CollateralTracker ct0,
    CollateralTracker ct1
)
    internal
    view
    returns (
        LeftRightUnsigned tokenData0,
        LeftRightUnsigned tokenData1,
        PositionBalance globalUtilizations
    )
{
    LeftRightUnsigned tokensRequired;
    LeftRightUnsigned creditAmounts;
    (tokensRequired, creditAmounts, globalUtilizations) = _getTotalRequiredCollateral(
        positionBalanceArray,
        positionIdList,
        atTick,
        longPremia
    );
    uint256 balance0;
    uint256 balance1;
    uint256 interest0;
    uint256 interest1;
    unchecked {
        (balance0, interest0) = ct0.assetsAndInterest(user);
        (balance1, interest1) = ct1.assetsAndInterest(user);
        // Insolvent-interest case: a user cannot pay more interest than their balance.
        // Cap interest to available balance and zero the balance so we don't treat the same funds
        // as both spendable collateral and interest payment.
        // The capped interest is later added to collateral requirements.
        if (interest0 > balance0) {
            interest0 = balance0; // Cap interest
            balance0 = 0; // Zero balance
        } else {
            balance0 -= interest0; // Subtract interest from balance
            interest0 = 0; // Zero interest (nothing to add to requirements)
        }
        if (interest1 > balance1) {
            interest1 = balance1;
            balance1 = 0;
        } else {
            balance1 -= interest1; // Subtract interest from balance
            interest1 = 0; // Zero interest (nothing to add to requirements)
        }
    }
    unchecked {
        balance0 += shortPremia.rightSlot();
        balance1 += shortPremia.leftSlot();
        balance0 += creditAmounts.rightSlot();
        balance1 += creditAmounts.leftSlot();
        tokensRequired = tokensRequired.addToRightSlot(uint128(interest0)).addToLeftSlot(
            uint128(interest1)
        );
    }
    tokenData0 = LeftRightUnsigned.wrap(balance0.toUint128()).addToLeftSlot(
        tokensRequired.rightSlot()
    );
    tokenData1 = LeftRightUnsigned.wrap(balance1.toUint128()).addToLeftSlot(
        tokensRequired.leftSlot()
    );
}

```

```

}
/// @notice Gets the highest pool utilization (for token0 and token1) from an array of positions.
/// @dev Iterates through all of a user's positions to find the maximum `utilization0` and maximum `utilization1`
/// recorded at the time of minting. These "global" max utilizations are then used for
/// portfolio-level margin calculations, ensuring a more conservative risk assessment.
/// @param positionBalanceArray The array of a user's `PositionBalance` structs.
/// @return globalUtilizations A packed PositionBalance that contains only the utilization data, recoverable as .uti...
function _getGlobalUtilization(
    PositionBalance[] calldata positionBalanceArray
) internal pure returns (PositionBalance globalUtilizations) {
    int256 utilization0;
    int256 utilization1;
    uint256 pLength = positionBalanceArray.length;
    for (uint256 i; i < pLength; ) {
        PositionBalance positionBalance = positionBalanceArray[i];
        int256 _utilization0 = positionBalance.utilization0();
        int256 _utilization1 = positionBalance.utilization1();
        // utilizations are always positive, so can compare directly here
        utilization0 = _utilization0 > utilization0 ? _utilization0 : utilization0;
        utilization1 = _utilization1 > utilization1 ? _utilization1 : utilization1;
        unchecked {
            ++i;
        }
    }
    unchecked {
        // can never miscast because utilization < 10_000
        globalUtilizations = PositionBalanceLibrary.storeBalanceData(
            0,
            uint32(uint256(utilization0) + (uint256(utilization1) << 16)),
            0
        );
    }
}

/// @notice Get the total required amount of collateral tokens of a user/account across all active positions to stay...
/// @dev Returns the token amounts required for the entire account with active positions in `positionIdList` (list o...
/// @param positionBalanceArray The list of all open positions held by the `optionOwner`, stored as `[balance/poolUt...
/// @param positionIdList The list of all option positions held by `owner`
/// @param atTick The tick at which to evaluate the account's positions
/// @return tokensRequired The amount of token0 (right) and token1 (left) required to stay above the margin threshol...
/// @return creditAmounts The amount of credit token0 (right) and token1 (left) in the user's portfolio
function _getTotalRequiredCollateral(
    PositionBalance[] calldata positionBalanceArray,
    TokenId[] calldata positionIdList,
    int24 atTick,
    LeftRightUnsigned longPremia
) internal
view
returns (
    LeftRightUnsigned tokensRequired,
    LeftRightUnsigned creditAmounts,
    PositionBalance globalUtilizations
)
{
    // get the global utilizations, which is the max utilizations for all open positions
    globalUtilizations = _getGlobalUtilization(positionBalanceArray);
    // add long premia to tokens required
    tokensRequired = tokensRequired.add(longPremia);
    for (uint256 i; i < positionBalanceArray.length; ) {
        uint256 _tokenRequired0;
        uint256 _credits0;
        uint256 _tokenRequired1;
        uint256 _credits1;
        {
            TokenId tokenId = positionIdList[i];
            PositionBalance positionBalance = positionBalanceArray[i];
            uint128 positionSize = positionBalance.positionSize();
            int24 _atTick = atTick;
            unchecked {
                // can never miscast because utilization < 10_000
                // Use the global utilizations for all positions
                int16 utilization0 = int16(globalUtilizations.utilization0());
                (_tokenRequired0, _credits0) = _getRequiredCollateralAtTickSinglePosition(
                    tokenId,
                    positionSize,
                    _atTick,
                    utilization0,
                    true
                );
            }
            unchecked {
                // can never miscast because utilization < 10_000
                // Use the global utilizations for all positions
                int16 utilization1 = int16(globalUtilizations.utilization1());
            }
        }
    }
}

```

```

        (_tokenRequired1, _credits1) = _getRequiredCollateralAtTickSinglePosition(
            tokenId,
            positionSize,
            _atTick,
            utilization1,
            false
        );
    }
}
tokensRequired = tokensRequired
    .addToRightSlot(_tokenRequired0.toUint128())
    .addToLeftSlot(_tokenRequired1.toUint128());
creditAmounts = creditAmounts.addToRightSlot(_credits0.toUint128()).addToLeftSlot(
    _credits1.toUint128()
);
unchecked {
    ++i;
}
}
}
}
/// @notice Get the required amount of collateral tokens corresponding to a specific single position `tokenId` at a ...
/// @param tokenId The option position
/// @param positionSize The size of the option position
/// @param atTick The tick at which to evaluate the account's positions
/// @param poolUtilization The utilization of the collateral vault (balance of buying and selling)
/// @param underlyingIsToken0 Cached `s_underlyingIsToken0` value for this CollateralTracker instance
/// @return tokenRequired Total required tokens for all legs of the specified tokenId.
function _getRequiredCollateralAtTickSinglePosition(
    TokenId tokenId,
    uint128 positionSize,
    int24 atTick,
    int16 poolUtilization,
    bool underlyingIsToken0
) internal view returns (uint256 tokenRequired, uint256 credits) {
    uint256 numLegs = tokenId.countLegs();
    unchecked {
        for (uint256 index = 0; index < numLegs; ++index) {
            // bypass the collateral calculation if tokenType doesn't match the requested token (underlyingIsToken0)
            if (tokenId.tokenType(index) != (underlyingIsToken0 ? 0 : 1)) continue;
            if (tokenId.width(index) == 0 && tokenId.isLong(index) == 1) {
                LeftRightUnsigned amountsMoved = PanopticMath.getAmountsMoved(
                    tokenId,
                    positionSize,
                    index,
                    false
                );
                credits = tokenId.tokenType(index) == 0
                    ? amountsMoved.rightSlot()
                    : amountsMoved.leftSlot();
            }
            // Increment the tokenRequired accumulator
            tokenRequired += _getRequiredCollateralSingleLeg(
                tokenId,
                index,
                positionSize,
                atTick,
                poolUtilization
            );
        }
    }
}
}
/// @notice Calculate the required amount of collateral for a single leg `index` of position `tokenId`.
/// @param tokenId The option position
/// @param index The leg index (associated with a liquidity chunk) to compute the required collateral for
/// @param positionSize The size of the position
/// @param atTick The tick at which to evaluate the account's positions
/// @param poolUtilization The pool utilization: how much funds are in the Panoptic pool versus the AMM pool
/// @return required The required amount collateral needed for this leg `index`
function _getRequiredCollateralSingleLeg(
    TokenId tokenId,
    uint256 index,
    uint128 positionSize,
    int24 atTick,
    int16 poolUtilization
) internal view returns (uint256 required) {
    return
        tokenId.riskPartner(index) == index // does this leg have a risk partner? Affects required collateral
        ? _getRequiredCollateralSingleLegNoPartner(
            tokenId,
            index,
            positionSize,
            atTick,
            poolUtilization
        )

```

```

    : _getRequiredCollateralSingleLegPartner(
        tokenId,
        index,
        positionSize,
        atTick,
        poolUtilization
    );
}
/// @notice Calculate the required amount of collateral for leg `index` of position `tokenId` when the leg does not ...
/// @param tokenId The option position
/// @param index The leg index (associated with a liquidity chunk) to consider a partner for
/// @param positionSize The size of the position
/// @param atTick The tick at which to evaluate the account's positions
/// @param poolUtilization The pool utilization: ratio of how much funds are in the Panoptic pool versus the AMM poo...
/// @return required The required amount collateral needed for this leg `index`
function _getRequiredCollateralSingleLegNoPartner(
    TokenId tokenId,
    uint256 index,
    uint128 positionSize,
    int24 atTick,
    int16 poolUtilization
) internal view returns (uint256 required) {
    // extract the tokenType (token0 or token1)
    uint256 tokenType = tokenId.tokenType(index);
    // compute the total amount of funds moved for that position
    // Since this is a collateral check, we want the amounts moved upon closure, not upon opening
    LeftRightUnsigned amountsMoved = PanopticMath.getAmountsMoved(
        tokenId,
        positionSize,
        index,
        false
    );
    // amount moved is right slot if tokenType=0, left slot otherwise
    uint128 amountMoved = tokenType == 0 ? amountsMoved.rightSlot() : amountsMoved.leftSlot();
    uint256 isLong = tokenId.isLong(index);
    unchecked {
        // if the width is 0, then this is a loan/credit
        if (tokenId.width(index) == 0) {
            if (isLong == 0) {
                // buying power requirement for a Loan position is 100% + MAINT_MARGIN_RATE
                required = Math.mulDivRoundingUp(
                    amountMoved,
                    MAINT_MARGIN_RATE + DECIMALS,
                    DECIMALS
                );
            } else {
                // buying power requirement for a Credit position is 0
                // this is not netted against other legs unless it has a partner
                required = 0;
            }
        } else {
            // required collateral is at least 1
            required = 1;
            uint256 baseCollateralRatio;
            {
                uint256 baseRequired;
                // start with base requirement, which is based on isLong value
                (baseRequired, baseCollateralRatio) = _getRequiredCollateralAtUtilization(
                    amountMoved,
                    isLong,
                    poolUtilization
                );
                required += baseRequired;
            }
            (int24 tickLower, int24 tickUpper) = tokenId.asTicks(index);
            int24 strike = tokenId.strike(index);
            if (isLong == 0) {
                // if position is short, check whether the position is out-the-money
                // if position is ITM or ATM, then the collateral requirement depends on price:
                // We must first get the ratio of strike to price for calls (or price to strike for puts).
                // Both of these ratios decrease as the position becomes deeper ITM.
                // We must clamp the difference between atTick and strike to the min & max Uniswap ticks,
                // to conform with what getSqrtRatioAtTick can support.
                // This is acceptable because a higher ratio will result in an increased slope for the collateral re...
                // (- and * 2 in tick space are / and ^ 2 in price space so sqrtRatioAtTick(2 *(a - b)) = a/b (*2^96...
                uint160 ratio = tokenType == 1 // tokenType
                    ? Math.getSqrtRatioAtTick(
                        int24(
                            Math.bound(
                                2 * (atTick - strike),
                                Constants.MIN_POOL_TICK,
                                Constants.MAX_POOL_TICK
                            )
                        )
                    ) : 0;
            }
        }
    }
}

```

```

    ) // puts -> price/strike
    : Math.getSqrtRatioAtTick(
        uint24(
            Math.bound(
                2 * (strike - atTick),
                Constants.MIN_POOL_TICK,
                Constants.MAX_POOL_TICK
            )
        )
    ); // calls -> strike/price
// Following Reg-T guidelines, the collateral requirement is the max of:
// - 10% of the notional value at the strike price (r0)
// - 20% of the underlying price MINUS the out-the-money amount (r1)
// Note that we over-estimate the capital composition between the LP position's range.
uint256 r0 = required / 2;
uint256 r1;
{
    uint256 p0 = amountMoved + Math.mulDiv96RoundingUp(required, ratio);
    uint256 p1 = Math.mulDiv96RoundingUp(amountMoved, ratio);
    r1 = p0 > p1 ? p0 - p1 : 0;
}
uint256 r2;
if ((atTick < tickUpper) && (atTick >= tickLower)) {
    // position is in-range (ie. current tick is between upper+lower tick): we draw a line between t...
    // collateral requirement at the lowerTick and the one at the upperTick. We use that interpolati...
    // the collateral requirement when in-range, which always over-estimates the amount of token req...
    // Specifically:
    uint160 scaleFactor = Math.getSqrtRatioAtTick((tickUpper - tickLower));
    r2 =
        Math.mulDivRoundingUp(
            amountMoved * (DECIMALS - baseCollateralRatio),
            (scaleFactor - ratio),
            DECIMALS * (scaleFactor + Constants.FP96)
        ) +
        r0;
}
required = Math.max(Math.max(r2, r1), r0);
} else {
    uint256 positionWidth = uint256(uint24(tickUpper - tickLower));
    uint256 distanceFromStrike = Math.max(
        positionWidth / 2,
        atTick > strike
            ? uint256(uint24(atTick - strike))
            : uint256(uint24(strike - atTick))
    );
    // Calculate the exponent: distance / width
    uint256 expValue;
    {
        uint256 scaledRatio = (distanceFromStrike * DECIMALS) / (positionWidth);
        // Divide by ln(2) to get the number of doublings
        // LN2_SCALED = ln(2) * DECIMALS
        uint256 shifts = scaledRatio / LN2_SCALED;
        uint256 remainder = scaledRatio % LN2_SCALED;
        // Calculate e^(remainder/DECIMALS) - now always less than e^(ln(2)) = 2
        // This means Taylor expansion is very accurate
        uint256 expFractional = Math.sTaylorCompounded(remainder, DECIMALS);
        // Combine: e^x = 2^shifts * e^remainder
        // We divide by DECIMALS at the end to maintain precision
        if (shifts < 128) {
            // Prevent overflow
            expValue = (expFractional << shifts);
        } else {
            expValue = type(uint128).max; // Cap at uint128 max value
        }
    }
    // Apply the exponential decay to required collateral
    uint256 _required = required;
    required = Math.min(
        required,
        (DECIMALS * _required * positionWidth) /
        (distanceFromStrike * expValue) +
        TEN_BPS
    );
}
}
}

/// @notice Calculate the required amount of collateral for leg `index` for position `tokenId` accounting for its pa...
/// @dev If the two `isLong` fields are different (i.e., a short leg and a long leg are partnered) but the tokenType...
/// @dev A spread is a defined risk position which has a max loss given by difference between the long and short str...
/// @dev If the two `isLong` fields are the same but the tokenTypes are different (one is a call, the other a put, e...
/// a strangle benefits from enhanced capital efficiency because only one side can be ITM at any given time.
/// @param tokenId The option position
/// @param index The leg index (associated with a liquidity chunk) to consider a partner for

```



```

/// @param positionSize The size of the position
/// @param atTick The tick at which to evaluate the account's positions
/// @param poolUtilization The pool utilization: how much funds are in the Panoptic pool versus the AMM pool
/// @return required The required amount of collateral needed for this leg `index`
function _getRequiredCollateralSingleLegPartner(
    tokenId tokenId,
    uint256 index,
    uint128 positionSize,
    int24 atTick,
    int16 poolUtilization
) internal view returns (uint256) {
    // extract partner index (associated with another liquidity chunk)
    uint256 partnerIndex = tokenId.riskPartner(index);
    // In the following, we check whether the risk partner of this leg is itself
    // or another leg in this position.
    // Handles case where riskPartner(i) != i ==> leg i has a risk partner that is another leg
    // @dev In summary, the allowed risk partners:
    //
    // PURE OPTIONS
    // -Short Strangles/Straddles (short put + short call) = each leg's basic requirement is 50% less
    // -Vertical Spreads and Calendar Spreads (short put + long put) or (short call + long call) = requirement is ma...
    // -Synthetic Stocks (short put + long call) or (short call + long put) = requirement is short leg only
    //
    // FUNDED OPTIONS
    // -Prepaid long option (long put or call + credit) "Purchases pre-pays for the cost of the option" = requiremen...
    // -Upfront short option (short put or call + loan) "Upfront payment to seller" = requirement is max(loan, short...
    // -Option-protected loan (long put or call + loan) "Get a loan with an embedded long option for capital protect...
    // -Cash-Secured Option (short put or call + credit) "Allocate collateral to that specific option" = requirement...
    //
    // TOKEN TRANSFERS
    // - Delayed Swap (credit at one strike, loan at another; different amounts = effective swap) = requirement is m...
    {
        // only proceed if the partners have the same asset
        if (
            tokenId.asset(partnerIndex) == tokenId.asset(index) &&
            tokenId.optionRatio(partnerIndex) == tokenId.optionRatio(index)
        ) {
            // width of associated legs, true if greater than 0 (ie. it is an option leg)
            bool _width = tokenId.width(index) > 0;
            bool widthP = tokenId.width(partnerIndex) > 0;
            // long/short status of associated legs
            uint256 _isLong = tokenId.isLong(index);
            uint256 isLongP = tokenId.isLong(partnerIndex);
            // token type status of associated legs (call/put)
            uint256 _tokenType = tokenId.tokenType(index);
            uint256 tokenTypeP = tokenId.tokenType(partnerIndex);
            // if both legs are options
            if (_width && widthP) {
                if (_tokenType != tokenTypeP) {
                    if (_isLong == 0 && isLongP == 0) {
                        // STRANGLES: different token types, both short
                        return
                            _computeStrangle(
                                tokenId,
                                index,
                                positionSize,
                                atTick,
                                poolUtilization
                            );
                    } else if (
                        _isLong != isLongP &&
                        tokenId.strike(index) == tokenId.strike(partnerIndex)
                    ) {
                        // SYNTHETIC STOCK: different token types, one is long and the other is short. MUST BE AT TH...
                        return
                            // return the collateral requirement of the short leg only (the long leg comes for free"...
                            _isLong == 0
                            ? _getRequiredCollateralSingleLegNoPartner(
                                tokenId,
                                index,
                                positionSize,
                                atTick,
                                poolUtilization
                            )
                            : 0;
                    }
                }
            } else {
                if (_isLong != isLongP) {
                    // SPREADS: same token type, one is long and the other is short
                    return
                        // only return the requirement once for the first leg it encounters
                        index < partnerIndex
                        ? _computeSpread(
                            tokenId,

```

```

        positionSize,
        index,
        partnerIndex,
        atTick,
        poolUtilization
    )
    : 0;
    }
} else if (_width != widthP) {
    if (_tokenType == tokenTypeP) {
        if (isLongP == 1) {
            // CASH-SECURED OPTION
            // PREPAID LONG OPTION
            // only compute it once for the option leg
            return
                _width
                ? _computeCreditOptionComposite(
                    tokenId,
                    positionSize,
                    index,
                    atTick
                )
                : 0;
        } else {
            // OPTION-PROTECTED LOAN
            // UPFRONT SHORT OPTION
            // only compute it once for the option leg
            return
                _width
                ? _computeLoanOptionComposite(
                    tokenId,
                    positionSize,
                    index,
                    partnerIndex,
                    atTick,
                    poolUtilization
                )
                : 0;
        }
    }
} else {
    if (_tokenType != tokenTypeP) {
        // TOKEN TRANSFERS
        if (_isLong != isLongP) {
            // DELAYED SWAP
            // only compute it once for the loan side
            return
                _isLong == 0
                ? _computeDelayedSwap(
                    tokenId,
                    positionSize,
                    index,
                    partnerIndex,
                    atTick
                )
                : 0;
        }
    }
}
}
// otherwise, not a list of allowed strategies. Return the single-leg collateral requirement
return
    _getRequiredCollateralSingleLegNoPartner(
        tokenId,
        index,
        positionSize,
        atTick,
        poolUtilization
    );
}
/// @notice Get the base collateral requirement for a position of notional value `amount` at the current Panoptic po...
/// @param amount The amount to multiply by the base collateral ratio
/// @param isLong Whether the position is long (=1) or short (=0)
/// @param utilization The utilization of the Panoptic pool (balance between sellers and buyers)
/// @return required The base collateral requirement corresponding to the incoming `amount`
function _getRequiredCollateralAtUtilization(
    uint128 amount,
    uint256 isLong,
    int16 utilization
) internal view returns (uint256 required, uint256 baseCollateralRatio) {
    // if position is short, use sell collateral ratio
    if (isLong == 0) {

```

```

        // compute the sell collateral ratio, which depends on the pool utilization
        baseCollateralRatio = _sellCollateralRatio(utilization);
        // compute required as amount*collateralRatio
        // can use unsafe because denominator is always nonzero
        unchecked {
            required = Math.unsafeDivRoundingUp(amount * baseCollateralRatio, DECIMALS);
        }
    } else if (isLong == 1) {
        // if options is long, use buy collateral ratio
        // compute the buy collateral ratio, which depends on the pool utilization
        baseCollateralRatio = _buyCollateralRatio();
        // compute required as amount*collateralRatio
        // can use unsafe because denominator is always nonzero
        unchecked {
            required = Math.unsafeDivRoundingUp(amount * baseCollateralRatio, DECIMALS);
        }
    }
}

/// @notice Calculates the total collateral requirement for a defined-risk spread position.
/// @dev A spread's collateral is the minimum of its defined max loss or the sum of its legs' individual (unpartnere...
/// @dev This provides capital efficiency, as deep OTM spreads may require less collateral than their max loss due t...
/// @param tokenId The option position
/// @param positionSize The size of the position
/// @param index The leg index of the LONG leg in the spread position
/// @param partnerIndex The index of the partnered SHORT leg in the spread position
/// @param atTick the tick the requirement is evaluated at
/// @param poolUtilization The pool utilization: how much funds are in the Panoptic pool versus the AMM pool
/// @return spreadRequirement The required amount of collateral needed for the spread
function _computeSpread(
    TokenId tokenId,
    uint128 positionSize,
    uint256 index,
    uint256 partnerIndex,
    int24 atTick,
    int16 poolUtilization
) internal view returns (uint256 spreadRequirement) {
    spreadRequirement = 1;
    uint256 splitRequirement;
    unchecked {
        uint256 _required = _getRequiredCollateralSingleLegNoPartner(
            tokenId,
            index,
            positionSize,
            atTick,
            poolUtilization
        );
        uint256 requiredPartner = _getRequiredCollateralSingleLegNoPartner(
            tokenId,
            partnerIndex,
            positionSize,
            atTick,
            poolUtilization
        );
        splitRequirement = _required + requiredPartner;
    }
    uint128 moved0;
    uint128 moved1;
    uint128 moved0Partner;
    uint128 moved1Partner;
    uint256 tokenType = tokenId.tokenType(index);
    {
        // compute the total amount of funds moved for the position's current leg
        // Since this is returning a collateral requirement, we want to return the amounts moved upon closure, not o...
        LeftRightUnsigned amountsMoved = PanopticMath.getAmountsMoved(
            tokenId,
            positionSize,
            index,
            false
        );
        unchecked {
            // This is a CALENDAR SPREAD adjustment, where the collateral requirement is the max loss of the positio...
            // real formula is contractSize * (1/(sqrt(r1)+1) - 1/(sqrt(r2)+1))
            // Taylor expand to get a rough approximation of: contractSize * Δwidth * tickSpacing / 40000
            // This is strictly larger than the real one, so OK to use that for a collateral requirement.
            TokenId _tokenId = tokenId;
            int24 deltaWidth = _tokenId.width(index) - _tokenId.width(partnerIndex);
            // TODO check if same strike and same width is allowed -&gt; Think not from TokenId.sol?
            if (deltaWidth < 0) deltaWidth = -deltaWidth;
            if (tokenType == 0) {
                spreadRequirement +=
                    (amountsMoved.rightSlot() *
                     uint256(int256(deltaWidth * _tokenId.tickSpacing()))) /
                    80000;
            } else {

```

```

        spreadRequirement +=
            (amountsMoved.leftSlot() *
             uint256(int256(deltaWidth * _tokenId.tickSpacing())) /
             80000;
    }
}
moved0 = amountsMoved.rightSlot();
moved1 = amountsMoved.leftSlot();
{
    // compute the total amount of funds moved for the position's partner leg
    LeftRightUnsigned amountsMovedPartner = PanopticMath.getAmountsMoved(
        tokenId,
        positionSize,
        partnerIndex,
        false
    );
    moved0Partner = amountsMovedPartner.rightSlot();
    moved1Partner = amountsMovedPartner.leftSlot();
}
}
// compute the max loss of the spread
// if asset is NOT the same as the tokenType, the required amount is simply the difference in notional values
// ie. asset = 1, tokenType = 0:
if (tokenId.asset(index) != tokenType) {
    unchecked {
        // always take the absolute values of the difference of amounts moved
        if (tokenType == 0) {
            spreadRequirement += moved0 &lt; moved0Partner
                ? moved0Partner - moved0
                : moved0 - moved0Partner;
        } else {
            spreadRequirement += moved1 &lt; moved1Partner
                ? moved1Partner - moved1
                : moved1 - moved1Partner;
        }
    }
} else {
    unchecked {
        uint256 notional;
        uint256 notionalP;
        uint128 contracts;
        if (tokenType == 1) {
            notional = moved0;
            notionalP = moved0Partner;
            contracts = moved1;
        } else {
            notional = moved1;
            notionalP = moved1Partner;
            contracts = moved0;
        }
        // the required amount is the amount of contracts multiplied by (notional1 - notional2)/max(notional1, n...
        // can use unsafe because denominator is always nonzero
        spreadRequirement += (notional &lt; notionalP)
            ? Math.unsafeDivRoundingUp((notionalP - notional) * contracts, notionalP)
            : Math.unsafeDivRoundingUp((notional - notionalP) * contracts, notional);
    }
}
spreadRequirement = Math.min(splitRequirement, spreadRequirement);
}
/// @notice Calculate the required amount of collateral for a strangle leg.
/// @dev The base collateral requirement is halved for short strangles.
/// @dev A strangle can only have only one of its legs ITM at any given time, so this reduces the total risk and col...
/// @param tokenId The option position
/// @param positionSize The size of the position
/// @param index The leg index (associated with a liquidity chunk) to consider a partner for
/// @param atTick The tick at which to evaluate the account's positions
/// @param poolUtilization The pool utilization: how much funds are in the Panoptic pool versus the AMM pool
/// @return strangleRequired The required amount of collateral needed for the strangle leg
function _computeStrangle(
    TokenId tokenId,
    uint256 index,
    uint128 positionSize,
    int24 atTick,
    int16 poolUtilization
) internal view returns (uint256 strangleRequired) {
    // If both tokenTypes are the same, then this is a short strangle.
    // A strangle is an options strategy in which the investor holds a position
    // in both a call and a put option with different strike prices,
    // but with the same expiration date and underlying asset.
    // collateral requirement is for short strangles depicted:
    /**
        Put side of a short strangle, BPR = 100% - (100% - SCR/2)*(price/strike)
        BUYING
        POWER

```

```

REQUIREMENT
    ^
    |
    |      <- ITM      . OTM ->;
100% -|-----
    |      |
SCR/2 -|-----> <----- base collateral is half that of a single-leg
    |      |
    +-----+-----> current
    0      strike      price
*/
unchecked {
    // A negative pool utilization is used to denote a position which is a strangle
    // add 1 to handle poolUtilization = 0
    poolUtilization = -(poolUtilization == 0 ? int16(1) : poolUtilization);
    return
        strangleRequired = _getRequiredCollateralSingleLegNoPartner(
            tokenId,
            index,
            positionSize,
            atTick,
            poolUtilization
        );
}
}
function _computeLoanOptionComposite(
    TokenId tokenId,
    uint128 positionSize,
    uint256 index,
    uint256 partnerIndex,
    int24 atTick,
    int16 poolUtilization
) internal view returns (uint256) {
    // compute both token requirements. Can directly compare them because they have the same tokenType
    uint256 _required = _getRequiredCollateralSingleLegNoPartner(
        tokenId,
        index,
        positionSize,
        atTick,
        poolUtilization
    );
    uint256 requiredPartner = _getRequiredCollateralSingleLegNoPartner(
        tokenId,
        partnerIndex,
        positionSize,
        atTick,
        poolUtilization
    );
    unchecked {
        if (tokenId.isLong(index) == 0) {
            return _required + requiredPartner;
        } else {
            // return the max of the requirement between a loan and the long option position
            return Math.max(_required, requiredPartner);
        }
    }
}
}
function _computeCreditOptionComposite(
    TokenId tokenId,
    uint128 positionSize,
    uint256 index,
    int24 atTick
) internal view returns (uint256) {
    // can only be called when partnerIndex is the credit
    // required amount for the option leg
    // Assume 100% utilization, which means
    // - 100% collateralization for sold options (cash account requirement)
    uint256 _required = _getRequiredCollateralSingleLegNoPartner(
        tokenId,
        index,
        positionSize,
        atTick,
        MAX_UTILIZATION
    );
    return _required;
}
}
function _computeDelayedSwap(
    TokenId tokenId,
    uint128 positionSize,
    uint256 index,
    uint256 partnerIndex,
    int24 atTick
) internal view returns (uint256) {
    unchecked {
        // can only be called when partnerIndex is the credit
    }
}

```

```

LeftRightUnsigned amountsMoved = PanopticMath.getAmountsMoved(
    tokenId,
    positionSize,
    index,
    false
);
LeftRightUnsigned amountsMovedP = PanopticMath.getAmountsMoved(
    tokenId,
    positionSize,
    partnerIndex,
    false
);
uint256 loanAmount = tokenId.tokenType(index) == 0
    ? amountsMoved.rightSlot()
    : amountsMoved.leftSlot();
uint256 required = Math.mulDivRoundingUp(
    loanAmount,
    SELLER_COLLATERAL_RATIO + DECIMALS,
    DECIMALS
);
uint256 creditAmount = tokenId.tokenType(partnerIndex) == 0
    ? amountsMovedP.rightSlot()
    : amountsMovedP.leftSlot();
uint256 convertedCredit = tokenId.tokenType(partnerIndex) == 0
    ? PanopticMath.convert0to1RoundingUp(creditAmount, Math.getSqrtRatioAtTick(atTick))
    : PanopticMath.convert1to0RoundingUp(creditAmount, Math.getSqrtRatioAtTick(atTick));
if (required > convertedCredit) {
    return required;
} else {
    return convertedCredit;
}
}

}
/// @notice Get the base collateral requirement for a short leg at a given pool utilization.
/// @dev This is computed at the time the position is minted.
/// @param utilization The pool utilization of this collateral vault at the time the position is minted
/// @return sellCollateralRatio The sell collateral ratio at `utilization`
function _sellCollateralRatio(
    int256 utilization
) internal view returns (uint256 sellCollateralRatio) {
    // the sell ratio is on a straight line defined between two points (x0,y0) and (x1,y1):
    // (x0,y0) = (targetPoolUtilization,min_sell_ratio) and
    // (x1,y1) = (saturatedPoolUtilization,max_sell_ratio)
    // the line's formula: y = a * (x - x0) + y0, where a = (y1 - y0) / (x1 - x0)
    /*
    SELL
    COLLATERAL
    RATIO
    ^
    100% - |
          |
          | max ratio = 100%
    20% - |-----
          |
          |
          +-----+-----> POOL_
                50%   90% 100% UTILIZATION
    */
    uint256 min_sell_ratio = SELLER_COLLATERAL_RATIO;
    /// if utilization is less than zero, this is the calculation for a strangle, which gets 2x the capital efficien...
    if (utilization < 0) {
        unchecked {
            min_sell_ratio /= 2;
            utilization = -utilization;
        }
    }
    unchecked {
        utilization *= 1_000;
    }
    // return the basal sell ratio if pool utilization is lower than target
    if (uint256(utilization) < TARGET_POOL_UTIL) {
        return min_sell_ratio;
    }
    // return 100% collateral ratio if utilization is above saturated pool utilization
    if (uint256(utilization) > SATURATED_POOL_UTIL) {
        return DECIMALS;
    }
    unchecked {
        return
            min_sell_ratio +
            ((DECIMALS - min_sell_ratio) * (uint256(utilization) - TARGET_POOL_UTIL)) /
            (SATURATED_POOL_UTIL - TARGET_POOL_UTIL);
    }
}
}

```

```

/// @notice Get the base collateral requirement for a given pool utilization.
/// @dev This is computed at the time the position is minted.
/// @return buyCollateralRatio The buy collateral ratio at `utilization`
function _buyCollateralRatio() internal view returns (uint256 buyCollateralRatio) {
    return BUYER_COLLATERAL_RATIO;
}

/// @notice Get the cross buffer ration for a given utilization
/// @dev This is computed using the global utilization of the user.
/// @param utilization The pool utilization of this collateral vault at the time the position is minted
/// @return crossBufferRatio The cross buffer ratio at `utilization`
function _crossBufferRatio(
    int256 utilization,
    uint256 crossBuffer
) internal view returns (uint256 crossBufferRatio) {
    // linear from crossBuffer to 0 between 50% and 90%
    // the buy ratio is on a straight line defined between two points (x0,y0) and (x1,y1):
    //   (x0,y0) = (targetPoolUtilization, crossBuffer) and
    //   (x1,y1) = (saturatedPoolUtilization, 0)
    // note that y1<y0 so the slope is negative:
    // aka the cross buffer starts high and drops to zero with increased utilization
    // the line's formula: y = a * (x - x0) + y0, where a = (y1 - y0) / (x1 - x0)
    // but since a<0, we rewrite as:
    // y = a' * (x0 - x) + y0, where a' = (y0 - y1) / (x1 - x0)
    /*
        CROSS
        BUFFER
        RATIO
            ^
            |      cross_buffer = 80%
    80% - |-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
            |               .         .         .         .         .         .         .         .         .         .
    0% -  |-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
            |               50%       90%     100%          POOL_
                                UTILIZATION
    */
    unchecked {
        uint256 utilizationScaled = uint256(utilization * 1_000);
        // return the basal cross buffer ratio if pool utilization is lower than target
        if (utilizationScaled < TARGET_POOL_UTIL) {
            return crossBuffer;
        }
        // return 0 if pool utilization is above saturated pool utilization
        if (utilizationScaled > SATURATED_POOL_UTIL) {
            return 0;
        }
        return ((crossBuffer * (SATURATED_POOL_UTIL - utilizationScaled)) /
            (SATURATED_POOL_UTIL - TARGET_POOL_UTIL));
    }
}

//////////////////////////////////////////////////////////////
ADAPTIVE INTEREST RATE MODEL
//////////////////////////////////////////////////////////////
/// @notice Calculates the current interest rate based on utilization
/// @param utilization The current pool utilization
/// @param interestRateAccumulator The current state of the interest rate accumulator
/// @return The calculated interest rate per second
function interestRate(
    uint256 utilization,
    MarketState interestRateAccumulator
) external view returns (uint128) {
    (uint256 avgRate, ) = _borrowRate(utilization, interestRateAccumulator);
    return uint128(avgRate);
}

/// @notice Calculates both the average interest rate and the new rate at target
/// @param utilization The current pool utilization
/// @param interestRateAccumulator The current state of the interest rate accumulator
/// @return The average interest rate
/// @return The new rate at target
function updateInterestRate(
    uint256 utilization,
    MarketState interestRateAccumulator
) external view returns (uint128, uint256) {
    (uint256 avgRate, int256 endRateAtTarget) = _borrowRate(
        utilization,
        interestRateAccumulator
    );
    return (uint128(avgRate), uint256(endRateAtTarget));
}

/// @dev Returns avgRate and endRateAtTarget.
/// @dev Assumes that the inputs `marketParams` and `id` match.
function _borrowRate(
    uint256 utilization,
    MarketState interestRateAccumulator
) internal view returns (uint256, int256) {

```

```

unchecked {
    // Safe "unchecked" cast because the utilization is smaller than 1 (scaled by WAD).
    int256 _utilization = int256(utilization);
    int256 errNormFactor = int256(_utilization) > TARGET_UTILIZATION
        ? WAD - TARGET_UTILIZATION
        : TARGET_UTILIZATION;
    int256 err = Math.wDivToZero(_utilization - TARGET_UTILIZATION, errNormFactor);
    // 38-bit rateAtTarget, 32-bit epoch<&&2 in accumulator
    int256 startRateAtTarget = int256(uint256(interestRateAccumulator.rateAtTarget()));
    // convert from epoch to time. Used to avoid Y2K38
    uint256 previousTime = interestRateAccumulator.marketEpoch() <&& 2;
    int256 avgRateAtTarget;
    int256 endRateAtTarget;
    if (startRateAtTarget == 0) {
        // First interaction.
        avgRateAtTarget = INITIAL_RATE_AT_TARGET;
        endRateAtTarget = INITIAL_RATE_AT_TARGET;
    } else {
        // The speed is assumed constant between two updates, but it is in fact not constant because of interest...
        // So the rate is always underestimated.
        int256 speed = Math.wMulToZero(ADJUSTMENT_SPEED, err);
        // Safe "unchecked" cast because block.timestamp - market.lastUpdate <= block.timestamp <= type(in...
        // Cap the elapsed time to prevent IRM drift
        int256 elapsed = Math.min(
            int256(block.timestamp) - int256(previousTime),
            IRM_MAX_ELAPSED_TIME
        );
        int256 linearAdaptation = speed * elapsed;
        if (linearAdaptation == 0) {
            // If linearAdaptation == 0, avgRateAtTarget = endRateAtTarget = startRateAtTarget;
            avgRateAtTarget = startRateAtTarget;
            endRateAtTarget = startRateAtTarget;
        } else {
            // Formula of the average rate that should be returned to Morpho Blue:
            // avg = 1/T * ∫0T curve(startRateAtTarget*exp(speed*x), err) dx
            // The integral is approximated with the trapezoidal rule:
            // avg ~ 1/T * ∑i=1N [curve(f((i-1) * T/N), err) + curve(f(i * T/N), err)] / 2 * T/N
            // Where f(x) = startRateAtTarget*exp(speed*x)
            // avg ~ ∑i=1N [curve(f((i-1) * T/N), err) + curve(f(i * T/N), err)] / (2 * N)
            // As curve is linear in its first argument:
            // avg ~ curve([∑i=1N [f((i-1) * T/N) + f(i * T/N)] / (2 * N), err)
            // avg ~ curve([(f(0) + f(T))/2 + ∑i=1N-1 f(i * T/N)] / N, err)
            // avg ~ curve([(startRateAtTarget + endRateAtTarget)/2 + ∑i=1N-1 f(i * T/N)] / N, err)
            // With N = 2:
            // avg ~ curve([(startRateAtTarget + endRateAtTarget)/2 + startRateAtTarget*exp(speed*T/2)] / 2, er...
            // avg ~ curve([startRateAtTarget + endRateAtTarget + 2*startRateAtTarget*exp(speed*T/2)] / 4, err)
            endRateAtTarget = _newRateAtTarget(startRateAtTarget, linearAdaptation);
            int256 midRateAtTarget = _newRateAtTarget(
                startRateAtTarget,
                linearAdaptation / 2
            );
            avgRateAtTarget =
                (startRateAtTarget + endRateAtTarget + 2 * midRateAtTarget) /
                4;
        }
    }
    // Safe "unchecked" cast because avgRateAtTarget >= 0.
    return (uint256(_curve(avgRateAtTarget, err)), endRateAtTarget);
}

}

/// @dev Returns the rate for a given `_rateAtTarget` and an `err`.
/// The formula of the curve is the following:
/// r = ((1-1/C)*err + 1) * rateAtTarget if err < 0
/// ((C-1)*err + 1) * rateAtTarget else.
function _curve(int256 _rateAtTarget, int256 err) private pure returns (int256) {
    // Non negative because 1 - 1/C >= 0, C - 1 >= 0.
    unchecked {
        int256 coeff = err < 0
            ? WAD - Math.wDivToZero(WAD, CURVE_STEEPNESS)
            : CURVE_STEEPNESS - WAD;
        // Non negative if _rateAtTarget >= 0 because if err < 0, coeff <= 1.
        return Math.wMulToZero(Math.wMulToZero(coeff, err) + WAD, _rateAtTarget);
    }
}

/// @dev Returns the new rate at target, for a given `startRateAtTarget` and a given `linearAdaptation`.
/// The formula is: max(min(startRateAtTarget * exp(linearAdaptation), maxRateAtTarget), minRateAtTarget).
function _newRateAtTarget(
    int256 startRateAtTarget,
    int256 linearAdaptation
) private pure returns (int256) {
    // Non negative because MIN_RATE_AT_TARGET >= 0.
    return
        Math.bound(
            Math.wMulToZero(startRateAtTarget, Math.wExp(linearAdaptation)),

```



```

        MIN_RATE_AT_TARGET,
        MAX_RATE_AT_TARGET
    );
}
/*/////////////////////////////////////////////////////////////
                        QUERY HELPERS
////////////////////////////////////////////////////////////*/
/// @notice Returns the stored VEGOID parameter
function vegoid() external view returns (uint8) {
    return uint8(VEGOID);
}
}
/*/////////////////////////////////////////////////////////////
                        BUILDER WALLETS
////////////////////////////////////////////////////////////*/
interface IERC20 {
    function balanceOf(address) external view returns (uint256);
    function transfer(address to, uint256 amount) external returns (bool);
}
contract BuilderWallet {
    address public immutable FACTORY;
    address public builderAdmin;
    constructor(address factory) {
        FACTORY = factory;
    }
    function init(address _builderAdmin) external {
        builderAdmin = _builderAdmin;
    }
    function sweep(address token, address to) external {
        if (msg.sender != builderAdmin) revert Errors.NotBuilder();
        uint256 bal = IERC20(token).balanceOf(address(this));
        if (bal == 0) return;
        bool ok = IERC20(token).transfer(to, bal);
        if (!ok) {
            // `from` is this wallet, `balance` is pre-transfer token balance
            revert Errors.TransferFailed(token, address(this), bal, bal);
        }
    }
}
}
library Create2Lib {
    function deploy(
        uint256 value,
        bytes32 salt,
        bytes memory code
    ) internal returns (address addr) {
        assembly {
            addr := create2(value, add(code, 0x20), mload(code), salt)
        }
        require(addr != address(0), "CREATE2 failed");
    }
}
}
contract BuilderFactory {
    using Create2Lib for uint256;
    address public immutable OWNER;
    constructor(address owner) {
        if (owner == address(0)) revert Errors.ZeroAddress();
        OWNER = owner;
    }
    modifier onlyOwner() {
        _onlyOwner();
        _;
    }
    function _onlyOwner() internal {
        require(msg.sender == OWNER, "NOT_OWNER");
    }
}
/**
 * @notice Deploys a BuilderWallet contract using CREATE2.
 * @param builderCode The uint256 used as the CREATE2 salt (must match caller's referral code).
 * @param builderAdmin The EOA/multisig allowed to sweep tokens from the wallet.
 * @return wallet The deployed wallet address (deterministic).
 */
function deployBuilder(
    uint48 builderCode,
    address builderAdmin
) external onlyOwner returns (address wallet) {
    bytes32 salt = bytes32(uint256(builderCode));
    // Constructor args are part of the init code and therefore part of the CREATE2 address.
    bytes memory initCode = abi.encodePacked(
        type(BuilderWallet).creationCode,
        abi.encode(address(this))
    );
    wallet = Create2Lib.deploy(0, salt, initCode);
    // now set the admin in storage (not part of init code)
    BuilderWallet(wallet).init(builderAdmin);
}

```

```

    }
    /**
     * @notice Computes the CREATE2 address for (builderCode, builderAdmin).
     * @dev Must match the formula used in the RiskEngine.
     */
    function predictBuilderWallet(uint48 builderCode) external view returns (address) {
        bytes32 salt = bytes32(uint256(builderCode));
        bytes32 initCodeHash = keccak256(
            abi.encodePacked(type(BuilderWallet).creationCode, abi.encode(address(this)))
        );
        bytes32 h = keccak256(abi.encodePacked(bytes1(0xff), address(this), salt, initCodeHash));
        return address(uint160(uint256(h)));
    }
}

```

[3/12] contracts/types/OraclePack.sol

```

// SPDX-License-Identifier: GPL-2.0-or-later
pragma solidity ^0.8.24;
import {Constants} from "@libraries/Constants.sol";
type OraclePack is uint256;
using OraclePackLibrary for OraclePack global;
/// @title A Panoptic OraclePack. Tracks a set of 8 price observations, 4 EMAs, and a timestamp to compute the internal ...
/// @author Axicon Labs Limited
//
//
//
// PACKING RULES FOR A ORACLEPACK:
// =====
// From the LSB to the MSB:
// (0) residual0      12bits : The last recorded residual.
// (1) residual1      12bits : The second last recorded residual.
// (2) residual2      12bits : The third last recorded residual.
// (3) residual3      12bits : The forth last residual.
// (4) residual4      12bits : The fifth last residual.
// (5) residual5      12bits : The sixth last residual.
// (6) residual6      12bits : The seventh last residual.
// (7) residual7      12bits : The eight last residual.
// (8) referenceTick   22bits : The reference tick used to reconstruce the obsercations as: last recorded tick = refer...
// (9) lockMode        2 bits : The externally controllable safe mode override
// (10) eonseEMA       22bits : The value of the exponential moving average (EMA) tick determined using the longest ti...
// (11) slowEMA        22bits : The value of the EMA tick determined using the second longest timescale
// (12) fastEMA        22bits : The value of EMA tick determined using the shortest timescale
// (13) spotEMA        22bits : The value of spot tick determined using the near instant timescale
// (14) orderMap       24bits : A map of the ordered residuals (see details below)
// (15) epoch          24bits : The latest epoch as recorded using a 64s epoch-based timekeeping
// Total              256bits : Total bits used by a OraclePack.
// =====
//
// The bit pattern is therefore:
//
// timestamp      orderMap      spotEMA      fastEMA      slowEMA      eonseEMA      lockMode      referenceTick      ...
// |<- 24 bits ->|<- 24 bits ->|<- 22 bits ->|<- 22 bits ->|<- 22 bits ->|<- 22 bits ->|<- 22 bits ->|<- 22 bits ->|<- 22 bits ->|
//
// The data for the last 8 interactions is stored as such:
// LAST UPDATED BLOCK TIMESTAMP (22 bits) -> 22 bits (use 28 bits for the timestamp and truncate the lower 6 bits to ...
// [BLOCK.TIMESTAMP]
// (000000000000000000000000) // dynamic
//
// ORDERING of tick indices least --> greatest (24 bits)
// The value of the bit codon ([#]) is a pointer to a tick index in the tick array.
// The position of the bit codon from most to least significant is the ordering of the
// tick index it points to from least to greatest.
//
// rank:  0   1   2   3   4   5   6   7
// slot: [7] [5] [3] [1] [0] [2] [4] [6]
//      111 101 011 001 000 010 100 110
//
//
//
//
library OraclePackLibrary {
    /*//////////////////////////////////////////////////////////////
                                ENCODING
    /*//////////////////////////////////////////////////////////////
    uint256 internal constant BITMASK_UINT22 = 0x3FFFFFF;
    uint256 internal constant BITMASK_UINT88 = 0xFFFFFFFFFFFFFFFFFFFFFFFF;
    uint256 internal constant UPPER_118BITS_MASK =
        0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF;
    uint256 internal constant LOCK_MODE_MASK = ~(uint256(3) <&& 118);

```

```

uint256 internal constant LOCK_MODE_ON = uint256(3) &lt;&lt; 118;
uint256 internal constant LOCK_MODE_OFF = 0;
/// @notice Create a new `OraclePack` given the relevant parameters.
/// @param _currentEpoch The current epoch timestamp
/// @param _newOrderMap The new order map for the observations
/// @param _updatedEMAs The updated EMA values
/// @param _referenceTick The reference tick
/// @param _currentResiduals The current residual ticks
/// @param _latestResidual The latest residual tick
/// @param _lockMode The lock mode state
/// @return The new OraclePack
function storeOraclePack(
    uint256 _currentEpoch,
    uint256 _newOrderMap,
    uint256 _updatedEMAs,
    int24 _referenceTick,
    uint96 _currentResiduals,
    int24 _latestResidual,
    uint256 _lockMode
) internal pure returns (OraclePack) {
    unchecked {
        return
            OraclePack.wrap(
                (_currentEpoch &lt;&lt; 232) +
                (_newOrderMap &lt;&lt; 208) +
                (_updatedEMAs &lt;&lt; 120) +
                ((_lockMode & 3) &lt;&lt; 118) +
                (uint256(uint24(_referenceTick) & 0xFFFF) &lt;&lt; 96) +
                uint256(_currentResiduals &lt;&lt; 12) +
                uint256(uint16(uint24(_latestResidual) & 0xFFFF))
            );
    }
}
/// @notice Concatenate all oracle ticks into a single uint96.
/// @param _spotEMA The spot EMA tick
/// @param _fastEMA The fast EMA tick
/// @param _slowEMA The slow EMA tick
/// @param _eonsEMA The eons EMA tick
/// @return A 96bit word concatenating all 4 input ticks
function packEMAs(
    int24 _spotEMA,
    int24 _fastEMA,
    int24 _slowEMA,
    int24 _eonsEMA
) internal pure returns (uint96) {
    unchecked {
        return
            uint96(
                (uint256(uint24(_spotEMA)) & 0xFFFF) &lt;&lt; 22 +
                ((uint256(uint24(_fastEMA)) & 0xFFFF) &lt;&lt; 22) +
                ((uint256(uint24(_slowEMA)) & 0xFFFF) &lt;&lt; 44) +
                ((uint256(uint24(_eonsEMA)) & 0xFFFF) &lt;&lt; 66)
            );
    }
}
/// @notice Lock the oracle pack.
/// @param self The OraclePack to lock
/// @return The locked OraclePack
function lock(OraclePack self) internal pure returns (OraclePack) {
    unchecked {
        return OraclePack.wrap((OraclePack.unwrap(self) & 0xFFFF) + (LOCK_MODE_ON));
    }
}
/// @notice Unlock the oracle pack.
/// @param self The OraclePack to unlock
/// @return The unlocked OraclePack
function unlock(OraclePack self) internal pure returns (OraclePack) {
    unchecked {
        return OraclePack.wrap((OraclePack.unwrap(self) & 0xFFFF) + (LOCK_MODE_OFF));
    }
}
}
//////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
//                                DECODING
//////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
/// @notice Get the EMAs of `self`.
/// @param self The OraclePack to retrieve the EMAs from
/// @return The EMAs of `self`
function EMAs(OraclePack self) internal pure returns (uint256) {
    unchecked {
        return (OraclePack.unwrap(self) >>> 120) & 0xFFFF;
    }
}
/// @notice Get the lastTick of `self`.
/// @param self The OraclePack to retrieve the lastTick from

```

```

/// @return _lastTick The lastTick of `self`
function lastTick(OraclePack self) internal pure returns (int24 _lastTick) {
    unchecked {
        _lastTick = self.referenceTick() + self.residualTick(0);
    }
}

/// @notice Get the spotEMA of `self`.
/// @param self The OraclePack to retrieve the spotEMA from
/// @return _spotEMA The spotEMA of `self`
function spotEMA(OraclePack self) internal pure returns (int24 _spotEMA) {
    unchecked {
        (_spotEMA, , , ) = getEMAs(self);
    }
}

/// @notice Get the fastEMA of `self`.
/// @param self The OraclePack to retrieve the fastEMA from
/// @return _fastEMA The fastEMA of `self`
function fastEMA(OraclePack self) internal pure returns (int24 _fastEMA) {
    unchecked {
        (, _fastEMA, , ) = getEMAs(self);
    }
}

/// @notice Get the slowEMA of `self`.
/// @param self The OraclePack to retrieve the slowEMA from
/// @return _slowEMA The slowEMA of `self`
function slowEMA(OraclePack self) internal pure returns (int24 _slowEMA) {
    unchecked {
        (, , _slowEMA, ) = getEMAs(self);
    }
}

/// @notice Get the eonseEMA of `self`.
/// @param self The OraclePack to retrieve the eonseEMA from
/// @return _eonseEMA The eonseEMA of `self`
function eonseEMA(OraclePack self) internal pure returns (int24 _eonseEMA) {
    unchecked {
        (, , , _eonseEMA, ) = getEMAs(self);
    }
}

/// @notice Get the all the EMA ticks of `self`.
/// @param self The OraclePack to retrieve the EMAs from
/// @return _spotEMA The spotEMA of `self`
/// @return _fastEMA The fastEMA of `self`
/// @return _slowEMA The slowEMA of `self`
/// @return _eonseEMA The eonseEMA of `self`
/// @return _medianTick The median tick of `self`
function getEMAs(
    OraclePack self
)
    internal
    pure
    returns (int24 _spotEMA, int24 _fastEMA, int24 _slowEMA, int24 _eonseEMA, int24 _medianTick)
{
    unchecked {
        uint256 _EMAs = self.EMAs();
        _spotEMA = int22toInt24((_EMAs & BITMASK_UINT22));
        _fastEMA = int22toInt24((_EMAs >> 22) & BITMASK_UINT22);
        _slowEMA = int22toInt24((_EMAs >> 44) & BITMASK_UINT22);
        _eonseEMA = int22toInt24((_EMAs >> 66) & BITMASK_UINT22);
        _medianTick = getMedianTick(self);
    }
}

/// @notice Get the order map of `self`.
/// @param self The OraclePack to retrieve the order map from
/// @return The order map of `self`
function orderMap(OraclePack self) internal pure returns (uint24) {
    unchecked {
        return uint24(OraclePack.unwrap(self) >> 208);
    }
}

/// @notice Get the reference tick of `self`.
/// @param self The OraclePack to retrieve the reference tick from
/// @return The last reference tick of `self`
function referenceTick(OraclePack self) internal pure returns (int24) {
    unchecked {
        return int22toInt24((OraclePack.unwrap(self) >> 96) & BITMASK_UINT22);
    }
}

/// @notice Get the residual tick of `self` at position i.
/// @param self The OraclePack to retrieve the residual tick from
/// @param i The position index
/// @return The residual tick of `self` at position i
function residualTickOrdered(OraclePack self, uint8 i) internal pure returns (int24) {
    unchecked {
        uint24 _orderMap = self.orderMap();

```

```

        uint8 index = uint8((_orderMap &gt;&gt; (i * 3)) & 7);
        return int12toInt24((OraclePack.unwrap(self) &gt;&gt; (index * 12)) & 0xFFF);
    }
}
/// @notice Get the residual tick of `self` at position i.
/// @param self The OraclePack to retrieve the residual tick from
/// @param i The position index
/// @return The residual tick of `self` at position i
function residualTick(OraclePack self, uint8 i) internal pure returns (int24) {
    unchecked {
        return int12toInt24((OraclePack.unwrap(self) &gt;&gt; (i * 12)) & 0xFFF);
    }
}
/// @notice Get the current residuals of `self`.
/// @param self The OraclePack to retrieve the current residuals from
/// @return The current residuals of `self`
function currentResiduals(OraclePack self) internal pure returns (uint96) {
    unchecked {
        return uint96(OraclePack.unwrap(self));
    }
}
/// @notice Get the lock mode of `self`.
/// @param self The OraclePack to retrieve the lock mode from
/// @return The lock mode of `self`
function lockMode(OraclePack self) internal pure returns (uint8) {
    unchecked {
        return uint8((OraclePack.unwrap(self) &gt;&gt; 118) & 3);
    }
}
/// @notice Get the timestamp of `self`.
/// @dev Returns a timestamp in seconds
/// @param self The OraclePack to retrieve the timestamp from.
/// @return The timestamp of `self`
function timestamp(OraclePack self) internal pure returns (uint24) {
    unchecked {
        return uint24((OraclePack.unwrap(self) &gt;&gt; 232) &lt;&lt; 6);
    }
}
/// @notice Get the epoch of `self`.
/// @dev Returns a timestamp in 64s based epochs
/// @param self The OraclePack to retrieve the epoch from.
/// @return The epoch of `self`
function epoch(OraclePack self) internal pure returns (uint24) {
    unchecked {
        return uint24((OraclePack.unwrap(self) &gt;&gt; 232));
    }
}
}
//////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
// HELPERS
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
/// @notice Converts a 12-bit signed integer to a 24-bit signed integer with proper sign extension
/// @dev Handles two's complement sign extension for 12-bit values stored in larger integer types
/// @dev The function checks bit 11 (the sign bit for 12-bit integers) and extends the sign
/// @dev if the number is negative by setting bits 12-15 to 1
/// @param x The input value containing a 12-bit signed integer in its lower 12 bits
/// @return The sign-extended 24-bit signed integer (as int24)
function int12toInt24(uint256 x) internal pure returns (int24) {
    unchecked {
        // Extract only the lower 12 bits
        uint16 u = uint16(x & 0xFFF);
        // Check if bit 11 is set
        // This is the sign bit for a 12-bit signed integer
        if ((u & 0x0800) != 0) {
            // Number is negative, extend the sign by setting bits 12-15 to 1
            u |= 0xF000;
        }
        return int24(int16(u));
    }
}
/// @notice Converts a 22-bit signed integer to a 24-bit signed integer with proper sign extension
/// @dev Handles two's complement sign extension for 22-bit values stored in larger integer types
/// @dev The function checks bit 21 (the sign bit for 22-bit integers) and extends the sign
/// @dev if the number is negative by setting bits 22-31 to 1
/// @param x The input value containing a 22-bit signed integer in its lower 22 bits
/// @return The sign-extended 24-bit signed integer (as int24)
function int22toInt24(uint256 x) internal pure returns (int24) {
    unchecked {
        // Extract only the lower 22 bits
        uint32 u = uint32(x & BITMASK_UINT22);
        // Check if bit 21 is set
        // This is the sign bit for a 22-bit signed integer
        if ((u & 0x200000) != 0) {
            // Number is negative, extend the sign by setting bits 22-31 to 1
            u |= 0xFFC00000;
        }
    }
}

```

```

    }
    return int24(int32(u));
}
}
/// @notice Updates exponential moving averages (EMAs) at multiple timescales with a new tick observation
/// @dev Implements a cascading time delta cap to prevent excessive convergence after periods of inactivity
/// @dev EMAs converge at most 75% toward the new tick value using linear approximation:  $\exp(-x) \approx 1-x$ 
/// @dev The function modifies timeDelta in cascade: longer periods cap it first, affecting shorter periods
/// @param oraclePack The packed median data containing current EMA values
/// @param timeDelta Time elapsed since last update in seconds (at least 64s since observations have to be in differ...
/// @param newTick The new tick observation to update EMAs toward
/// @param EMAsPeriods The packed EMA period values for spot, fast, slow, and eons EMAs
/// @return updatedEMAs The packed 88-bit value containing all four updated EMAs
function updateEMAs(
    OraclePack oraclePack,
    int256 timeDelta,
    int24 newTick,
    uint96 EMAsPeriods
) internal pure returns (uint256 updatedEMAs) {
    unchecked {
        int256 EMA_PERIOD_SPOT = int24(uint24(EMAsPeriods));
        int256 EMA_PERIOD_FAST = int24(uint24(EMAsPeriods >> 24));
        int256 EMA_PERIOD_SLOW = int24(uint24(EMAsPeriods >> 48));
        int256 EMA_PERIOD_EONS = int24(uint24(EMAsPeriods >> 72));
        // Extract current EMAs from oraclePack (88 bits starting at bit 120)
        uint256 _EMAs = oraclePack.EMAs();
        // Update eons EMA (bits 87-66)
        int24 _eonsEMA = int22toInt24((_EMAs >> 66) & BITMASK_UINT22);
        if (timeDelta > (3 * EMA_PERIOD_EONS) / 4) timeDelta = (3 * EMA_PERIOD_EONS) / 4;
        _eonsEMA = int24(_eonsEMA + (timeDelta * (newTick - _eonsEMA)) / EMA_PERIOD_EONS);
        // Update slow EMA (bits 65-44)
        int24 _slowEMA = int22toInt24((_EMAs >> 44) & BITMASK_UINT22);
        if (timeDelta > (3 * EMA_PERIOD_SLOW) / 4) timeDelta = (3 * EMA_PERIOD_SLOW) / 4;
        _slowEMA = int24(_slowEMA + (timeDelta * (newTick - _slowEMA)) / EMA_PERIOD_SLOW);
        // Update fast EMA (bits 43-22)
        int24 _fastEMA = int22toInt24((_EMAs >> 22) & BITMASK_UINT22);
        if (timeDelta > (3 * EMA_PERIOD_FAST) / 4) timeDelta = (3 * EMA_PERIOD_FAST) / 4;
        _fastEMA = int24(_fastEMA + (timeDelta * (newTick - _fastEMA)) / EMA_PERIOD_FAST);
        // Update spot EMA (bits 21-0)
        int24 _spotEMA = int22toInt24(_EMAs & BITMASK_UINT22);
        if (timeDelta > (3 * EMA_PERIOD_SPOT) / 4) timeDelta = (3 * EMA_PERIOD_SPOT) / 4;
        _spotEMA = int24(_spotEMA + (timeDelta * (newTick - _spotEMA)) / EMA_PERIOD_SPOT);
        // Pack updated EMAs back into 88-bit format
        updatedEMAs = packEMAs(_spotEMA, _fastEMA, _slowEMA, _eonsEMA);
    }
}
/// @notice Calculates the median tick from a packed median data structure
/// @dev Retrieves the 3rd and 4th ranked values from the sorted 8-slot queue and returns their average
/// @dev The median is calculated as: referenceTick + (rank3_residual + rank4_residual) / 2
/// @param oraclePack The packed structure containing:
/// - Order map indicating the rank of each slot
/// - Reference tick for absolute positioning
/// - 8 tick observations stored as 12-bit signed residuals relative to reference tick
/// @return medianTick The median tick value, representing the middle value of the sorted observations
function getMedianTick(OraclePack oraclePack) internal pure returns (int24) {
    unchecked {
        int24 rank3 = oraclePack.residualTickOrdered(3);
        int24 rank4 = oraclePack.residualTickOrdered(4);
        int24 _referenceTick = oraclePack.referenceTick();
        return _referenceTick + ((rank3) + (rank4)) / 2;
    }
}
/// @notice Inserts a new tick observation into the median data structure and updates EMAs
/// @dev Updates the sorted queue by finding the correct insertion point for the new tick residual
/// @dev The function maintains an 8-slot sorted queue using a 24-bit order map where each 3-bit segment
/// @dev represents the rank of the corresponding slot. Slot 7 is reserved for the new observation.
/// @param oraclePack The current packed median data structure containing:
/// - Bits 255-232: Current epoch timestamp
/// - Bits 231-208: 24-bit order map (8 slots × 3 bits each)
/// - Bits 207-128: Reserved for EMA data (88 bits): 10mins, 1hour, 8hour and 1day
/// - Bits 127-96: Reference tick (24 bits)
/// - Bits 95-12: Previous observations as 12-bit residuals (84 bits)
/// - Bits 11-0: Most recent observation residual (12 bits)
/// @param newTick The new tick observation to insert (as a residual relative to reference tick)
/// @param currentEpoch The current epoch timestamp ((block.timestamp >> 6) & 0xFFFFF)
/// @param timeDelta Time difference in seconds between current and last epoch (currentEpoch - recordedEpoch) * 64
/// @param EMAsPeriods The packed EMA period values for spot, fast, slow, and eons EMAs
/// @return newOraclePack The updated oraclePack with the new observation inserted
function insertObservation(
    OraclePack oraclePack,
    int24 newTick,
    uint256 currentEpoch,
    int256 timeDelta,
    uint96 EMAsPeriods

```

```

) internal pure returns (OraclePack newOraclePack) {
    unchecked {
        int24 _referenceTick = oraclePack.referenceTick();
        int24 lastResidual = newTick - _referenceTick;
        // update oracle pack and reference tick if the move is beyond residual threshold
        if (
            (lastResidual > Constants.MAX_RESIDUAL_THRESHOLD) ||
            (lastResidual < -Constants.MAX_RESIDUAL_THRESHOLD)
        ) {
            (_referenceTick, oraclePack) = rebaseOraclePack(oraclePack);
            lastResidual = newTick - _referenceTick;
        }
        uint24 _newOrderMap;
        {
            uint24 _orderMap = oraclePack.orderMap();
            uint256 _oraclePack = OraclePack.unwrap(oraclePack);
            uint24 shift = 1;
            bool below = true;
            uint24 rank;
            int24 entry;
            for (uint8 i; i < 8; ++i) {
                // read the rank from the existing ordering
                rank = (_orderMap >>> (3 * i)) & 7; // mod 2**3
                if (rank == 7) {
                    shift -= 1;
                    continue;
                }
                // read the corresponding entry
                entry = int12toInt24((_oraclePack >>> (rank * 12)) & 0xFFFF); // mod 2**12
                if ((below && (lastResidual > entry)) {
                    shift += 1;
                    below = false;
                }
                _newOrderMap = _newOrderMap + ((rank + 1) <<< (3 * (i + shift - 1)));
            }
        }
        {
            uint256 _EMAs = updateEMAs(oraclePack, timeDelta, newTick, EMAsPeriods);
            uint8 _lockMode = oraclePack.lockMode();
            uint96 _currentResiduals = oraclePack.currentResiduals();
            newOraclePack = storeOraclePack(
                currentEpoch,
                _newOrderMap,
                _EMAs,
                _referenceTick,
                _currentResiduals,
                lastResidual,
                _lockMode
            );
        }
    }
}

/// @notice Clamps a new tick observation to prevent large price movements that could manipulate the median
/// @dev Limits the new tick to be within `clampDelta` of the most recent tick observation
/// @dev This prevents flash loan attacks or other price manipulation attempts from skewing the median calculation
/// @param newTick The new tick observation from Uniswap TWAP that needs to be clamped
/// @param _oraclePack The current OraclePack containing the reference tick and most recent observation
/// @param clampDelta The maximum allowed tick deviation from the last observation
/// @return clamped The clamped tick value, guaranteed to be within `clampDelta` of the last observation
function clampTick(
    int24 newTick,
    OraclePack _oraclePack,
    int24 clampDelta
) internal pure returns (int24 clamped) {
    unchecked {
        int24 _lastTick = _oraclePack.lastTick();
        // Clamp lastObservedTick to be within clampDelta of lastTick
        if (newTick > _lastTick + clampDelta) {
            clamped = _lastTick + clampDelta;
        } else if (newTick < _lastTick - clampDelta) {
            clamped = _lastTick - clampDelta;
        } else {
            clamped = newTick;
        }
    }
}

/// @notice Takes a packed structure representing a sorted 8-slot queue of ticks and returns the median of those val...
/// @dev Also inserts the latest Uniswap observation into the buffer, resorts, and returns if the last entry is at l...
/// @param oraclePack The packed structure representing the sorted 8-slot queue of ticks
/// @param currentTick The current tick as return from slot0
/// @return _medianTick The median of the provided 8-slot queue of ticks in `oraclePack`
/// @return _updatedOraclePack The updated 8-slot queue of ticks with the latest observation inserted if the last en...
function computeInternalMedian(
    OraclePack oraclePack,

```

```

    int24 currentTick,
    uint96 EMAperiods,
    int24 clampDelta
) internal view returns (int24 _medianTick, OraclePack _updatedOraclePack) {
    unchecked {
        // return the average of the rank 3 and 4 values
        _medianTick = getMedianTick(oraclePack);
        uint256 currentEpoch;
        bool differentEpoch;
        int256 timeDelta;
        {
            currentEpoch = (block.timestamp >> 6) & 0xFFFF; // 64-long epoch, taken mod 2**24
            uint256 recordedEpoch = oraclePack.epoch();
            differentEpoch = currentEpoch != recordedEpoch;
            timeDelta = int256(uint256(uint24(currentEpoch - recordedEpoch))) * 64; // take a rough time delta, bas...
        }
        // only proceed if last entry is in a different epoch
        if (differentEpoch) {
            int24 clampedTick = clampTick(currentTick, oraclePack, clampDelta);
            _updatedOraclePack = insertObservation(
                oraclePack,
                clampedTick,
                currentEpoch,
                timeDelta,
                EMAperiods
            );
        }
    }
}

/// @notice Computes various oracle prices corresponding to a Uniswap pool.
/// @param self The packed structure representing the sorted 8-slot queue of internal median observations
/// @param _currentTick The current tick in the Uniswap pool
/// @param _EMAperiods A packed uint96 containing the EMA period data
/// @param clampDelta The max change in tick between updates
/// @return spotEMATick The spot tick, computed from the shortest timescale EMA
/// @return medianTick The median oracle tick computed from the last 8 observations
/// @return latestTick The latest observed tick in Panoptic before the current transaction
/// @return oraclePack The updated value for `s_oraclePack` (0 if not enough time has passed since last observation)
function getOracleTicks(
    OraclePack self,
    int24 _currentTick,
    uint96 _EMAperiods,
    int24 clampDelta
)
internal
view
returns (int24 spotEMATick, int24 medianTick, int24 latestTick, OraclePack oraclePack)
{
    // Extract the spot EMA from the lowest 22 bits of the packed EMAs value
    spotEMATick = self.spotEMA();
    // get the tick at the last protocol interaction
    latestTick = self.lastTick();
    // finally, get the median tick
    (medianTick, oraclePack) = computeInternalMedian(
        self,
        _currentTick,
        _EMAperiods,
        clampDelta
    );
}

/// @notice Rebases the median data structure when tick residuals exceed the 12-bit signed integer range
/// @dev When residuals become too large (>2047 or <-2048), this function shifts the reference tick
/// @dev to the current median and adjusts all stored residuals relative to the new reference
/// @dev This maintains precision while keeping residuals within the 12-bit storage constraint
/// @param oraclePack The current oraclePack with residuals that have exceeded the threshold
/// @return _newReferenceTick The new reference tick (set to the current median)
/// @return rebasedOraclePack The updated median data structure with:
///     - New reference tick set to the current median
///     - All residuals recalculated relative to the new reference
///     - All other data (order map, EMAs, epoch) preserved
function rebaseOraclePack(
    OraclePack oraclePack
) internal pure returns (int24 _newReferenceTick, OraclePack rebasedOraclePack) {
    unchecked {
        int24 _referenceTick = oraclePack.referenceTick();
        _newReferenceTick = getMedianTick(oraclePack);
        int24 deltaOffset = _newReferenceTick - _referenceTick;
        uint256 _newResiduals;
        for (uint8 i; i < 8; ++i) {
            int24 _residual = oraclePack.residualTick(i);
            int24 newEntry = _residual - deltaOffset;
            _newResiduals += (uint256(uint16(uint24(newEntry) & 0xFFF)) & 0xFFF) << (i * 12);
        }
        rebasedOraclePack = OraclePack.wrap(

```



```

) internal pure returns (MarketState result) {
    assembly {
        // 1. Clear the lowest 80 bits using not(BORROW_INDEX_MASK)
        let cleared := and(self, not(BORROW_INDEX_MASK))
        // 2. OR with the new value (no shift needed, it's at 0)
        result := or(cleared, newIndex)
    }
}

/// @notice Update the Market Epoch (Bits 80-111)
/// @param self The MarketState to update
/// @param newEpoch The new market epoch value
/// @return result The updated MarketState with the new market epoch
function updateMarketEpoch(
    MarketState self,
    uint32 newEpoch
) internal pure returns (MarketState result) {
    assembly {
        // 1. Clear bits 80-111
        let cleared := and(self, not(EPOCH_MASK))
        // 2. Shift new value to 80 and combine
        result := or(cleared, shl(80, newEpoch))
    }
}

/// @notice Update the Rate At Target (Bits 112-149)
/// @param self The MarketState to update
/// @param newRate The new rate at target value
/// @return result The updated MarketState with the new rate at target
function updateRateAtTarget(
    MarketState self,
    uint40 newRate
) internal pure returns (MarketState result) {
    assembly {
        // 1. Clear bits 112-149
        let cleared := and(self, not(TARGET_RATE_MASK))
        // 2. Safety: Mask the input to ensure it fits in 38 bits (0x3FFFFFFFFF)
        // This prevents 'newRate' from corrupting the neighbor if it > 38 bits.
        let safeRate := and(newRate, 0x3FFFFFFFFF)
        // 3. Shift to 112 and combine
        result := or(cleared, shl(112, safeRate))
    }
}

/// @notice Update the Unrealized Interest (Bits 150-255)
/// @param self The MarketState to update
/// @param newInterest The new unrealized interest value
/// @return result The updated MarketState with the new unrealized interest
function updateUnrealizedInterest(
    MarketState self,
    uint128 newInterest
) internal pure returns (MarketState result) {
    assembly {
        // 1. Clear bits 150-255
        let cleared := and(self, not(UNREALIZED_INTEREST_MASK))
        // 2. Safety: Mask input to 106 bits
        // (1 << 106) - 1
        let max106 := sub(shl(106, 1), 1)
        let safeInterest := and(newInterest, max106)
        // 3. Shift to 150 and combine
        result := or(cleared, shl(150, safeInterest))
    }
}

}

/*//////////////////////////////////////////////////////////////
                        DECODING
//////////////////////////////////////////////////////////////*/

/// @notice Get the borrowIndex of `self`.
/// @param self The MarketState to retrieve the borrowIndex state from
/// @return result The borrowIndex of `self`
function borrowIndex(MarketState self) internal pure returns (uint80 result) {
    assembly {
        result := and(self, 0xFFFFFFFFFFFFFFFFFFFF)
    }
}

/// @notice Get the marketEpoch of `self`.
/// @param self The MarketState to retrieve the marketEpoch from
/// @return result The marketEpoch of `self`
function marketEpoch(MarketState self) internal pure returns (uint32 result) {
    assembly {
        result := and(shr(80, self), 0xFFFFFFFF)
    }
}

/// @notice Get the rateAtTarget of `self`.
/// @param self The MarketState to retrieve the rateAtTarget from
/// @return result The rateAtTarget of `self`
function rateAtTarget(MarketState self) internal pure returns (uint40 result) {
    assembly {

```

```

        result := and(shr(112, self), 0x3FFFFFFF)
    }
}
/// @notice Get the unrealizedInterest of `self`.
/// @param self The MarketState to retrieve the unrealizedInterest from
/// @return result The unrealizedInterest of `self`
function unrealizedInterest(MarketState self) internal pure returns (uint128 result) {
    assembly {
        result := shr(150, self)
    }
}
}

```

[5/12] contracts/types/PoolData.sol

```

// SPDX-License-Identifier: GPL-2.0-or-later
pragma solidity ^0.8.24;
type PoolData is uint256;
using PoolDataLibrary for PoolData global;
/// @title A Panoptic Pool Data. Tracks the Uniswap Pool, the minEnforcedTick, and the maxEnforcedTick
/// @author Axicon Labs Limited
//
//
// PACKING RULES FOR A POOLDATA:
// =====
// From the LSB to the MSB:
// (1) maxLiquidityPerTick 128bits : The max liquidity per tick
// (2) poolId 64bits : The poolId
// (3) minEnforcedTick 24bits : The current minimum enforced tick for the pool in the SFPM (int24).
// (4) maxEnforcedTick 24bits : The current maximum enforced tick for the pool in the SFPM (int24).
// (5) initialized 1bit : Whether the pool has been initialized
// Total 241bits : Total bits used by a PoolData.
// =====
//
// The bit pattern is therefore:
//
// (5) (4) (3) (2) (1)
// &lt;--- 1 bit ---&gt;&lt;--- 24 bits ---&gt; &lt;--- 24 bits ---&gt; &lt;--- 64 bits ---&gt; &lt;--- 128 bits...
// initialized maxEnforcedTick minEnforcedTick poolId maxLiquidityPerTick
//
// &lt;--- most significant bit least significant bit ---&gt;
//
library PoolDataLibrary {
    /*//////////////////////////////////////////////////////////////
    ENCODING
    //////////////////////////////////////////////////////////////////////*/
    /// @notice Create a new `PoolData` given by UniswapV3Pool, min/maxEnforcedTick.
    /// @param _maxLiquidityPerTick The max liquidity per tick
    /// @param _poolId The poolId of the Uniswap pool
    /// @param _minEnforcedTick The current minimum enforced tick for the pool in the SFPM
    /// @param _maxEnforcedTick The current maximum enforced tick for the pool in the SFPM
    /// @return The new PoolData with the given IUniswapV3Pool and min/maxEnforcedTick
    function storePoolData(
        uint128 _maxLiquidityPerTick,
        uint64 _poolId,
        int24 _minEnforcedTick,
        int24 _maxEnforcedTick,
        bool _initialized
    ) internal pure returns (PoolData) {
        unchecked {
            return
                PoolData.wrap(
                    _maxLiquidityPerTick +
                    (uint256(_poolId) &lt;&lt; 128) +
                    (uint256(uint24(_minEnforcedTick)) &lt;&lt; 192) +
                    (uint256(uint24(_maxEnforcedTick)) &lt;&lt; 216) +
                    (uint256(_initialized ? 1 : 0) &lt;&lt; 240)
                );
        }
    }
}
/*//////////////////////////////////////////////////////////////
DECODING
////////////////////////////////////////////////////////////////////*/
/// @notice Get the maxLiquidityPerTick of `self`.
/// @param self The PoolData to retrieve the max liquidity from
/// @return The maxLiquidityPerTick of `self`
function maxLiquidityPerTick(PoolData self) internal pure returns (uint128) {
    unchecked {
        return uint128(PoolData.unwrap(self));
    }
}

```

```

}
/// @notice Get the poolId of `self`.
/// @param self The PoolData to retrieve the poolId from
/// @return The poolId of `self`
function poolId(PoolData self) internal pure returns (uint64) {
    unchecked {
        return uint64(PoolData.unwrap(self) >>> 128);
    }
}
/// @notice Get the min enforced tick of `self`.
/// @param self The PoolData to retrieve the min enforced tick from
/// @return The min enforced tick of `self`
function minEnforcedTick(PoolData self) internal pure returns (int24) {
    unchecked {
        return int24(uint24(PoolData.unwrap(self) >>> 192));
    }
}
/// @notice Get the max enforced tick of `self`.
/// @param self The PoolData to retrieve the max enforced tick from
/// @return The max enforced tick of `self`
function maxEnforcedTick(PoolData self) internal pure returns (int24) {
    unchecked {
        return int24(uint24(PoolData.unwrap(self) >>> 216));
    }
}
/// @notice Get the initialized bool of `self`.
/// @param self The PoolData to retrieve initialized flag from
/// @return The initialized flag of `self`
function initialized(PoolData self) internal pure returns (bool) {
    unchecked {
        return ((PoolData.unwrap(self) >>> 240) % 2) > 0;
    }
}
}
}

```

[6/12] contracts/types/RiskParameters.sol

```

// SPDX-License-Identifier: GPL-2.0-or-later
pragma solidity ^0.8.24;
type RiskParameters is uint256;
using RiskParametersLibrary for RiskParameters global;
/// @title A Panoptic Risk Parameters. Tracks the data outputted from the RiskEngine, like the safeMode, commission fees...
/// @author Axicon Labs Limited
//
//
// PACKING RULES FOR A RISKPARAMETERS:
// =====
// From the LSB to the MSB:
// (1) safeMode          4 bits : The safeMode state
// (2) notionalFee       14 bits : The fee to be charged on notional at mint
// (3) premiumFee        14 bits : The fee to be charged on the premium at burn
// (4) protocolSplit     14 bits : The part of the fee that goes to the protocol w/ buildercodes
// (5) builderSplit      14 bits : The part of the fee that goes to the builder w/ buildercodes
// (6) tickDeltaLiquidation 13 bits : The MAX_TWAP_DELTA_LIQUIDATION. Tick deviation = 1.0001**(2**13) = +/- 126%
// (7) maxSpread         22 bits : The MAX_SPREAD, in bps. Max fraction removed = 2**22/(2**22 + 10_000) = 99.76%
// (8) bpDecreaseBuffer  26 bits : The BP_DECREASE_BUFFER, in millitick
// (9) maxLegs           7 bits : The MAX_OPEN_LEGS (constrained to be <128)
// (9) feeRecipient      128bits : The recipient of the commission fee split
// Total                256bits : Total bits used by a RiskParameters.
// =====
//
// The bit pattern is therefore:
//
//      (9)      (8)      (7)      (6)      (5)      (4)      (3)      (...)
//      &lt;--- 128 bits --&gt;&lt;--- 7 bits --&gt;&lt;--- 26 bits --&gt;&lt;--- 22 bits --&gt;&lt;--- 13 bits --&gt;&lt;--- 14...
//      feeRecipient  maxLegs  bpDecrease  maxSpread  tickDelta  builderSplit  protocolSplit  premiu...
//
//      &lt;--- most significant bit                                     least significant bi...
//
library RiskParametersLibrary {
    /*//////////////////////////////////////////////////////////////
    ENCODING
    //////////////////////////////////////////////////////////////////////*/
    /// @notice Create a new `RiskParameters` object.
    /// @param _safeMode The safe mode state (uint6)
    /// @param _notionalFee The commission fee (uint14)
    /// @param _premiumFee The commission fee (uint14)
    /// @param _protocolSplit The part of the fee that goes to the protocol w/ buildercodes (uint14)
    /// @param _builderSplit The part of the fee that goes to the builder w/ buildercodes (uint14)
    /// @param _tickDeltaLiquidation The MAX_TWAP_DELTA_LIQUIDATION (uint16)

```

```

/// @param _maxSpread The MAX_SPREAD, in bps (uint24)
/// @param _bpDecreaseBuffer The BP_DECREASE_BUFFER, in millitick (uint26)
/// @param _maxLegs The maximum allowed number of legs across all open positions for a user
/// @param _feeRecipient The recipient of the commission fee split (uint128)
/// @return result The new RiskParameters object
function storeRiskParameters(
    uint256 _safeMode,
    uint256 _notionalFee,
    uint256 _premiumFee,
    uint256 _protocolSplit,
    uint256 _builderSplit,
    uint256 _tickDeltaLiquidation,
    uint256 _maxSpread,
    uint256 _bpDecreaseBuffer,
    uint256 _maxLegs,
    uint256 _feeRecipient
) internal pure returns (RiskParameters result) {
    assembly {
        result := add(
            add(
                add(
                    add(_safeMode, shl(4, _notionalFee)),
                    add(shl(18, _premiumFee), shl(32, _protocolSplit))
                ),
                add(shl(46, _builderSplit), shl(60, _tickDeltaLiquidation))
            ),
            add(
                add(shl(73, _maxSpread), add(shl(95, _bpDecreaseBuffer), shl(121, _maxLegs))),
                shl(128, _feeRecipient)
            )
        )
    }
}

}

/*//////////////////////////////////////
                                DECODING
//////////////////////////////////////*/
/// @notice Get the safeMode state of `self`.
/// @param self The RiskParameters to retrieve the safeMode state from
/// @return result The safeMode of `self`
function safeMode(RiskParameters self) internal pure returns (uint8 result) {
    assembly {
        result := and(self, 0xF)
    }
}

/// @notice Get the notionalFee of `self`.
/// @param self The RiskParameters to retrieve the notionalFee from
/// @return result The notionalFee of `self`
function notionalFee(RiskParameters self) internal pure returns (uint16 result) {
    assembly {
        result := and(shr(4, self), 0x3FFF)
    }
}

/// @notice Get the premiumFee of `self`.
/// @param self The RiskParameters to retrieve the premiumFee from
/// @return result The premiumFee of `self`
function premiumFee(RiskParameters self) internal pure returns (uint16 result) {
    assembly {
        result := and(shr(18, self), 0x3FFF)
    }
}

/// @notice Get the protocolSplit of `self`.
/// @param self The RiskParameters to retrieve the protocolSplit from
/// @return result The protocolSplit of `self`
function protocolSplit(RiskParameters self) internal pure returns (uint16 result) {
    assembly {
        result := and(shr(32, self), 0x3FFF)
    }
}

/// @notice Get the builderSplit of `self`.
/// @param self The RiskParameters to retrieve the builderSplit from
/// @return result The builderSplit of `self`
function builderSplit(RiskParameters self) internal pure returns (uint16 result) {
    assembly {
        result := and(shr(46, self), 0x3FFF)
    }
}

/// @notice Get the tickDeltaLiquidation of `self`.
/// @param self The RiskParameters to retrieve the tickDeltaLiquidation from
/// @return result The tickDeltaLiquidation of `self`
function tickDeltaLiquidation(RiskParameters self) internal pure returns (uint16 result) {
    assembly {
        result := and(shr(60, self), 0x1FFF)
    }
}
}

```


[illegible]

```

// @dev Used to verify callbacks and initialize pools.
IUniswapV3Factory internal immutable FACTORY;
/// @notice The approximate minimum amount of tokens it should require to fill `maxLiquidityPerTick` at the minimum ...
uint256 internal immutable MIN_ENFORCED_TICKFILL_COST;
/// @notice The multiplier, in basis points, to apply to the token supply and set as the minimum enforced tick fill ...
uint256 internal immutable SUPPLY_MULTIPLIER_TICKFILL;
/*
////////////////////////////////////
STORAGE
////////////////////////////////////
*/
/// @notice Retrieve the corresponding poolId for a given Uniswap V3 pool address.
mapping(address univ3pool => mapping(uint256 vegoid => PoolData poolData))
    internal s_addressToPoolData;
/// @notice Retrieve the PoolData struct corresponding to a given poolId.
mapping(uint64 poolId => address univ3pool) internal s_poolIdToAddress;
/*
    We're tracking the amount of net and removed liquidity for the specific region:
    net amount
    received minted
    ▲ for isLong=0
    ■■■■■■-T amount
    moved out actual amount
    due isLong=1 in the UniswapV3Pool
    mints
    ■■■■■■-(T-R)
    ■■■■■■-R
    total=T removed=R net=(T-R)
    removed liquidity r net liquidity N=(T-R)
    * |&lt;----- 128 bits -----&gt;|&lt;----- 128 bits -----&gt;|
    * |&lt;----- 256 bits -----&gt;|
    */
/// @notice Retrieve the current liquidity state in a chunk for a given user.
/// @dev `removedAndNetLiquidity` is a LeftRight. The right slot represents the liquidity currently sold (added) in ...
// the left slot represents the amount of liquidity currently bought (removed) that has been removed from the AMM - ...
// The reason why it is called "removedLiquidity" is because long options are created by removed liquidity - ie. sho...
mapping(bytes32 positionKey => LeftRightUnsigned removedAndNetLiquidity)
    internal s_accountLiquidity;
/*
    Any liquidity that has been deposited in the AMM using the SFPM will collect fees over
    time, we call this the gross premia. If that liquidity has been removed, we also need to
    keep track of the amount of fees that *would have been collected*, we call this the owed
    premia. The gross and owed premia are tracked per unit of liquidity by the
    s_accountPremiumGross and s_accountPremiumOwed accumulators.
    Here is how we can use the accumulators to compute the Gross, Net, and Owed fees collected
    by any position.
    Let's say Charlie the smart contract deposited T into the AMM and later removed R from that
    same tick using a tokenId with a isLong=1 parameter. Because the netLiquidity is only (T-R),
    the AMM will collect fees equal to:
        net_feesCollectedX128 = feeGrowthX128 * (T - R)
                             = feeGrowthX128 * N
    where N = netLiquidity = T-R. Had that liquidity never been removed, we want the gross
    premia to be given by:
        gross_feesCollectedX128 = feeGrowthX128 * T
    So we must keep track of fees for the removed liquidity R so that the long premia exactly
    compensates for the fees that would have been collected from the initial liquidity.
    In addition to tracking, we also want to track those fees plus a small spread. Specifically,
    we want:
        gross_feesCollectedX128 = net_feesCollectedX128 + owed_feesCollectedX128
    where
        owed_feesCollectedX128 = feeGrowthX128 * R * (1 + spread)
    A very opinionated definition for the spread is:
        spread = v*(liquidity removed from that strike)/(netLiquidity remaining at that strike)
              = v*R/N
    For an arbitrary parameter 0 &lt;= v &lt;= 1 (v = 1/VEG0ID). This way, the gross_feesCollectedX128 will be given...
        gross_feesCollectedX128 = feeGrowthX128 * N + feeGrowthX128*R*(1 + v*R/N)
                             = feeGrowthX128 * T + feesGrowthX128*v*R^2/N
                             = feeGrowthX128 * T * (1 + v*R^2/(N*T))
    The s_accountPremiumOwed accumulator tracks the feeGrowthX128 * R * (1 + spread) term
    per unit of removed liquidity R every time the position touched:
        s_accountPremiumOwed += feeGrowthX128 * R * (1 + v*R/N) / R
                             += feeGrowthX128 * (T - R + v*R)/N
                             += feeGrowthX128 * T/N * (1 - R/T + v*R/T)
    Note that the value of feeGrowthX128 can be extracted from the amount of fees collected by
    the smart contract since the amount of feesCollected is related to feeGrowthX128 according
    to:
        feesCollected = feesGrowthX128 * (T-R)
    So that we get:
        feesGrowthX128 = feesCollected/N
    And the accumulator is computed from the amount of collected fees according to:
        s_accountPremiumOwed += feesCollected * T/N^2 * (1 - R/T + v*R/T)
    So, the amount of owed premia for a position of size r minted at time t1 and burnt at
    time t2 is:
        owedPremia(t1, t2) = (s_accountPremiumOwed_t2-s_accountPremiumOwed_t1) * r
                          = ΔfeesGrowthX128 * r * T/N * (1 - R/T + v*R/T)

```


$$\begin{aligned}
&= \Delta \text{feesGrowthX128} * r * (T - R + v * R) / N \\
&= \Delta \text{feesGrowthX128} * r * (N + v * R) / N \\
&= \Delta \text{feesGrowthX128} * r * (1 + v * R / N) \quad (\text{same as Eqn 1})
\end{aligned}$$

This way, the amount of premia owed for a position will match Eqn 1 exactly.

Similarly, the amount of gross fees for the total liquidity is tracked in a similar manner by the `s_accountPremiumGross` accumulator.

However, since we require that Eqn 2 holds up-- ie. the gross fees collected should be equal to the net fees collected plus the over fees plus the small spread, the expression for the `s_accountPremiumGross` accumulator has to be given by (you'll see why in a minute):

$$s_accountPremiumGross += feesCollected * T / N^2 * (1 - R / T + v * R^2 / T^2) \quad (\text{Eqn 4})$$

This expression can be used to calculate the fees collected by a position of size `t` between times `t1` and `t2` according to:

$$\begin{aligned}
grossPremia(t1, t2) &= \Delta(s_accountPremiumGross) * t \\
&= \Delta feeGrowthX128 * t * T / N * (1 - R / T + v * R^2 / T^2) \\
&= \Delta feeGrowthX128 * t * (T - R + v * R^2 / T) / N \\
&= \Delta feeGrowthX128 * t * (N + v * R^2 / T) / N \\
&= \Delta feeGrowthX128 * t * (1 + v * R^2 / (N * T)) \quad (\text{same as Eqn 2})
\end{aligned}$$

where the last expression matches Eqn 2 exactly.

In summary, the `s_accountPremium` accumulators allow smart contracts that need to handle long+short liquidity to guarantee that liquidity deposited always receives the correct premia, whether that liquidity has been removed from the AMM or not.

Note that the expression for the spread is extremely opinionated, and may not fit the specific risk management profile of every smart contract. And simply setting the `v` parameter to zero would get rid of the "spread logic".

```

*/
/// @notice Per-liquidity accumulator for the premium owed by buyers on a given chunk, tokenType and account.
mapping(bytes32 positionKey => LeftRightUnsigned accountPremium) private s_accountPremiumOwed;
/// @notice Per-liquidity accumulator for the premium earned by sellers on a given chunk, tokenType and account.
mapping(bytes32 positionKey => LeftRightUnsigned accountPremium) private s_accountPremiumGross;
/// @notice Per-liquidity accumulator for the fees collected on an account for a given chunk.
/// @dev Base fees are stored as `int128((feeGrowthInsideLastX128 * liquidity) / 2**128)`, which allows us to store ...
/// @dev Right slot: int128 token0 base fees, Left slot: int128 token1 base fees.
/// @dev feesBase represents the baseline fees collected by the position last time it was updated - this is recalcul...
mapping(bytes32 positionKey => LeftRightSigned baseFees0And1) internal s_accountFeesBase;
/////////////////////////////////////////////////////////////
INITIALIZATION
/////////////////////////////////////////////////////////////
/// @notice Set the canonical Uniswap V3 Factory address.
/// @param _factory The canonical Uniswap V3 Factory address
/// @param _minEnforcedTickFillCost The minimum amount of tokens it should require to fill `maxLiquidityPerTick` at ...
/// @param _supplyMultiplierTickFill The multiplier, in basis points, to apply to the token supply and set as the mi...
constructor(
    IUniswapV3Factory _factory,
    uint256 _minEnforcedTickFillCost,
    uint256 _supplyMultiplierTickFill
) {
    FACTORY = _factory;
    MIN_ENFORCED_TICKFILL_COST = _minEnforcedTickFillCost;
    SUPPLY_MULTIPLIER_TICKFILL = _supplyMultiplierTickFill;
}
/// @notice Initialize a Uniswap V3 pool in the SFPM.
/// @dev Revert if already initialized.
/// @param token0 The contract address of token0 of the pool
/// @param token1 The contract address of token1 of the pool
/// @param fee The fee level of the of the underlying Uniswap V3 pool, denominated in hundredths of bips
function initializeAMMPool(
    address token0,
    address token1,
    uint24 fee,
    uint8 vegoid
) external returns (uint64 poolId) {
    // sort the tokens, if necessary:
    (token0, token1) = token0 < token1 ? (token0, token1) : (token1, token0);
    // compute the address of the Uniswap V3 pool for the given token0, token1, and fee tier
    address univ3pool = FACTORY.getPool(token0, token1, fee);
    // reverts if the Uniswap V3 pool has not been initialized
    if (univ3pool == address(0)) revert Errors.PoolNotInitialized();
    // return if the pool has already been initialized in SFPM
    // pools can be initialized from the Panoptic Factory or by calling initializeAMMPool directly, so reverting
    // could prevent a PanopticPool from being deployed on a previously initialized but otherwise valid pool
    if (s_addressToPoolData[univ3pool][vegoid].initialized())
        return uint64(s_addressToPoolData[univ3pool][vegoid].poolId());
    int24 tickSpacing = IUniswapV3Pool(univ3pool).tickSpacing();
    // The base poolId is composed as follows:
    // [tickSpacing][pool pattern]
    // [16 bit tickSpacing][most significant 48 bits of the pool address]
    poolId = _getPoolId(univ3pool, tickSpacing, vegoid);
    // There are 1,099,511,627,776 possible pool patterns.
    // A modern GPU can generate a collision in such a space relatively quickly,
    // so if a collision is detected increment the pool pattern until a unique poolId is found
    while (address(s_poolIdToAddress[poolId]) != address(0)) {
        poolId = PanopticMath.incrementPoolPattern(poolId);
    }
    uint256 maxLiquidityPerTick = Math.getMaxLiquidityPerTick(tickSpacing);

```

```

int24 minEnforcedTick;
int24 maxEnforcedTick;
unchecked {
    minEnforcedTick = int24(
        -Math.getApproxTickWithMaxAmount(
            Math.max(
                MIN_ENFORCED_TICKFILL_COST,
                (IERC20Partial(token1).totalSupply() * SUPPLY_MULTIPLIER_TICKFILL) / 10_000
            ),
            tickSpacing,
            maxLiquidityPerTick
        )
    );
    maxEnforcedTick = int24(
        Math.getApproxTickWithMaxAmount(
            Math.max(
                MIN_ENFORCED_TICKFILL_COST,
                (IERC20Partial(token0).totalSupply() * SUPPLY_MULTIPLIER_TICKFILL) / 10_000
            ),
            tickSpacing,
            maxLiquidityPerTick
        )
    );
}
s_addressToPoolData[univ3pool][vegoId] = PoolDataLibrary.storePoolData(
    uint128(maxLiquidityPerTick),
    poolId,
    minEnforcedTick,
    maxEnforcedTick,
    true
);
s_poolIdToAddress[poolId] = univ3pool;
emit PoolInitialized(univ3pool, poolId, minEnforcedTick, maxEnforcedTick);
}
/// @notice Given an address to a Uniswap V3 pool, return its 64-bit ID as used in the `TokenId` of Panoptic.
/// Example:
///     the 64 bits are the 40 *last* (most significant) bits - and thus corresponds to the *first* 10 hex character...
///     of the Uniswap V3 pool address, with the vegoId written in the next 8 bits and the tickSpacing written in th...
///     e.g.:
///     [16-bit tickSpacing][8-bits vegoId][first 40 bits of Uniswap V3 pool address] = poolId
///     univ3pool    = 0x8ad599c3A0ff1De082011EFDDc58f1908eb6e6D8
///     vegoId       = 42 (0x2a)
///     tickSpacing  = 60 (0x3c)
///     the returned id is then:
///     poolPattern = 0x0000008ad599c3A0
///     vegoId      = 0x00002a0000000000
///     tickSpacing = 0x003c000000000000 +
///     -----
///     poolId      = 0x003c2a8ad599c3A0
///
/// @param univ3pool The address of the Uniswap V3 pool to get the ID of
/// @param tickSpacing The tick spacing of `univ3pool`
/// @return A uint64 representing a fingerprint of the Uniswap V3 pool address
function _getPoolId(
    address univ3pool,
    int24 tickSpacing,
    uint256 vegoId
) internal pure returns (uint64) {
    unchecked {
        uint64 poolId = uint40(uint160(univ3pool) >>> 112);
        poolId += (uint64(uint8(vegoId)) <<< 40);
        poolId += uint64(uint24(tickSpacing)) <<< 48;
        return poolId;
    }
}
/// @notice Recomputes and decreases `minEnforcedTick` and/or increases `maxEnforcedTick` for a given `poolId` if ce...
/// @dev This function will only have an effect if both conditions are met:
/// - The token supply for one of the tokens was greater than MIN_ENFORCED_TICKFILL_COST at the last `initializeAMMP...
/// - The token supply for one of the tokens meeting the first condition has *decreased* significantly since the las...
/// @dev This function *cannot* decrease the absolute value of either enforced tick, i.e., it can only widen the ran...
/// @dev The purpose of this function is to prevent pools created while a large amount of one of the tokens was flas...
/// @param poolId The poolId on which to expand the enforced tick range
function expandEnforcedTickRange(uint64 poolId) external {
    address univ3pool = s_poolIdToAddress[poolId];
    uint256 vegoId = uint8(poolId >>> 40);
    PoolData dataOld = s_addressToPoolData[univ3pool][vegoId];
    if (!dataOld.initialized()) revert Errors.PoolNotInitialized();
    // tick spacing is stored in the highest 16 bits of the poolId
    int24 tickSpacing = int24(uint24(poolId >>> 48));
    uint128 maxLiquidityPerTick = dataOld.maxLiquidityPerTick();
    int24 minEnforcedTick;
    int24 maxEnforcedTick;
    unchecked {
        minEnforcedTick = int24(

```

```

        Math.min(
            dataOld.minEnforcedTick(),
            -Math.getApproxTickWithMaxAmount(
                Math.max(
                    MIN_ENFORCED_TICKFILL_COST,
                    (IERC20Partial(IUniswapV3Pool(univ3pool).token1()).totalSupply() *
                     SUPPLY_MULTIPLIER_TICKFILL) / 10_000
                ),
                tickSpacing,
                maxLiquidityPerTick
            )
        )
    );
    maxEnforcedTick = int24(
        Math.max(
            dataOld.maxEnforcedTick(),
            Math.getApproxTickWithMaxAmount(
                Math.max(
                    MIN_ENFORCED_TICKFILL_COST,
                    (IERC20Partial(IUniswapV3Pool(univ3pool).token0()).totalSupply() *
                     SUPPLY_MULTIPLIER_TICKFILL) / 10_000
                ),
                tickSpacing,
                maxLiquidityPerTick
            )
        )
    );
}
s_addressToPoolData[univ3pool][vegoId] = PoolDataLibrary.storePoolData(
    maxLiquidityPerTick,
    poolId,
    minEnforcedTick,
    maxEnforcedTick,
    dataOld.initialized()
);
emit EnforcedTicksUpdated(univ3pool, minEnforcedTick, maxEnforcedTick);
}
/*//////////////////////
CALLBACK HANDLERS
////////////////////////////////*/
/// @notice Called after minting liquidity to a position.
/// @dev Pays the pool tokens owed for the minted liquidity from the payer (always the caller).
/// @param amount0Owed The amount of token0 due to the pool for the minted liquidity
/// @param amount1Owed The amount of token1 due to the pool for the minted liquidity
/// @param data Contains the payer address and the pool features required to validate the callback
function uniswapV3MintCallback(
    uint256 amount0Owed,
    uint256 amount1Owed,
    bytes calldata data
) external {
    // Decode the mint callback data
    CallbackLib.CallbackData memory decoded = abi.decode(data, (CallbackLib.CallbackData));
    // Validate caller to ensure we got called from the AMM pool
    CallbackLib.validateCallback(msg.sender, FACTORY, decoded.poolFeatures);
    // Sends the amount0Owed and amount1Owed quantities provided
    if (amount0Owed > 0)
        SafeTransferLib.safeTransferFrom(
            decoded.poolFeatures.token0,
            decoded.payer,
            msg.sender,
            amount0Owed
        );
    if (amount1Owed > 0)
        SafeTransferLib.safeTransferFrom(
            decoded.poolFeatures.token1,
            decoded.payer,
            msg.sender,
            amount1Owed
        );
}
/// @notice Called by the pool after executing a swap during an ITM option mint/burn.
/// @dev Pays the pool tokens owed for the swap from the payer (always the caller).
/// @param amount0Delta The amount of token0 that was sent (negative) or must be received (positive) by the pool by
/// the end of the swap. If positive, the callback must send that amount of token0 to the pool
/// @param amount1Delta The amount of token1 that was sent (negative) or must be received (positive) by the pool by
/// the end of the swap. If positive, the callback must send that amount of token1 to the pool
/// @param data Contains the payer address and the pool features required to validate the callback
function uniswapV3SwapCallback(
    int256 amount0Delta,
    int256 amount1Delta,
    bytes calldata data
) external {
    // Decode the swap callback data, checks that the UniswapV3Pool has the correct address.
    CallbackLib.CallbackData memory decoded = abi.decode(data, (CallbackLib.CallbackData));

```

```

    // Validate caller to ensure we got called from the AMM pool
    CallbackLib.validateCallback(msg.sender, FACTORY, decoded.poolFeatures);
    // Extract the address of the token to be sent (amount0 -> token0, amount1 -> token1)
    address token = amount0Delta > 0
        ? address(decoded.poolFeatures.token0)
        : address(decoded.poolFeatures.token1);
    // Transform the amount to pay to uint256 (take positive one from amount0 and amount1)
    // the pool will always pass one delta with a positive sign and one with a negative sign or zero,
    // so this logic always picks the correct delta to pay
    uint256 amountToPay = amount0Delta > 0 ? uint256(amount0Delta) : uint256(amount1Delta);
    // Pay the required token from the payer to the caller of this contract
    SafeTransferLib.safeTransferFrom(token, decoded.payer, msg.sender, amountToPay);
}
}

/*//////////////////////
                PUBLIC MINT/BURN FUNCTIONS
//////////////////////*/
/// @notice Burn a new position containing up to 4 legs wrapped in a ERC1155 token.
/// @dev Auto-collect all accumulated fees.
/// @param poolKey The abi.encode(address) of the Uniswap V3 Pool
/// @param tokenId The tokenId of the minted position, which encodes information about up to 4 legs
/// @param positionSize The number of contracts minted, expressed in terms of the asset
/// @param tickLimitLow The lower bound of an acceptable open interval for the ending price
/// @param tickLimitHigh The upper bound of an acceptable open interval for the ending price
/// @return An array of LeftRight encoded words containing the amount of token0 and token1 collected as fees for eac...
/// @return The net amount of token0 and token1 moved to/from the Uniswap V3 pool
/// @return currentTick The current tick of the Uniswap pool after the burn
function burnTokenizedPosition(
    bytes calldata poolKey,
    TokenId tokenId,
    uint128 positionSize,
    int24 tickLimitLow,
    int24 tickLimitHigh
) external nonReentrant returns (LeftRightUnsigned[4] memory, LeftRightSigned, int24) {
    _burn(msg.sender, TokenId.unwrap(tokenId), positionSize);
    emit TokenizedPositionBurnt(msg.sender, tokenId, positionSize);
    TokenId flippedTokenId = tokenId.flipToBurnToken();
    bool invertedLimits = tickLimitLow > tickLimitHigh;
    (LeftRightUnsigned[4] memory collected, LeftRightSigned totalMoved) = _createPositionInAMM(
        poolKey,
        invertedLimits,
        positionSize,
        flippedTokenId,
        BURN
    );
    if (invertedLimits) (tickLimitLow, tickLimitHigh) = (tickLimitHigh, tickLimitLow);
    // Get the current tick of the Uniswap pool, check slippage
    int24 currentTick = getCurrentTick(poolKey);
    if ((currentTick >= tickLimitHigh) || (currentTick <= tickLimitLow))
        revert Errors.PriceBoundFail(currentTick);
    return (collected, totalMoved, currentTick);
}

/// @notice Create a new position `tokenId` containing up to 4 legs.
/// @param poolKey The abi.encode(address) of the Uniswap V3 Pool
/// @param tokenId The tokenId of the minted position, which encodes information for up to 4 legs
/// @param positionSize The number of contracts minted, expressed in terms of the asset
/// @param tickLimitLow The lower bound of an acceptable open interval for the ending price
/// @param tickLimitHigh The upper bound of an acceptable open interval for the ending price
/// @return An array of LeftRight encoded words containing the amount of token0 and token1 collected as fees for eac...
/// @return The net amount of token0 and token1 moved to/from the Uniswap V3 pool
/// @return currentTick The current tick of the Uniswap pool after the mint
function mintTokenizedPosition(
    bytes calldata poolKey,
    TokenId tokenId,
    uint128 positionSize,
    int24 tickLimitLow,
    int24 tickLimitHigh
) external nonReentrant returns (LeftRightUnsigned[4] memory, LeftRightSigned, int24) {
    _mint(msg.sender, TokenId.unwrap(tokenId), positionSize);
    emit TokenizedPositionMinted(msg.sender, tokenId, positionSize);
    // verify that the tokenId is correctly formatted and conforms to all enforced constraints
    tokenId.validate();
    bool invertedLimits = tickLimitLow > tickLimitHigh;
    (LeftRightUnsigned[4] memory collected, LeftRightSigned totalMoved) = _createPositionInAMM(
        poolKey,
        invertedLimits,
        positionSize,
        tokenId,
        MINT
    );
    if (invertedLimits) (tickLimitLow, tickLimitHigh) = (tickLimitHigh, tickLimitLow);
    // Get the current tick of the Uniswap pool, check slippage
    int24 currentTick = getCurrentTick(poolKey);
    if ((currentTick >= tickLimitHigh) || (currentTick <= tickLimitLow))
        revert Errors.PriceBoundFail(currentTick);
}

```

```
} return (collected, totalMoved, currentTick);  
/*/////////////////////////////////////////////////////////////  
                TRANSFER HOOK IMPLEMENTATIONS  
////////////////////////////////////////////////////////////*/  
/// @notice All ERC1155 transfers are disabled.  
function safeTransferFrom(  
    address,  
    address,  
    uint256,  
    uint256,  
    bytes calldata  
) public pure override {  
    revert();  
}  
  
/// @notice All ERC1155 transfers are disabled.  
function safeBatchTransferFrom(  
    address,  
    address,  
    uint256[] calldata,  
    uint256[] calldata,  
    bytes calldata  
) public pure override {  
    revert();  
}  
  
/*/////////////////////////////////////////////////////////////  
            AMM INTERACTION AND POSITION UPDATE HELPERS  
////////////////////////////////////////////////////////////*/  
/// @notice Called to perform an ITM swap in the Uniswap pool to resolve any non-tokenType token deltas.  
/// @dev When a position is minted or burnt in-the-money (ITM) we are *not* 100% token0 or 100% token1: we have a mix...  
/// @dev The swapping for ITM options is needed because only one of the tokens are "borrowed" by a user to create th...  
/// This is an ITM situation below (price within the range of the chunk):  
//  
//          AMM      strike  
// liquidity price tick  
// ▲             ■  
//   █           █████▼█████  
//   █         █       █liquidity chunk  
//   █ ███████▲██████████████████  
//   █ █     █        █              █  
//   █ █ :               █  
//   █ █ :               █  
//   █ █ :               █  
//   ███████▲██████████████████ price  
//                   ■  
//          current price  
//          in-the-money: mix of tokens 0 and 1 within the chunk  
//  
// If we take token0 as an example, we deploy it to the AMM pool and *then* swap to get the right mix of token0 an...  
// to be correctly in the money at that strike.  
// It that position is burnt, then we remove a mix of the two tokens and swap one of them so that the user receive...  
/// @param univ3pool The Uniswap pool in which to swap.  
/// @param itmAmounts How much to swap (i.e. how many tokens are ITM)  
/// @param asset The asset of the first leg of the tokenId (determines which token to swap into)  
/// @return totalSwapped The token deltas swapped in the AMM  
function swapInAMM(  
    IUniswapV3Pool univ3pool,  
    LeftRightSigned itmAmounts,  
    uint256 asset  
) internal returns (LeftRightSigned totalSwapped) {  
    bool zeroForOne; // The direction of the swap, true for token0 to token1, false for token1 to token0  
    int256 swapAmount; // The amount of token0 or token1 to swap  
    bytes memory data;  
    IUniswapV3Pool _univ3pool = univ3pool;  
    unchecked {  
        // unpack the in-the-money amounts  
        int128 itm0 = itmAmounts.rightSlot();  
        int128 itm1 = itmAmounts.leftSlot();  
        // construct the swap callback struct  
        data = abi.encode(  
            CallbackLib.CallbackData({  
                poolFeatures: CallbackLib.PoolFeatures({  
                    token0: _univ3pool.token0(),  
                    token1: _univ3pool.token1(),  
                    fee: _univ3pool.fee()  
                }),  
                payer: msg.sender  
            })  
        );  
        ;  
        /// NOTE: upstream users of this function such as the Panoptic Pool should ensure users always compensate for...  
        /// the netting swap is not perfectly accurate, and it is possible for swaps to run out of liquidity, so we d...  
        /// this is simply a convenience feature, and should be treated as such  
        if ((itm0 != 0) && (itm1 != 0)) {  
            // ensure the tokens are swapped from the correct asset.
```

```

        if (asset == 0) {
            zeroForOne = itm0 < 0;
            swapAmount = -itm0;
        } else {
            zeroForOne = itm1 > 0;
            swapAmount = -itm1;
        }
    } else if (itm0 != 0) {
        zeroForOne = itm0 < 0;
        swapAmount = -itm0;
    } else {
        zeroForOne = itm1 > 0;
        swapAmount = -itm1;
    }
    // NOTE: can occur if itm0 and itm1 have the same value
    // in that case, swapping would be pointless so skip
    if (swapAmount == 0) return LeftRightSigned.wrap(0);
    // swap tokens in the Uniswap pool
    // NOTE: this triggers our swap callback function
    (int256 swap0, int256 swap1) = _univ3pool.swap(
        msg.sender,
        zeroForOne,
        swapAmount,
        zeroForOne ? Constants.MIN_POOL_SQRT_RATIO + 1 : Constants.MAX_POOL_SQRT_RATIO - 1,
        data
    );
    // Add amounts swapped to totalSwapped variable
    totalSwapped = LeftRightSigned.wrap(0).addToRightSlot(swap0.toInt128()).addToLeftSlot(
        swap1.toInt128()
    );
}

}

/// @notice Create the position in the AMM defined by `tokenId`.
/// @dev Loops over each leg in the tokenId and calls _createLegInAMM for each, which does the mint/burn in the AMM.
/// @param poolKey The abi.encode(address) of the Uniswap V3 Pool
/// @param invertedLimits Whether the inputted lower limit > upper limit
/// @param positionSize The size of the option position
/// @param tokenId The option position
/// @param isBurn Whether a position is being minted (false) or burned (true)
/// @return collectedByLeg An array of LeftRight encoded words containing the amount of token0 and token1 collected ...
/// @return totalMoved The net amount of funds moved to/from Uniswap
function _createPositionInAMM(
    bytes calldata poolKey,
    bool invertedLimits,
    uint128 positionSize,
    TokenId tokenId,
    bool isBurn
) internal returns (LeftRightUnsigned[4] memory collectedByLeg, LeftRightSigned totalMoved) {
    // get Uni v3 pool from
    IUniswapV3Pool univ3Pool = IUniswapV3Pool(address(abi.decode(poolKey, (address))));
    // Extract univ3pool from the poolId map to Uniswap Pool
    PoolData poolData = s_addressToPoolData[address(univ3Pool)][tokenId.vegoId()];
    // Revert if the pool not been previously initialized or poolId is wrong
    if (
        address(univ3Pool) == address(0) ||
        poolData.poolId() != tokenId.poolId() ||
        !poolData.initialized()
    ) revert Errors.WrongUniswapPool();
    // upper bound on amount of tokens contained across all legs of the position at any given tick
    uint256 amount0;
    uint256 amount1;
    LeftRightSigned itmAmounts;
    for (uint256 leg = 0; leg < tokenId.countLegs(); ) {
        if (tokenId.width(leg) == 0) {
            uint256 isLong = tokenId.isLong(leg);
            LeftRightUnsigned amountsMoved = PanopticMath.getAmountsMoved(
                tokenId,
                positionSize,
                leg,
                true
            );
            int128 signMultiplier = isLong == 0 ? int128(-1) : int128(1);
            {
                uint256 tokenType = tokenId.tokenType(leg);
                int128 itm0 = tokenType == 1
                    ? int128(0)
                    : signMultiplier * int128(amountsMoved.rightSlot());
                int128 itm1 = tokenType == 0
                    ? int128(0)
                    : signMultiplier * int128(amountsMoved.leftSlot());
                itmAmounts = itmAmounts.addToRightSlot(itm0).addToLeftSlot(itm1);
            }
        } else {
            LiquidityChunk liquidityChunk = PanopticMath.getLiquidityChunk(

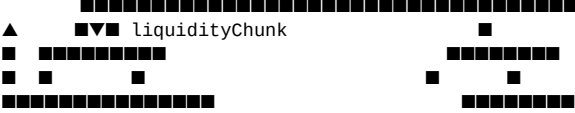
```

```

        tokenId,
        leg,
        positionSize
    );
    // validate tick range for newly minted positions
    if (!isBurn) {
        int24 tickSpacing = tokenId.tickSpacing();
        int24 tickLower = liquidityChunk.tickLower();
        int24 tickUpper = liquidityChunk.tickUpper();
        // Revert if the upper/lower ticks are not multiples of tickSpacing
        // This is an invalid state, and would revert silently later in `univ3Pool.mint`
        // Revert if the tick range extends from the strike outside of the enforced tick range
        if (
            tickLower % tickSpacing != 0 ||
            tickUpper % tickSpacing != 0 ||
            tickLower &lt; poolData.minEnforcedTick() ||
            tickUpper &gt; poolData.maxEnforcedTick()
        ) revert Errors.InvalidTickBound();
    }
    unchecked {
        // increment accumulators of the upper bound on tokens contained across all legs of the position at ...
        amount0 += Math.getAmount0ForLiquidity(liquidityChunk);
        amount1 += Math.getAmount1ForLiquidity(liquidityChunk);
    }
    LeftRightSigned movedLeg;
    (movedLeg, collectedByLeg[leg]) = _createLegInAMM(
        univ3Pool,
        tokenId,
        leg,
        liquidityChunk,
        isBurn
    );
    totalMoved = totalMoved.add(movedLeg);
    // if tokenType is 1, and we transacted some token0: then this leg is ITM
    // if tokenType is 0, and we transacted some token1: then this leg is ITM
    itmAmounts = itmAmounts.add(
        tokenId.tokenType(leg) == 0
            ? LeftRightSigned.wrap(0).addToLeftSlot(movedLeg.leftSlot())
            : LeftRightSigned.wrap(0).addToRightSlot(movedLeg.rightSlot())
    );
}
unchecked {
    ++leg;
}
}
// Ensure upper bound on amount of tokens contained across all legs of the position on any given tick does not e...
// This is the maximum value of the `int128` type we frequently use to hold token amounts, so a given position's...
// fit within that limit at all times.
if (amount0 &gt; uint128(type(int128).max - 4) || amount1 &gt; uint128(type(int128).max - 4))
    revert Errors.PositionTooLarge();
if (invertedLimits) {
    // if the in-the-money amount is not zero (i.e. positions were minted ITM) and the user did provide tick lim...
    if ((LeftRightSigned.unwrap(itmAmounts) != 0)) {
        totalMoved = swapInAMM(univ3Pool, itmAmounts, tokenId.asset(0)).add(totalMoved);
    }
}
}
}
/// @notice Create the position in the AMM for a specific leg in the tokenId.
/// @dev For the leg specified by the _leg input:
/// @dev - mints any new liquidity in the AMM needed (via _mintLiquidity)
/// @dev - burns any new liquidity in the AMM needed (via _burnLiquidity)
/// @dev - tracks all amounts minted and burned
/// @dev To burn a position, the opposing position is "created" through this function,
/// but we need to pass in a flag to indicate that so the removedLiquidity is updated.
/// @param univ3pool The Uniswap pool
/// @param tokenId The option position
/// @param leg The leg index that needs to be modified
/// @param liquidityChunk The liquidity chunk in Uniswap represented by the leg
/// @param isBurn Whether a position is being burned (true) or minted (false)
/// @return moved The net amount of funds moved to/from Uniswap
/// @return collectedSingleLeg LeftRight encoded words containing the amount of token0 and token1 collected as fees
function _createLegInAMM(
    IUniswapV3Pool univ3pool,
    TokenId tokenId,
    uint256 leg,
    LiquidityChunk liquidityChunk,
    bool isBurn
) internal returns (LeftRightSigned moved, LeftRightUnsigned collectedSingleLeg) {
    // unique key to identify the liquidity chunk in this Uniswap pool
    bytes32 positionKey = EfficientHash.efficientKeccak256(
        abi.encodePacked(
            address(univ3pool),
            msg.sender,
            tokenId.tokenType(leg),

```

```

        liquidityChunk.tickLower(),
        liquidityChunk.tickUpper()
    )
};
// update our internal bookkeeping of how much liquidity we have deployed in the AMM
// for example: if this leg is short, we add liquidity to the amm, make sure to add that to our tracking
uint128 updatedLiquidity;
uint256 isLong = tokenId.isLong(leg);
LeftRightUnsigned currentLiquidity = s_accountLiquidity[positionKey];
{
    // s_accountLiquidity is a LeftRight. The right slot represents the liquidity currently sold (added) in the ...
    // the left slot represents the amount of liquidity currently bought (removed) that has been removed from th...
    // the reason why it is called "removedLiquidity" is because long options are created by removing - ie. shor...
    uint128 startingLiquidity = currentLiquidity.rightSlot();
    uint128 removedLiquidity = currentLiquidity.leftSlot();
    uint128 chunkLiquidity = liquidityChunk.liquidity();
    // 0-liquidity interactions are asymmetrical in Uniswap (burning 0 liquidity is permitted and functions as a...
    // thus, we prohibit all 0-liquidity chunks to prevent users from creating positions that cannot be closed
    if (chunkLiquidity == 0) revert Errors.ChunkHasZeroLiquidity();
    if (isLong == 0) {
        // selling/short: so move from msg.sender *to* uniswap
        // we're minting more liquidity in uniswap: so add the incoming liquidity chunk to the existing liquidit...
        updatedLiquidity = startingLiquidity + chunkLiquidity;
        /// @dev If the isLong flag is 0=short but the position was burnt, then this is closing a long position
        /// @dev so the amount of removed liquidity should decrease.
        if (isBurn) {
            removedLiquidity -= chunkLiquidity;
        }
    } else {
        // the _leg is long (buying: moving *from* uniswap to msg.sender)
        // so we seek to move the incoming liquidity chunk *out* of uniswap - but was there sufficient liquidity...
        // in the first place?
        if (startingLiquidity < chunkLiquidity) {
            // the amount we want to move (liquidityChunk.legLiquidity()) out of uniswap is greater than
            // what the account that owns the liquidity in uniswap has (startingLiquidity)
            // we must ensure that an account can only move its own liquidity out of uniswap
            // so we revert in this case
            revert Errors.NotEnoughLiquidityInChunk();
        } else {
            // startingLiquidity is >= chunkLiquidity, so no possible underflow
            unchecked {
                // we want to move less than what already sits in uniswap, no problem:
                updatedLiquidity = startingLiquidity - chunkLiquidity;
            }
        }
        /// @dev If the isLong flag is 1=long and the position is minted, then this is opening a long position
        /// @dev so the amount of removed liquidity should increase.
        if (!isBurn) {
            removedLiquidity += chunkLiquidity;
        }
    }
}
// update the starting liquidity for this position for next time around
s_accountLiquidity[positionKey] = LeftRightUnsigned
    .wrap(updatedLiquidity)
    .addToLeftSlot(removedLiquidity);
}
// track how much liquidity we need to collect from uniswap
// add the fees that accumulated in uniswap within the liquidityChunk:
/* if the position is NOT long (selling a put or a call), then _mintLiquidity to move liquidity
    from the msg.sender to the Uniswap V3 pool:
    Selling(isLong=0): Mint chunk of liquidity in Uniswap (defined by upper tick, lower tick, and amount)
    
    else: the position is long (buying a put or a call), then _burnLiquidity to remove liquidity from Uniswap V3
    Buying(isLong=1): Burn in Uniswap
    
    */
moved = isLong == 0
    ? _mintLiquidity(liquidityChunk, univ3pool)
    : _burnLiquidity(liquidityChunk, univ3pool);
// if there was liquidity at that tick before the transaction, collect any accumulated fees
if (currentLiquidity.rightSlot() > 0) {
    uint256 vegoid = tokenId.vegoid();
    collectedSingleLeg = _collectAndWritePositionData(
        liquidityChunk,

```



```

        univ3pool,
        currentLiquidity,
        positionKey,
        moved,
        isLong,
        vegoid
    );
}
// position has been touched, update s_accountFeesBase with the latest values from the pool.positions
// round up the stored feesbase to minimize Δfeesbase when we next calculate it
s_accountFeesBase[positionKey] = _getFeesBase(
    univ3pool,
    updatedLiquidity,
    liquidityChunk,
    true
);
}
/// @notice Updates the premium accumulators for a chunk with the latest collected tokens.
/// @param positionKey A key representing a liquidity chunk/range in Uniswap
/// @param currentLiquidity The total amount of liquidity in the AMM for the specified chunk
/// @param collectedAmounts The amount of tokens (token0 and token1) collected from Uniswap
function _updateStoredPremia(
    bytes32 positionKey,
    LeftRightUnsigned currentLiquidity,
    LeftRightUnsigned collectedAmounts,
    uint256 vegoid
) private {
    (
        LeftRightUnsigned deltaPremiumOwed,
        LeftRightUnsigned deltaPremiumGross
    ) = _getPremiaDeltas(currentLiquidity, collectedAmounts, vegoid);
    // add deltas to accumulators and freeze both accumulators (for a token) if one of them overflows
    // (i.e if only token0 (right slot) of the owed premium overflows, then stop accumulating both token0 owed prem...
    // this prevents situations where the owed premium gets out of sync with the gross premium due to one of them ov...
    (s_accountPremiumOwed[positionKey], s_accountPremiumGross[positionKey]) = LeftRightLibrary
        .addCapped(
            s_accountPremiumOwed[positionKey],
            deltaPremiumOwed,
            s_accountPremiumGross[positionKey],
            deltaPremiumGross
        );
}
/// @notice Compute an up-to-date feeGrowth value without a poke.
/// @dev Stored fees base is rounded up and the current fees base is rounded down to minimize the amount of fees col...
/// @param univ3pool The Uniswap pool
/// @param liquidity The total amount of liquidity in the AMM for the specific position
/// @param liquidityChunk The liquidity chunk in Uniswap to compute the feesBase for
/// @param roundUp If true, round up the feesBase, otherwise round down
function _getFeesBase(
    IUniswapV3Pool univ3pool,
    uint128 liquidity,
    LiquidityChunk liquidityChunk,
    bool roundUp
) private view returns (LeftRightSigned feesBase) {
    // read the latest feeGrowth directly from the Uniswap pool
    (, uint256 feeGrowthInside0LastX128, uint256 feeGrowthInside1LastX128, , ) = univ3pool
        .positions(
            EfficientHash.efficientKeccak256(
                abi.encodePacked(
                    address(this),
                    liquidityChunk.tickLower(),
                    liquidityChunk.tickUpper()
                )
            )
        );
    // (feeGrowth * liquidity) / 2 ** 128
    // here we're converting the value to an int128 even though all values (feeGrowth, liquidity, Q128) are strictly...
    // That's because of the way feeGrowthInside works in Uniswap V3, where it can underflow when stored for the fir...
    // This is not a problem in Uniswap V3 because the fees are always calculated by taking the difference of the fe...
    // so that the net different is always positive.
    // So by using int128 instead of uint128, we remove the need to handle extremely large underflows and simply all...
    feesBase = roundUp
        ? LeftRightSigned
            .wrap(0)
            .addToRightSlot(
                int128(int256(Math.mulDiv128RoundingUp(feeGrowthInside0LastX128, liquidity)))
            )
            .addToLeftSlot(
                int128(int256(Math.mulDiv128RoundingUp(feeGrowthInside1LastX128, liquidity)))
            )
        : LeftRightSigned
            .wrap(0)
            .addToRightSlot(int128(int256(Math.mulDiv128(feeGrowthInside0LastX128, liquidity))))
            .addToLeftSlot(int128(int256(Math.mulDiv128(feeGrowthInside1LastX128, liquidity))));
}

```

```

}
/// @notice Mint a chunk of liquidity (`liquidityChunk`) in the Uniswap V3 pool; return the amount moved.
/// @param liquidityChunk The liquidity chunk in Uniswap to mint
/// @param univ3pool The Uniswap V3 pool to mint liquidity in/to
/// @return movedAmounts How many tokens were moved from `msg.sender` to Uniswap
function _mintLiquidity(
    LiquidityChunk liquidityChunk,
    IUniswapV3Pool univ3pool
) internal returns (LeftRightSigned movedAmounts) {
    // build callback data
    bytes memory mintdata = abi.encode(
        CallbackLib.CallbackData({
            poolFeatures: CallbackLib.PoolFeatures({
                token0: univ3pool.token0(),
                token1: univ3pool.token1(),
                fee: univ3pool.fee()
            }),
            payer: msg.sender
        })
    );
    // mint the required amount in the Uniswap pool
    // this triggers the uniswap mint callback function
    (uint256 amount0, uint256 amount1) = univ3pool.mint(
        address(this),
        liquidityChunk.tickLower(),
        liquidityChunk.tickUpper(),
        liquidityChunk.liquidity(),
        mintdata
    );
    // amount0 The amount of token0 that was paid to mint the given amount of liquidity
    // amount1 The amount of token1 that was paid to mint the given amount of liquidity
    // no need to safecast to int from uint here as the max position size is int128
    movedAmounts = LeftRightSigned
        .wrap(0)
        .addToLeftSlot(int128(int256(amount0)))
        .addToRightSlot(int128(int256(amount1)));
}
/// @notice Burn a chunk of liquidity (`liquidityChunk`) in the Uniswap V3 pool and send to msg.sender; return the a...
/// @param liquidityChunk The liquidity chunk in Uniswap to burn
/// @param univ3pool The Uniswap V3 pool to burn liquidity in/from
/// @return movedAmounts How many tokens were moved from Uniswap to `msg.sender`
function _burnLiquidity(
    LiquidityChunk liquidityChunk,
    IUniswapV3Pool univ3pool
) internal returns (LeftRightSigned movedAmounts) {
    // burn that option's liquidity in the Uniswap Pool.
    // This will send the underlying tokens back to the Panoptic Pool (msg.sender)
    (uint256 amount0, uint256 amount1) = univ3pool.burn(
        liquidityChunk.tickLower(),
        liquidityChunk.tickUpper(),
        liquidityChunk.liquidity()
    );
    // amount0 The amount of token0 that was sent back to the Panoptic Pool
    // amount1 The amount of token1 that was sent back to the Panoptic Pool
    // no need to safecast to int from uint here as the max position size is int128
    // decrement the amountsOut with burnt amounts. amountsOut = notional value of tokens moved
    unchecked {
        movedAmounts = LeftRightSigned
            .wrap(0)
            .addToLeftSlot(-int128(int256(amount0)))
            .addToRightSlot(-int128(int256(amount1)));
    }
}
/// @notice Helper to collect amounts between msg.sender and Uniswap and also to update the Uniswap fees collected t...
/// @param liquidityChunk The liquidity chunk in Uniswap to collect from
/// @param univ3pool The Uniswap pool where the position is deployed
/// @param currentLiquidity The existing liquidity msg.sender owns in the AMM for this chunk before the SFPM was cal...
/// @param positionKey The unique key to identify the liquidity chunk/tokenType pairing in this Uniswap pool
/// @param movedInLeg How many tokens have been moved between msg.sender and Uniswap before this function call
/// @param isLong Whether the leg in question is long (=1) or short (=0)
/// @return collectedChunk The amount of tokens collected from Uniswap
function _collectAndWritePositionData(
    LiquidityChunk liquidityChunk,
    IUniswapV3Pool univ3pool,
    LeftRightUnsigned currentLiquidity,
    bytes32 positionKey,
    LeftRightSigned movedInLeg,
    uint256 isLong,
    uint256 vegoid
) internal returns (LeftRightUnsigned collectedChunk) {
    LeftRightUnsigned amountToCollect;
    {
        uint128 startingLiquidity = currentLiquidity.rightSlot();
        // round down current fees base to minimize Δfeesbase
    }
}

```

```

        // If the current feesBase is close or identical to the stored one, the amountToCollect can be negative.
        // This is because the stored feesBase is rounded up, and the current feesBase is rounded down.
        // When this is the case, we want to behave as if there are 0 fees, so we just rectify the values.
        amountToCollect = _getFeesBase(univ3pool, startingLiquidity, liquidityChunk, false)
            .subRect(s_accountFeesBase[positionKey]);
    }
    if (isLong == 1) {
        // movedInLeg deltas are always represented as negative during liquidity burns (for long positions), so the ...
        amountToCollect = LeftRightUnsigned.wrap(
            uint256(
                LeftRightSigned.unwrap(
                    LeftRightSigned.wrap(int256(LeftRightUnsigned.unwrap(amountToCollect))).sub(
                        movedInLeg
                    )
                )
            )
        );
    }
    if (LeftRightUnsigned.unwrap(amountToCollect) != 0) {
        // Collects tokens owed to a liquidity chunk
        (uint128 receivedAmount0, uint128 receivedAmount1) = univ3pool.collect(
            msg.sender,
            liquidityChunk.tickLower(),
            liquidityChunk.tickUpper(),
            amountToCollect.rightSlot(),
            amountToCollect.leftSlot()
        );
        // moved will be negative if the leg was long (funds left the caller, don't count it in collected fees)
        uint128 collected0;
        uint128 collected1;
        unchecked {
            collected0 = movedInLeg.rightSlot() < 0
                ? receivedAmount0 - uint128(-movedInLeg.rightSlot())
                : receivedAmount0;
            collected1 = movedInLeg.leftSlot() < 0
                ? receivedAmount1 - uint128(-movedInLeg.leftSlot())
                : receivedAmount1;
        }
        // CollectedOut is the amount of fees accumulated+collected (received - burnt)
        // That's because receivedAmount contains the burnt tokens and whatever amount of fees collected
        collectedChunk = LeftRightUnsigned.wrap(collected0).addToLeftSlot(collected1);
        // record the collected amounts in the s_accountPremiumOwed and s_accountPremiumGross accumulators
        _updateStoredPremia(positionKey, currentLiquidity, collectedChunk, vegoid);
    }
}
/// @notice Compute deltas for Owed/Gross premium given quantities of tokens collected from Uniswap.
/// @dev Returned accumulators are capped at the max value (2^128 - 1) for each token if they overflow.
/// @param currentLiquidity NetLiquidity (right) and removedLiquidity (left) at the start of the transaction
/// @param collectedAmounts Total amount of tokens (token0 and token1) collected from Uniswap
/// @return deltaPremiumOwed The extra premium (per liquidity X64) to be added to the owed accumulator for token0 (r...
/// @return deltaPremiumGross The extra premium (per liquidity X64) to be added to the gross accumulator for token0 ...
function _getPremiaDeltas(
    LeftRightUnsigned currentLiquidity,
    LeftRightUnsigned collectedAmounts,
    uint256 vegoid
)
private
pure
returns (LeftRightUnsigned deltaPremiumOwed, LeftRightUnsigned deltaPremiumGross)
{
    // extract liquidity values
    uint256 removedLiquidity = currentLiquidity.leftSlot();
    uint256 netLiquidity = currentLiquidity.rightSlot();
    // premia spread equations are graphed and documented here: https://www.desmos.com/calculator/mdeqob2m04
    // explains how we get from the premium per liquidity (calculated here) to the total premia collected and the mu...
    // as well as how the value of VEGOID affects the premia
    // note that the "base" premium is just a common factor shared between the owed (long) and gross (short)
    // premia, and is only separated to simplify the calculation
    // (the graphed equations include this factor without separating it)
    unchecked {
        uint256 totalLiquidity = netLiquidity + removedLiquidity;
        uint256 premium0X64_base;
        uint256 premium1X64_base;
        {
            uint128 collected0 = collectedAmounts.rightSlot();
            uint128 collected1 = collectedAmounts.leftSlot();
            // compute the base premium as collected * total / net^2 (from Eqn 3)
            premium0X64_base = Math.mulDiv(
                collected0,
                totalLiquidity * 2 ** 64,
                netLiquidity ** 2
            );
            premium1X64_base = Math.mulDiv(
                collected1,

```

[illegible]

```

    address owner,
    uint256 tokenType,
    int24 tickLower,
    int24 tickUpper,
    int24 atTick,
    uint256 isLong,
    uint256 vegoid
) external view returns (uint128, uint128) {
    IUniswapV3Pool univ3pool = IUniswapV3Pool(abi.decode(poolKey, (address)));
    bytes32 positionKey = EfficientHash.efficientKeccak256(
        abi.encodePacked(univ3pool, owner, tokenType, tickLower, tickUpper)
    );
    LeftRightUnsigned acctPremia;
    LeftRightUnsigned accountLiquidities = s_accountLiquidities[positionKey];
    uint128 netLiquidity = accountLiquidities.rightSlot();
    // Compute the premium up to the current block (ie. after last touch until now). Do not proceed if `atTick == (t...
    if (atTick < type(int24).max && netLiquidity != 0) {
        // unique key to identify the liquidity chunk in this Uniswap pool
        LeftRightUnsigned amountToCollect;
        {
            IUniswapV3Pool _univ3pool = IUniswapV3Pool(univ3pool);
            int24 _tickLower = tickLower;
            int24 _tickUpper = tickUpper;
            int24 _atTick = atTick;
            // how much fees have been accumulated within the liquidity chunk since last time we updated this chunk?
            // Compute (currentFeesGrowth - oldFeesGrowth), the amount to collect
            // currentFeesGrowth (calculated from FeesCalc.calculateAMMSwapFeesLiquidityChunk) is (ammFeesCollectedP...
            // oldFeesGrowth is the last stored update of fee growth within the position range in the past (feeGrowt...
            LeftRightSigned feesBase = FeesCalc.calculateAMMSwapFees(
                _univ3pool,
                _atTick,
                _tickLower,
                _tickUpper,
                netLiquidity
            );
            // If the current feesBase is close or identical to the stored one, the amountToCollect can be negative.
            // This is because the stored feesBase is rounded up, and the current feesBase is rounded down.
            // When this is the case, we want to behave as if there are 0 fees, so we just rectify the values.
            // Guaranteed to be positive, so swap to unsigned type
            amountToCollect = feesBase.subRect(s_accountFeesBase[positionKey]);
        }
        (LeftRightUnsigned premiumOwed, LeftRightUnsigned premiumGross) = _getPremiaDeltas(
            accountLiquidities,
            amountToCollect,
            vegoid
        );
        // add deltas to accumulators and freeze both accumulators (for a token) if one of them overflows
        // (i.e if only token0 (right slot) of the owed premium overflows, then stop accumulating both token0 owed p...
        // this prevents situations where the owed premium gets out of sync with the gross premium due to one of the...
        (premiumOwed, premiumGross) = LeftRightLibrary.addCapped(
            s_accountPremiumOwed[positionKey],
            premiumOwed,
            s_accountPremiumGross[positionKey],
            premiumGross
        );
        acctPremia = isLong == 1 ? premiumOwed : premiumGross;
    } else {
        // Extract the account liquidity for a given Uniswap pool, owner, token type, and ticks
        acctPremia = isLong == 1
            ? s_accountPremiumOwed[positionKey]
            : s_accountPremiumGross[positionKey];
    }
    return (acctPremia.rightSlot(), acctPremia.leftSlot());
}

/// @notice Return the feesBase associated with a given liquidity chunk.
/// @param poolKey The abi.encode(address) of the Uniswap V3 Pool
/// @param owner The address of the account that is queried
/// @param tokenType The tokenType of the position (the token it started as)
/// @param tickLower The lower end of the tick range for the position
/// @param tickUpper The upper end of the tick range for the position
/// @return feesBase0 The feesBase of the position for token0
/// @return feesBase1 The feesBase of the position for token1
function getAccountFeesBase(
    bytes calldata poolKey,
    address owner,
    uint256 tokenType,
    int24 tickLower,
    int24 tickUpper
) external view returns (int128 feesBase0, int128 feesBase1) {
    IUniswapV3Pool univ3pool = IUniswapV3Pool(abi.decode(poolKey, (address)));
    // Get accumulated fees for token0 (rightSlot) and token1 (leftSlot)
    LeftRightSigned feesBase = s_accountFeesBase[
        EfficientHash.efficientKeccak256(
            abi.encodePacked(univ3pool, owner, tokenType, tickLower, tickUpper)
        )
    ];
    return (feesBase.leftSlot(), feesBase.rightSlot());
}

```


[illegible]

```

/// @notice The canonical Uniswap V4 Pool Manager address.
IPoolManager internal immutable POOL_MANAGER_V4;
/// @notice The approximate minimum amount of tokens it should require to fill `maxLiquidityPerTick` at the minimum ...
uint256 internal immutable MIN_ENFORCED_TICKFILL_COST;
/// @notice The approximate minimum amount of tokens it should require to fill `maxLiquidityPerTick` at the minimum ...
uint256 internal immutable NATIVE_ENFORCED_TICKFILL_COST;
/// @notice The multiplier, in basis points, to apply to the token supply and set as the minimum enforced tick fill ...
uint256 internal immutable SUPPLY_MULTIPLIER_TICKFILL;
//*****
STORAGE
//*****
/// @notice Retrieve the SFPM PoolIdData struct associated with a given Uniswap V4 poolId.
mapping(PoolId idV4 => mapping(uint256 vegoid => PoolData poolData)) internal s_V4toSFPMIdData;
/// @notice Retrieve the Uniswap V4 pool key corresponding to a given poolId.
mapping(uint64 poolId => PoolKey key) internal s_poolIdToKey;
/*
We're tracking the amount of net and removed liquidity for the specific region:
    net amount
    received minted
    ▲ for isLong=0
    ■■■■■■-T      amount
    ■ moved out    actual amount
    ■ due isLong=1 in the UniswapV4 Pool
    ■ mints
    ■■■■■■-(T-R)
    ■■■■■■-R
    ■■■■■■
    total=T      removed=R      net=(T-R)
*
* removed liquidity r      net liquidity N=(T-R)
* |&lt;----- 128 bits -----&gt;|&lt;----- 128 bits -----&gt;|
* |&lt;----- 256 bits -----&gt;|
*/
/// @notice Retrieve the current liquidity state in a chunk for a given user.
/// @dev `removedAndNetLiquidity` is a LeftRight. The right slot represents the liquidity currently sold (added) in ...
/// the left slot represents the amount of liquidity currently bought (removed) that has been removed from the AMM - ...
/// The reason why it is called "removedLiquidity" is because long options are created by removed liquidity - ie. sho...
mapping(bytes32 positionKey => LeftRightUnsigned removedAndNetLiquidity)
internal s_accountLiquidity;
/*
Any liquidity that has been deposited in the AMM using the SFPM will collect fees over
time, we call this the gross premia. If that liquidity has been removed, we also need to
keep track of the amount of fees that *would have been collected*, we call this the owed
premia. The gross and owed premia are tracked per unit of liquidity by the
s_accountPremiumGross and s_accountPremiumOwed accumulators.
Here is how we can use the accumulators to compute the Gross, Net, and Owed fees collected
by any position.
Let's say Charlie the smart contract deposited T into the AMM and later removed R from that
same tick using a tokenId with a isLong=1 parameter. Because the netLiquidity is only (T-R),
the AMM will collect fees equal to:
    net_feesCollectedX128 = feeGrowthX128 * (T - R)
                          = feeGrowthX128 * N
where N = netLiquidity = T-R. Had that liquidity never been removed, we want the gross
premia to be given by:
    gross_feesCollectedX128 = feeGrowthX128 * T
So we must keep track of fees for the removed liquidity R so that the long premia exactly
compensates for the fees that would have been collected from the initial liquidity.
In addition to tracking, we also want to track those fees plus a small spread. Specifically,
we want:
    gross_feesCollectedX128 = net_feesCollectedX128 + owed_feesCollectedX128
where
    owed_feesCollectedX128 = feeGrowthX128 * R * (1 + spread) (Eqn 1)
A very opinionated definition for the spread is:
    spread = v*(liquidity removed from that strike)/(netLiquidity remaining at that strike)
           = v*R/N
For an arbitrary parameter 0 &lt;= v &lt;= 1 (v = 1/VEG0ID). This way, the gross_feesCollectedX128 will be given...
    gross_feesCollectedX128 = feeGrowthX128 * N + feeGrowthX128*R*(1 + v*R/N)
                          = feeGrowthX128 * T + feesGrowthX128*v*R^2/N
                          = feeGrowthX128 * T * (1 + v*R^2/(N*T)) (Eqn 2)
The s_accountPremiumOwed accumulator tracks the feeGrowthX128 * R * (1 + spread) term
per unit of removed liquidity R every time the position touched:
    s_accountPremiumOwed += feeGrowthX128 * R * (1 + v*R/N) / R
                       += feeGrowthX128 * (T - R + v*R)/N
                       += feeGrowthX128 * T/N * (1 - R/T + v*R/T)
Note that the value of feeGrowthX128 can be extracted from the amount of fees collected by
the smart contract since the amount of feesCollected is related to feeGrowthX128 according
to:
    feesCollected = feesGrowthX128 * (T-R)
So that we get:
    feesGrowthX128 = feesCollected/N
And the accumulator is computed from the amount of collected fees according to:
    s_accountPremiumOwed += feesCollected * T/N^2 * (1 - R/T + v*R/T) (Eqn 3)
So, the amount of owed premia for a position of size r minted at time t1 and burnt at
time t2 is:
    owedPremia(t1, t2) = (s_accountPremiumOwed_t2-s_accountPremiumOwed_t1) * r

```


$$\begin{aligned}
&= \Delta \text{feesGrowthX128} * r * T/N * (1 - R/T + v*R/T) \\
&= \Delta \text{feesGrowthX128} * r * (T - R + v*R)/N \\
&= \Delta \text{feesGrowthX128} * r * (N + v*R)/N \\
&= \Delta \text{feesGrowthX128} * r * (1 + v*R/N) \quad (\text{same as Eqn 1})
\end{aligned}$$

This way, the amount of premia owed for a position will match Eqn 1 exactly.

Similarly, the amount of gross fees for the total liquidity is tracked in a similar manner by the `s_accountPremiumGross` accumulator.

However, since we require that Eqn 2 holds up-- ie. the gross fees collected should be equal to the net fees collected plus the over fees plus the small spread, the expression for the `s_accountPremiumGross` accumulator has to be given by (you'll see why in a minute):

$$s_accountPremiumGross += feesCollected * T/N^2 * (1 - R/T + v*R^2/T^2) \quad (\text{Eqn 4})$$

This expression can be used to calculate the fees collected by a position of size `t` between times `t1` and `t2` according to:

$$\begin{aligned}
grossPremia(t1, t2) &= \Delta(s_accountPremiumGross) * t \\
&= \Delta \text{feeGrowthX128} * t * T/N * (1 - R/T + v*R^2/T^2) \\
&= \Delta \text{feeGrowthX128} * t * (T - R + v*R^2/T) / N \\
&= \Delta \text{feeGrowthX128} * t * (N + v*R^2/T) / N \\
&= \Delta \text{feeGrowthX128} * t * (1 + v*R^2/(N*T)) \quad (\text{same as Eqn 2})
\end{aligned}$$

where the last expression matches Eqn 2 exactly.

In summary, the `s_accountPremium` accumulators allow smart contracts that need to handle long+short liquidity to guarantee that liquidity deposited always receives the correct premia, whether that liquidity has been removed from the AMM or not.

Note that the expression for the spread is extremely opinionated, and may not fit the specific risk management profile of every smart contract. And simply setting the `v` parameter to zero would get rid of the "spread logic".

```

*/
/// @notice Per-liquidity accumulator for the premium owed by buyers on a given chunk, tokenType and account.
mapping(bytes32 positionKey => LeftRightUnsigned accountPremium) private s_accountPremiumOwed;
/// @notice Per-liquidity accumulator for the premium earned by sellers on a given chunk, tokenType and account.
mapping(bytes32 positionKey => LeftRightUnsigned accountPremium) private s_accountPremiumGross;
/////////////////////////////////////////////////////////////
INITIALIZATION
/////////////////////////////////////////////////////////////
/// @notice Set the canonical Uniswap V4 pool manager address and tick fill parameters.
/// @param poolManager The canonical Uniswap V4 pool manager address
/// @param _minEnforcedTickFillCost The minimum amount of tokens it should require to fill `maxLiquidityPerTick` at ...
/// @param _nativeEnforcedTickFillCost The minimum amount of tokens it should require to fill `maxLiquidityPerTick` ...
/// @param _supplyMultiplierTickFill The multiplier, in basis points, to apply to the token supply and set as the mi...
constructor(
    IPoolManager poolManager,
    uint256 _minEnforcedTickFillCost,
    uint256 _nativeEnforcedTickFillCost,
    uint256 _supplyMultiplierTickFill
) {
    POOL_MANAGER_V4 = poolManager;
    MIN_ENFORCED_TICKFILL_COST = _minEnforcedTickFillCost;
    NATIVE_ENFORCED_TICKFILL_COST = _nativeEnforcedTickFillCost;
    SUPPLY_MULTIPLIER_TICKFILL = _supplyMultiplierTickFill;
}
/// @notice Initialize a Uniswap V4 pool in the SFPM.
/// @dev Revert if already initialized.
/// @param key An identifying key for a Uniswap V4 pool
function initializeAMMPool(
    PoolKey calldata key,
    uint8 vegoid
) external returns (uint64 poolId) {
    PoolId idV4 = key.toId();
    if (V4StateReader.getSqrtPriceX96(POOL_MANAGER_V4, idV4) == 0)
        revert Errors.PoolNotInitialized();
    // return if the pool has already been initialized in SFPM
    // pools can be initialized from the Panoptic Factory or by calling initializeAMMPool directly, so reverting
    // could prevent a PanopticPool from being deployed on a previously initialized but otherwise valid pool
    if (s_V4toSFPMIdData[idV4][vegoid].initialized())
        return s_V4toSFPMIdData[idV4][vegoid].poolId();
    // The base poolId is composed as follows:
    // [tickSpacing][pool pattern]
    // [16 bit tickSpacing][most significant 48 bits of the V4 poolId]
    poolId = _getPoolId(idV4, key.tickSpacing, vegoid);
    // There are 1,099,511,627,776 possible pool patterns.
    // A modern GPU can generate a collision in such a space relatively quickly,
    // so if a collision is detected increment the pool pattern until a unique poolId is found
    while (s_poolIdToKey[poolId].tickSpacing != 0) {
        poolId = PanopticMath.incrementPoolPattern(poolId);
    }
    uint128 maxLiquidityPerTick = Math.getMaxLiquidityPerTick(key.tickSpacing);
    int24 minEnforcedTick;
    int24 maxEnforcedTick;
    unchecked {
        minEnforcedTick = int24(
            -Math.getApproxTickWithMaxAmount(
                Math.max(
                    MIN_ENFORCED_TICKFILL_COST,
                    (IERC20Partial(Currency.unwrap(key.currency1)).totalSupply() *
                        SUPPLY_MULTIPLIER_TICKFILL) / 10_000
                )
            )
        );
    }
}

```

```

        ),
        key.tickSpacing,
        maxLiquidityPerTick
    )
);
maxEnforcedTick = int24(
    Math.getApproxTickWithMaxAmount(
        Currency.unwrap(key.currency0) == address(0)
        ? NATIVE_ENFORCED_TICKFILL_COST
        : Math.max(
            MIN_ENFORCED_TICKFILL_COST,
            (IERC20Partial(Currency.unwrap(key.currency0)).totalSupply() *
            SUPPLY_MULTIPLIER_TICKFILL) / 10_000
        ),
        key.tickSpacing,
        maxLiquidityPerTick
    )
);
}
s_V4toSFPMIdData[idV4][vegoId] = PoolDataLibrary.storePoolData(
    maxLiquidityPerTick,
    poolId,
    minEnforcedTick,
    maxEnforcedTick,
    true
);
s_poolIdToKey[poolId] = key;
emit PoolInitialized(idV4, poolId, minEnforcedTick, maxEnforcedTick);
}
/// @notice Given a 256-bit Uniswap V4 pool ID (hash) and the corresponding `tickSpacing`, return its 64-bit ID as u...
/// Example:
/// [16-bit tickSpacing][8-bits vegoid][last 40 bits of Uniswap V4 pool ID] = poolId
/// e.g.:
/// idV4      = 0x9c33e1937fe23c3ff82d7725f2bb5af696db1c89a9b8cae141cb0e986847638a
/// vegoid    = 42 (0x2a)
/// tickSpacing = 60 (0x3c)
/// the returned id is then:
/// poolPattern = 0x000000986847638a
/// vegoid      = 0x00002a0000000000
/// tickSpacing = 0x003c000000000000 +
/// -----
/// poolId      = 0x0032a986847638a
/// @param idV4 The 256-bit Uniswap V4 pool ID
/// @param tickSpacing The tick spacing of the Uniswap V4 pool identified by `idV4`
/// @param vegoid The vegoid of the SFPM, must be 8 bits
/// @return A fingerprint representing the Uniswap V4 pool
function _getPoolId(
    PoolId idV4,
    int24 tickSpacing,
    uint256 vegoid
) internal pure returns (uint64) {
    unchecked {
        return
            uint40(uint256(PoolId.unwrap(idV4))) +
            (uint64(uint8(vegoid)) <<< 40) +
            (uint64(uint24(tickSpacing)) <<< 48);
    }
}
/// @notice Recomputes and decreases `minEnforcedTick` and/or increases `maxEnforcedTick` for a given V4 pool `key` ...
/// @dev This function will only have an effect if both conditions are met:
/// - The token supply for one of the (non-native) tokens was greater than MIN_ENFORCED_TICKFILL_COST at the last `i...
/// - The token supply for one of the tokens meeting the first condition has *decreased* significantly since the las...
/// @dev This function *cannot* decrease the absolute value of either enforced tick, i.e., it can only widen the ran...
/// @dev The purpose of this function is to prevent pools created while a large amount of one of the tokens was flas...
/// @param poolId The poolId on which to expand the enforced tick range
function expandEnforcedTickRange(uint64 poolId) external {
    PoolKey memory key = s_poolIdToKey[poolId];
    PoolId idV4 = key.toId();
    uint256 vegoid = uint8(poolId >>> 40);
    PoolData dataOld = s_V4toSFPMIdData[idV4][vegoId];
    if (!dataOld.initialized()) revert Errors.PoolNotInitialized();
    // tick spacing is stored in the highest 16 bits of the poolId
    int24 tickSpacing = int24(uint24(dataOld.poolId() >>> 48));
    uint128 maxLiquidityPerTick = dataOld.maxLiquidityPerTick();
    int24 minEnforcedTick;
    int24 maxEnforcedTick;
    unchecked {
        minEnforcedTick = int24(
            Math.min(
                dataOld.minEnforcedTick(),
                -Math.getApproxTickWithMaxAmount(
                    Math.max(
                        MIN_ENFORCED_TICKFILL_COST,
                        (IERC20Partial(Currency.unwrap(key.currency1)).totalSupply() *

```

```

        SUPPLY_MULTIPLIER_TICKFILL) / 10_000
    ),
    tickSpacing,
    maxLiquidityPerTick
)
)
);
maxEnforcedTick = int24(
    Math.max(
        dataOld.maxEnforcedTick(),
        Math.getApproxTickWithMaxAmount(
            Currency.unwrap(key.currency0) == address(0)
            ? NATIVE_ENFORCED_TICKFILL_COST
            : Math.max(
                MIN_ENFORCED_TICKFILL_COST,
                (IERC20Partial(Currency.unwrap(key.currency0)).totalSupply() *
                SUPPLY_MULTIPLIER_TICKFILL) / 10_000
            ),
        tickSpacing,
        maxLiquidityPerTick
    )
);
}
s_V4toSFPMDData[idV4][vegoId] = PoolDataLibrary.storePoolData(
    maxLiquidityPerTick,
    dataOld.poolId(),
    minEnforcedTick,
    maxEnforcedTick,
    dataOld.initialized()
);
emit EnforcedTicksUpdated(idV4, minEnforcedTick, maxEnforcedTick);
}
/*////////////////////
                UNISWAP V4 LOCK CALLBACK
////////////////////*/
/// @notice Executes the corresponding operations and state updates required to mint `tokenId` of `positionSize` in ...
/// @param key The Uniswap V4 pool key in which to mint `tokenId`
/// @param tickLimitLow The lower bound of an acceptable open interval for the ending price
/// @param tickLimitHigh The upper bound of an acceptable open interval for the ending price
/// @param positionSize The number of contracts minted, expressed in terms of the asset
/// @param tokenId The tokenId of the minted position, which encodes information about up to 4 legs
/// @param isBurn Flag indicating if the position is being burnt
/// @return An array of LeftRight encoded words containing the amount of currency0 and currency1 collected as fees f...
/// @return The net amount of currency0 and currency1 moved to/from the Uniswap V4 pool
function _unlockAndCreatePositionInAMM(
    PoolKey memory key,
    int24 tickLimitLow,
    int24 tickLimitHigh,
    uint128 positionSize,
    TokenId tokenId,
    bool isBurn
) internal returns (LeftRightUnsigned[4] memory, LeftRightSigned, int24) {
    return
        abi.decode(
            POOL_MANAGER_V4.unlock(
                abi.encode(
                    msg.sender,
                    key,
                    tickLimitLow,
                    tickLimitHigh,
                    positionSize,
                    tokenId,
                    isBurn
                )
            ),
            (LeftRightUnsigned[4], LeftRightSigned, int24)
        );
}
/// @notice Uniswap V4 unlock callback implementation.
/// @dev Parameters are `(address account, PoolKey key, int24 tickLimitLow, int24 tickLimitHigh, uint128 positionSiz...
/// @dev Executes the corresponding operations and state updates required to mint `tokenId` of `positionSize` in `ke...
/// @dev (shorts/longs are reversed before calling this function at burn)
/// @param data The encoded data containing the input parameters
/// @return `(LeftRightUnsigned[4] collectedByLeg, LeftRightSigned totalMoved)` An array of LeftRight encoded words ...
function unlockCallback(bytes calldata data) external returns (bytes memory) {
    if (msg.sender != address(POOL_MANAGER_V4)) revert Errors.UnauthorizedUniswapCallback();
    (
        address account,
        PoolKey memory key,
        int24 tickLimitLow,
        int24 tickLimitHigh,
        uint128 positionSize,
        TokenId tokenId,

```

```

        bool isBurn
    ) = abi.decode(data, (address, PoolKey, int24, int24, uint128, TokenId, bool));
    bool invertedLimits = tickLimitLow > tickLimitHigh;
    (
        LeftRightUnsigned[4] memory collectedByLeg,
        LeftRightSigned totalMoved
    ) = _createPositionInAMM(account, key, invertedLimits, positionSize, tokenId, isBurn);
    // Get the current tick of the Uniswap pool, check slippage
    int24 currentTick = getCurrentTick(abi.encode(key));
    if (invertedLimits) (tickLimitLow, tickLimitHigh) = (tickLimitHigh, tickLimitLow);
    if ((currentTick >= tickLimitHigh) || (currentTick <= tickLimitLow))
        revert Errors.PriceBoundFail(currentTick);
    return abi.encode(collectedByLeg, totalMoved, currentTick);
}
/*/////////////////////////////////////////////////////////////
                PUBLIC MINT/BURN FUNCTIONS
////////////////////////////////////////////////////////////*/
/// @notice Burn a new position containing up to 4 legs wrapped in a ERC1155 token.
/// @dev Auto-collect all accumulated fees.
/// @param poolKey The Uniswap V4 pool key in which to burn `tokenId`
/// @param tokenId The tokenId of the minted position, which encodes information about up to 4 legs
/// @param positionSize The number of contracts minted, expressed in terms of the asset
/// @param tickLimitLow The lower bound of an acceptable open interval for the ending price
/// @param tickLimitHigh The upper bound of an acceptable open interval for the ending price
/// @return An array of LeftRight encoded words containing the amount of currency0 and currency1 collected as fees f...
/// @return The net amount of currency0 and currency1 moved to/from the Uniswap V4 pool
function burnTokenizedPosition(
    bytes calldata poolKey,
    TokenId tokenId,
    uint128 positionSize,
    int24 tickLimitLow,
    int24 tickLimitHigh
) external nonReentrant returns (LeftRightUnsigned[4] memory, LeftRightSigned, int24) {
    _burn(msg.sender, TokenId.unwrap(tokenId), positionSize);
    emit TokenizedPositionBurnt(msg.sender, tokenId, positionSize);
    PoolKey memory _key = abi.decode(poolKey, (PoolKey));
    return
        _unlockAndCreatePositionInAMM(
            _key,
            tickLimitLow,
            tickLimitHigh,
            positionSize,
            tokenId.flipToBurnToken(),
            BURN
        );
}
/// @notice Create a new position `tokenId` containing up to 4 legs.
/// @param poolKey The Uniswap V4 pool key in which to mint `tokenId`
/// @param tokenId The tokenId of the minted position, which encodes information for up to 4 legs
/// @param positionSize The number of contracts minted, expressed in terms of the asset
/// @param tickLimitLow The lower bound of an acceptable open interval for the ending price
/// @param tickLimitHigh The upper bound of an acceptable open interval for the ending price
/// @return An array of LeftRight encoded words containing the amount of currency0 and currency1 collected as fees f...
/// @return The net amount of currency0 and currency1 moved to/from the Uniswap V4 pool
function mintTokenizedPosition(
    bytes calldata poolKey,
    TokenId tokenId,
    uint128 positionSize,
    int24 tickLimitLow,
    int24 tickLimitHigh
) external nonReentrant returns (LeftRightUnsigned[4] memory, LeftRightSigned, int24) {
    _mint(msg.sender, TokenId.unwrap(tokenId), positionSize);
    emit TokenizedPositionMinted(msg.sender, tokenId, positionSize);
    // verify that the tokenId is correctly formatted and conforms to all enforced constraints
    tokenId.validate();
    PoolKey memory _key = abi.decode(poolKey, (PoolKey));
    return
        _unlockAndCreatePositionInAMM(
            _key,
            tickLimitLow,
            tickLimitHigh,
            positionSize,
            tokenId,
            MINT
        );
}
/*/////////////////////////////////////////////////////////////
                TRANSFER HOOK IMPLEMENTATIONS
////////////////////////////////////////////////////////////*/
/// @notice All ERC1155 transfers are disabled.
function safeTransferFrom(
    address,
    address,
    uint256,

```

```

uint256,
bytes calldata
) public pure override {
    revert();
}
/// @notice All ERC1155 transfers are disabled.
function safeBatchTransferFrom(
    address,
    address,
    uint256[] calldata,
    uint256[] calldata,
    bytes calldata
) public pure override {
    revert();
}
/////////////////////////////////////////////////////////////////
// AMM INTERACTION AND POSITION UPDATE HELPERS
/////////////////////////////////////////////////////////////////
/// @notice Called to perform an ITM swap in the Uniswap pool to resolve any non-tokenType token deltas.
/// @dev When a position is minted or burnt in-the-money (ITM) we are *not* 100% currency0 or 100% currency1: we hav...
/// @dev The swapping for ITM options is needed because only one of the tokens are "borrowed" by a user to create th...
/// This is an ITM situation below (price within the range of the chunk):
//
//
// AMM      strike
// liquidity price tick
//
// ▲          ■
// ■          ■■■■■▼■■■■
// ■          ■          ■ liquidity chunk
// ■ ■■■■■■■▲■■■■■■■■■■
// ■ ■          ■          ■
// ■ ■          :          ■
// ■ ■          :          ■
// ■ ■          :          ■
// ■■■■■■■■■▲■■■■■■■■■■ price
//
//          ■
//          current price
//          in-the-money: mix of tokens 0 and 1 within the chunk
//
// If we take currency0 as an example, we deploy it to the AMM pool and *then* swap to get the right mix of curren...
// to be correctly in the money at that strike.
// It that position is burnt, then we remove a mix of the two tokens and swap one of them so that the user receive...
/// @param key The Uniswap V4 pool key in which to perform the swap
/// @param itmAmounts How much to swap (i.e. how many tokens are ITM)
/// @param asset The asset of the first leg of the tokenId (determines which token to swap into)
/// @return The token deltas swapped in the AMM
function swapInAMM(
    PoolKey memory key,
    LeftRightSigned itmAmounts,
    uint256 asset
) internal returns (LeftRightSigned) {
    unchecked {
        bool zeroForOne; // The direction of the swap, true for currency0 to currency1, false for currency1 to curre...
        int256 swapAmount; // The amount of currency0 or currency1 to swap
        // unpack the in-the-money amounts
        int128 itm0 = itmAmounts.rightSlot();
        int128 itm1 = itmAmounts.leftSlot();
        // NOTE: upstream users of this function such as the Panoptic Pool should ensure users always compensate for...
        // the netting swap is not perfectly accurate, and it is possible for swaps to run out of liquidity, so we d...
        // this is simply a convenience feature, and should be treated as such
        if ((itm0 != 0) && (itm1 != 0)) {
            // ensure the tokens are swapped from the correct asset.
            if (asset == 0) {
                zeroForOne = itm0 < 0;
                swapAmount = itm0;
            } else {
                zeroForOne = itm1 > 0;
                swapAmount = itm1;
            }
        } else if (itm0 != 0) {
            zeroForOne = itm0 < 0;
            swapAmount = itm0;
        } else {
            zeroForOne = itm1 > 0;
            swapAmount = itm1;
        }
        // NOTE: can occur if itm0 and itm1 have the same value
        // in that case, swapping would be pointless so skip
        if (swapAmount == 0) return LeftRightSigned.wrap(0);
        BalanceDelta swapDelta = POOL_MANAGER_V4.swap(
            key,
            IPoolManager.SwapParams(
                zeroForOne,
                swapAmount,
                zeroForOne
            )
        );
    }
}

```

```

        ? Constants.MIN_POOL_SQRT_RATIO + 1
        : Constants.MAX_POOL_SQRT_RATIO - 1

    ),
    ""
);
// return token deltas
return
    LeftRightSigned.wrap(0).addToLeftSlot(-swapDelta.amount0()).addToLeftSlot(
        -swapDelta.amount1()
    );
}

}
/// @notice Create the position in the AMM defined by `tokenId`.
/// @dev Loops over each leg in the tokenId and calls _createLegInAMM for each, which does the mint/burn in the AMM.
/// @param account The address of the user creating the position
/// @param key The Uniswap V4 pool key in which to create the position
/// @param invertedLimits Whether the inputted lower limit > upper limit
/// @param positionSize The size of the option position
/// @param tokenId The option position
/// @param isBurn Whether a position is being minted (false) or burned (true)
/// @return collectedByLeg An array of LeftRight encoded words containing the amount of currency0 and currency1 coll...
/// @return totalMoved The net amount of funds moved to/from Uniswap
function _createPositionInAMM(
    address account,
    PoolKey memory key,
    bool invertedLimits,
    uint128 positionSize,
    TokenId tokenId,
    bool isBurn
) internal returns (LeftRightUnsigned[4] memory collectedByLeg, LeftRightSigned totalMoved) {
    // upper bound on amount of tokens contained across all legs of the position at any given tick
    uint256 amount0;
    uint256 amount1;
    LeftRightSigned itmAmounts;
    LeftRightUnsigned totalCollected;
    {
        PoolData poolData = s_V4toSFPmIdData[key.toId()][tokenId.vegoid()];
        if (poolData.poolId() != tokenId.poolId() || !poolData.initialized())
            revert Errors.WrongUniswapPool();
        for (uint256 leg = 0; leg < tokenId.countLegs(); ) {
            if (tokenId.width(leg) == 0) {
                uint256 isLong = tokenId.isLong(leg);
                LeftRightUnsigned amountsMoved = PanopticMath.getAmountsMoved(
                    tokenId,
                    positionSize,
                    leg,
                    true
                );
                int128 signMultiplier = isLong == 0 ? int128(-1) : int128(1);
                {
                    uint256 tokenType = tokenId.tokenType(leg);
                    int128 itm0 = tokenType == 1
                        ? int128(0)
                        : signMultiplier * int128(amountsMoved.rightSlot());
                    int128 itm1 = tokenType == 0
                        ? int128(0)
                        : signMultiplier * int128(amountsMoved.leftSlot());
                    itmAmounts = itmAmounts.addToRightSlot(itm0).addToLeftSlot(itm1);
                }
            } else {
                address _account = account;
                PoolKey memory _key = key;
                LiquidityChunk liquidityChunk;
                {
                    uint128 _positionSize = positionSize;
                    liquidityChunk = PanopticMath.getLiquidityChunk(
                        tokenId,
                        leg,
                        _positionSize
                    );
                }
            }
            // validate tick range for newly minted positions
            if (!isBurn) {
                int24 tickSpacing = tokenId.tickSpacing();
                int24 tickLower = liquidityChunk.tickLower();
                int24 tickUpper = liquidityChunk.tickUpper();
                if (
                    tickLower % tickSpacing != 0 ||
                    tickUpper % tickSpacing != 0 ||
                    tickLower < poolData.minEnforcedTick() ||
                    tickUpper > poolData.maxEnforcedTick()
                ) revert Errors.InvalidTickBound();
            }
            unchecked {

```

```

        // increment accumulators of the upper bound on tokens contained across all legs of the position...
        amount0 += Math.getAmount0ForLiquidity(liquidityChunk);
        amount1 += Math.getAmount1ForLiquidity(liquidityChunk);
    }
    LeftRightSigned movedLeg;
    TokenId _tokenId = tokenId;
    bool _isBurn = isBurn;
    (movedLeg, collectedByLeg[leg]) = _createLegInAMM(
        _account,
        _key,
        _tokenId,
        leg,
        liquidityChunk,
        _isBurn,
        _tokenId.vegoId()
    );
    totalMoved = totalMoved.add(movedLeg);
    totalCollected = totalCollected.add(collectedByLeg[leg]);
    // if tokenType is 1, and we transacted some currency0: then this leg is ITM
    // if tokenType is 0, and we transacted some currency1: then this leg is ITM
    itmAmounts = itmAmounts.add(
        _tokenId.tokenType(leg) == 0
        ? LeftRightSigned.wrap(0).addToLeftSlot(movedLeg.leftSlot())
        : LeftRightSigned.wrap(0).addToRightSlot(movedLeg.rightSlot())
    );
}
unchecked {
    ++leg;
}
}
}
// Ensure upper bound on amount of tokens contained across all legs of the position on any given tick does not e...
// This is the maximum value of the `int128` type we frequently use to hold token amounts, so a given position's...
// fit within that limit at all times.
if (amount0 > uint128(type(int128).max - 4) || amount1 > uint128(type(int128).max - 4))
    revert Errors.PositionTooLarge();
if (invertedLimits) {
    // if the in-the-money amount is not zero (i.e. positions were minted ITM) and the user did provide tick lim...
    if ((LeftRightSigned.unwrap(itmAmounts) != 0)) {
        totalMoved = swapInAMM(key, itmAmounts, tokenId.asset(0)).add(totalMoved);
    }
}
{
    LeftRightSigned cumulativeDelta = totalMoved.sub(totalCollected);
    if (cumulativeDelta.rightSlot() > 0) {
        POOL_MANAGER_V4.burn(
            account,
            uint160(Currency.unwrap(key.currency0)),
            uint128(cumulativeDelta.rightSlot())
        );
    } else if (cumulativeDelta.rightSlot() < 0) {
        POOL_MANAGER_V4.mint(
            account,
            uint160(Currency.unwrap(key.currency0)),
            uint128(-cumulativeDelta.rightSlot())
        );
    }
    if (cumulativeDelta.leftSlot() > 0) {
        POOL_MANAGER_V4.burn(
            account,
            uint160(Currency.unwrap(key.currency1)),
            uint128(cumulativeDelta.leftSlot())
        );
    } else if (cumulativeDelta.leftSlot() < 0) {
        POOL_MANAGER_V4.mint(
            account,
            uint160(Currency.unwrap(key.currency1)),
            uint128(-cumulativeDelta.leftSlot())
        );
    }
}
}
}
/// @notice Create the position in the AMM for a specific leg in the tokenId.
/// @dev For the leg specified by the _leg input:
/// @dev - mints any new liquidity in the AMM needed (via _mintLiquidity)
/// @dev - burns any new liquidity in the AMM needed (via _burnLiquidity)
/// @dev - tracks all amounts minted and burned
/// @dev To burn a position, the opposing position is "created" through this function,
/// but we need to pass in a flag to indicate that so the removedLiquidity is updated.
/// @param account The address of the user creating the position
/// @param key The Uniswap V4 pool key in which to create the position
/// @param tokenId The option position
/// @param leg The leg index that needs to be modified
/// @param liquidityChunk The liquidity chunk in Uniswap represented by the leg

```



```

        Uniswap V4                                account
else: the position is long (buying a put or a call), then _burnLiquidity to remove liquidity from Uniswap V4
Buying(isLong=1): Burn in Uniswap
    ▲      ■■■      ■
    ■ ■■■■■■■■      ■■■■■■■■
    ■ ■      ■      ■      ■
    ■■■■■■■■■■      ■■■■■■■■
        Uniswap V4                                account
*/
LiquidityChunk _liquidityChunk = liquidityChunk;
PoolKey memory _key = key;
(BalanceDelta delta, BalanceDelta feesAccrued) = POOL_MANAGER_V4.modifyLiquidity(
    _key,
    IPoolManager.ModifyLiquidityParams(
        _liquidityChunk.tickLower(),
        _liquidityChunk.tickUpper(),
        isLong == 0
        ? int256(uint256(_liquidityChunk.liquidity()))
        : -int256(uint256(_liquidityChunk.liquidity())),
        positionKey
    ),
    ""
);
unchecked {
    moved = LeftRightSigned
        .wrap(0)
        .addToRightSlot(feesAccrued.amount0() - delta.amount0())
        .addToLeftSlot(feesAccrued.amount1() - delta.amount1());
}
// (premium can only be collected if liquidity existed in the chunk prior to this mint)
if (currentLiquidity.rightSlot() > 0) {
    collectedSingleLeg = LeftRightUnsigned
        .wrap(0)
        .addToRightSlot(uint128(feesAccrued.amount0()))
        .addToLeftSlot(uint128(feesAccrued.amount1()));
    _updateStoredPremia(positionKey, currentLiquidity, collectedSingleLeg, vegoid);
}
}
/// @notice Updates the premium accumulators for a chunk with the latest collected tokens.
/// @param positionKey A key representing a liquidity chunk/range in Uniswap
/// @param currentLiquidity The total amount of liquidity in the AMM for the specified chunk
/// @param collectedAmounts The amount of tokens (currency0 and currency1) collected from Uniswap
function _updateStoredPremia(
    bytes32 positionKey,
    LeftRightUnsigned currentLiquidity,
    LeftRightUnsigned collectedAmounts,
    uint256 vegoid
) private {
    (
        LeftRightUnsigned deltaPremiumOwed,
        LeftRightUnsigned deltaPremiumGross
    ) = _getPremiaDeltas(currentLiquidity, collectedAmounts, vegoid);
    // add deltas to accumulators and freeze both accumulators (for a token) if one of them overflows
    // (i.e if only currency0 (right slot) of the owed premium overflows, then stop accumulating both currency0 owe...
    // this prevents situations where the owed premium gets out of sync with the gross premium due to one of them ov...
    (s_accountPremiumOwed[positionKey], s_accountPremiumGross[positionKey]) = LeftRightLibrary
        .addCapped(
            s_accountPremiumOwed[positionKey],
            deltaPremiumOwed,
            s_accountPremiumGross[positionKey],
            deltaPremiumGross
        );
}
/// @notice Compute deltas for Owed/Gross premium given quantities of tokens collected from Uniswap.
/// @dev Returned accumulators are capped at the max value (2^128 - 1) for each token if they overflow.
/// @param currentLiquidity NetLiquidity (right) and removedLiquidity (left) at the start of the transaction
/// @param collectedAmounts Total amount of tokens (currency0 and currency1) collected from Uniswap
/// @return deltaPremiumOwed The extra premium (per liquidity X64) to be added to the owed accumulator for currency0...
/// @return deltaPremiumGross The extra premium (per liquidity X64) to be added to the gross accumulator for currenc...
function _getPremiaDeltas(
    LeftRightUnsigned currentLiquidity,
    LeftRightUnsigned collectedAmounts,
    uint256 vegoid
)
private
pure
returns (LeftRightUnsigned deltaPremiumOwed, LeftRightUnsigned deltaPremiumGross)
{
    // extract liquidity values
    uint256 removedLiquidity = currentLiquidity.leftSlot();
    uint256 netLiquidity = currentLiquidity.rightSlot();
    // premia spread equations are graphed and documented here: https://www.desmos.com/calculator/mdeqob2m04
    // explains how we get from the premium per liquidity (calculated here) to the total premia collected and the mu...

```

```

// as well as how the value of VEGOID affects the premia
// note that the "base" premium is just a common factor shared between the owed (long) and gross (short)
// premia, and is only separated to simplify the calculation
// (the graphed equations include this factor without separating it)
unchecked {
    uint256 totalLiquidity = netLiquidity + removedLiquidity;
    uint256 premium0X64_base;
    uint256 premium1X64_base;
    {
        uint128 collected0 = collectedAmounts.rightSlot();
        uint128 collected1 = collectedAmounts.leftSlot();
        // compute the base premium as collected * total / net^2 (from Eqn 3)
        premium0X64_base = Math.mulDiv(
            collected0,
            totalLiquidity * 2 ** 64,
            netLiquidity ** 2
        );
        premium1X64_base = Math.mulDiv(
            collected1,
            totalLiquidity * 2 ** 64,
            netLiquidity ** 2
        );
    }
    {
        uint128 premium0X64_owed;
        uint128 premium1X64_owed;
        {
            // compute the owed premium (from Eqn 3)
            uint256 numerator = netLiquidity + (removedLiquidity / vegoid);
            premium0X64_owed = Math
                .mulDiv(premium0X64_base, numerator, totalLiquidity)
                .toUint128Capped();
            premium1X64_owed = Math
                .mulDiv(premium1X64_base, numerator, totalLiquidity)
                .toUint128Capped();
            deltaPremiumOwed = LeftRightUnsigned.wrap(premium0X64_owed).addToLeftSlot(
                premium1X64_owed
            );
        }
    }
    {
        uint128 premium0X64_gross;
        uint128 premium1X64_gross;
        {
            // compute the gross premium (from Eqn 4)
            uint256 numerator = totalLiquidity * 2 -
                totalLiquidity *
                removedLiquidity +
                ((removedLiquidity ** 2) / vegoid);
            premium0X64_gross = Math
                .mulDiv(premium0X64_base, numerator, totalLiquidity ** 2)
                .toUint128Capped();
            premium1X64_gross = Math
                .mulDiv(premium1X64_base, numerator, totalLiquidity ** 2)
                .toUint128Capped();
            deltaPremiumGross = LeftRightUnsigned.wrap(premium0X64_gross).addToLeftSlot(
                premium1X64_gross
            );
        }
    }
}
}

/*//////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
                                   QUERIES
//////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////*/
/// @notice Return the liquidity associated with a given liquidity chunk/tokenType for a user on a Uniswap pool.
/// @param poolKey the poolKey of the UniswapV4 pool
/// @param owner The address of the account that is queried
/// @param tokenType The tokenType of the position
/// @param tickLower The lower end of the tick range for the position
/// @param tickUpper The upper end of the tick range for the position
/// @return accountLiquidities The amount of liquidity that held in and removed from Uniswap for that chunk (netLiqu...
function getAccountLiquidity(
    bytes calldata poolKey,
    address owner,
    uint256 tokenType,
    int24 tickLower,
    int24 tickUpper
) external view returns (LeftRightUnsigned accountLiquidities) {
    PoolKey memory key = abi.decode(poolKey, (PoolKey));
    // Extract the account liquidity for a given Uniswap pool, owner, token type, and ticks
    // tokenType input here is the asset of the positions minted, this avoids put liquidity to be used for call, and...
    accountLiquidities = s_accountLiquidity[
        EfficientHash.efficientKeccak256(

```

```

        abi.encodePacked(key.toId(), owner, tokenType, tickLower, tickUpper)
    )
    ];
}
/// @notice Return the premium associated with a given position, where premium is an accumulator of feeGrowth for th...
/// @dev If an atTick parameter is provided that is different from `type(int24).max`, then it will update the premiu...
/// block at the provided atTick value. We do this because this may be called immediately after the Uniswap V4 pool ...
/// so no need to read the feeGrowth from the Uniswap V4 pool.
/// @param poolKey the poolKey of the UniswapV4 pool
/// @param owner The address of the account that is queried
/// @param tokenType The tokenType of the position
/// @param tickLower The lower end of the tick range for the position
/// @param tickUpper The upper end of the tick range for the position
/// @param atTick The current tick. Set `atTick < (type(int24).max = 8388608)` to get latest premium up to the cu...
/// @param isLong Whether the position is long (=1) or short (=0)
/// @return The amount of premium (per liquidity X64) for currency0 = `sum(feeGrowthLast0X128)` over every block whe...
/// @return The amount of premium (per liquidity X64) for currency1 = `sum(feeGrowthLast0X128)` over every block whe...
function getAccountPremium(
    bytes calldata poolKey,
    address owner,
    uint256 tokenType,
    int24 tickLower,
    int24 tickUpper,
    int24 atTick,
    uint256 isLong,
    uint256 vegoid
) external view returns (uint128, uint128) {
    PoolKey memory key = abi.decode(poolKey, (PoolKey));
    bytes32 positionKey = EfficientHash.efficientKeccak256(
        abi.encodePacked(key.toId(), owner, tokenType, tickLower, tickUpper)
    );
    LeftRightUnsigned acctPremia;
    LeftRightUnsigned accountLiquidities = s_accountLiquidities[positionKey];
    uint128 netLiquidity = accountLiquidities.rightSlot();
    // Compute the premium up to the current block (ie. after last touch until now). Do not proceed if `atTick == (t...
    if (atTick < type(int24).max && netLiquidity != 0) {
        // unique key to identify the liquidity chunk in this Uniswap pool
        LeftRightUnsigned amountToCollect;
        {
            PoolId _idV4 = key.toId();
            int24 _tickLower = tickLower;
            int24 _tickUpper = tickUpper;
            int24 _atTick = atTick;
            bytes32 _positionKey = positionKey;
            (uint256 feeGrowthInside0X128, uint256 feeGrowthInside1X128) = V4StateReader
                .getFeeGrowthInside(POOL_MANAGER_V4, _idV4, _atTick, _tickLower, _tickUpper);
            (uint256 feeGrowthInside0LastX128, uint256 feeGrowthInside1LastX128) = V4StateReader
                .getFeeGrowthInsideLast(
                    POOL_MANAGER_V4,
                    _idV4,
                    keccak256(
                        abi.encodePacked(address(this), _tickLower, _tickUpper, _positionKey)
                    )
                );
        }
        unchecked {
            amountToCollect = LeftRightUnsigned
                .wrap(
                    uint128(
                        Math.mulDiv128(
                            feeGrowthInside0X128 - feeGrowthInside0LastX128,
                            netLiquidity
                        )
                    )
                )
                .addToLeftSlot(
                    uint128(
                        Math.mulDiv128(
                            feeGrowthInside1X128 - feeGrowthInside1LastX128,
                            netLiquidity
                        )
                    )
                );
        }
    }
    (LeftRightUnsigned premiumOwed, LeftRightUnsigned premiumGross) = _getPremiaDeltas(
        accountLiquidities,
        amountToCollect,
        vegoid
    );
    // add deltas to accumulators and freeze both accumulators (for a token) if one of them overflows
    // (i.e if only currency0 (right slot) of the owed premium overflows, then stop accumulating both currency0 ...
    // this prevents situations where the owed premium gets out of sync with the gross premium due to one of the...
    (premiumOwed, premiumGross) = LeftRightLibrary.addCapped(
        s_accountPremiumOwed[positionKey],

```

```

        premiumOwed,
        s_accountPremiumGross[positionKey],
        premiumGross
    );
    acctPremia = isLong == 1 ? premiumOwed : premiumGross;
} else {
    // Extract the account liquidity for a given Uniswap pool, owner, token type, and ticks
    acctPremia = isLong == 1
        ? s_accountPremiumOwed[positionKey]
        : s_accountPremiumGross[positionKey];
}
return (acctPremia.rightSlot(), acctPremia.leftSlot());
}
/// @notice Returns the Uniswap V4 poolkey for a given `poolId`.
/// @param poolId The unique pool identifier for a Uni V4 pool in the SFPM
/// @return The Uniswap V4 pool key corresponding to `poolId`
function getUniswapV4PoolKeyFromId(uint64 poolId) external view returns (PoolKey memory) {
    return s_poolIdToKey[poolId];
}
/// @notice Returns the current enforced tick limits for a given idV4 `poolId`.
/// @param poolId The unique pool identifier for a Uniswap V4 pool
/// @return The minimum enforced tick for chunks created in the pool corresponding to `poolId`
/// @return The maximum enforced tick for chunks created in the pool corresponding to `poolId`
function getEnforcedTickLimits(uint64 poolId) external view returns (int24, int24) {
    PoolKey memory poolKey = s_poolIdToKey[poolId];
    uint256 vegoid = uint8(poolId >> 40);
    PoolData poolData = s_V4toSFPMIdData[poolKey.toId()][vegoid];
    return (poolData.minEnforcedTick(), poolData.maxEnforcedTick());
}
/// @notice Returns the `poolId` for a given Uniswap pool.
/// @param id The PoolId of the Uniswap V4 Pool
/// @return poolId The unique pool identifier corresponding to a idV4
function getPoolId(bytes memory id, uint8 vegoid) external view returns (uint64) {
    PoolId idV4 = abi.decode(id, (PoolId));
    return s_V4toSFPMIdData[idV4][vegoid].poolId();
}
/// @notice Returns the current tick of a given Uniswap V4 pool
/// @param poolKey the poolKey of the UniswapV4 pool
/// @return currentTick The current tick of the Uniswap pool
function getCurrentTick(bytes memory poolKey) public view returns (int24 currentTick) {
    PoolKey memory key = abi.decode(poolKey, (PoolKey));
    currentTick = V4StateReader.getTick(POOL_MANAGER_V4, key.toId());
}
}

```

[9/12] contracts/CollateralTracker.sol

```

// SPDX-License-Identifier: BUSL-1.1
pragma solidity ^0.8.24;
// Interfaces
import {PanopticPool} from "./PanopticPool.sol";
import {IRiskEngine} from "@contracts/interfaces/IRiskEngine.sol";
import {IPoolManager} from "v4-core/interfaces/IPoolManager.sol";
// Inherited implementations
import {Clone} from "clones-with-immutable-args/Clone.sol";
import {ERC20Minimal} from "@tokens/ERC20Minimal.sol";
import {Multicall} from "@base/Multicall.sol";
// Libraries
import {Errors} from "@libraries/Errors.sol";
import {InteractionHelper} from "@libraries/InteractionHelper.sol";
import {Math} from "@libraries/Math.sol";
import {SafeTransferLib} from "@libraries/SafeTransferLib.sol";
// Custom types
import {Currency} from "v4-core/types/Currency.sol";
import {LeftRightSigned} from "@types/LeftRight.sol";
import {TokenId} from "@types/TokenId.sol";
import {RiskParameters} from "@types/RiskParameters.sol";
import {MarketState, MarketStateLibrary} from "@types/MarketState.sol";
/// @title Collateral Tracking System / Margin Accounting used in conjunction with a Panoptic Pool.
/// @author Axicon Labs Limited
//
/// @notice Tracks collateral of users which is key to ensure the correct level of collateralization is achieved.
/// This is represented as an ERC20 share token. A Panoptic pool has 2 tokens, each issued by its own instance of a Coll...
/// All math within this contract pertains to a single token.
//
/// @notice This contract uses the ERC4626 standard allowing the minting and burning of "shares" (represented using ERC2...
/// Panoptic uses a collateral tracking system that is similar to TradFi margin accounts. While users can borrow and
/// effectively control funds several times larger than the collateral they deposited, they cannot withdraw those funds
/// from the Panoptic-Uniswap ecosystem. All funds are always owned by the Panoptic protocol, but users will:
//

```

```

/// @notice 1) collect any fees generated by selling an option.
//
/// @notice 2) get any gain in capital that results from buying an option that becomes in-the-money.
contract CollateralTracker is Clone, ERC20Minimal, Multicall {
    using Math for uint256;

    /*//////////////////////////////////////////////////////////////
                                EVENTS
    //////////////////////////////////////////////////////////////*/

    /// @notice Emitted when assets are deposited into the Collateral Tracker.
    /// @param sender The address of the caller
    /// @param owner The address of the recipient of the newly minted shares
    /// @param assets The amount of assets deposited by `sender` in exchange for `shares`
    /// @param shares The amount of shares minted to `owner`
    event Deposit(address indexed sender, address indexed owner, uint256 assets, uint256 shares);

    /// @notice Emitted when assets are withdrawn from the Collateral Tracker.
    /// @param sender The address of the caller
    /// @param receiver The address of the recipient of the withdrawn assets
    /// @param owner The address of the owner of the shares being burned
    /// @param assets The amount of assets withdrawn to `receiver`
    /// @param shares The amount of shares burned by `owner` in exchange for `assets`
    event Withdraw(
        address indexed sender,
        address indexed receiver,
        address indexed owner,
        uint256 assets,
        uint256 shares
    );

    /// @notice Emitted when shares are donated to the protocol.
    /// @param sender The address of the caller
    /// @param shares The amount of shares burned by the sender
    event Donate(address indexed sender, uint256 shares);

    /// @notice Emitted when a commission is paid.
    /// @param owner The address of the owner of the shares being used to pay for the commission
    /// @param builder The address of the account that received the commission if a builderCode is provided
    /// @param commissionPaidProtocol The amount of assets paid that goes to the PLPs (if builder == address(0)) or to t...
    /// @param commissionPaidBuilder The amount of assets paid that goes to the builder
    event CommissionPaid(
        address indexed owner,
        address indexed builder,
        uint128 commissionPaidProtocol,
        uint128 commissionPaidBuilder
    );

    /// @notice Emitted when a user attempts to settle interest but lacks sufficient shares to pay in full.
    /// @dev The user's borrow index is not updated, meaning they will need to pay this interest again in the future.
    /// @param owner The address of the insolvent user
    /// @param interestOwed The total amount of interest the user owed
    /// @param interestPaid The actual amount of interest paid (value of shares burned)
    /// @param sharesBurned The number of shares burned in the partial payment
    event InsolvencyPenaltyApplied(
        address indexed owner,
        uint256 interestOwed,
        uint256 interestPaid,
        uint256 sharesBurned
    );

    /*//////////////////////////////////////////////////////////////
                                CONSTANTS
    //////////////////////////////////////////////////////////////*/

    /// @notice Prefix for the token symbol (i.e. poUSDC).
    string internal constant TICKER_PREFIX = "po";
    /// @notice Prefix for the token name (i.e. POPT-V1 USDC LP on ETH/USDC 30bps).
    string internal constant NAME_PREFIX = "POPT-V1";
    /// @notice Decimals for computation (1 bps (1 basis point) precision: 0.01%).
    /// @dev uint type for composability with unsigned integer based mathematical operations.
    uint256 internal constant DECIMALS = 10_000;
    /// @notice Decimals for WAD calculations.
    uint256 internal constant WAD = 1e18;
    /// @notice Mask zero the value between bits 112 and 150);
    uint256 internal constant TARGET_RATE_MASK =
        0xFFFFFFFFFFFFFFFFFFFFFFFFC000000000FFFFFFFFFFFFFFFFFFFFFFFFFFFF;
    bool internal constant IS_NOT_DEPOSIT = false;
    bool internal constant IS_DEPOSIT = true;
    /// @notice Transient storage slot for the utilization
    bytes32 internal constant UTILIZATION_TRANSIENT_SLOT =
        keccak256("panoptic.utilization.snapshot");

    /*//////////////////////////////////////////////////////////////
                                PANOPTIC POOL DATA
    //////////////////////////////////////////////////////////////*/

    /// @notice Cached amount of assets accounted to be held by the Panoptic Pool – ignores donations, pending fee payout...
    uint128 internal s_depositedAssets;
    /// @notice Amount of assets moved from the Panoptic Pool to the AMM.
    uint128 internal s_assetsInAMM;
    /// @notice Amount of shares credited to the protocol, includes credits and purchased option liquidity above the reh...
    uint256 internal s_creditedShares;
}

```

```

UNISWAP POOL DATA
////////////////////////////////////////////////////////////*/
/// @notice Boolean which tracks whether this CollateralTracker has been initialized.
bool internal s_initialized;
////////////////////////////////////////////////////////////
POOL-SPECIFIC IMMUTABLE PARAMETERS
////////////////////////////////////////////////////////////*/
// The parameters will be encoded at `_getImmutableArgsOffset()` in calldata as follows:
// abi.encodePacked(address panopticPool, bool underlyingIsToken0, address underlyingToken, address token0, address ...
// bytes: 0                20                21                41                61                81 ...
//          |<&lt;---- 160 bits ---->&gt;|<&lt;---- 8 bits ---->&gt;|<&lt;---- 160 bits ---->&gt;|<&lt;---- 160 bits ---->&gt;|<&lt;---- 160 bits ---->&gt;...
//          panopticPool    underlyingIsToken0    underlyingToken    token0            token1            ...
/// @notice Retrieve the Panoptic Pool that this collateral token belongs to.
/// @return The Panoptic Pool associated with this collateral token
function panopticPool() public pure returns (PanopticPool) {
    return PanopticPool(_getArgAddress(0));
}
/// @notice Retrieve a boolean indicating whether the underlying token is token0 or token1 in the Uniswap V3 pool.
/// @return _underlyingIsToken0 True if the underlying token is token0, false if it is token1
function underlyingIsToken0() public pure returns (bool _underlyingIsToken0) {
    uint256 offset = _getImmutableArgsOffset();
    assembly ("memory-safe") {
        _underlyingIsToken0 := shr(0xf8, calldataload(add(offset, 20)))
    }
}
/// @notice Retrieve the address of the underlying token.
/// @return The address of the underlying token
function underlyingToken() public pure returns (address) {
    return _getArgAddress(21);
}
/// @notice Retrieve the address of token0 in the Uniswap V3 pool.
/// @return The address of token0 in the Uniswap V3 pool
function token0() public pure returns (address) {
    return _getArgAddress(41);
}
/// @notice Retrieve the address of token1 in the Uniswap V3 pool.
/// @return The address of token1 in the Uniswap V3 pool
function token1() public pure returns (address) {
    return _getArgAddress(61);
}
/// @notice Retrieve the RiskEngine associated with that CollateralTracker.
/// @return The RiskEngine instance associated with that CollateralTracker's uniswap pool
function riskEngine() public pure returns (IRiskEngine) {
    return IRiskEngine(_getArgAddress(81));
}
/// @notice Retrieve the PoolManager associated with that CollateralTracker.
/// @dev stored as zero if not a Uniswap v4 pool
/// @return The PoolManager instance associated with that CollateralTracker's uniswap v4 pool
function poolManager() public pure returns (IPoolManager) {
    return IPoolManager(_getArgAddress(101));
}
/// @notice Retrieve the fee of the Uniswap V3 pool.
/// @return _poolFee The fee of the Uniswap V3 pool
function poolFee() public pure returns (uint24 _poolFee) {
    uint256 offset = _getImmutableArgsOffset();
    assembly ("memory-safe") {
        _poolFee := shr(0xe8, calldataload(add(offset, 121)))
    }
}
}
////////////////////////////////////////////////////////////
STORAGE
////////////////////////////////////////////////////////////*/
/**
 * @notice How the Borrow Index Works
 *
 * The borrow index is a global accumulator that tracks how much $1 of debt
 * grows over time with compound interest. It starts at 1e18 (representing 1.0)
 * and increases continuously.
 *
 * Example:
 * - User borrows 100 tokens when globalIndex = 1.0e18
 * - Time passes, globalIndex grows to 1.2e18 (20% growth)
 * - User now owes: 100 * (1.2e18 / 1.0e18) = 120 tokens
 *
 * Each user stores their "checkpoint" index from their last interaction,
 * allowing efficient compound interest calculation without iteration.
 */
/// @notice Global interest rate accumulator packed into a single 256-bit value
/// @dev Layout:
/// - Left slot (106 bits): Accumulated unrealized interest that hasn't been distributed (max deposit is 2**104...
/// - Next 38 bits: the rateAtTarget value in WAD (2**38 = 800% interest rate)
/// - Next lowest 32 bits: Last interaction epoch (1 epoch = block.timestamp/4)
/// - Lowest 80 bits: Global borrow index in WAD (starts at 1e18). 2**80 = 1.75 years at 800% interest
/// The borrow index tracks the compound growth factor since protocol inception.

```

```

///      A user's current debt = originalDebt * (currentBorrowIndex / userBorrowIndexSnapshot)
MarketState internal s_marketState;
/// @notice Tracks each user's borrowing state and last interaction checkpoint
/// @dev Packed layout:
///      - Left slot (128 bits): Net borrows = netShorts - netLongs
///      Represents the user's net borrowed amount in tokens
///      Can be negative, in which case they purchased more options than they sold
///      - Right slot (128 bits): User's borrow index snapshot
///      The global borrow index value when this user last accrued interest
/// @dev Interest calculation: interestOwed = netBorrows * (currentIndex - userIndex) / userIndex
mapping(address account => LeftRightSigned interestState) internal s_interestState;
*/
////////////////////////////////////
RISK PARAMETERS
////////////////////////////////////
/// @notice The commission fee, in basis points, collected from PLPs at option mint.
/// @dev In Panoptic, options never expire, commissions are only paid when a new position is minted.
/// @dev We believe that this will eliminate the impact of the commission fee on the user's decision-making process ...
uint256 immutable COMMISSION_FEE;
*/
////////////////////////////////////
ACCESS CONTROL
////////////////////////////////////
/// @notice Reverts if the associated Panoptic Pool is not the caller.
modifier onlyPanopticPool() {
    _onlyPanopticPool();
    _;
}
function _onlyPanopticPool() internal view {
    if (msg.sender != address(panopticPool())) revert Errors.NotPanopticPool();
}
*/
////////////////////////////////////
INITIALIZATION & PARAMETER SETTINGS
////////////////////////////////////
/// @notice Set immutable parameters for the Collateral Tracker.
/// @param _commissionFee The commission fee, in basis points, collected from PLPs at option mint
constructor(uint256 _commissionFee) {
    COMMISSION_FEE = _commissionFee;
}
/// @notice Initializes a new `CollateralTracker` instance with 1 virtual asset and 10^6 virtual shares. Can only be...
function initialize() external {
    // fails if already initialized
    if (s_initialized) revert Errors.CollateralTokenAlreadyInitialized();
    s_initialized = true;
    // these virtual shares function as a multiplier for the capital requirement to manipulate the pool price
    // e.g. if the virtual shares are 10**6, then the capital requirement to manipulate the price to 10**12 is 10**1...
    _internalSupply = 10 ** 6;
    // set total assets to 1
    // the initial share price is defined by 1/virtualShares
    s_depositedAssets = 1;
    // store the initial block and initialize the borrowIndex
    s_marketState = MarketStateLibrary.storeMarketState(WAD, block.timestamp >> 2, 0, 0);
}
*/
////////////////////////////////////
COLLATERAL TOKEN INFORMATION
////////////////////////////////////
/// @notice Get information about the utilization of this collateral vault.
/// @return depositedAssets Cached amount of assets accounted to be held by the Panoptic Pool – ignores donations, p...
/// @return insideAMM The underlying token amount held in the AMM
/// @return creditedShares The amount of shares currently held as credit
/// @return currentPoolUtilization The pool utilization defined as `s_assetsInAMM * 10_000 / totalAssets()`,
/// where totalAssets is the total tracked assets in the AMM and PanopticPool minus fees and donations to the Panopt...
function getPoolData()
    external
    view
    returns (
        uint256 depositedAssets,
        uint256 insideAMM,
        uint256 creditedShares,
        uint256 currentPoolUtilization
    )
{
    depositedAssets = s_depositedAssets;
    insideAMM = s_assetsInAMM;
    creditedShares = s_creditedShares;
    currentPoolUtilization = _poolUtilizationView();
}
/// @notice Returns the global borrow index that tracks compound interest growth
/// @dev The index starts at 1e18 and compounds continuously. Represents how much 1 unit of debt has grown since inc...
/// @return The current global borrow index in WAD (18 decimals)
function borrowIndex() external view returns (uint80) {
    return s_marketState.borrowIndex();
}
/// @notice Returns the last time at which interest rates were compounded.
/// @return The last time at which the interest rates were compounded
function lastInteractionTimestamp() external view returns (uint256) {

```

```

return s_marketState.marketEpoch() &lt;&lt; 2;
}
/// @notice Returns the accumulated unrealized global interest
/// @return The total interest that has accumulated but not yet been distributed to lenders
function unrealizedGlobalInterest() external view returns (uint256) {
    return s_marketState.unrealizedInterest();
}
/// @notice Returns rateAtTarget of the market
/// @return The rateAtTarget
function rateAtTarget() external view returns (uint256) {
    return s_marketState.rateAtTarget();
}
}
/// @notice Returns the borrowing state for a specific user
/// @dev Returns both the user's borrow index snapshot and their net borrowed amount
/// @return userBorrowIndex The borrow index when the user last accrued interest (used as the basis for interest cal...)
/// @return netBorrows The net borrowed amount for the user (positive = borrower, zero/negative = no interest owed)
function interestState(
    address user
) external view returns (int128 userBorrowIndex, int128 netBorrows) {
    return (s_interestState[user].rightSlot(), s_interestState[user].leftSlot());
}
}
/// @notice Returns name of token composed of underlying token symbol and pool data.
/// @return The name of the token
function name() external view returns (string memory) {
    // this logic requires multiple external calls and error handling, so we do it in a delegatecall to a library to...
    return
        InteractionHelper.computeName(
            token0(),
            token1(),
            underlyingIsToken0(),
            poolFee(),
            NAME_PREFIX
        );
}
}
/// @notice Returns symbol as prefixed symbol of underlying token.
/// @return The symbol of the token
function symbol() external view returns (string memory) {
    // this logic requires multiple external calls and error handling, so we do it in a delegatecall to a library to...
    return InteractionHelper.computeSymbol(underlyingToken(), TICKER_PREFIX);
}
}
/// @notice Returns decimals of underlying token (0 if not present).
/// @return The decimals of the token
function decimals() external view returns (uint8) {
    // this logic requires multiple external calls and error handling, so we do it in a delegatecall to a library to...
    return InteractionHelper.computeDecimals(underlyingToken());
}
}
/*
LIMITED TRANSFER FUNCTIONS
*/
/// @dev See {IERC20-transfer}.
/// @dev Requirements:
/// - the caller must have a balance of at least `amount`.
/// - the caller must not have any open positions on the Panoptic Pool.
function transfer(
    address recipient,
    uint256 amount
) public override(ERC20Minimal) returns (bool) {
    _accrueInterest(msg.sender, IS_NOT_DEPOSIT);
    // make sure the caller does not have any open option positions
    // if they do: we don't want them sending panoptic pool shares to others
    // as this would reduce their amount of collateral against the opened positions
    if (panopticPool().numberOfLegs(msg.sender) != 0) revert Errors.PositionCountNotZero();
    return ERC20Minimal.transfer(recipient, amount);
}
}
/// @dev See {IERC20-transferFrom}.
/// @dev Requirements:
/// - the `from` must have a balance of at least `amount`.
/// - the caller must have allowance for `from` of at least `amount` tokens.
/// - `from` must not have any open positions on the Panoptic Pool.
function transferFrom(
    address from,
    address to,
    uint256 amount
) public override(ERC20Minimal) returns (bool) {
    _accrueInterest(from, IS_NOT_DEPOSIT);
    // make sure the sender does not have any open option positions
    // if they do: we don't want them sending panoptic pool shares to others
    // as this would reduce their amount of collateral against the opened positions
    if (panopticPool().numberOfLegs(from) != 0) revert Errors.PositionCountNotZero();
    return ERC20Minimal.transferFrom(from, to, amount);
}
}
/*
UNISWAP V4 LOCK CALLBACK
*/

```



```

/// @notice Initiates the unlock callback to wrap/unwrap `delta` amount of the underlying asset and transfer to/from...
/// @param account The address of the account to transfer the underlying asset to/from
/// @param delta The amount of the underlying asset to wrap/unwrap and transfer
function _settleCurrencyDelta(address account, int256 delta) internal {
    poolManager().unlock(abi.encode(account, delta, msg.value));
}
/// @notice Uniswap V4 unlock callback implementation.
/// @dev Parameters are `(address account, int256 delta, uint256 valueOrigin)`.
/// @dev Wraps/unwraps `delta` amount of the underlying asset and transfers to/from the Panoptic Pool.
/// @param data The encoded data containing the account, delta, and valueOrigin
/// @return This function returns no data
function unlockCallback(bytes calldata data) external returns (bytes memory) {
    if (msg.sender != address(poolManager())) revert Errors.UnauthorizedUniswapCallback();
    (address account, int256 delta, uint256 valueOrigin) = abi.decode(
        data,
        (address, int256, uint256)
    );
    address underlyingAsset = underlyingToken();
    if (delta > 0) {
        if (Currency.wrap(underlyingAsset).isAddressZero()) {
            poolManager().settle{value: uint256(delta)}();
            // keep checked to prevent underflows
            uint256 surplus = valueOrigin - uint256(delta);
            if (surplus > 0) SafeTransferLib.safeTransferETH(account, surplus);
        } else {
            poolManager().sync(Currency.wrap(underlyingAsset));
            SafeTransferLib.safeTransferFrom(
                underlyingAsset,
                account,
                address(poolManager()),
                uint256(delta)
            );
            poolManager().settle();
        }
        poolManager().mint(address(panopticPool()), uint160(underlyingAsset), uint256(delta));
    } else if (delta < 0) {
        unchecked {
            delta = -delta;
        }
        poolManager().burn(address(panopticPool()), uint160(underlyingAsset), uint256(delta));
        poolManager().take(Currency.wrap(underlyingAsset), account, uint256(delta));
    }
    return "";
}
}
/*//////////////////////////////////////////////////////////////
                STANDARD ERC4626 INTERFACE
//////////////////////////////////////////////////////////////*/
/// @notice Get the token contract address of the underlying asset being managed.
/// @return assetTokenAddress The address of the underlying asset
function asset() external pure returns (address assetTokenAddress) {
    return underlyingToken();
}
/// @notice Get the total amount of assets managed by the CollateralTracker vault.
/// @dev This returns the total tracked assets in the AMM and PanopticPool,
/// @dev - EXCLUDING the amount of collected fees (because they are reserved for short options)
/// @dev - EXCLUDING any donations that have been made to the pool
/// @return The total amount of assets managed by the CollateralTracker vault
function totalAssets() public view returns (uint256) {
    unchecked {
        return uint256(s_depositedAssets) + s_assetsInAMM + s_marketState.unrealizedInterest();
    }
}
/// @notice Returns the total supply of shares including credited shares
/// @return The total supply of shares (internal supply + credited shares)
function totalSupply() public view returns (uint256) {
    unchecked {
        return _internalSupply + s_creditedShares;
    }
}
}
/// @notice Returns the amount of shares that can be minted for the given amount of assets.
/// @param assets The amount of assets to be deposited
/// @return shares The amount of shares that can be minted
function convertToShares(uint256 assets) public view returns (uint256 shares) {
    return Math.mulDiv(assets, totalSupply(), totalAssets());
}
}
/// @notice Returns the amount of assets that can be redeemed for the given amount of shares.
/// @param shares The amount of shares to be redeemed
/// @return assets The amount of assets that can be redeemed
function convertToAssets(uint256 shares) public view returns (uint256 assets) {
    return Math.mulDiv(shares, totalAssets(), totalSupply());
}
}
/// @notice Returns the amount of assets that can be redeemed by the user.
/// @param owner The redeeming address
/// @return assets The amount of assets that can be redeemed

```

```

function assetsOf(address owner) external view returns (uint256 assets) {
    return convertToAssets(balanceOf[owner]);
}
/// @notice Returns the maximum deposit amount.
/// @return maxAssets The maximum amount of assets that can be deposited
function maxDeposit(address) external pure returns (uint256 maxAssets) {
    return type(uint104).max;
}
/// @notice Returns shares received for depositing given amount of assets.
/// @param assets The amount of assets to be deposited
/// @return shares The amount of shares that can be minted
function previewDeposit(uint256 assets) public view returns (uint256 shares) {
    shares = Math.mulDiv(assets, totalSupply(), totalAssets());
}
/// @notice Deposit underlying tokens (assets) to the Panoptic pool from the LP and mint corresponding amount of sha...
/// @dev There is a maximum asset deposit limit of  $2^{104} - 1$ .
/// @dev Shares are minted and sent to the LP (`receiver`).
/// @param assets Amount of assets deposited
/// @param receiver User to receive the shares
/// @return shares The amount of Panoptic pool shares that were minted to the recipient
function deposit(uint256 assets, address receiver) external payable returns (uint256 shares) {
    _accrueInterest(msg.sender, IS_DEPOSIT);
    if (assets > type(uint104).max) revert Errors.DepositTooLarge();
    if (assets == 0) revert Errors.BelowMinimumRedemption();
    shares = previewDeposit(assets);
    address _poolManager = address(poolManager());
    if (_poolManager == address(0)) {
        // transfer assets (underlying token funds) from the user/the LP to the PanopticPool
        // in return for the shares to be minted
        SafeTransferLib.safeTransferFrom(
            underlyingToken(),
            msg.sender,
            address(panopticPool()),
            assets
        );
    }
    // mint collateral shares of the Panoptic Pool funds (this ERC20 token)
    _mint(receiver, shares);
    // update tracked asset balance
    s_depositedAssets += uint128(assets);
    if (_poolManager != address(0)) {
        // transfer assets from the user/the LP to the PanopticPool
        // in return for the shares to be minted
        _settleCurrencyDelta(msg.sender, int256(assets));
    }
    emit Deposit(msg.sender, receiver, assets, shares);
}
/// @notice Returns the maximum shares received for a deposit.
/// @return maxShares The maximum amount of shares that can be minted
function maxMint(address) external view returns (uint256 maxShares) {
    return convertToShares(type(uint104).max);
}
/// @notice Returns the amount of assets that would be deposited to mint a given amount of shares.
/// @param shares The amount of shares to be minted
/// @return assets The amount of assets required to mint `shares`
function previewMint(uint256 shares) public view returns (uint256 assets) {
    // round up depositing assets to avoid protocol loss
    // This prevents minting of shares where the assets provided is rounded down to zero
    assets = Math.mulDivRoundingUp(shares, totalAssets(), totalSupply());
}
/// @notice Deposit required amount of assets to receive specified amount of shares.
/// @dev There is a maximum asset deposit limit of  $2^{104} - 1$ .
/// @dev Shares are minted and sent to the LP (`receiver`).
/// @param shares Amount of shares to be minted
/// @param receiver User to receive the shares
/// @return assets The amount of assets deposited to mint the desired amount of shares
function mint(uint256 shares, address receiver) external payable returns (uint256 assets) {
    _accrueInterest(msg.sender, IS_DEPOSIT);
    assets = previewMint(shares);
    if (assets > type(uint104).max) revert Errors.DepositTooLarge();
    if (assets == 0) revert Errors.BelowMinimumRedemption();
    address _poolManager = address(poolManager());
    if (_poolManager == address(0)) {
        // transfer assets (underlying token funds) from the user/the LP to the PanopticPool
        // in return for the shares to be minted
        SafeTransferLib.safeTransferFrom(
            underlyingToken(),
            msg.sender,
            address(panopticPool()),
            assets
        );
    }
    // mint collateral shares of the Panoptic Pool funds (this ERC20 token)
    _mint(receiver, shares);
}

```

```

    // update tracked asset balance
    s_depositedAssets += uint128(assets);
    if (_poolManager != address(0)) {
        // transfer assets from the user/the LP to the PanopticPool
        // in return for the shares to be minted
        _settleCurrencyDelta(msg.sender, int256(assets));
    }
    emit Deposit(msg.sender, receiver, assets, shares);
}

/// @notice Returns The maximum amount of assets that can be withdrawn for a given user.
/// If the user has any open positions, the max withdrawable balance is zero.
/// @dev Calculated from the balance of the user; limited by the assets the pool has available.
/// @param owner The address being withdrawn for
/// @return maxAssets The maximum amount of assets that can be withdrawn
function maxWithdraw(address owner) public view returns (uint256 maxAssets) {
    uint256 depositedAssets = s_depositedAssets;
    unchecked {
        uint256 available = depositedAssets > 0 ? depositedAssets - 1 : 0;
        uint256 balance = convertToAssets(balanceOf[owner]);
        return panopticPool().numberOfLegs(owner) == 0 ? Math.min(available, balance) : 0;
    }
}

/// @notice Returns The maximum amount of assets that can be withdrawn for a given user with open positions.
/// If the user has any open positions, the max withdrawable balance is zero.
/// @dev Calculated from the balance of the user; limited by the assets the pool has available.
/// @param owner The address being withdrawn for
/// @return maxAssets The maximum amount of assets that can be withdrawn
function _maxWithdrawWithPositions(address owner) internal view returns (uint256 maxAssets) {
    uint256 depositedAssets = s_depositedAssets;
    unchecked {
        uint256 available = depositedAssets > 0 ? depositedAssets - 1 : 0;
        uint256 balance = convertToAssets(balanceOf[owner]);
        return Math.min(available, balance);
    }
}

/// @notice Redeem the amount of shares required to withdraw the specified amount of assets.
/// @dev We can only use this standard 4626 function if the user has no open positions.
/// @dev Shares are burned and assets are sent to the LP (`receiver`).
/// @param assets Amount of assets to be withdrawn
/// @param receiver User to receive the assets
/// @param owner User to burn the shares from
/// @return shares The amount of shares burned to withdraw the desired amount of assets
function withdraw(
    uint256 assets,
    address receiver,
    address owner
) external returns (uint256 shares) {
    _accrueInterest(owner, IS_NOT_DEPOSIT);
    if (assets > maxWithdraw(owner)) revert Errors.ExceedsMaximumRedemption();
    if (assets == 0) revert Errors.BelowMinimumRedemption();
    shares = previewWithdraw(assets);
    // check/update allowance for approved withdraw
    if (msg.sender != owner) {
        uint256 allowed = allowance[owner][msg.sender];
        if (allowed != type(uint256).max) allowance[owner][msg.sender] = allowed - shares; // Saves gas for unlimite...
    }
    // burn collateral shares of the Panoptic Pool funds (this ERC20 token)
    _burn(owner, shares);
    // update tracked asset balance
    // keep checked to prevent underflows
    s_depositedAssets -= uint128(assets);
    address _poolManager = address(poolManager());
    if (_poolManager == address(0)) {
        // transfer assets (underlying token funds) from the PanopticPool to the LP
        SafeTransferLib.safeTransferFrom(
            underlyingToken(),
            address(panopticPool()),
            receiver,
            assets
        );
    } else {
        // transfer assets from the PanopticPool to the LP
        unchecked {
            _settleCurrencyDelta(receiver, -int256(assets));
        }
    }
    emit Withdraw(msg.sender, receiver, owner, assets, shares);
}

```

```

}
/// @notice Redeem the amount of shares required to withdraw the specified amount of assets.
/// @dev Reverts if the account is not solvent with the given `positionIdList`.
/// @dev Shares are burned and assets are sent to the LP (`receiver`).
/// @param assets Amount of assets to be withdrawn
/// @param receiver User to receive the assets
/// @param owner User to burn the shares from
/// @param positionIdList The list of all option positions held by `owner`
/// @param usePremiaAsCollateral Whether to compute accumulated premia for all legs held by the user for collateral ...
/// @return shares The amount of shares burned to withdraw the desired amount of assets
function withdraw(
    uint256 assets,
    address receiver,
    address owner,
    TokenId[] calldata positionIdList,
    bool usePremiaAsCollateral
) external returns (uint256 shares) {
    _accrueInterest(owner, IS_NOT_DEPOSIT);
    if (assets == 0) revert Errors.BelowMinimumRedemption();
    if (assets > _maxWithdrawWithPositions(owner)) revert Errors.ExceedsMaximumRedemption();
    shares = previewWithdraw(assets);
    // check/update allowance for approved withdraw
    if (msg.sender != owner) {
        uint256 allowed = allowance[owner][msg.sender];
        if (allowed != type(uint256).max) allowance[owner][msg.sender] = allowed - shares; // Saves gas for unlimite...
    }
    // burn collateral shares of the Panoptic Pool funds (this ERC20 token)
    _burn(owner, shares);
    // update tracked asset balance
    s_depositedAssets -= uint128(assets);
    // reverts if account is not solvent/eligible to withdraw
    panopticPool().validateCollateralWithdrawable(owner, positionIdList, usePremiaAsCollateral);
    address _poolManager = address(poolManager());
    if (_poolManager == address(0)) {
        // transfer assets (underlying token funds) from the PanopticPool to the LP
        SafeTransferLib.safeTransferFrom(
            underlyingToken(),
            address(panopticPool()),
            receiver,
            assets
        );
    } else {
        // transfer assets from the PanopticPool to the LP
        unchecked {
            _settleCurrencyDelta(receiver, -int256(assets));
        }
    }
    emit Withdraw(msg.sender, receiver, owner, assets, shares);
}

/// @notice Returns the maximum amount of shares that can be redeemed for a given user.
/// @dev If the user has any open positions, the max redeemable balance is zero.
/// @param owner The redeeming address
/// @return maxShares The maximum amount of shares that can be redeemed by `owner`
function maxRedeem(address owner) public view returns (uint256 maxShares) {
    uint256 depositedAssets = s_depositedAssets;
    unchecked {
        uint256 available = convertToShares(depositedAssets > 0 ? depositedAssets - 1 : 0);
        uint256 balance = balanceOf[owner];
        return panopticPool().numberOfLegs(owner) == 0 ? Math.min(available, balance) : 0;
    }
}

/// @notice Returns the amount of assets resulting from a given amount of shares being redeemed.
/// @param shares The amount of shares to be redeemed
/// @return assets The amount of assets resulting from the redemption
function previewRedeem(uint256 shares) public view returns (uint256 assets) {
    return convertToAssets(shares);
}

/// @notice Redeem exact shares for underlying assets.
/// @dev We can only use this standard 4626 function if the user has no open positions.
/// @param shares Amount of shares to be redeemed
/// @param receiver User to receive the assets
/// @param owner User to burn the shares from
/// @return assets The amount of assets resulting from the redemption
function redeem(
    uint256 shares,
    address receiver,
    address owner
) external returns (uint256 assets) {
    _accrueInterest(owner, IS_NOT_DEPOSIT);
    if (shares > maxRedeem(owner)) revert Errors.ExceedsMaximumRedemption();
    // check/update allowance for approved redeem
    if (msg.sender != owner) {
        uint256 allowed = allowance[owner][msg.sender];
        if (allowed != type(uint256).max) allowance[owner][msg.sender] = allowed - shares; // Saves gas for unlimite...
    }
}

```

```

    }
    assets = previewRedeem(shares);
    if (assets == 0) revert Errors.BelowMinimumRedemption();
    // burn collateral shares of the Panoptic Pool funds (this ERC20 token)
    _burn(owner, shares);
    // update tracked asset balance
    // keep checked to avoid underflows
    s_depositedAssets -= uint128(assets);
    address _poolManager = address(poolManager());
    if (_poolManager == address(0)) {
        // transfer assets (underlying token funds) from the PanopticPool to the LP
        SafeTransferLib.safeTransferFrom(
            underlyingToken(),
            address(panopticPool()),
            receiver,
            assets
        );
    } else {
        // transfer assets from the PanopticPool to the LP
        unchecked {
            _settleCurrencyDelta(receiver, -int256(assets));
        }
    }
    emit Withdraw(msg.sender, receiver, owner, assets, shares);
}

/// @notice Donate exact shares to all shareholders.
/// @dev Can only be used when the user has no open positions
/// @param shares Amount of shares to be donated
function donate(uint256 shares) external {
    _accrueInterest(msg.sender, IS_NOT_DEPOSIT);
    if (shares > maxRedeem(msg.sender)) revert Errors.ExceedsMaximumRedemption();
    uint256 assets = previewRedeem(shares);
    if (assets == 0) revert Errors.BelowMinimumRedemption();
    // burn collateral shares of the Panoptic Pool funds (this ERC20 token)
    _burn(msg.sender, shares);
    emit Donate(msg.sender, shares);
}

/// @notice Accrues protocol-wide interest for the calling user
/// @dev Updates global interest state and settles any outstanding interest for msg.sender
function accrueInterest() external {
    _accrueInterest(msg.sender, IS_NOT_DEPOSIT);
}

/// @notice Accrues protocol-wide interest and settles a specific user's interest.
/// @dev This function should be called before any user action that affects their borrow balance.
/// @param owner the account which calls accrue interest
function _accrueInterest(address owner, bool isDeposit) internal {
    uint128 _assetsInAMM = s_assetsInAMM;
    (
        uint128 currentBorrowIndex,
        uint128 _unrealizedGlobalInterest,
        uint256 currentEpoch
    ) = _calculateCurrentInterestState(_assetsInAMM, _updateInterestRate());
    // USER
    LeftRightSigned userState = s_interestState[owner];
    int128 netBorrows = userState.leftSlot();
    int128 userBorrowIndex = int128(currentBorrowIndex);
    if (netBorrows > 0) {
        uint128 userInterestOwed = _getUserInterest(userState, currentBorrowIndex);
        if (userInterestOwed != 0) {
            uint256 _totalAssets;
            unchecked {
                _totalAssets = s_depositedAssets + _assetsInAMM + _unrealizedGlobalInterest;
            }
            uint256 shares = Math.mulDivRoundingUp(
                userInterestOwed,
                totalSupply(),
                _totalAssets
            );
            uint128 burntInterestValue = userInterestOwed;
            address _owner = owner;
            uint256 userBalance = balanceOf[_owner];
            if (shares > userBalance) {
                if (!isDeposit) {
                    // update the accrual of interest paid
                    burntInterestValue = Math
                        .mulDiv(userBalance, _totalAssets, totalSupply())
                        .toUint128();
                    emit InsolvencyPenaltyApplied(
                        owner,
                        userInterestOwed,
                        burntInterestValue,
                        userBalance
                    );
                }
                /// Insolvent case: Pay what you can
            }
        }
    }
}

```

```

        _burn(_owner, userBalance);
        /// @dev DO NOT update index. By keeping the user's old baseIndex, their debt continues to compo...
        userBorrowIndex = userState.rightSlot();
    } else {
        // set interest paid to zero
        burntInterestValue = 0;
        // we effectively **did not settle** this user:
        // we keep their old baseIndex so future interest is computed correctly.
        userBorrowIndex = userState.rightSlot();
    }
} else {
    // Solvent case: Pay in full.
    _burn(_owner, shares);
}
// Due to repeated rounding up when:
// - compounding the global borrow index (multiplicative propagation of rounding error), and
// - converting a user's interest into shares,
// burntInterestValue can exceed _unrealizedGlobalInterest by a few wei (because that accumulator calcul...
// In that case, treat all remaining unrealized interest as consumed
// and clamp the bucket to zero; otherwise subtract normally.
if (burntInterestValue > _unrealizedGlobalInterest) {
    _unrealizedGlobalInterest = 0;
} else {
    unchecked {
        // can never underflow because burntInterestValue <= _unrealizedGlobalInterest
        _unrealizedGlobalInterest = _unrealizedGlobalInterest - burntInterestValue;
    }
}
}
}
s_interestState[owner] = LeftRightSigned
    .wrap(0)
    .addToRightSlot(userBorrowIndex)
    .addToLeftSlot(netBorrows);
s_marketState = MarketStateLibrary.storeMarketState(
    currentBorrowIndex,
    currentEpoch,
    s_marketState.rateAtTarget(),
    _unrealizedGlobalInterest
);
}
/// @notice Calculates the current interest state without modifying storage
/// @dev Simulates interest accrual from last interaction to current epoch
/// @param _assetsInAMM Amount of assets currently deployed in AMM positions
/// @param interestRateSnapshot The current interest rate to evaluate at
/// @return currentBorrowIndex Updated global borrow index after simulated accrual
/// @return _unrealizedGlobalInterest Total unrealized interest including new accrual
/// @return currentEpoch Current epoch = block timestamp / 4
function _calculateCurrentInterestState(
    uint128 _assetsInAMM,
    uint128 interestRateSnapshot
)
internal
view
returns (
    uint128 currentBorrowIndex,
    uint128 _unrealizedGlobalInterest,
    uint256 currentEpoch
)
{
    MarketState accumulator = s_marketState;
    currentEpoch = block.timestamp >> 2;
    uint256 previousEpoch = accumulator.marketEpoch();
    uint128 deltaTime;
    unchecked {
        deltaTime = uint32(currentEpoch - previousEpoch) << 2;
    }
    currentBorrowIndex = accumulator.borrowIndex();
    _unrealizedGlobalInterest = accumulator.unrealizedInterest();
    if (deltaTime > 0) {
        // Calculate interest growth
        uint128 rawInterest = (Math.wTaylorCompounded(interestRateSnapshot, uint128(deltaTime)))
            .toUint128();
        // Calculate interest owed on borrowed amount
        uint128 interestOwed = Math.mulDivWadRoundingUp(_assetsInAMM, rawInterest).toUint128();
        // keep checked to prevent overflows
        _unrealizedGlobalInterest += interestOwed;
        // Update borrow index
        unchecked {
            uint128 _borrowIndex = (WAD + rawInterest).toUint128();
            currentBorrowIndex = Math
                .mulDivWadRoundingUp(currentBorrowIndex, _borrowIndex)
                .toUint128();
        }
    }
}

```

```

    }
}
/*/////////////////////////////////////////////////////////////
                ADAPTIVE INTEREST RATE MODEL
////////////////////////////////////////////////////////////*/
function _interestRateView(uint256 utilization) internal view returns (uint128) {
    uint128 avgRate = riskEngine().interestRate(utilization, s_marketState);
    return avgRate;
}
/// @notice Returns the current interest rate per second based on pool utilization
/// @return The current interest rate per second in WAD (18 decimal precision)
function interestRate() public view returns (uint128) {
    uint128 avgRate = riskEngine().interestRate(_poolUtilizationWadView(), s_marketState);
    return avgRate;
}
/// @notice Returns the interest rate per second based on pool utilization
/// @dev uses the maximum utilization during this transaction, users to prevent flash deposits from lowering the int...
/// @return The interest rate per second in 18 decimal precision
function _updateInterestRate() internal returns (uint128) {
    (uint128 avgRate, uint256 endRateAtTarget) = riskEngine().updateInterestRate(
        _poolUtilizationWad(),
        s_marketState
    );
    s_marketState = s_marketState.updateRateAtTarget(uint40(endRateAtTarget));
    return avgRate;
}
/// @notice Calculates interest owed by a user based on their borrow state
/// @dev Uses the difference between current and user's last borrow index to compute compound interest
/// @param userState Packed state containing user's net borrows (left slot) and last borrow index (right slot)
/// @param currentBorrowIndex The current global borrow index
/// @return interestOwed Amount of interest the user owes, returns 0 if user is a lender or indices match
function _getUserInterest(
    LeftRightSigned userState,
    uint256 currentBorrowIndex
) internal pure returns (uint128 interestOwed) {
    int128 netBorrows = userState.leftSlot();
    uint128 userBorrowIndex = uint128(userState.rightSlot());
    if (netBorrows <= 0 || userBorrowIndex == 0 || currentBorrowIndex == userBorrowIndex) {
        return 0;
    }
    // keep checked to catch currentBorrowIndex < userBorrowIndex
    interestOwed = Math
        .mulDivRoundingUp(
            uint128(netBorrows),
            currentBorrowIndex - userBorrowIndex,
            userBorrowIndex
        )
        .toUint128();
}
/// @notice Returns the current interest owed by a specific user in assets
/// @param owner Address of the user to check
/// @return The amount of interest currently owed by the user in assets
function owedInterest(address owner) external view returns (uint128) {
    return _owedInterest(owner);
}
/// @notice Returns the assets and interest owed for a specific user
/// @param owner Address of the user to check
/// @return The amount of assets owned by the user (in token units)
/// @return The amount of interest currently owed by the user (in token units)
function assetsAndInterest(address owner) external view returns (uint256, uint256) {
    return (convertToAssets(balanceOf[owner]), _owedInterest(owner));
}
/// @notice Internal function to calculate interest owed by a user
/// @dev Retrieves user state and current borrow index from storage
/// @param owner Address of the user to check
/// @return Amount of interest owed based on last compounded index
function _owedInterest(address owner) internal view returns (uint128) {
    LeftRightSigned userState = s_interestState[owner];
    (uint128 currentBorrowIndex, ) = _calculateCurrentInterestState(
        s_assetsInAMM,
        _interestRateView(_poolUtilizationWadView())
    );
    return _getUserInterest(userState, currentBorrowIndex);
}
/// @notice Calculates the current borrow index including un-compounded time
/// @dev Simulates interest accrual up to the current block epoch
/// @return The borrow index as if interest was compounded at current epoch
function _calculateCurrentBorrowIndex() internal view returns (uint256) {
    (uint128 currentBorrowIndex, ) = _calculateCurrentInterestState(
        s_assetsInAMM,
        _interestRateView(_poolUtilizationWadView())
    );
    return currentBorrowIndex;
}
}

```

[illegible]


```

/// @notice Increase the share balance of a user by `2^248 - 1` without updating the total supply.
/// @dev This is controlled by the Panoptic Pool - not individual users.
/// @dev When the user owes more interest than their balance, we reduce the delegation amount
/// by their entire balance. This accounts for the fact that `_accrueInterest` will consume
/// their real shares for interest payment, preventing the delegated virtual shares from
/// being incorrectly used to pay interest obligations.
/// @param delegatee The account to increase the balance of
function delegate(address delegatee) external onlyPanopticPool {
    // Round up to match `_accrueInterest`'s share calculation
    uint256 interestShares = previewWithdraw(_owedInterest(delegatee));
    uint256 balance = balanceOf[delegatee];
    // If user owes more interest than they have, their entire balance will be consumed
    // paying interest. Reduce delegation by this amount so virtual shares aren't used
    // for interest payment.
    uint256 balanceConsumedByInterest = interestShares > balance ? balance : 0;
    // keep checked to catch overflows
    balanceOf[delegatee] += type(uint248).max - balanceConsumedByInterest;
}

/// @notice Decrease the share balance of a user by `2^248 - 1` without updating the total supply.
/// @dev This is controlled by the Panoptic Pool - not individual users.
/// @dev If the user's balance is less than `2^248 - 1` (i.e., some phantom shares were consumed
/// during the delegation period, e.g., by interest payments), their balance is zeroed and
/// `_internalSupply` is increased to compensate for the phantom shares that were incorrectly
/// deducted by `_burn` operations during the delegation period.
/// @param delegatee The account to decrease the balance of
function revoke(address delegatee) external onlyPanopticPool {
    uint256 balance = balanceOf[delegatee];
    if (type(uint248).max > balance) {
        // Phantom shares were consumed during delegation (e.g., burned for interest).
        // This can happen when the user owed more interest than their real balance
        // at the time `delegate()` was called. Zero the balance and restore
        // `_internalSupply` for the overcounted burn.
        balanceOf[delegatee] = 0;
        _internalSupply += type(uint248).max - balance;
    } else {
        // Normal case: user still has all phantom shares plus any real shares
        balanceOf[delegatee] = balance - type(uint248).max;
    }
}

/// @notice Settles liquidation bonus and returns remaining virtual shares to the protocol.
/// @dev This function is where protocol loss is realized, if it exists.
/// @param liquidator The account performing the liquidation of `liquidatee`
/// @param liquidatee The liquidated account to settle
/// @param bonus The liquidation bonus, in assets, to be paid to `liquidator`. May be negative
function settleLiquidation(
    address liquidator,
    address liquidatee,
    int256 bonus
) external payable onlyPanopticPool {
    if (bonus < 0) {
        uint256 bonusAbs;
        unchecked {
            bonusAbs = uint256(-bonus);
        }
        address _poolManager = address(poolManager());
        if (_poolManager == address(0)) {
            uint256 underlyingTokenBalance = ERC20Minimal(underlyingToken()).balanceOf(
                liquidator
            );
            if (underlyingTokenBalance < bonusAbs)
                revert Errors.NotEnoughTokens(
                    underlyingToken(),
                    bonusAbs,
                    underlyingTokenBalance
                );
            SafeTransferLib.safeTransferFrom(
                underlyingToken(),
                liquidator,
                msg.sender,
                bonusAbs
            );
        }
        _mint(liquidatee, convertToShares(bonusAbs));
        s_depositedAssets += uint128(bonusAbs);
        uint256 liquidateeBalance = balanceOf[liquidatee];
        if (type(uint248).max > liquidateeBalance) {
            balanceOf[liquidatee] = 0;
            // keep checked to catch under/overflows
            _internalSupply += type(uint248).max - liquidateeBalance;
        } else {
            // keep checked to catch under/overflows
            balanceOf[liquidatee] = liquidateeBalance - type(uint248).max;
        }
    }
    if (_poolManager != address(0)) {

```

```

        _settleCurrencyDelta(liquidator, int256(bonusAbs));
    }
} else {
    uint256 liquidateeBalance = balanceOf[liquidatee];
    if (type(uint248).max > liquidateeBalance) {
        // keep checked to catch under/overflows
        _internalSupply += type(uint248).max - liquidateeBalance;
        liquidateeBalance = 0;
    } else {
        // keep checked to catch under/overflows
        liquidateeBalance -= type(uint248).max;
    }
    balanceOf[liquidatee] = liquidateeBalance;
    uint256 bonusShares = convertToShares(uint256(bonus));
    // if requested amount is larger than user balance, transfer their balance and mint the remaining shares
    if (bonusShares > liquidateeBalance) {
        _transferFrom(liquidatee, liquidator, liquidateeBalance);
        // this is paying out protocol loss, so correct for that in the amount of shares to be minted
        // X: total assets in vault
        // Y: total supply of shares
        // Z: desired value (assets) of shares to be minted
        // N: total shares corresponding to Z
        // T: transferred shares from liquidatee which are a component of N but do not contribute toward protoco...
        //  $Z = N * X / (Y + N - T)$ 
        //  $Z * (Y + N - T) = N * X$ 
        //  $ZY + ZN - ZT = NX$ 
        //  $ZY - ZT = N(X - Z)$ 
        //  $N = (ZY - ZT) / (X - Z)$ 
        //  $N = Z(Y - T) / (X - Z)$ 
        // subtract delegatee balance from N since it was already transferred to the delegator
        uint256 _totalSupply = totalSupply();
        // keep checked to catch any casting/math errors
        _mint(
            liquidator,
            Math.min(
                Math.mulDivCapped(
                    uint256(bonus),
                    _totalSupply - liquidateeBalance,
                    uint256(Math.max(1, int256(totalAssets()) - bonus))
                ) - liquidateeBalance,
                _totalSupply * DECIMALS
            )
        );
    } else {
        _transferFrom(liquidatee, liquidator, bonusShares);
    }
    // refund liquidator if they attached value expecting to settle a negative bonus in the native currency
    if (msg.value > 0) SafeTransferLib.safeTransferETH(liquidator, msg.value);
}
}

/// @notice Refunds tokens to `refunder` from `refundee`.
/// @dev Assumes that the refunder has enough money to pay for the refund.
/// @param refunder The account refunding tokens to `refundee`
/// @param refundee The account being refunded to
/// @param assets The amount of assets to refund. Positive means a transfer from refunder to refundee, vice versa fo...
function refund(address refunder, address refundee, int256 assets) external onlyPanopticPool {
    if (assets > 0) {
        _transferFrom(refunder, refundee, convertToShares(uint256(assets)));
    } else {
        uint256 sharesToTransfer = convertToShares(uint256(-assets));
        if (balanceOf[refundee] < sharesToTransfer)
            revert Errors.NotEnoughTokens(
                address(this),
                uint256(-assets),
                convertToAssets(balanceOf[refundee])
            );
        _transferFrom(refundee, refunder, sharesToTransfer);
    }
}

}

/*//////////////////////
OPTION EXERCISE AND COMMISSION
//////////////////////*/
/// @notice Internal function to handle all balance and state updates for position creation and closing.
/// @param isCreation A boolean flag to indicate if this is for option creation (true) or closing (false).
/// @param optionOwner The user minting the option
/// @param longAmount The amount of longs
/// @param shortAmount The amount of shorts
/// @param ammDeltaAmount The amount of tokens moved during creation of the option position
///
function _updateBalancesAndSettle(
    address optionOwner,
    bool isCreation,
    int128 longAmount,
    int128 shortAmount,

```

```

    int128 ammDeltaAmount,
    int128 realizedPremium
) internal returns (uint32, int128, uint256, uint256) {
    _accrueInterest(optionOwner, IS_NOT_DEPOSIT);
    /// Snapshot state variables to compute the price per share
    uint256 _totalAssets = totalAssets();
    uint256 _totalSupply = totalSupply();
    int128 netBorrows;
    int256 tokenToPay;
    unchecked {
        // cannot miscast because all values are larger than 0
        netBorrows = isCreation ? shortAmount - longAmount : longAmount - shortAmount;
        tokenToPay = int256(ammDeltaAmount) - netBorrows - realizedPremium;
    }
    {
        // compute creditDelta with the snapshotted values
        uint256 creditDelta;
        if (longAmount > 0) {
            unchecked {
                // cannot miscast because longAmount != 0
                creditDelta = isCreation
                    ? Math.mulDivRoundingUp(
                        uint256(uint128(longAmount)),
                        _totalSupply,
                        _totalAssets
                    )
                    : Math.mulDiv(uint256(uint128(longAmount)), _totalSupply, _totalAssets);
            }
        }
        if (!isCreation) {
            if (creditDelta > 0) {
                // update s_creditedShares: add long amounts == tokens moved into AMM or received when the position ...
                //
                // An underflow is possible because Uniswap rounds long position DOWN when minting and UP when burni...
                // For examples, for a long position, the amount of credited shares at MINT will be lower than the o...
                // which means the s_creditedShares tracker will become negative (the protocol lost ~1 asset worth o...
                // Consequently, those shares must also be burnt, and we're making those shares come out of the opti...
                uint256 _creditedShares = s_creditedShares;
                if (_creditedShares < creditDelta) {
                    s_creditedShares = 0;
                    // add the rounding haircut paid by the option owner at close
                    // rounding up again during conversion potentially add another `1` extra share as ceil*ceil is n...
                    unchecked {
                        // can never miscast because creditDelta > _creditedShares
                        tokenToPay += int256(
                            uint256(
                                Math
                                    .mulDivRoundingUp(
                                        creditDelta - _creditedShares,
                                        _totalAssets,
                                        _totalSupply
                                    )
                                    .toUint128()
                            )
                        );
                    }
                }
            } else {
                // keep unchecked to catch underflows
                s_creditedShares -= creditDelta;
            }
        }
    }
    } else {
        if (creditDelta > 0) {
            // update s_creditedShares: Add long amounts == tokens moved out of AMM or paid when creating credit...
            // keep unchecked to catch overflows
            s_creditedShares += creditDelta;
        }
        // pay commission only when opening a new position, return notional value
    }
}

address _optionOwner = optionOwner;
// Mint/Burn Shares
if (tokenToPay > 0) {
    uint256 sharesToBurn = Math.mulDivRoundingUp(
        uint256(tokenToPay),
        _totalSupply,
        _totalAssets
    );
    if (balanceOf[_optionOwner] < sharesToBurn)
        revert Errors.NotEnoughTokens(
            address(this),
            uint256(tokenToPay),
            convertToAssets(balanceOf[_optionOwner])
        );
};

```

```

        _burn(_optionOwner, sharesToBurn);
    } else if (tokenToPay > 0) {
        uint256 sharesToMint = Math.mulDiv(uint256(-tokenToPay), _totalSupply, _totalAssets);
        _mint(_optionOwner, sharesToMint);
    }
    // Update Pool Assets
    // use current available assets belonging to PLPs (updated after settlement)
    /// @dev realizedPremium is 0 for mints, so can add it here
    // keep checked to prevent under/overflow
    s_depositedAssets = uint256(
        int256(uint256(s_depositedAssets)) - ammDeltaAmount + realizedPremium
    ).toUint128();
    // Update s_assetsInAMM:
    // isCreation: Add short amounts == tokens moved into the AMM or used to create loans
    // !isCreation: remove short amounts == tokens moved out of the AMM or repaid when the position is closed
    // keep checked to catch miscast
    {
        int256 newAssetsInAmm = int256(uint256(s_assetsInAMM));
        newAssetsInAmm += isCreation ? int256(shortAmount) : -int256(shortAmount);
        s_assetsInAMM = uint256(newAssetsInAmm).toUint128();
    }
    {
        // add new netBorrows to the left slot
        s_interestState[_optionOwner] = s_interestState[_optionOwner].addToLeftSlot(netBorrows);
    }
    // get the utilization, store the current one in transient storage
    uint32 utilization = uint32(_poolUtilization());
    return (utilization, int128(tokenToPay), _totalAssets, _totalSupply);
}
/// @notice Take commission and settle ITM amounts on option creation.
/// @param optionOwner The user minting the option
/// @param longAmount The amount of longs
/// @param shortAmount The amount of shorts
/// @param ammDeltaAmount The amount of tokens moved during creation of the option position
/// @param riskParameters The RiskEngine's core parameters
/// @return utilization The final utilization of the collateral vault (in basis points)
/// @return tokenPaid The total amount of tokens paid by the option owner (negative if tokens were received)
function settleMint(
    address optionOwner,
    int128 longAmount,
    int128 shortAmount,
    int128 ammDeltaAmount,
    RiskParameters riskParameters
) external onlyPanopticPool returns (uint32, int128) {
    (
        uint32 utilization,
        int128 tokenPaid,
        uint256 _totalAssets,
        uint256 _totalSupply
    ) = _updateBalancesAndSettle(
        optionOwner,
        true, // isCreation = true
        longAmount,
        shortAmount,
        ammDeltaAmount,
        0 // realizedPremium not used
    );
    {
        uint128 commission = uint256(int256(shortAmount) + int256(longAmount)).toUint128();
        uint128 commissionFee = Math
            .mulDivRoundingUp(commission, riskParameters.notionalFee(), DECIMALS)
            .toUint128();
        uint256 sharesToBurn = Math.mulDivRoundingUp(commissionFee, _totalSupply, _totalAssets);
        if (riskParameters.feeRecipient() == 0) {
            _burn(optionOwner, sharesToBurn);
            emit CommissionPaid(optionOwner, address(0), commissionFee, 0);
        } else {
            unchecked {
                _transferFrom(
                    optionOwner,
                    address(riskEngine()),
                    (sharesToBurn * riskParameters.protocolSplit()) / DECIMALS
                );
                _transferFrom(
                    optionOwner,
                    address(uint160(riskParameters.feeRecipient())),
                    (sharesToBurn * riskParameters.builderSplit()) / DECIMALS
                );
                emit CommissionPaid(
                    optionOwner,
                    address(uint160(riskParameters.feeRecipient())),
                    uint128((commissionFee * riskParameters.protocolSplit()) / DECIMALS),
                    uint128((commissionFee * riskParameters.protocolSplit()) / DECIMALS)
                );
            }
        }
    }
}

```

```

    }
    }
    return (utilization, tokenPaid);
}
/// @notice Exercise an option and pay to the seller what is owed from the buyer.
/// @dev Called when a position is burnt because it may need to be exercised.
/// @param optionOwner The owner of the option being burned
/// @param longAmount The notional value of the long legs of the position (if any)
/// @param shortAmount The notional value of the short legs of the position (if any)
/// @param ammDeltaAmount The amount of tokens moved during the option close
/// @param realizedPremium Premium to settle on the current positions
/// @param riskParameters The RiskEngine's core risk parameters
/// @return The amount of tokens paid when closing that position
function settleBurn(
    address optionOwner,
    int128 longAmount,
    int128 shortAmount,
    int128 ammDeltaAmount,
    int128 realizedPremium,
    RiskParameters riskParameters
) external onlyPanopticPool returns (int128) {
    (, int128 tokenPaid, uint256 _totalAssets, uint256 _totalSupply) = _updateBalancesAndSettle(
        optionOwner,
        false, // isCreation = false
        longAmount,
        shortAmount,
        ammDeltaAmount,
        realizedPremium
    );
    if (realizedPremium != 0) {
        uint128 commissionFee;
        // compute the minimum of the notionalFee and the premiumFee
        {
            uint128 commissionP;
            unchecked {
                commissionP = realizedPremium > 0
                    ? uint128(realizedPremium)
                    : uint128(-realizedPremium);
            }
            uint128 commissionFeeP = Math
                .mulDivRoundingUp(commissionP, riskParameters.premiumFee(), DECIMALS)
                .toUint128();
            uint128 commissionN = uint256(int256(shortAmount) + int256(longAmount)).toUint128();
            uint128 commissionFeeN;
            unchecked {
                commissionFeeN = Math
                    .mulDivRoundingUp(commissionN, 10 * riskParameters.notionalFee(), DECIMALS)
                    .toUint128();
            }
            commissionFee = Math.min(commissionFeeP, commissionFeeN).toUint128();
        }
        uint256 sharesToBurn = Math.mulDivRoundingUp(commissionFee, _totalSupply, _totalAssets);
        if (riskParameters.feeRecipient() == 0) {
            _burn(optionOwner, sharesToBurn);
            emit CommissionPaid(optionOwner, address(0), commissionFee, 0);
        } else {
            unchecked {
                _transferFrom(
                    optionOwner,
                    address(riskEngine()),
                    (sharesToBurn * riskParameters.protocolSplit()) / DECIMALS
                );
                _transferFrom(
                    optionOwner,
                    address(uint160(riskParameters.feeRecipient())),
                    (sharesToBurn * riskParameters.builderSplit()) / DECIMALS
                );
                emit CommissionPaid(
                    optionOwner,
                    address(uint160(riskParameters.feeRecipient())),
                    uint128((commissionFee * riskParameters.protocolSplit()) / DECIMALS),
                    uint128((commissionFee * riskParameters.builderSplit()) / DECIMALS)
                );
            }
        }
    }
    return tokenPaid;
}
}

```

[10/12] contracts/libraries/PanopticMath.sol

```
// SPDX-License-Identifier: BUSL-1.1
pragma solidity ^0.8.24;
// Interfaces
import {IERC20Metadata} from "@openzeppelin/contracts/token/ERC20/extensions/IERC20Metadata.sol";
import {IUniswapV3Pool} from "univ3-core/interfaces/IUniswapV3Pool.sol";
// Libraries
import {Constants} from "@libraries/Constants.sol";
import {Math} from "@libraries/Math.sol";
import {EfficientHash} from "@libraries/EfficientHash.sol";
import {Errors} from "@libraries/Errors.sol";
// OpenZeppelin libraries
import {Strings} from "@openzeppelin/contracts/utils/Strings.sol";
// Custom types
import {LeftRightUnsigned, LeftRightSigned} from "@types/LeftRight.sol";
import {LiquidityChunk} from "@types/LiquidityChunk.sol";
import {TokenId} from "@types/TokenId.sol";
import {RiskParameters} from "@types/RiskParameters.sol";
/// @title Compute general math quantities relevant to Panoptic and AMM pool management.
/// @notice Contains Panoptic-specific helpers and math functions.
/// @author Axicon Labs Limited
library PanopticMath {
    using Math for uint256;
    /// @notice This is equivalent to `type(uint256).max` - used in assembly blocks as a replacement.
    uint256 internal constant MAX_UINT256 = 2 ** 256 - 1;
    /// @notice Masks 16-bit tickSpacing and 8 bits of vegoid out of 64-bit `[16-bit tickspacing][8-bit vegoid][40-bit p...`
    uint64 internal constant TICKSPACING_VEGOID_MASK = 0xFFFFF00000000000;
    uint256 internal constant PRIME_MODULUS_248 =
        0xffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff13;
    uint256 internal constant PRIME_MODULUS_124_0 = 0xfffffffffffffffffffffffffffffc5; // 2**124 - 59
    uint256 internal constant PRIME_MODULUS_124_1 = 0xffffffffffffffffffffffffffffd99; // 2**124 - 615
    // Mask for isolating a 124-bit lane
    uint256 internal constant LANE_MASK_124 = 0xffffffffffffffffffffffffffff;
    uint256 internal constant UPPER_120BITS_MASK =
        0xfffffffffffffffffffffffffffff00000000000000000000000000000000;
    uint256 internal constant BITMASK_UINT88 = 0xffffffffffffffffffffffff;
    uint256 internal constant BITMASK_UINT22 = 0x3fffff;
    /*//////////////////////////////////////////////////////////////
                                UTILITIES
    /*//////////////////////////////////////////////////////////////
    /// @notice Increments the pool pattern (first 48 bits) of a poolId by 1.
    /// @param poolId The 64-bit pool ID
    /// @return The provided `poolId` with its pool pattern slot incremented by 1
    function incrementPoolPattern(uint64 poolId) internal pure returns (uint64) {
        unchecked {
            return (poolId & TICKSPACING_VEGOID_MASK) + (uint40(poolId + 1));
        }
    }
    /// @notice Get the number of leading hex characters in an address.
    //      0x0000bababaab...      0xababababab...
    //      ▲                      ▲
    //      ■                      ■
    //      4 leading hex          0 leading hex
    //      character zeros        character zeros
    //
    /// @param addr The address to get the number of leading zero hex characters for
    /// @return The number of leading zero hex characters in the address
    function numberOfLeadingHexZeros(address addr) external pure returns (uint256) {
        unchecked {
            return addr == address(0) ? 40 : 39 - Math.mostSignificantNibble(uint160(addr));
        }
    }
    /// @notice Returns ERC20 symbol of `token`.
    /// @param token The address of the token to get the symbol of
    /// @return The symbol of `token` or "???" if not supported
    function safeERC20Symbol(address token) external view returns (string memory) {
        // not guaranteed that token supports metadata extension
        // so we need to let call fail and return placeholder if not
        try IERC20Metadata(token).symbol() returns (string memory symbol) {
            return symbol;
        } catch {
            return "???";
        }
    }
    /// @notice Converts `fee` to a string with "bps" appended.
    /// @dev The lowest supported value of `fee` is 1 (`="0.01bps"`).
    /// @param fee The fee to convert to a string (in hundredths of basis points)
    /// @return Stringified version of `fee` with "bps" appended
    function uniswapFeeToString(uint24 fee) internal pure returns (string memory) {
        return
            string.concat(
```

```

        Strings.toString(fee / 100),
        fee % 100 == 0
        ? ""
        : string.concat(
            ".",
            Strings.toString((fee / 10) % 10),
            Strings.toString(fee % 10)
        ),
        "bps"
    );
}

/// @notice Update an existing account's "positions hash" with a new `tokenId`.
/// @notice The positions hash contains a fingerprint of all open positions created by an account/user and a count 0...
/// @dev The "fingerprint" portion of the hash is given by XORing the hashed `tokenId` of each position the user has...
/// @param existingHash The existing position hash representing a list of positions and the count of the legs across...
/// @param tokenId The new position to modify the existing hash with: `existingHash = uint248(existingHash) ^ uint24...
/// @param addFlag Whether to mint (add) the tokenId to the count of positions or burn (subtract) it from the count ...
/// @return newHash The updated position hash with the new tokenId XORed in and the leg count incremented/decremente...
function updatePositionsHash(
    uint256 existingHash,
    TokenId tokenId,
    bool addFlag
) internal pure returns (uint256) {
    // update hash by using the homomorphicHash method
    uint256 updatedHash = homomorphicHash(existingHash, tokenId.unwrap(tokenId), addFlag);
    // increment the upper 8 bits (leg counter) if addFlag=true, decrement otherwise
    uint8 numberOfLegs = uint8(tokenId.countLegs());
    if (numberOfLegs == 0) revert Errors.TokenIdHasZeroLegs();
    // unchecked, so reverts if overflow
    uint256 newLegCount = addFlag
        ? uint8(existingHash >>> 248) + numberOfLegs
        : uint8(existingHash >>> 248) - numberOfLegs;
    unchecked {
        return uint256(updatedHash) + (newLegCount <<< 248);
    }
}

/// @notice Computes a homomorphic hash by adding or subtracting an item from an existing hash
/// @dev Uses XOR-based homomorphic hashing (XHASH). The hash of the item is XORed with the
/// existing hash. Since XOR is its own inverse ( $A \oplus B \oplus B = A$ ), both addition and
/// subtraction operations use the same XOR operation. This ensures the operation is
/// reversible and order-independent for the same set of items.
/// OR
/// Uses additive homomorphic hashing (AdHash) over a 248-bit prime field. The hash of the item
/// is either added to or subtracted from the existing hash using modular arithmetic.
/// Subtraction is implemented as addition of the modular inverse:  $\text{hash} + (\text{PRIME} - \text{itemHash}) \bmod \text{PRIME}$ .
/// This ensures the operation is reversible and order-independent for the same set of items.
/// OR
/// Uses LtHash (Lattice-based Hash) with k=2 lanes for improved collision resistance.
/// The 248-bit hash space is divided into two 124-bit lanes, each operating under
/// modular arithmetic with a 124-bit prime. The item hash is split into two 124-bit
/// chunks and each chunk is added/subtracted from its corresponding lane independently.
/// Subtraction is implemented as addition of the modular inverse:  $\text{lane} + (\text{PRIME} - \text{chunk}) \bmod \text{PRIME}$ .
/// This parallel lane approach provides better security properties than single-lane hashing
/// while maintaining homomorphic properties (order-independence and reversibility).
/// @param hash The existing hash value (only lower 248 bits are used)
/// @param item The item to be hashed and added/subtracted (typically a TokenId cast to uint256)
/// @param addFlag True to add the item to the hash, false to subtract it
/// @return The updated homomorphic hash as a uint256 (but only lower 248 bits contain the hash)
function homomorphicHash(
    uint256 hash,
    uint256 item,
    bool addFlag
) internal pure returns (uint256) {
    /*
    {
        // XHASH
        return
            uint248(hash) ^
            (uint248(uint256(EfficientHash.efficientKeccak256(abi.encode(item)))));
    }
    {
        // AdHash
        uint256 itemHash = uint256(EfficientHash.efficientKeccak256(abi.encode(item)));
        return
            addFlag
            ? addmod(uint248(hash), uint248(itemHash), PRIME_MODULUS_248)
            : addmod(
                uint248(hash),
                PRIME_MODULUS_248 - (itemHash % PRIME_MODULUS_248),
                PRIME_MODULUS_248
            );
    }
    */
    unchecked {

```

```

    // LtHash, k=2
    uint256 itemHash = uint256(EfficientHash.efficientKeccak256(abi.encode(item)));
    // Pre-calculate the 124-bit chunks for the item to be added/removed
    uint256 item_h0 = itemHash & LANE_MASK_124;
    uint256 item_h1 = (itemHash >> 124) & LANE_MASK_124;
    uint256 lane0 = hash & LANE_MASK_124;
    uint256 newItem_h0 = addFlag
        ? item_h0
          : PRIME_MODULUS_124_0 - (item_h0 % PRIME_MODULUS_124_0);
    uint256 hash0 = addmod(lane0, newItem_h0, PRIME_MODULUS_124_0);
    uint256 lane1 = (hash >> 124) & LANE_MASK_124;
    uint256 newItem_h1 = addFlag
        ? item_h1
          : PRIME_MODULUS_124_1 - (item_h1 % PRIME_MODULUS_124_1);
    uint256 hash1 = addmod(lane1, newItem_h1, PRIME_MODULUS_124_1);
    return hash0 + (hash1 <<< 124);
}
}
/// @notice Checks if an array of TokenIds contains any duplicate values
/// @dev Uses assembly for gas optimization. Performs O(n^2) comparison by checking each element
///      against all subsequent elements. Returns false immediately upon finding the first duplicate.
///      Arrays with 0 or 1 elements are considered to have no duplicates.
/// @param arr The array of TokenIds to check for duplicates
/// @return True if the array contains no duplicate TokenIds, false if duplicates are found
function hasNoDuplicateTokenIds(TokenId[] calldata arr) external pure returns (bool) {
    assembly {
        let len := arr.length
        let offset := arr.offset
        // Early return for 0 or 1 elements
        if lt(len, 2) {
            mstore(0x00, 1)
            return(0x00, 0x20)
        }
        // Check for duplicates
        for {
            let i := 0
        } lt(i, len) {
            i := add(i, 1)
        } {
            let val := calldataload(add(offset, mul(i, 0x20)))
            for {
                let j := add(i, 1)
            } lt(j, len) {
                j := add(j, 1)
            } {
                if eq(val, calldataload(add(offset, mul(j, 0x20)))) {
                    mstore(0x00, 0)
                    return(0x00, 0x20)
                }
            }
        }
        mstore(0x00, 1)
        return(0x00, 0x20)
    }
}
}
/////////////////////////////////////////////////////////////
ORACLE CALCULATIONS
/////////////////////////////////////////////////////////////
/// @notice Returns the median of the last `cardinality` average prices over `period` observations from `univ3pool`.
/// @dev Used when we need a manipulation-resistant TWAP price.
/// @dev Uniswap observations snapshot the closing price of the last block before the first interaction of a given b...
/// @dev The maximum frequency of observations is 1 per block, but there is no guarantee that the pool will be obser...
/// @dev Each period has a minimum length of `blocktime * period`, but may be longer if the Uniswap pool is relative...
/// @dev The final price used in the array (of length `cardinality`) is the average of `cardinality` observations sp...
/// @dev Thus, the minimum total time window is `cardinality * period * blocktime`.
/// @param univ3pool The Uniswap pool to get the median observation from
/// @param observationIndex The index of the last observation in the pool
/// @param observationCardinality The number of observations in the pool
/// @param cardinality The number of `periods` to in the median price array, should be odd
/// @param period The number of observations to average to compute one entry in the median price array
/// @return The median of `cardinality` observations spaced by `period` in the Uniswap pool
/// @return The latest observation in the Uniswap pool
function computeMedianObservedPrice(
    IUniswapV3Pool univ3pool,
    uint256 observationIndex,
    uint256 observationCardinality,
    uint256 cardinality,
    uint256 period
) internal view returns (int24, int24) {
    unchecked {
        int256[] memory tickCumulatives = new int256[](cardinality + 1);
        uint256[] memory timestamps = new uint256[](cardinality + 1);
        // get the last "cardinality" timestamps/tickCumulatives (if observationIndex < cardinality, the index wi...
        for (uint256 i = 0; i < cardinality + 1; ++i) {

```



```
(timestamps[i], tickCumulatives[i], , ) = univ3pool.observations(
    uint256(
        (int256(observationIndex) - int256(i * period)) +
        int256(observationCardinality)
    ) % observationCardinality
);
}
int256[] memory ticks = new int256[](cardinality);
// use cardinality periods given by cardinality + 1 accumulator observations to compute the last cardinality...
for (uint256 i = 0; i <&t; cardinality; ++i) {
    ticks[i] =
        (tickCumulatives[i] - tickCumulatives[i + 1]) /
        int256(timestamps[i] - timestamps[i + 1]);
}
// the `ticks` array descends from the most recent Uniswap observation prior to the sort
int24 latestTick = int24(ticks[0]);
// get the median of the `ticks` array (assuming `cardinality` is odd)
return (int24(Math.sort(ticks)[cardinality / 2]), latestTick);
}
}

/// @notice Computes the TWAP of a Uniswap V3 pool using data from its oracle.
/// @dev Note that our definition of TWAP differs from a typical mean of prices over a time window.
/// @dev We instead observe the average price over a series of time intervals, and define the TWAP as the median of ...
/// @param univ3pool The Uniswap pool from which to compute the TWAP
/// @param twapWindow The time window to compute the TWAP over
/// @return The final calculated TWAP tick
function twapFilter(IUniswapV3Pool univ3pool, uint32 twapWindow) external view returns (int24) {
    uint32[] memory secondsAgos = new uint32[](20);
    int256[] memory twapMeasurement = new int256[](19);
    unchecked {
        // construct the time slots
        for (uint256 i = 0; i <&t; 20; ++i) {
            secondsAgos[i] = uint32(((i + 1) * twapWindow) / 20);
        }
        // observe the tickCumulative at the 20 pre-defined time slots
        (int56[] memory tickCumulatives, ) = univ3pool.observe(secondsAgos);
        // compute the average tick per 30s window
        for (uint256 i = 0; i <&t; 19; ++i) {
            twapMeasurement[i] = int24(
                (tickCumulatives[i] - tickCumulatives[i + 1]) / int56(uint56(twapWindow) / 20))
        );
        // sort the tick measurements
        int256[] memory sortedTicks = Math.sort(twapMeasurement);
        // Get the median value
        return int24(sortedTicks[9]);
    }
}

}

/*//////////////////////
LIQUIDITY CHUNK MATH
////////////////////////////////*/
////////////////////////////////////////*//
/// @notice For a given option position (`tokenId`), leg index within that position (`legIndex`), and `positionSize`...
/// liquidity (share ownership) in the Uniswap V3 pool; this is a liquidity chunk.
/// Liquidity chunk (defined by tick upper, tick lower, and its size/amount: the liquidity)
/// liquidity
/// ▲
/// ■
/// ■ ▼▼
/// ■■■■■■■■
/// ■ ■
/// ■ ■
/// ■■■■■■■■■■■■■■■■■ price
/// Uniswap V3 Pool
/// @param tokenId The option position id
/// @param legIndex The leg index of the option position, can be {0,1,2,3}
/// @param positionSize The number of contracts held by this leg
/// @return A LiquidityChunk with `tickLower`, `tickUpper`, and `liquidity`
function getLiquidityChunk(
    TokenId tokenId,
    uint256 legIndex,
    uint128 positionSize
) internal pure returns (LiquidityChunk) {
    // get the tick range for this leg
    (int24 tickLower, int24 tickUpper) = tokenId.asTicks(legIndex);
    // Get the amount of liquidity owned by this leg in the Uniswap V3 pool in the above tick range
    // Background:
    // In Uniswap V3, the amount of liquidity received for a given amount of token0 when the price is
    // not in range is given by:
    // Liquidity = amount0 * (sqrt(upper) * sqrt(lower)) / (sqrt(upper) - sqrt(lower))
    // For token1, it is given by:
    // Liquidity = amount1 / (sqrt(upper) - sqrt(lower))
    // However, in Panoptic, each position has a asset parameter. The asset is the "basis" of the position.
    // In TradFi, the asset is always cash and selling a $1000 put requires the user to lock $1000, and selling
```

```

// a call requires the user to lock 1 unit of asset.
//
// Because Uniswap V3 chooses token0 and token1 from the alphanumeric order, there is no consistency as to whet...
// stablecoin, ETH, or an ERC20. Some pools may want ETH to be the asset (e.g. ETH-DAI) and some may wish the s...
// be the asset (e.g. DAI-ETH) so that K asset is moved for puts and 1 asset is moved for calls.
// But since the convention is to force the order always we have no say in this.
//
// To solve this, we encode the asset value in tokenId. This parameter specifies which of token0 or token1 is t...
// asset, such that:
//   when asset=0, then amount0 moved at strike K = 1.0001**currentTick is 1, amount1 moved to strike K is K
//   when asset=1, then amount1 moved at strike K = 1.0001**currentTick is K, amount0 moved to strike K is 1/K
//
// The following function takes this into account when computing the liquidity of the leg and switches between
// the definition for getLiquidityForAmount0 or getLiquidityForAmount1 when relevant.
uint256 amount = positionSize * tokenId.optionRatio(legIndex);
if (tokenId.asset(legIndex) == 0) {
    return Math.getLiquidityForAmount0(tickLower, tickUpper, amount);
} else {
    return Math.getLiquidityForAmount1(tickLower, tickUpper, amount);
}
}

/// @notice Extract the tick range specified by `strike` and `width` for the given `tickSpacing`.
/// @param strike The strike price of the option
/// @param width The width of the option
/// @param tickSpacing The tick spacing of the underlying Uniswap V3 pool
/// @return The lower tick of the liquidity chunk
/// @return The upper tick of the liquidity chunk
function getTicks(
    int24 strike,
    int24 width,
    int24 tickSpacing
) internal pure returns (int24, int24) {
    (int24 rangeDown, int24 rangeUp) = PanopticMath.getRangesFromStrike(width, tickSpacing);
    unchecked {
        return (strike - rangeDown, strike + rangeUp);
    }
}

/// @notice Returns the distances of the upper and lower ticks from the strike for a position with the given width a...
/// @dev Given `r = (width * tickSpacing) / 2`, `tickLower = strike - floor(r)` and `tickUpper = strike + ceil(r)`.
/// @param width The width of the leg
/// @param tickSpacing The tick spacing of the underlying pool
/// @return The distance of the lower tick from the strike
/// @return The distance of the upper tick from the strike
function getRangesFromStrike(
    int24 width,
    int24 tickSpacing
) internal pure returns (int24, int24) {
    return (
        (width * tickSpacing) / 2,
        int24(int256(Math.unsafeDivRoundingUp(uint24(width) * uint24(tickSpacing), 2)))
    );
}

/// @notice Computes the chunk key for a given leg of a position.
/// @dev The chunk key uniquely identifies a liquidity chunk by its strike, width, and token type.
/// @param tokenId The option position
/// @param leg The leg index within the position
/// @return chunkKey The keccak256 hash identifying this chunk
function getChunkKey(TokenId tokenId, uint256 leg) internal pure returns (bytes32 chunkKey) {
    chunkKey = EfficientHash.efficientKeccak256(
        abi.encodePacked(tokenId.strike(leg), tokenId.width(leg), tokenId.tokenType(leg))
    );
}

}

//////////////////////////////////////////////////
// TOKEN CONVERSION LOGIC
//////////////////////////////////////////////////

/// @notice Compute the amount of notional value underlying an option position.
/// @param tokenId The option position id
/// @param positionSize The number of contracts of the option
/// @param opening Whether you need the token0s and token1s moved while opening the position, or while closing
/// @return longAmounts Left-right packed word where rightSlot = token0 and leftSlot = token1 held against borrowed ...
/// @return shortAmounts Left-right packed word where where rightSlot = token0 and leftSlot = token1 borrowed to cre...
function computeExercisedAmounts(
    TokenId tokenId,
    uint128 positionSize,
    bool opening
) internal pure returns (LeftRightSigned longAmounts, LeftRightSigned shortAmounts) {
    uint256 numLegs = tokenId.countLegs();
    for (uint256 leg = 0; leg < numLegs; ) {
        (LeftRightSigned longs, LeftRightSigned shorts) = calculateIOAmounts(
            tokenId,
            positionSize,
            leg,
            opening
        );
    }
}

```

```

        longAmounts = longAmounts.add(longs);
        shortAmounts = shortAmounts.add(shorts);
        unchecked {
            ++leg;
        }
    }
}

/// @notice Convert an amount of token0 into an amount of token1 given the sqrtPriceX96 in a Uniswap pool defined as...
/// @dev Uses reduced precision after tick 443636 in order to accommodate the full range of ticks
/// @param amount The amount of token0 to convert into token1
/// @param sqrtPriceX96 The square root of the price at which to convert `amount` of token0 into token1
/// @return The converted `amount` of token0 represented in terms of token1
function convert0to1(uint256 amount, uint160 sqrtPriceX96) internal pure returns (uint256) {
    unchecked {
        // the tick 443636 is the maximum price where (price) * 2**192 fits into a uint256 (< 2**256-1)
        // above that tick, we are forced to reduce the amount of decimals in the final price by 2**64 to 2**128
        if (sqrtPriceX96 < type(uint128).max) {
            return Math.mulDiv192(amount, uint256(sqrtPriceX96) ** 2);
        } else {
            return Math.mulDiv128(amount, Math.mulDiv64(sqrtPriceX96, sqrtPriceX96));
        }
    }
}

/// @notice Convert an amount of token0 into an amount of token1 given the sqrtPriceX96 in a Uniswap pool defined as...
/// @dev Uses reduced precision after tick 443636 in order to accommodate the full range of ticks
/// @param amount The amount of token0 to convert into token1
/// @param sqrtPriceX96 The square root of the price at which to convert `amount` of token0 into token1
/// @return The converted `amount` of token0 represented in terms of token1
function convert0to1RoundingUp(
    uint256 amount,
    uint160 sqrtPriceX96
) internal pure returns (uint256) {
    unchecked {
        // the tick 443636 is the maximum price where (price) * 2**192 fits into a uint256 (< 2**256-1)
        // above that tick, we are forced to reduce the amount of decimals in the final price by 2**64 to 2**128
        if (sqrtPriceX96 < type(uint128).max) {
            return Math.mulDiv192RoundingUp(amount, uint256(sqrtPriceX96) ** 2);
        } else {
            return Math.mulDiv128RoundingUp(amount, Math.mulDiv64(sqrtPriceX96, sqrtPriceX96));
        }
    }
}

/// @notice Convert an amount of token1 into an amount of token0 given the sqrtPriceX96 in a Uniswap pool defined as...
/// @dev Uses reduced precision after tick 443636 in order to accommodate the full range of ticks.
/// @param amount The amount of token1 to convert into token0
/// @param sqrtPriceX96 The square root of the price at which to convert `amount` of token1 into token0
/// @return The converted `amount` of token1 represented in terms of token0
function convert1to0(uint256 amount, uint160 sqrtPriceX96) internal pure returns (uint256) {
    unchecked {
        // the tick 443636 is the maximum price where (price) * 2**192 fits into a uint256 (< 2**256-1)
        // above that tick, we are forced to reduce the amount of decimals in the final price by 2**64 to 2**128
        if (sqrtPriceX96 < type(uint128).max) {
            return Math.mulDiv(amount, 2 ** 192, uint256(sqrtPriceX96) ** 2);
        } else {
            return Math.mulDiv(amount, 2 ** 128, Math.mulDiv64(sqrtPriceX96, sqrtPriceX96));
        }
    }
}

/// @notice Convert an amount of token1 into an amount of token0 given the sqrtPriceX96 in a Uniswap pool defined as...
/// @dev Uses reduced precision after tick 443636 in order to accommodate the full range of ticks.
/// @param amount The amount of token1 to convert into token0
/// @param sqrtPriceX96 The square root of the price at which to convert `amount` of token1 into token0
/// @return The converted `amount` of token1 represented in terms of token0
function convert1to0RoundingUp(
    uint256 amount,
    uint160 sqrtPriceX96
) internal pure returns (uint256) {
    unchecked {
        // the tick 443636 is the maximum price where (price) * 2**192 fits into a uint256 (< 2**256-1)
        // above that tick, we are forced to reduce the amount of decimals in the final price by 2**64 to 2**128
        if (sqrtPriceX96 < type(uint128).max) {
            return Math.mulDivRoundingUp(amount, 2 ** 192, uint256(sqrtPriceX96) ** 2);
        } else {
            return
                Math.mulDivRoundingUp(
                    amount,
                    2 ** 128,
                    Math.mulDiv64(sqrtPriceX96, sqrtPriceX96)
                );
        }
    }
}

/// @notice Convert an amount of token0 into an amount of token1 given the sqrtPriceX96 in a Uniswap pool defined as...
/// @dev Uses reduced precision after tick 443636 in order to accommodate the full range of ticks.

```

```

/// @param amount The amount of token0 to convert into token1
/// @param sqrtPriceX96 The square root of the price at which to convert `amount` of token0 into token1
/// @return The converted `amount` of token0 represented in terms of token1
function convert0to1(int256 amount, uint160 sqrtPriceX96) internal pure returns (int256) {
    unchecked {
        // the tick 443636 is the maximum price where (price) * 2**192 fits into a uint256 (< 2**256-1)
        // above that tick, we are forced to reduce the amount of decimals in the final price by 2**64 to 2**128
        if (sqrtPriceX96 <= type(uint128).max) {
            int256 absResult = Math
                .mulDiv192(Math.absUint(amount), uint256(sqrtPriceX96) ** 2)
                .toInt256();
            return amount < 0 ? -absResult : absResult;
        } else {
            int256 absResult = Math
                .mulDiv128(Math.absUint(amount), Math.mulDiv64(sqrtPriceX96, sqrtPriceX96))
                .toInt256();
            return amount < 0 ? -absResult : absResult;
        }
    }
}

/// @notice Convert an amount of token0 into an amount of token1 given the sqrtPriceX96 in a Uniswap pool defined as...
/// @dev Uses reduced precision after tick 443636 in order to accommodate the full range of ticks.
/// @param amount The amount of token0 to convert into token1
/// @param sqrtPriceX96 The square root of the price at which to convert `amount` of token0 into token1
/// @return The converted `amount` of token0 represented in terms of token1
function convert0to1RoundingUp(
    int256 amount,
    uint160 sqrtPriceX96
) internal pure returns (int256) {
    unchecked {
        // the tick 443636 is the maximum price where (price) * 2**192 fits into a uint256 (< 2**256-1)
        // above that tick, we are forced to reduce the amount of decimals in the final price by 2**64 to 2**128
        if (sqrtPriceX96 <= type(uint128).max) {
            int256 absResult = Math
                .mulDiv192RoundingUp(Math.absUint(amount), uint256(sqrtPriceX96) ** 2)
                .toInt256();
            return amount < 0 ? -absResult : absResult;
        } else {
            int256 absResult = Math
                .mulDiv128RoundingUp(
                    Math.absUint(amount),
                    Math.mulDiv64(sqrtPriceX96, sqrtPriceX96)
                )
                .toInt256();
            return amount < 0 ? -absResult : absResult;
        }
    }
}

/// @notice Convert an amount of token1 into an amount of token0 given the sqrtPriceX96 in a Uniswap pool defined as...
/// @dev Uses reduced precision after tick 443636 in order to accommodate the full range of ticks.
/// @param amount The amount of token1 to convert into token0
/// @param sqrtPriceX96 The square root of the price at which to convert `amount` of token1 into token0
/// @return The converted `amount` of token1 represented in terms of token0
function convert1to0(int256 amount, uint160 sqrtPriceX96) internal pure returns (int256) {
    unchecked {
        // the tick 443636 is the maximum price where (price) * 2**192 fits into a uint256 (< 2**256-1)
        // above that tick, we are forced to reduce the amount of decimals in the final price by 2**64 to 2**128
        if (sqrtPriceX96 <= type(uint128).max) {
            int256 absResult = Math
                .mulDiv(Math.absUint(amount), 2 ** 192, uint256(sqrtPriceX96) ** 2)
                .toInt256();
            return amount < 0 ? -absResult : absResult;
        } else {
            int256 absResult = Math
                .mulDiv(
                    Math.absUint(amount),
                    2 ** 128,
                    Math.mulDiv64(sqrtPriceX96, sqrtPriceX96)
                )
                .toInt256();
            return amount < 0 ? -absResult : absResult;
        }
    }
}

/// @notice Convert an amount of token1 into an amount of token0 given the sqrtPriceX96 in a Uniswap pool defined as...
/// @dev Uses reduced precision after tick 443636 in order to accommodate the full range of ticks.
/// @param amount The amount of token1 to convert into token0
/// @param sqrtPriceX96 The square root of the price at which to convert `amount` of token1 into token0
/// @return The converted `amount` of token1 represented in terms of token0
function convert1to0RoundingUp(
    int256 amount,
    uint160 sqrtPriceX96
) internal pure returns (int256) {
    unchecked {

```

```

// the tick 443636 is the maximum price where (price) * 2**192 fits into a uint256 (< 2**256-1)
// above that tick, we are forced to reduce the amount of decimals in the final price by 2**64 to 2**128
if (sqrtPriceX96 < type(uint128).max) {
    int256 absResult = Math
        .mulDivRoundingUp(Math.absUint(amount), 2 ** 192, uint256(sqrtPriceX96) ** 2)
        .toInt256();
    return amount < 0 ? -absResult : absResult;
} else {
    int256 absResult = Math
        .mulDivRoundingUp(
            Math.absUint(amount),
            2 ** 128,
            Math.mulDiv64(sqrtPriceX96, sqrtPriceX96)
        )
        .toInt256();
    return amount < 0 ? -absResult : absResult;
}
}

/// @notice Get a single collateral balance and requirement in terms of the lowest-priced token for a given set of (...)
/// @param tokenData0 LeftRight encoded word with balance of token0 in the right slot, and required balance in left ...
/// @param tokenData1 LeftRight encoded word with balance of token1 in the right slot, and required balance in left ...
/// @param sqrtPriceX96 The price at which to compute the collateral value and requirements
/// @return The combined collateral balance of `tokenData0` and `tokenData1` in terms of (token0 if `price(token1/to...
/// @return The combined required collateral threshold of `tokenData0` and `tokenData1` in terms of (token0 if `pric...
function getCrossBalances(
    LeftRightUnsigned tokenData0,
    LeftRightUnsigned tokenData1,
    uint160 sqrtPriceX96
) internal pure returns (uint256, uint256) {
    // convert values to the highest precision (lowest price) of the two tokens (token0 if price token1/token0 < ...
    if (sqrtPriceX96 < Constants.FP96) {
        return (
            tokenData0.rightSlot() +
            PanopticMath.convert1to0(tokenData1.rightSlot(), sqrtPriceX96),
            tokenData0.leftSlot() +
            PanopticMath.convert1to0RoundingUp(tokenData1.leftSlot(), sqrtPriceX96)
        );
    }
    return (
        PanopticMath.convert0to1(tokenData0.rightSlot(), sqrtPriceX96) + tokenData1.rightSlot(),
        PanopticMath.convert0to1RoundingUp(tokenData0.leftSlot(), sqrtPriceX96) +
        tokenData1.leftSlot()
    );
}

/// @notice Compute the notional value (for `tokenType = 0` and `tokenType = 1`) represented by a given leg in an op...
/// @param tokenId The option position identifier
/// @param positionSize The number of option contracts held in this position (each contract can control multiple tok...
/// @param legIndex The leg index of the option contract, can be {0,1,2,3}
/// @param opening Whether this position is being opened or closed
/// @return A LeftRight encoded variable containing the amount0 and the amount1 value controlled by this option posi...
function getAmountsMoved(
    TokenId tokenId,
    uint128 positionSize,
    uint256 legIndex,
    bool opening
) internal pure returns (LeftRightUnsigned) {
    uint128 amount0;
    uint128 amount1;
    bool hasWidth = tokenId.width(legIndex) != 0;
    // if the width is zero, add 1 to the width to allow liquidity amounts to be computes
    /// @dev this is just for accounting purposes, the actual tokenId will remain with a width = 0
    if (!hasWidth) {
        tokenId = tokenId.addWidth(2, legIndex);
    }
    LiquidityChunk liquidityChunk = getLiquidityChunk(tokenId, legIndex, positionSize);
    // Shorts round UP to ensure user pays enough (conservative for protocol)
    // Longs round DOWN to ensure user receives correct amount (conservative for protocol)
    if (
        (tokenId.isLong(legIndex) == 0 && opening) ||
        (tokenId.isLong(legIndex) != 0 && !opening) ||
        !hasWidth
    ) {
        amount0 = uint128(Math.getAmount0ForLiquidityUp(liquidityChunk));
        amount1 = uint128(Math.getAmount1ForLiquidityUp(liquidityChunk));
    } else {
        amount0 = uint128(Math.getAmount0ForLiquidity(liquidityChunk));
        amount1 = uint128(Math.getAmount1ForLiquidity(liquidityChunk));
    }
    return LeftRightUnsigned.wrap(amount0).addToLeftSlot(amount1);
}

/// @notice Compute the amount of funds that are moved to or removed from the Panoptic Pool when `tokenId` is create...
/// @param tokenId The option position identifier
/// @param positionSize The number of positions minted

```

```

/// @param legIndex The leg index minted in this position, can be {0,1,2,3}
/// @param opening Whether this position is being opened or closed
/// @return longs A LeftRight-packed word containing the total amount of long positions
/// @return shorts A LeftRight-packed word containing the amount of short positions
function calculateIOAmounts(
    TokenId tokenId,
    uint128 positionSize,
    uint256 legIndex,
    bool opening
) internal pure returns (LeftRightSigned longs, LeftRightSigned shorts) {
    LeftRightUnsigned amountsMoved = getAmountsMoved(tokenId, positionSize, legIndex, opening);
    bool isShort = tokenId.isLong(legIndex) == 0;
    if (tokenId.tokenType(legIndex) == 0) {
        if (isShort) {
            // if option is short, increment shorts by contracts
            shorts = LeftRightSigned.wrap(0).addToRightSlot(
                Math.toInt128(amountsMoved.rightSlot())
            );
        } else {
            // is option is long, increment longs by contracts
            longs = LeftRightSigned.wrap(0).addToRightSlot(
                Math.toInt128(amountsMoved.rightSlot())
            );
        }
    } else {
        if (isShort) {
            // if option is short, increment shorts by notional
            shorts = LeftRightSigned.wrap(0).addToLeftSlot(
                Math.toInt128(amountsMoved.leftSlot())
            );
        } else {
            // if option is long, increment longs by notional
            longs = LeftRightSigned.wrap(0).addToLeftSlot(
                Math.toInt128(amountsMoved.leftSlot())
            );
        }
    }
}
}
}

```

[11/12] contracts/libraries/Math.sol

```

// SPDX-License-Identifier: GPL-2.0-or-later
pragma solidity ^0.8.24;
// Libraries
import {Errors} from "@libraries/Errors.sol";
import {Constants} from "@libraries/Constants.sol";
import {FixedPointMathLib} from "solady/utils/FixedPointMathLib.sol";
// Custom types
import {LiquidityChunk, LiquidityChunkLibrary} from "@types/LiquidityChunk.sol";
/// @title Core math library.
/// @author Axicon Labs Limited
/// @notice Contains general math helpers and functions
library Math {
    /// @notice This is equivalent to `type(uint256).max` – used in assembly blocks as a replacement.
    uint256 internal constant MAX_UINT256 = 2 ** 256 - 1;
    uint256 constant WAD = 1e18;
    int256 constant WAD_INT = int256(1e18);
    /// @dev ln(2).
    int256 internal constant LN_2_INT = 0.693147180559945309 ether;
    /// @dev ln(1e-18).
    int256 internal constant LN_WEI_INT = -41.446531673892822312 ether;
    /// @dev Above this bound, `wExp` is clipped to avoid overflowing when multiplied with 1 ether.
    /// @dev This upper bound corresponds to: ln(type(int256).max / 1e36) (scaled by WAD, floored).
    int256 internal constant WEXP_UPPER_BOUND = 93.859467695000404319 ether;
    /// @dev The value of wExp(`WEXP_UPPER_BOUND`).
    int256 internal constant WEXP_UPPER_VALUE =
        57716089161558943949701069502944508345128.422502756744429568 ether;
    /*//////////////////////////////////////////////////////////////////
                                GENERAL MATH HELPERS
    //////////////////////////////////////////////////////////////////////////*/
    /// @notice Compute the min of the incoming int24s `a` and `b`.
    /// @param a The first number
    /// @param b The second number
    /// @return The min of `a` and `b`: min(a, b), e.g.: min(4, 1) = 1
    function min24(int24 a, int24 b) internal pure returns (int24) {
        return a < b ? a : b;
    }
    /// @notice Compute the max of the incoming int24s `a` and `b`.
    /// @param a The first number

```

```

/// @param b The second number
/// @return The max of `a` and `b`: max(a, b), e.g.: max(4, 1) = 4
function max24(uint24 a, uint24 b) internal pure returns (uint24) {
    return a > b ? a : b;
}

/// @notice Compute the min of the incoming `a` and `b`.
/// @param a The first number
/// @param b The second number
/// @return The min of `a` and `b`: min(a, b), e.g.: min(4, 1) = 1
function min(uint256 a, uint256 b) internal pure returns (uint256) {
    return a < b ? a : b;
}

/// @notice Compute the min of the incoming `a` and `b`.
/// @param a The first number
/// @param b The second number
/// @return The min of `a` and `b`: min(a, b), e.g.: min(4, 1) = 1
function min(int256 a, int256 b) internal pure returns (int256) {
    return a < b ? a : b;
}

/// @notice Compute the max of the incoming `a` and `b`.
/// @param a The first number
/// @param b The second number
/// @return The max of `a` and `b`: max(a, b), e.g.: max(4, 1) = 4
function max(uint256 a, uint256 b) internal pure returns (uint256) {
    return a > b ? a : b;
}

/// @notice Compute the max of the incoming `a` and `b`.
/// @param a The first number
/// @param b The second number
/// @return The max of `a` and `b`: max(a, b), e.g.: max(4, 1) = 4
function max(int256 a, int256 b) internal pure returns (int256) {
    return a > b ? a : b;
}

/// @notice Compute the absolute value of an integer.
/// @param x The incoming *signed* integer to take the absolute value of
/// @dev Does not support `type(int256).min` and will revert (`type(int256).max = abs(type(int256).min) - 1`).
/// @return The absolute value of `x`, e.g. abs(-4) = 4
function abs(int256 x) internal pure returns (int256) {
    return x > 0 ? x : -x;
}

/// @notice Compute the absolute value of an integer.
/// @param x The incoming *signed* integer to take the absolute value of
/// @dev Supports `type(int256).min` because the corresponding value can fit in a uint (unlike `type(int256).max`).
/// @return The absolute value of `x`, e.g. abs(-4) = 4
function absUint(uint256 x) internal pure returns (uint256) {
    unchecked {
        return x > 0 ? uint256(x) : uint256(-x);
    }
}

/// @notice Returns the index of the most significant nibble of the 160-bit number,
/// where the least significant nibble is at index 0 and the most significant nibble is at index 39.
/// @param x The value for which to compute the most significant nibble
/// @return r The index of the most significant nibble (default: 0)
function mostSignificantNibble(uint160 x) internal pure returns (uint256 r) {
    unchecked {
        if (x >= 0x100000000000000000000000000000000) {
            x >>= 128;
            r += 32;
        }
        if (x >= 0x1000000000000000000000000) {
            x >>= 64;
            r += 16;
        }
        if (x >= 0x100000000000000000000000) {
            x >>= 32;
            r += 8;
        }
        if (x >= 0x10000000000000000000000) {
            x >>= 16;
            r += 4;
        }
        if (x >= 0x1000000000000000000000) {
            x >>= 8;
            r += 2;
        }
        if (x >= 0x100000000000000000000) {
            x >>= 4;
            r += 1;
        }
    }
}

}

//*****
// TICK MATH
//*****
/// @notice Computes a tick that will require approximately `amount` of token0 to create a `tickSpacing`-wide position.

```

```

/// @dev This function can have a maximum of two ticks of error from one of the ticks with `amount(tickRes + 2) &lt;...
/// @dev `tickSpacing` is assumed to be within the range (0, 32768)
/// @dev `maxLiquidityPerTick` for `s=tickSpacing` should be defined by `(2^128 - 1) / ((887272/s) - (-887272/s) + 1...
/// @param amount The desired amount of token0 required to fill the returned tick
/// @param tickSpacing The spacing between initializable ticks in the Uniswap pool
/// @param maxLiquidityPerTick The maximum liquidity that can reference any given tick in the Uniswap pool
/// @return A tick that will require approximately `amount` of token0 to create a `tickSpacing`-wide position with `...
function getApproxTickWithMaxAmount(
    uint256 amount,
    int24 tickSpacing,
    uint256 maxLiquidityPerTick
) internal pure returns (int24) {
    unchecked {
        // abs(max_error) ≈ 2^-13 * log(√(1.0001))^1 ≈ -1.70234
        return
            int24(
                int256(
                    Math.log_Sqrt1p0001MantissaRect(
                        Math.mulDivCapped(
                            amount,
                            2 ** 224,
                            (maxLiquidityPerTick *
                                (Math.getSqrtRatioAtTick(tickSpacing) - 2 ** 96)),
                            128
                        ),
                        13
                    )
                )
            );
    }
}

/// @notice Computes the maximum liquidity that is allowed to reference any given tick in a Uniswap V3 pool with `ti...
/// @param tickSpacing The spacing between initializable ticks in the Uniswap V3 pool
/// @return The maximum liquidity that can reference any given tick in the Uniswap V3 pool
function getMaxLiquidityPerTick(int24 tickSpacing) internal pure returns (uint128) {
    unchecked {
        // forge-lint: disable-next-line(divide-before-multiply)
        return type(uint128).max / uint24((Constants.MAX_POOL_TICK / tickSpacing) * 2 + 1);
    }
}

/// @notice Calculates `1.0001^(tick/2)` as an X96 number.
/// @dev Will revert if `abs(tick) > 887272`.
/// @param tick Value of the tick for which `sqrt(1.0001^tick)` is calculated
/// @return A Q64.96 number representing the sqrt price at the provided tick
function getSqrtRatioAtTick(int24 tick) internal pure returns (uint160) {
    unchecked {
        uint256 absTick = tick &lt; 0 ? uint256(-int256(tick)) : uint256(int256(tick));
        if (absTick > uint256(int256(Constants.MAX_POOL_TICK))) revert Errors.InvalidTick();
        // sqrt(1.0001^(-absTick)) = Π sqrt(1.0001^(-bit_i))
        // ex: absTick = 100 = binary 1100100, so sqrt(1.0001^-100) = sqrt(1.0001^-64) * sqrt(1.0001^-32) * sqrt(1.0...
        // constants are 2^128/(sqrt(1.0001)^bit_i) rounded half-up
        // if the first bit is 0, initialize sqrtR to 1 (2^128)
        uint256 sqrtR = absTick & 0x1 != 0
            ? 0xffffcb933bd6fad37aa2d162d1a594001
            : 0x100000000000000000000000000000000;
        if (absTick & 0x2 != 0) sqrtR = (sqrtR * 0xffff9727373d413259a46990580e213a) & & 128;
        if (absTick & 0x4 != 0) sqrtR = (sqrtR * 0xffff2e50f5f656932ef12357cf3c7fdcc) & & 128;
        if (absTick & 0x8 != 0) sqrtR = (sqrtR * 0xffe5caca7e10e4e61c3624eaa0941cd0) & & 128;
        if (absTick & 0x10 != 0) sqrtR = (sqrtR * 0xffcb9843d60f6159c9db58835c926644) & & 128;
        if (absTick & 0x20 != 0) sqrtR = (sqrtR * 0xff973b41fa98c081472e6896dfb254c0) & & 128;
        if (absTick & 0x40 != 0) sqrtR = (sqrtR * 0xff2ea16466c96a3843ec78b326b52861) & & 128;
        if (absTick & 0x80 != 0) sqrtR = (sqrtR * 0xfe5dee046a99a2a811c461f1969c3053) & & 128;
        if (absTick & 0x100 != 0) sqrtR = (sqrtR * 0xfcbe86c7900a88aedcfc83b479aa3a4) & & 128;
        if (absTick & 0x200 != 0) sqrtR = (sqrtR * 0xf987a7253ac413176f2b074cf7815e54) & & 128;
        if (absTick & 0x400 != 0) sqrtR = (sqrtR * 0xf3392b0822b70005940c7a398e4b70f3) & & 128;
        if (absTick & 0x800 != 0) sqrtR = (sqrtR * 0xe7159475a2c29b7443b29c7fa6e889d9) & & 128;
        if (absTick & 0x1000 != 0) sqrtR = (sqrtR * 0xd097f3bdfd2022b8845ad8f792aa5825) & & 128;
        if (absTick & 0x2000 != 0) sqrtR = (sqrtR * 0xa9f746462d870fdf8a65dc1f90e061e5) & & 128;
        if (absTick & 0x4000 != 0) sqrtR = (sqrtR * 0x70d869a156d2a1b890bb3df62baf32f7) & & 128;
        if (absTick & 0x8000 != 0) sqrtR = (sqrtR * 0x31be135f97d08fd981231505542fcfa6) & & 128;
        if (absTick & 0x10000 != 0) sqrtR = (sqrtR * 0x9aa508b5b7a84e1c677de54f3e99bc9) & & 128;
        if (absTick & 0x20000 != 0) sqrtR = (sqrtR * 0x5d6af8dedb81196699c329225ee604) & & 128;
        if (absTick & 0x40000 != 0) sqrtR = (sqrtR * 0x2216e584f5fa1ea926041bedfe98) & & 128;
        if (absTick & 0x80000 != 0) sqrtR = (sqrtR * 0x48a170391f7dc42444e8fa2) & & 128;
        // 2^128 * sqrt(1.0001^x) = 2^128 / sqrt(1.0001^-x)
        if (tick > 0) sqrtR = type(uint256).max / sqrtR;
        // Downcast + rounding up to keep is consistent with Uniswap's
        return uint160((sqrtR & & 32) + (sqrtR % (1 & & 32) == 0 ? 0 : 1));
    }
}

/// @notice Approximates the absolute value of log base `sqrt(1.0001)` for a number in (0, 1) (`argX128/2^128`) with...
/// @param argX128 The Q128.128 fixed-point number in the range (0, 1) to calculate the log of
/// @param precision The bits of precision with which to compute the result, max 63 (`err &lt;= 2^-precision * log(1...
/// @return The absolute value of log with base `sqrt(1.0001)` for `argX128/2^128`

```



```

function log_Sqrt1p0001MantissaRect(
    uint256 argX128,
    uint256 precision
) internal pure returns (uint256) {
    unchecked {
        // =[log(x)] =MSB(x)
        uint256 log2_res = FixedPointMathLib.log2(argX128);
        // Normalize argX128 to [1, 2)
        // x_normal = x / 2^[log(x)]
        // = 1.a...a... = 2^(0.b...b...)
        // log(x_normal) = log(x / 2^[log(x)])
        // log(x_normal) = log(x) - log(2^[log(x)])
        // log(x_normal) = log(x) - log(x)
        // log(x) = log(x_normal) + log(x)
        argX128 &lt;&lt;= (127 - log2_res);
        // =[log(x)] * 2^64
        log2_res = (128 - log2_res) &lt;&lt; 64;
        // log(x_normal) = 0.b...b...
        // x_normal = (1.a...a...) = 2^(0.b...b...)
        // x_normal^2 = (1.a...a...)^2 = (2^(0.b...b...))^2
        // = 2^(0.b...b... * 2)
        // = 2^(b... + 0.b...b...)
        // if b... = 1, renormalize x_normal^2 to [1, 2):
        // 2^(b... + 0.b...b...) / 2^b... = 2^((b... - 1).b...b...)
        // = 2^(0.b...b...)
        // error = [0, 2^b...)
        uint256 iterBound = 63 - precision;
        for (uint256 i = 63; i > iterBound; i--) {
            argX128 = (argX128 ** 2) >> 127;
            uint256 bit = argX128 >> 128;
            log2_res -= bit &lt;&lt; i;
            argX128 >>= bit;
        }
        // log(sqrt(1.0001))(x) = log(x) / log(sqrt(1.0001))
        // 2^64 / log(sqrt(1.0001)) = 255738959000112593413423
        return (log2_res * 255738959000112593413423) / 2 ** 128;
    }
}

////////////////////////////////////////////////////////////
// LIQUIDITY AMOUNTS
////////////////////////////////////////////////////////////
/// @notice Calculates the amount of token0 received for a given LiquidityChunk.
/// @param liquidityChunk A specification for a liquidity chunk in Uniswap containing `liquidity`, `tickLower`, and ...
/// @return The amount of token0 represented by `liquidityChunk` when `currentTick > tickLower`
function getAmount0ForLiquidityUp(
    LiquidityChunk liquidityChunk
) internal pure returns (uint256) {
    uint160 lowPriceX96 = getSqrtRatioAtTick(liquidityChunk.tickLower());
    uint160 highPriceX96 = getSqrtRatioAtTick(liquidityChunk.tickUpper());
    unchecked {
        return
            mulDivRoundingUp(
                mulDivRoundingUp(
                    uint256(liquidityChunk.liquidity()) &lt;&lt; 96,
                    highPriceX96 - lowPriceX96,
                    highPriceX96
                ),
                1,
                lowPriceX96
            );
    }
}

/// @notice Calculates the amount of token1 received for a given LiquidityChunk.
/// @param liquidityChunk A specification for a liquidity chunk in Uniswap containing `liquidity`, `tickLower`, and ...
/// @return The amount of token1 represented by `liquidityChunk` when `currentTick > tickUpper`
function getAmount1ForLiquidityUp(
    LiquidityChunk liquidityChunk
) internal pure returns (uint256) {
    uint160 lowPriceX96 = getSqrtRatioAtTick(liquidityChunk.tickLower());
    uint160 highPriceX96 = getSqrtRatioAtTick(liquidityChunk.tickUpper());
    unchecked {
        return mulDiv96RoundingUp(liquidityChunk.liquidity(), highPriceX96 - lowPriceX96);
    }
}

/// @notice Calculates the amount of token0 received for a given LiquidityChunk.
/// @param liquidityChunk A specification for a liquidity chunk in Uniswap containing `liquidity`, `tickLower`, and ...
/// @return The amount of token0 represented by `liquidityChunk` when `currentTick > tickLower`
function getAmount0ForLiquidity(LiquidityChunk liquidityChunk) internal pure returns (uint256) {
    uint160 lowPriceX96 = getSqrtRatioAtTick(liquidityChunk.tickLower());
    uint160 highPriceX96 = getSqrtRatioAtTick(liquidityChunk.tickUpper());
    unchecked {
        return
            mulDiv(
                uint256(liquidityChunk.liquidity()) &lt;&lt; 96,

```

```

        highPriceX96 - lowPriceX96,
        highPriceX96
    ) / lowPriceX96;
}
}
/// @notice Calculates the amount of token1 received for a given LiquidityChunk.
/// @param liquidityChunk A specification for a liquidity chunk in Uniswap containing `liquidity`, `tickLower`, and ...
/// @return The amount of token1 represented by `liquidityChunk` when `currentTick > tickUpper`
function getAmount1ForLiquidity(LiquidityChunk liquidityChunk) internal pure returns (uint256) {
    uint160 lowPriceX96 = getSqrtRatioAtTick(liquidityChunk.tickLower());
    uint160 highPriceX96 = getSqrtRatioAtTick(liquidityChunk.tickUpper());
    unchecked {
        return mulDiv96(liquidityChunk.liquidity(), highPriceX96 - lowPriceX96);
    }
}
/// @notice Calculates the amount of token0 and token1 received for a given LiquidityChunk at the provided `currentT...
/// @param currentTick The tick at which to evaluate `liquidityChunk`
/// @param liquidityChunk A specification for a liquidity chunk in Uniswap containing `liquidity`, `tickLower`, and ...
/// @return amount0 The amount of token0 represented by `liquidityChunk` at `currentTick`
/// @return amount1 The amount of token1 represented by `liquidityChunk` at `currentTick`
function getAmountsForLiquidity(
    int24 currentTick,
    LiquidityChunk liquidityChunk
) internal pure returns (uint256 amount0, uint256 amount1) {
    if (currentTick <= liquidityChunk.tickLower()) {
        amount0 = getAmount0ForLiquidity(liquidityChunk);
    } else if (currentTick >= liquidityChunk.tickUpper()) {
        amount1 = getAmount1ForLiquidity(liquidityChunk);
    } else {
        amount0 = getAmount0ForLiquidity(liquidityChunk.updateTickLower(currentTick));
        amount1 = getAmount1ForLiquidity(liquidityChunk.updateTickUpper(currentTick));
    }
}
}
/// @notice Returns a LiquidityChunk at the provided tick range with `liquidity` corresponding to `amount0`.
/// @param tickLower The lower tick of the chunk
/// @param tickUpper The upper tick of the chunk
/// @param amount0 The amount of token0
/// @return A LiquidityChunk with `tickLower`, `tickUpper`, and the calculated amount of liquidity for `amount0`
function getLiquidityForAmount0(
    int24 tickLower,
    int24 tickUpper,
    uint256 amount0
) internal pure returns (LiquidityChunk) {
    unchecked {
        uint160 lowPriceX96 = getSqrtRatioAtTick(tickLower);
        uint160 highPriceX96 = getSqrtRatioAtTick(tickUpper);
        uint256 liquidity = mulDiv(
            amount0,
            mulDiv96(highPriceX96, lowPriceX96),
            highPriceX96 - lowPriceX96
        );
        // This check guarantees the following uint128 cast is safe.
        if (liquidity > type(uint128).max) revert Errors.LiquidityTooHigh();
        // casting to 'uint128' is safe because of the liquidity > type(uint128).max check above
        // forge-lint: disable-next-line(unsafe-typecast)
        return LiquidityChunkLibrary.createChunk(tickLower, tickUpper, uint128(liquidity));
    }
}
}
/// @notice Returns a LiquidityChunk at the provided tick range with `liquidity` corresponding to `amount1`.
/// @param tickLower The lower tick of the chunk
/// @param tickUpper The upper tick of the chunk
/// @param amount1 The amount of token1
/// @return A LiquidityChunk with `tickLower`, `tickUpper`, and the calculated amount of liquidity for `amount1`
function getLiquidityForAmount1(
    int24 tickLower,
    int24 tickUpper,
    uint256 amount1
) internal pure returns (LiquidityChunk) {
    unchecked {
        uint160 lowPriceX96 = getSqrtRatioAtTick(tickLower);
        uint160 highPriceX96 = getSqrtRatioAtTick(tickUpper);
        uint256 liquidity = mulDiv(amount1, Constants.FP96, highPriceX96 - lowPriceX96);
        // This check guarantees the following uint128 cast is safe.
        if (liquidity > type(uint128).max) revert Errors.LiquidityTooHigh();
        return LiquidityChunkLibrary.createChunk(tickLower, tickUpper, uint128(liquidity));
    }
}
}
}
/*//////////////////////////////////////////////////////////////
                        CASTING
//////////////////////////////////////////////////////////////*/
/// @notice Downcast uint256 to uint128. Revert on overflow or underflow.
/// @param toDowncast The uint256 to be downcasted
/// @return downcastedInt `toDowncast` downcasted to uint128
function toUint128(uint256 toDowncast) internal pure returns (uint128 downcastedInt) {

```

```

    if ((downcastedInt = uint128(toDowncast)) != toDowncast) revert Errors.CastingError();
}
/// @notice Downcast uint256 to uint128, but cap at type(uint128).max on overflow.
/// @param toDowncast The uint256 to be downcasted
/// @return downcastedInt `toDowncast` downcasted to uint128
function toUint128Capped(uint256 toDowncast) internal pure returns (uint128 downcastedInt) {
    if ((downcastedInt = uint128(toDowncast)) != toDowncast) {
        downcastedInt = type(uint128).max;
    }
}
/// @notice Downcast uint128 to int128.
/// @param toCast The uint256 to be downcasted
/// @return downcastedInt `toCast` downcasted to int128
function toInt128(uint128 toCast) internal pure returns (int128 downcastedInt) {
    if ((downcastedInt = int128(toCast)) &lt; 0) revert Errors.CastingError();
}
/// @notice Cast an int256 to an int128, revert on overflow or underflow.
/// @param toCast The int256 to be downcasted
/// @return downcastedInt `toCast` downcasted to int128
function toInt128(int256 toCast) internal pure returns (int128 downcastedInt) {
    if (!((downcastedInt = int128(toCast)) == toCast)) revert Errors.CastingError();
}
/// @notice Cast a uint256 to an int256, revert on overflow.
/// @param toCast The uint256 to be downcasted
/// @return `toCast` downcasted to int256
function toInt256(uint256 toCast) internal pure returns (int256) {
    if (toCast > uint256(type(int256).max)) revert Errors.CastingError();
    return int256(toCast);
}
}
/*//////////////////////////////////////////////////////////////
                        MULDIV
//////////////////////////////////////////////////////////////*/
/// @notice Calculates `floor(a*b+denominator)` with full precision. Throws if result overflows a uint256 or `denomi...
/// @param a The multiplicand
/// @param b The multiplier
/// @param denominator The divisor
/// @return result The 256-bit result
/// @dev Credit to Remco Bloemen under MIT license https://xn--2-umb.com/21/muldiv for this and all following `mulDi...
function mulDiv(
    uint256 a,
    uint256 b,
    uint256 denominator
) internal pure returns (uint256 result) {
    unchecked {
        // 512-bit multiply [prod1 prod0] = a * b
        // Compute the product mod 2**256 and mod 2**256 - 1
        // then use the Chinese Remainder Theorem to reconstruct
        // the 512 bit result. The result is stored in two 256
        // variables such that product = prod1 * 2**256 + prod0
        uint256 prod0; // Least significant 256 bits of the product
        uint256 prod1; // Most significant 256 bits of the product
        assembly ("memory-safe") {
            let mm := mulmod(a, b, not(0))
            prod0 := mul(a, b)
            prod1 := sub(sub(mm, prod0), lt(mm, prod0))
        }
        // Handle non-overflow cases, 256 by 256 division
        if (prod1 == 0) {
            require(denominator > 0);
            assembly ("memory-safe") {
                result := div(prod0, denominator)
            }
            return result;
        }
        // Make sure the result is less than 2**256.
        // Also prevents denominator == 0
        require(denominator > prod1);
        ////////////////////////////////////////////////////////////////
        // 512 by 256 division.
        ////////////////////////////////////////////////////////////////
        // Make division exact by subtracting the remainder from [prod1 prod0]
        // Compute remainder using mulmod
        uint256 remainder;
        assembly ("memory-safe") {
            remainder := mulmod(a, b, denominator)
        }
        // Subtract 256 bit number from 512 bit number
        assembly ("memory-safe") {
            prod1 := sub(prod1, gt(remainder, prod0))
            prod0 := sub(prod0, remainder)
        }
        // Factor powers of two out of denominator
        // Compute largest power of two divisor of denominator.
        // Always >= 1.
    }
}

```

```

uint256 twos = (0 - denominator) & denominator;
// Divide denominator by power of two
assembly ("memory-safe") {
    denominator := div(denominator, twos)
}
// Divide [prod1 prod0] by the factors of two
assembly ("memory-safe") {
    prod0 := div(prod0, twos)
}
// Shift in bits from prod1 into prod0. For this we need
// to flip `twos` such that it is  $2^{256} / \text{twos}$ .
// If twos is zero, then it becomes one
assembly ("memory-safe") {
    twos := add(div(sub(0, twos), twos), 1)
}
prod0 |= prod1 * twos;
// Invert denominator mod  $2^{256}$ 
// Now that denominator is an odd number, it has an inverse
// modulo  $2^{256}$  such that  $\text{denominator} * \text{inv} = 1 \bmod 2^{256}$ .
// Compute the inverse by starting with a seed that is correct
// correct for four bits. That is,  $\text{denominator} * \text{inv} = 1 \bmod 2^{256}$ 
uint256 inv = (3 * denominator) ^ 2;
// Now use Newton-Raphson iteration to improve the precision.
// Thanks to Hensel's lifting lemma, this also works in modular
// arithmetic, doubling the correct bits in each step.
inv *= 2 - denominator * inv; // inverse mod  $2^{256}$ 
inv *= 2 - denominator * inv; // inverse mod  $2^{512}$ 
inv *= 2 - denominator * inv; // inverse mod  $2^{768}$ 
inv *= 2 - denominator * inv; // inverse mod  $2^{1024}$ 
inv *= 2 - denominator * inv; // inverse mod  $2^{1280}$ 
inv *= 2 - denominator * inv; // inverse mod  $2^{1536}$ 
// Because the division is now exact we can divide by multiplying
// with the modular inverse of denominator. This will give us the
// correct result modulo  $2^{256}$ . Since the preconditions guarantee
// that the outcome is less than  $2^{256}$ , this is the final result.
// We don't need to compute the high bits of the result and prod1
// is no longer required.
result = prod0 * inv;
}
}
/// @notice Calculates  $\text{min}(\text{floor}(a \cdot b / \text{denominator}), 2^{256} - 1)$  with full precision.
/// @param a The multiplicand
/// @param b The multiplier
/// @param denominator The divisor
/// @return result The 256-bit result
function mulDivCapped(
    uint256 a,
    uint256 b,
    uint256 denominator
) internal pure returns (uint256 result) {
    unchecked {
        // 512-bit multiply [prod1 prod0] = a * b
        // Compute the product mod  $2^{256}$  and mod  $2^{256} - 1$ 
        // then use the Chinese Remainder Theorem to reconstruct
        // the 512 bit result. The result is stored in two 256
        // variables such that  $\text{product} = \text{prod1} * 2^{256} + \text{prod0}$ 
        uint256 prod0; // Least significant 256 bits of the product
        uint256 prod1; // Most significant 256 bits of the product
        assembly ("memory-safe") {
            let mm := mulmod(a, b, not(0))
            prod0 := mul(a, b)
            prod1 := sub(sub(mm, prod0), lt(mm, prod0))
        }
        // Handle non-overflow cases, 256 by 256 division
        if (prod1 == 0) {
            require(denominator > 0);
            assembly ("memory-safe") {
                result := div(prod0, denominator)
            }
            return result;
        }
        if (denominator <= prod1) return type(uint256).max;
        //////////////////////////////////////
        // 512 by 256 division.
        //////////////////////////////////////
        // Make division exact by subtracting the remainder from [prod1 prod0]
        // Compute remainder using mulmod
        uint256 remainder;
        assembly ("memory-safe") {
            remainder := mulmod(a, b, denominator)
        }
        // Subtract 256 bit number from 512 bit number
        assembly ("memory-safe") {
            prod1 := sub(prod1, gt(remainder, prod0))

```

```

    prod0 := sub(prod0, remainder)
  }
  // Factor powers of two out of denominator
  // Compute largest power of two divisor of denominator.
  // Always >= 1.
  uint256 twos = (0 - denominator) & denominator;
  // Divide denominator by power of two
  assembly ("memory-safe") {
    denominator := div(denominator, twos)
  }
  // Divide [prod1 prod0] by the factors of two
  assembly ("memory-safe") {
    prod0 := div(prod0, twos)
  }
  // Shift in bits from prod1 into prod0. For this we need
  // to flip `twos` such that it is  $2^{256} / \text{twos}$ .
  // If twos is zero, then it becomes one
  assembly ("memory-safe") {
    twos := add(div(sub(0, twos), twos), 1)
  }
  prod0 |= prod1 * twos;
  // Invert denominator mod  $2^{256}$ 
  // Now that denominator is an odd number, it has an inverse
  // modulo  $2^{256}$  such that denominator * inv = 1 mod  $2^{256}$ .
  // Compute the inverse by starting with a seed that is correct
  // correct for four bits. That is, denominator * inv = 1 mod  $2^4$ 
  uint256 inv = (3 * denominator) ^ 2;
  // Now use Newton-Raphson iteration to improve the precision.
  // Thanks to Hensel's lifting lemma, this also works in modular
  // arithmetic, doubling the correct bits in each step.
  inv = 2 - denominator * inv; // inverse mod  $2^8$ 
  inv = 2 - denominator * inv; // inverse mod  $2^{16}$ 
  inv = 2 - denominator * inv; // inverse mod  $2^{32}$ 
  inv = 2 - denominator * inv; // inverse mod  $2^{64}$ 
  inv = 2 - denominator * inv; // inverse mod  $2^{128}$ 
  inv = 2 - denominator * inv; // inverse mod  $2^{256}$ 
  // Because the division is now exact we can divide by multiplying
  // with the modular inverse of denominator. This will give us the
  // correct result modulo  $2^{256}$ . Since the preconditions guarantee
  // that the outcome is less than  $2^{256}$ , this is the final result.
  // We don't need to compute the high bits of the result and prod1
  // is no longer required.
  result = prod0 * inv;
}
}

/// @notice Calculates  $\min(\text{floor}(a \cdot b / \text{denominator}), 2^{\text{power}-1})$  with full precision.
/// @param a The multiplicand
/// @param b The multiplier
/// @param denominator The divisor
/// @param power The upper bound of the open interval representing the range of this function, given by  $2^{\text{power}}$ 
/// @return result The 256-bit result
function mulDivCapped(
  uint256 a,
  uint256 b,
  uint256 denominator,
  uint256 power
) internal pure returns (uint256 result) {
  unchecked {
    // 512-bit multiply [prod1 prod0] = a * b
    // Compute the product mod  $2^{256}$  and mod  $2^{256} - 1$ 
    // then use the Chinese Remainder Theorem to reconstruct
    // the 512 bit result. The result is stored in two 256
    // variables such that product = prod1 *  $2^{256}$  + prod0
    uint256 prod0; // Least significant 256 bits of the product
    uint256 prod1; // Most significant 256 bits of the product
    assembly ("memory-safe") {
      let mm := mulmod(a, b, not(0))
      prod0 := mul(a, b)
      prod1 := sub(sub(mm, prod0), lt(mm, prod0))
    }
    // Handle non-overflow cases, 256 by 256 division
    if (prod1 == 0) {
      require(denominator > 0);
      assembly ("memory-safe") {
        result := div(prod0, denominator)
      }
      return Math.min(result, 2 ** power - 1);
    }
  }
  if (denominator >>> (256 - power) &lt;= prod1) return 2 ** power - 1;
  // 512 by 256 division.
  // Make division exact by subtracting the remainder from [prod1 prod0]
  // Compute remainder using mulmod

```

```

uint256 remainder;
assembly ("memory-safe") {
    remainder := mulmod(a, b, denominator)
}
// Subtract 256 bit number from 512 bit number
assembly ("memory-safe") {
    prod1 := sub(prod1, gt(remainder, prod0))
    prod0 := sub(prod0, remainder)
}
// Factor powers of two out of denominator
// Compute largest power of two divisor of denominator.
// Always >= 1.
uint256 twos = (0 - denominator) & denominator;
// Divide denominator by power of two
assembly ("memory-safe") {
    denominator := div(denominator, twos)
}
// Divide [prod1 prod0] by the factors of two
assembly ("memory-safe") {
    prod0 := div(prod0, twos)
}
// Shift in bits from prod1 into prod0. For this we need
// to flip `twos` such that it is 2**256 / twos.
// If twos is zero, then it becomes one
assembly ("memory-safe") {
    twos := add(div(sub(0, twos), twos), 1)
}
prod0 |= prod1 * twos;
// Invert denominator mod 2**256
// Now that denominator is an odd number, it has an inverse
// modulo 2**256 such that denominator * inv = 1 mod 2**256.
// Compute the inverse by starting with a seed that is correct
// correct for four bits. That is, denominator * inv = 1 mod 2**4
uint256 inv = (3 * denominator) ^ 2;
// Now use Newton-Raphson iteration to improve the precision.
// Thanks to Hensel's lifting lemma, this also works in modular
// arithmetic, doubling the correct bits in each step.
inv *= 2 - denominator * inv; // inverse mod 2**8
inv *= 2 - denominator * inv; // inverse mod 2**16
inv *= 2 - denominator * inv; // inverse mod 2**32
inv *= 2 - denominator * inv; // inverse mod 2**64
inv *= 2 - denominator * inv; // inverse mod 2**128
inv *= 2 - denominator * inv; // inverse mod 2**256
// Because the division is now exact we can divide by multiplying
// with the modular inverse of denominator. This will give us the
// correct result modulo 2**256. Since the preconditions guarantee
// that the outcome is less than 2**256, this is the final result.
// We don't need to compute the high bits of the result and prod1
// is no longer required.
result = prod0 * inv;
}
}
/// @notice Calculates `ceil(a*b+denominator)` with full precision. Throws if result overflows a uint256 or `denomin...
/// @param a The multiplicand
/// @param b The multiplier
/// @param denominator The divisor
/// @return result The 256-bit result
function mulDivRoundingUp(
    uint256 a,
    uint256 b,
    uint256 denominator
) internal pure returns (uint256 result) {
    unchecked {
        result = mulDiv(a, b, denominator);
        if (mulmod(a, b, denominator) > 0) {
            require(result < type(uint256).max);
            result++;
        }
    }
}
/// @notice Calculates `floor(a*b+2^64)` with full precision. Throws if result overflows a uint256.
/// @param a The multiplicand
/// @param b The multiplier
/// @return The 256-bit result
function mulDiv64(uint256 a, uint256 b) internal pure returns (uint256) {
    unchecked {
        // 512-bit multiply [prod1 prod0] = a * b
        // Compute the product mod 2**256 and mod 2**256 - 1
        // then use the Chinese Remainder Theorem to reconstruct
        // the 512 bit result. The result is stored in two 256
        // variables such that product = prod1 * 2**256 + prod0
        uint256 prod0; // Least significant 256 bits of the product
        uint256 prod1; // Most significant 256 bits of the product
        assembly ("memory-safe") {

```

```

        let mm := mulmod(a, b, not(0))
        prod0 := mul(a, b)
        prod1 := sub(sub(mm, prod0), lt(mm, prod0))
    }
    // Handle non-overflow cases, 256 by 256 division
    if (prod1 == 0) {
        uint256 res;
        assembly ("memory-safe") {
            // Right shift by n is equivalent and 2 gas cheaper than division by 2^n
            res := shr(64, prod0)
        }
        return res;
    }
    // Make sure the result is less than 2**256.
    require(2 ** 64 > prod1);
    // 512 by 256 division.
    // Make division exact by subtracting the remainder from [prod1 prod0]
    // Compute remainder using mulmod
    uint256 remainder;
    assembly ("memory-safe") {
        remainder := mulmod(a, b, 0x1000000000000000000000000)
    }
    // Subtract 256 bit number from 512 bit number
    assembly ("memory-safe") {
        prod1 := sub(prod1, gt(remainder, prod0))
        prod0 := sub(prod0, remainder)
    }
    // Divide [prod1 prod0] by the factors of two (note that this is just 2**96 since the denominator is a power...
    assembly ("memory-safe") {
        // Right shift by n is equivalent and 2 gas cheaper than division by 2^n
        prod0 := shr(64, prod0)
    }
    // Shift in bits from prod1 into prod0. For this we need
    // to flip `twos` such that it is 2**256 / twos.
    // If twos is zero, then it becomes one
    // Note that this is just 2**192 since 2**256 over the fixed denominator (2**64) equals 2**192
    prod0 |= prod1 * 2 ** 192;
    return prod0;
}

}

/// @notice Calculates `floor(a*b+2^96)` with full precision. Throws if result overflows a uint256.
/// @param a The multiplicand
/// @param b The multiplier
/// @return The 256-bit result
function mulDiv96(uint256 a, uint256 b) internal pure returns (uint256) {
    unchecked {
        // 512-bit multiply [prod1 prod0] = a * b
        // Compute the product mod 2**256 and mod 2**256 - 1
        // then use the Chinese Remainder Theorem to reconstruct
        // the 512 bit result. The result is stored in two 256
        // variables such that product = prod1 * 2**256 + prod0
        uint256 prod0; // Least significant 256 bits of the product
        uint256 prod1; // Most significant 256 bits of the product
        assembly ("memory-safe") {
            let mm := mulmod(a, b, not(0))
            prod0 := mul(a, b)
            prod1 := sub(sub(mm, prod0), lt(mm, prod0))
        }
        // Handle non-overflow cases, 256 by 256 division
        if (prod1 == 0) {
            uint256 res;
            assembly ("memory-safe") {
                // Right shift by n is equivalent and 2 gas cheaper than division by 2^n
                res := shr(96, prod0)
            }
            return res;
        }
        // Make sure the result is less than 2**256.
        require(2 ** 96 > prod1);
        // 512 by 256 division.
        // Make division exact by subtracting the remainder from [prod1 prod0]
        // Compute remainder using mulmod
        uint256 remainder;
        assembly ("memory-safe") {
            remainder := mulmod(a, b, 0x100000000000000000000000000000000)
        }
        // Subtract 256 bit number from 512 bit number
        assembly ("memory-safe") {
            prod1 := sub(prod1, gt(remainder, prod0))
            prod0 := sub(prod0, remainder)
        }
    }
}

```

```

    }
    // Divide [prod1 prod0] by the factors of two (note that this is just 2**96 since the denominator is a power...
    assembly ("memory-safe") {
        // Right shift by n is equivalent and 2 gas cheaper than division by 2^n
        prod0 := shr(96, prod0)
    }
    // Shift in bits from prod1 into prod0. For this we need
    // to flip `twos` such that it is 2**256 / twos.
    // If twos is zero, then it becomes one
    // Note that this is just 2**160 since 2**256 over the fixed denominator (2**96) equals 2**160
    prod0 |= prod1 * 2 ** 160;
    return prod0;
}

}

/// @notice Calculates `ceil(a*b÷2^96)` with full precision. Throws if result overflows a uint256.
/// @param a The multiplicand
/// @param b The multiplier
/// @return result The 256-bit result
function mulDiv96RoundingUp(uint256 a, uint256 b) internal pure returns (uint256 result) {
    unchecked {
        result = mulDiv96(a, b);
        if (mulmod(a, b, 2 ** 96) > 0) {
            require(result < type(uint256).max);
            result++;
        }
    }
}

/// @notice Calculates `floor(a*b÷2^128)` with full precision. Throws if result overflows a uint256.
/// @param a The multiplicand
/// @param b The multiplier
/// @return The 256-bit result
function mulDiv128(uint256 a, uint256 b) internal pure returns (uint256) {
    unchecked {
        // 512-bit multiply [prod1 prod0] = a * b
        // Compute the product mod 2**256 and mod 2**256 - 1
        // then use the Chinese Remainder Theorem to reconstruct
        // the 512 bit result. The result is stored in two 256
        // variables such that product = prod1 * 2**256 + prod0
        uint256 prod0; // Least significant 256 bits of the product
        uint256 prod1; // Most significant 256 bits of the product
        assembly ("memory-safe") {
            let mm := mulmod(a, b, not(0))
            prod0 := mul(a, b)
            prod1 := sub(sub(mm, prod0), lt(mm, prod0))
        }
        // Handle non-overflow cases, 256 by 256 division
        if (prod1 == 0) {
            uint256 res;
            assembly ("memory-safe") {
                // Right shift by n is equivalent and 2 gas cheaper than division by 2^n
                res := shr(128, prod0)
            }
            return res;
        }
        // Make sure the result is less than 2**256.
        require(2 ** 128 > prod1);
        // 512 by 256 division.
        // Make division exact by subtracting the remainder from [prod1 prod0]
        // Compute remainder using mulmod
        uint256 remainder;
        assembly ("memory-safe") {
            remainder := mulmod(a, b, 0x100000000000000000000000000000000)
        }
        // Subtract 256 bit number from 512 bit number
        assembly ("memory-safe") {
            prod1 := sub(prod1, gt(remainder, prod0))
            prod0 := sub(prod0, remainder)
        }
        // Divide [prod1 prod0] by the factors of two (note that this is just 2**128 since the denominator is a powe...
        assembly ("memory-safe") {
            // Right shift by n is equivalent and 2 gas cheaper than division by 2^n
            prod0 := shr(128, prod0)
        }
        // Shift in bits from prod1 into prod0. For this we need
        // to flip `twos` such that it is 2**256 / twos.
        // If twos is zero, then it becomes one
        // Note that this is just 2**160 since 2**256 over the fixed denominator (2**128) equals 2**128
        prod0 |= prod1 * 2 ** 128;
        return prod0;
    }
}

}

/// @notice Calculates `ceil(a*b÷2^128)` with full precision. Throws if result overflows a uint256.

```



```

/// @param a The multiplicand
/// @param b The multiplier
/// @return result The 256-bit result
function mulDiv128RoundingUp(uint256 a, uint256 b) internal pure returns (uint256 result) {
    unchecked {
        result = mulDiv128(a, b);
        if (mulmod(a, b, 2 ** 128) > 0) {
            require(result < type(uint256).max);
            result++;
        }
    }
}

/// @notice Calculates `floor(a×b÷2^192)` with full precision. Throws if result overflows a uint256.
/// @param a The multiplicand
/// @param b The multiplier
/// @return The 256-bit result
function mulDiv192(uint256 a, uint256 b) internal pure returns (uint256) {
    unchecked {
        // 512-bit multiply [prod1 prod0] = a * b
        // Compute the product mod 2**256 and mod 2**256 - 1
        // then use the Chinese Remainder Theorem to reconstruct
        // the 512 bit result. The result is stored in two 256
        // variables such that product = prod1 * 2**256 + prod0
        uint256 prod0; // Least significant 256 bits of the product
        uint256 prod1; // Most significant 256 bits of the product
        assembly ("memory-safe") {
            let mm := mulmod(a, b, not(0))
            prod0 := mul(a, b)
            prod1 := sub(sub(mm, prod0), lt(mm, prod0))
        }
        // Handle non-overflow cases, 256 by 256 division
        if (prod1 == 0) {
            uint256 res;
            assembly ("memory-safe") {
                // Right shift by n is equivalent and 2 gas cheaper than division by 2^n
                res := shr(192, prod0)
            }
            return res;
        }
        // Make sure the result is less than 2**256.
        require(2 ** 192 < prod1);
        //////////////////////////////////////
        // 512 by 256 division.
        //////////////////////////////////////
        // Make division exact by subtracting the remainder from [prod1 prod0]
        // Compute remainder using mulmod
        uint256 remainder;
        assembly ("memory-safe") {
            remainder := mulmod(a, b, 0x10000000000000000000000000000000000000000000000000000000000000000)
        }
        // Subtract 256 bit number from 512 bit number
        assembly ("memory-safe") {
            prod1 := sub(prod1, gt(remainder, prod0))
            prod0 := sub(prod0, remainder)
        }
        // Divide [prod1 prod0] by the factors of two (note that this is just 2**96 since the denominator is a power...)
        assembly ("memory-safe") {
            // Right shift by n is equivalent and 2 gas cheaper than division by 2^n
            prod0 := shr(192, prod0)
        }
        // Shift in bits from prod1 into prod0. For this we need
        // to flip `twos` such that it is 2**256 / twos.
        // If twos is zero, then it becomes one
        // Note that this is just 2**64 since 2**256 over the fixed denominator (2**192) equals 2**64
        prod0 |= prod1 * 2 ** 64;
        return prod0;
    }
}

/// @notice Calculates `ceil(a×b÷2^192)` with full precision.
/// @param a The multiplicand
/// @param b The multiplier
/// @return result The 256-bit result
function mulDiv192RoundingUp(uint256 a, uint256 b) internal pure returns (uint256 result) {
    unchecked {
        result = mulDiv192(a, b);
        if (mulmod(a, b, 2 ** 192) > 0) {
            require(result < type(uint256).max);
            result++;
        }
    }
}

/// @notice Calculates `floor(a×b÷10^18)` with full precision. Throws if result overflows a uint256.
/// @param a The multiplicand
/// @param b The multiplier

```

```

/// @return The 256-bit result
/// @dev Optimized version of mulDiv for WAD (10^18) denominator
function mulDivWad(uint256 a, uint256 b) internal pure returns (uint256) {
    unchecked {
        // 512-bit multiply [prod1 prod0] = a * b
        // Compute the product mod 2**256 and mod 2**256 - 1
        // then use the Chinese Remainder Theorem to reconstruct
        // the 512 bit result. The result is stored in two 256
        // variables such that product = prod1 * 2**256 + prod0
        uint256 prod0; // Least significant 256 bits of the product
        uint256 prod1; // Most significant 256 bits of the product
        assembly ("memory-safe") {
            let mm := mulmod(a, b, not(0))
            prod0 := mul(a, b)
            prod1 := sub(sub(mm, prod0), lt(mm, prod0))
        }
        // Handle non-overflow cases, 256 by 256 division
        if (prod1 == 0) {
            uint256 res;
            assembly ("memory-safe") {
                res := div(prod0, 1000000000000000000)
            }
            return res;
        }
        // Make sure the result is less than 2**256.
        require(1000000000000000000 > prod1);
        // 512 by 256 division.
        // Make division exact by subtracting the remainder from [prod1 prod0]
        // Compute remainder using mulmod
        uint256 remainder;
        assembly ("memory-safe") {
            remainder := mulmod(a, b, 1000000000000000000)
        }
        // Subtract 256 bit number from 512 bit number
        assembly ("memory-safe") {
            prod1 := sub(prod1, gt(remainder, prod0))
            prod0 := sub(prod0, remainder)
        }
        // Since 10^18 = 2^18 * 5^18, we need to handle both factors
        // First divide by 2^18
        assembly ("memory-safe") {
            // Right shift by 18 is equivalent to division by 2^18
            prod0 := shr(18, prod0)
        }
        // Shift in bits from prod1 into prod0
        // We need 2^256 / 2^18 = 2^238
        prod0 |= prod1 << 238;
        // Now divide by 5^18 using modular inverse
        // Precomputed modular inverse of 5^18 modulo 2^256
        // This means: (5^18 * inv) mod 2^256 = 1
        uint256 inv = 0xaccb18165bd6fe31ae1cf318dc5b51eee0e1ba569b88cd74c1773b91fac10669;
        return prod0 * inv;
    }
}

/// @notice Calculates `ceil(a÷b÷10^18)` with full precision.
/// @param a The multiplicand
/// @param b The multiplier
/// @return result The 256-bit result
function mulDivWadRoundingUp(uint256 a, uint256 b) internal pure returns (uint256 result) {
    unchecked {
        result = mulDivWad(a, b);
        if (mulmod(a, b, 10 ** 18) > 0) {
            require(result < type(uint256).max);
            result++;
        }
    }
}

/// @notice Calculates `ceil(a÷b)`, returning 0 if `b == 0`.
/// @param a The numerator
/// @param b The denominator
/// @return result The 256-bit result
function unsafeDivRoundingUp(uint256 a, uint256 b) internal pure returns (uint256 result) {
    assembly ("memory-safe") {
        result := add(div(a, b), gt(mod(a, b), 0))
    }
}

/*//////////////////////////////////////////////////////////////
                        SORTING
//////////////////////////////////////////////////////////////
*/

/// @notice QuickSort is a sorting algorithm that employs the Divide and Conquer strategy. It selects a pivot elemen...
/// this pivot by correctly positioning it within the sorted array.
/// @param arr The elements that must be sorted

```

```

/// @param left The starting index
/// @param right The ending index
function quickSort(int256[] memory arr, int256 left, int256 right) internal pure {
    unchecked {
        int256 i = left;
        int256 j = right;
        if (i == j) return;
        int256 pivot = arr[uint256(left + (right - left) / 2)];
        while (i < j) {
            while (arr[uint256(i)] < pivot) i++;
            while (pivot < arr[uint256(j)]) j--;
            if (i < j) {
                (arr[uint256(i)], arr[uint256(j)]) = (arr[uint256(j)], arr[uint256(i)]);
                i++;
                j--;
            }
        }
        if (left < j) quickSort(arr, left, j);
        if (i < right) quickSort(arr, i, right);
    }
}

/// @notice Calls `quickSort` with default starting index of 0 and ending index of the last element in the array.
/// @param data The elements that must be sorted
/// @return The sorted array
function sort(int256[] memory data) internal pure returns (int256[] memory) {
    unchecked {
        quickSort(data, int256(0), int256(data.length - 1));
    }
    return data;
}

/*//////////////////////////////////////////////////////////////////
                        EXPONENTIAL MATH
//////////////////////////////////////////////////////////////////*/

/// @dev Returns the sum of the first three non-zero terms of a Taylor expansion of  $e^{(nx)} - 1$ , to approximate a
/// continuous compound interest rate. Source: https://github.com/morpho-org/morpho-blue/blob/main/src/libraries/Mat...
function wTaylorCompounded(uint256 x, uint256 n) internal pure returns (uint256) {
    uint256 firstTerm = x * n;
    uint256 secondTerm = mulDiv(firstTerm, firstTerm, 2 * WAD);
    uint256 thirdTerm = mulDiv(secondTerm, firstTerm, 3 * WAD);
    return firstTerm + secondTerm + thirdTerm;
}

/// @dev Returns the sum of the first three non-zero terms of a Taylor expansion of  $e^{(nx)}$ , to approximate a
/// continuous compound interest rate for a custom scale s. Source: https://github.com/morpho-org/morpho-blue/blob/m...
function sTaylorCompounded(uint256 x, uint256 s) internal pure returns (uint256) {
    uint256 zerothTerm = s;
    uint256 firstTerm = x;
    uint256 secondTerm = mulDiv(firstTerm, firstTerm, 2 * s);
    uint256 thirdTerm = mulDiv(secondTerm, firstTerm, 3 * s);
    uint256 fourthTerm = mulDiv(thirdTerm, firstTerm, 4 * s);
    return zerothTerm + firstTerm + secondTerm + thirdTerm + fourthTerm;
}

/// @dev Returns the multiplication of `x` by `y` (in WAD) rounded towards 0.
function wMulToZero(int256 x, int256 y) internal pure returns (int256) {
    return (x * y) / WAD_INT;
}

/// @dev Returns the division of `x` by `y` (in WAD) rounded towards 0.
function wDivToZero(int256 x, int256 y) internal pure returns (int256) {
    return (x * WAD_INT) / y;
}

/// @dev Bounds `x` between `low` and `high`.
/// @dev Assumes that `low` <= `high`. If it is not the case it returns `low`.
function bound(int256 x, int256 low, int256 high) internal pure returns (int256 z) {
    assembly {
        // z = min(x, high).
        z := xor(x, mul(xor(x, high), slt(high, x)))
        // z = max(z, low).
        z := xor(z, mul(xor(z, low), sgt(low, z)))
    }
}

/// @dev Returns an approximation of exp.
function wExp(int256 x) internal pure returns (int256) {
    unchecked {
        // If x < ln(1e-18) then exp(x) < 1e-18 so it is rounded to zero.
        if (x < LN_WEI_INT) return 0;
        // `wExp` is clipped to avoid overflowing when multiplied with 1 ether.
        if (x >= WEXP_UPPER_BOUND) return WEXP_UPPER_VALUE;
        // Decompose x as x = q * ln(2) + r with q an integer and -ln(2)/2 <= r <= ln(2)/2.
        // q = x / ln(2) rounded half toward zero.
        int256 roundingAdjustment = (x < 0) ? -(LN_2_INT / 2) : (LN_2_INT / 2);
        // Safe unchecked because x is bounded.
        int256 q = (x + roundingAdjustment) / LN_2_INT;
        // Safe unchecked because |q * ln(2) - x| <= ln(2)/2.
        int256 r = x - q * LN_2_INT;
        // Compute e^r with a 2nd-order Taylor polynomial.
    }
}

```

```

        // Safe unchecked because |r| < 1e18.
        int256 expR = WAD_INT + r + (r * r) / WAD_INT / 2;
        // Return e^x = 2^q * e^r.
        if (q >= 0) return expR <<< uint256(q);
        else return expR >>> uint256(-q);
    }
}
}

```

[12/12] contracts/types/TokenId.sol

```

// SPDX-License-Identifier: GPL-2.0-or-later
pragma solidity ^0.8.24;
// Libraries
import {Constants} from "@libraries/Constants.sol";
import {Errors} from "@libraries/Errors.sol";
import {PanopticMath} from "@libraries/PanopticMath.sol";
type TokenId is uint256;
using TokenIdLibrary for TokenId global;
/// @title Panoptic's tokenId: the fundamental options position.
/// @author Axicon Labs Limited
/// @notice This is the token ID used in the ERC1155 representation of the option position in the SFPM.
/// @notice The SFPM "overloads" the ERC1155 `id` by storing all option information in said `id`.
/// @notice Contains methods for packing and unpacking a Panoptic options position into a uint256 bit pattern.
// PACKING RULES FOR A TOKENID:
// this is how the token ID is packed into its bit-constituents containing position information.
// the following is a diagram to be read top-down in a little endian format
// (so (1) below occupies the first 64 least significant bits, e.g.):
// From the LSB to the MSB:
// ===== 1 time (same for all legs) =====
//      Property      Size      Offset      Comment
// (0) univ3pool      40bits     0bits       : first 5 bytes representing the Uniswap pool (first 40 bits; little-endi...
// (1) vegoid          8bits      40bits      : vegoid for the sfpm pool
// (2) tickSpacing     16bits     48bits      : tickSpacing for the univ3pool. Up to 16 bits
// ===== 4 times (one for each leg) =====
// (3) asset           1bit       0bits       : Specifies the asset (0: token0, 1: token1)
// (4) optionRatio     7bits      1bits       : number of contracts per leg
// (5) isLong          1bit       8bits       : long==1 means liquidity is removed, long==0 -> liquidity is added
// (6) tokenType       1bit       9bits       : put/call: which token is moved when deployed (0 -> token0, 1 -> token1)
// (7) riskPartner     2bits      10bits      : normally its own index. Partner in defined risk position otherwise
// (8) strike          24bits     12bits      : strike price; defined as (tickUpper + tickLower) / 2
// (9) width           12bits     36bits      : width; defined as (tickUpper - tickLower) / tickSpacing
// Total              48bits
// =====
//
// The bit pattern is therefore, in general:
//
//      (strike price tick of the 3rd leg)
//      |                               |
//      |                               | (width of the 2nd leg)
//      |                               |
// (9)(8)(7)(6)(5)(4)(3) (9)(8)(7)(6)(5)(4)(3) (9)(8)(7)(6)(5)(4)(3) (9)(8)(7)(6)(5)(4)(3) (2) (1) ...
// <--- 48 bits ---> <--- 48 bits ---> <--- 48 bits ---> <--- 48 bits ---> ...
//      Leg 4          Leg 3          Leg 2          Leg 1          tickSpacing vegoid ...
//
// <--- most significant bit                                     least signi...
//
// Some rules of how legs behave (we enforce these in a `validate()` function):
// - a leg is inactive if it's not part of the position. Technically it means that all bits are zero.
// - a leg is active if it has an optionRatio > 0 since this must always be set for an active leg.
// - if a leg is active (e.g. leg 1) there can be no gaps in other legs meaning: if leg 1 is active then leg 3 cannot ...
//
// Examples:
// We can think of the bit pattern as an array starting at bit index 0 going to bit index 255 (so 256 total bits)
// We also refer to the legs via their index, so leg number 2 has leg index 1 (legIndex) (counting from zero), and in g...
// - the underlying strike price of the 2nd leg (leg index = 1) in this option position starts at bit index (64 + 12 + ...
// - the tokenType of the 4th leg in this option position starts at bit index 64+9+48*3=217
// - the Uniswap V3 pool id starts at bit index 0 and ends at bit index 63 (and thus takes up 64 bits).
// - the width of the 3rd leg in this option position starts at bit index 64+36+48*2=196
library TokenIdLibrary {
    /// @notice AND mask to extract all `isLong` bits for each leg from a TokenId.
    uint256 internal constant LONG_MASK =
        0x100_000000000100_000000000100_000000000100_0000000000000000;
    /// @notice AND mask to clear `poolId` from a TokenId.
    uint256 internal constant CLEAR_POOLID_MASK =
        0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF_0000000000000000;
    /// @notice AND mask to clear all bits except for the option ratios of the legs.
    uint256 internal constant OPTION_RATIO_MASK =
        0x0000000000FE_0000000000FE_0000000000FE_0000000000FE_0000000000000000;
    /// @notice AND mask to clear all bits except for the components of the chunk key (strike, width, tokenType) for eac...
    uint256 internal constant CHUNK_MASK =

```

```

0xFFFFFFFF200_FFFFFFFF200_FFFFFFFF200_FFFFFFFF200_0000000000000000;
/// @notice AND mask to cut a sign-extended int256 back to an int24.
int256 internal constant BITMASK_INT24 = 0xFFFFF;
//*****
                        DECODING
//*****
/// @notice The full poolId (Uniswap pool identifier + pool pattern) of this option position.
/// @param self The TokenId to extract `poolId` from
/// @return The `poolId` (Panoptic's pool fingerprint, contains the whole 64 bit sequence with the tickSpacing) of t...
function poolId(TokenId self) internal pure returns (uint64) {
    unchecked {
        return uint64(TokenId.unwrap(self));
    }
}
/// @notice The vegoid of this option position.
/// @param self The TokenId to extract `vegoid` from
/// @return The `vegoid` of the Uniswap V3 pool
function vegoid(TokenId self) internal pure returns (uint8) {
    unchecked {
        return uint8((TokenId.unwrap(self) >>> 40) % 2 ** 8);
    }
}
/// @notice The tickSpacing of this option position.
/// @param self The TokenId to extract `tickSpacing` from
/// @return The `tickSpacing` of the Uniswap V3 pool
function tickSpacing(TokenId self) internal pure returns (int24) {
    unchecked {
        return int24(uint24((TokenId.unwrap(self) >>> 48) % 2 ** 16));
    }
}
/// @notice Get the asset basis for this TokenId.
/// @dev Which token is the asset - can be token0 (return 0) or token1 (return 1).
/// @param self The TokenId to extract `asset` from
/// @param legIndex The leg index of this position (in {0,1,2,3}) to extract `asset` from
/// @return 0 if asset is token0, 1 if asset is token1
function asset(TokenId self, uint256 legIndex) internal pure returns (uint256) {
    unchecked {
        return uint256((TokenId.unwrap(self) >>> (64 + legIndex * 48)) % 2);
    }
}
/// @notice Get the number of contracts multiplier for leg `legIndex`.
/// @param self The TokenId to extract `optionRatio` at `legIndex` from
/// @param legIndex The leg index of this position (in {0,1,2,3})
/// @return The number of contracts multiplier for leg `legIndex`
function optionRatio(TokenId self, uint256 legIndex) internal pure returns (uint256) {
    unchecked {
        return uint256((TokenId.unwrap(self) >>> (64 + legIndex * 48 + 1)) % 128);
    }
}
/// @notice Return 1 if the nth leg (leg index `legIndex`) is a long position.
/// @param self The TokenId to extract `isLong` at `legIndex` from
/// @param legIndex The leg index of this position (in {0,1,2,3})
/// @return 1 if long; 0 if not long
function isLong(TokenId self, uint256 legIndex) internal pure returns (uint256) {
    unchecked {
        return uint256((TokenId.unwrap(self) >>> (64 + legIndex * 48 + 8)) % 2);
    }
}
/// @notice Get the type of token moved for a given leg (implies a call or put). Either Token0 or Token1.
/// @param self The TokenId to extract `tokenType` at `legIndex` from
/// @param legIndex The leg index of this position (in {0,1,2,3})
/// @return 1 if the token moved is token1 or 0 if the token moved is token0
function tokenType(TokenId self, uint256 legIndex) internal pure returns (uint256) {
    unchecked {
        return uint256((TokenId.unwrap(self) >>> (64 + legIndex * 48 + 9)) % 2);
    }
}
/// @notice Get the associated risk partner of the leg index (generally another leg index in the position if enabled...
/// @param self The TokenId to extract `riskPartner` at `legIndex` from
/// @param legIndex The leg index of this position (in {0,1,2,3})
/// @return The leg index of `legIndex`'s risk partner
function riskPartner(TokenId self, uint256 legIndex) internal pure returns (uint256) {
    unchecked {
        return uint256((TokenId.unwrap(self) >>> (64 + legIndex * 48 + 10)) % 4);
    }
}
/// @notice Get the strike price tick of the nth leg (with index `legIndex`).
/// @param self The TokenId to extract `strike` at `legIndex` from
/// @param legIndex The leg index of this position (in {0,1,2,3})
/// @return The strike price tick of the leg
function strike(TokenId self, uint256 legIndex) internal pure returns (int24) {
    unchecked {
        return int24(int256(TokenId.unwrap(self) >>> (64 + legIndex * 48 + 12)));
    }
}

```

```

}
/// @notice Get the width (distance between upper and lower ticks) of the nth leg (index `legIndex`).
/// @dev The width is always positive; it is returned as an int24 for internal consistency with strike operations.
/// @param self The TokenId to extract `width` at `legIndex` from
/// @param legIndex The leg index of this position (in {0,1,2,3})
/// @return The width of the position
function width(TokenId self, uint256 legIndex) internal pure returns (int24) {
    unchecked {
        return int24(int256((TokenId.unwrap(self) >>> (64 + legIndex * 48 + 36)) % 4096));
    } // "% 4096" = take last (2 ** 12 = 4096) 12 bits
}
//*****
//                               ENCODING
//*****
/// @notice Add the Uniswap pool identifier corresponding to this option position (contains the entropy and tickSpac...
/// @param self The TokenId to add `_poolId` to
/// @param _poolId The PoolID to add to `self`
/// @return `self` with `_poolId` added to the PoolID slot
function addPoolId(TokenId self, uint64 _poolId) internal pure returns (TokenId) {
    unchecked {
        return TokenId.wrap(TokenId.unwrap(self) + _poolId);
    }
}
/// @notice Add the `tickSpacing` to the PoolID for `self`.
/// @param self The TokenId to add `_tickSpacing` to
/// @param _tickSpacing The tickSpacing to add to `self`
/// @return `self` with `_tickSpacing` added to the TickSpacing slot in the PoolID
function addTickSpacing(TokenId self, int24 _tickSpacing) internal pure returns (TokenId) {
    unchecked {
        return TokenId.wrap(TokenId.unwrap(self) + (uint256(uint24(_tickSpacing)) <<< 48));
    }
}
/// @notice Add the asset basis for this position.
/// @param self The TokenId to add `_asset` to
/// @param _asset The asset to add to the Asset slot in `self` for `legIndex`
/// @param legIndex The leg index of this position (in {0,1,2,3})
/// @return `self` with `_asset` added to the Asset slot
function addAsset(
    TokenId self,
    uint256 _asset,
    uint256 legIndex
) internal pure returns (TokenId) {
    unchecked {
        return
            TokenId.wrap(TokenId.unwrap(self) + (uint256(_asset % 2) <<< (64 + legIndex * 48)));
    }
}
/// @notice Add the number of contracts multiplier to leg index `legIndex`.
/// @param self The TokenId to add `_optionRatio` to
/// @param _optionRatio The number of contracts multiplier to add to the OptionRatio slot in `self` for LegIndex
/// @param legIndex The leg index of the position (in {0,1,2,3})
/// @return `self` with `_optionRatio` added to the OptionRatio slot for `legIndex`
function addOptionRatio(
    TokenId self,
    uint256 _optionRatio,
    uint256 legIndex
) internal pure returns (TokenId) {
    unchecked {
        return
            TokenId.wrap(
                TokenId.unwrap(self) + (uint256(_optionRatio % 128) <<< (64 + legIndex * 48 + 1))
            );
    }
}
/// @notice Add "isLong" parameter indicating whether a leg is long (isLong=1) or short (isLong=0).
/// @param self The TokenId to add `_isLong` to
/// @param _isLong The isLong parameter to add to the IsLong slot in `self` for `legIndex`
/// @param legIndex The leg index of this position (in {0,1,2,3})
/// @return `self` with `_isLong` added to the IsLong slot for `legIndex`
function addIsLong(
    TokenId self,
    uint256 _isLong,
    uint256 legIndex
) internal pure returns (TokenId) {
    unchecked {
        return TokenId.wrap(TokenId.unwrap(self) + ((_isLong % 2) <<< (64 + legIndex * 48 + 8)));
    }
}
/// @notice Add the type of token moved for a given leg (implies a call or put). Either Token0 or Token1.
/// @param self The TokenId to add `_tokenType` to
/// @param _tokenType The tokenType to add to the TokenType slot in `self` for `legIndex`
/// @param legIndex The leg index of this position (in {0,1,2,3})
/// @return `self` with `_tokenType` added to the TokenType slot for `legIndex`
function addTokenType(

```

```

        tokenId self,
        uint256 _tokenType,
        uint256 legIndex
    ) internal pure returns (tokenId) {
        unchecked {
            return
                tokenId.wrap(
                    tokenId.unwrap(self) + (uint256(_tokenType % 2) <<< (64 + legIndex * 48 + 9))
                );
        }
    }

    /// @notice Add the associated risk partner of the leg index.
    /// @param self The tokenId to add `_riskPartner` to
    /// @param _riskPartner The riskPartner to add to the RiskPartner slot in `self` for `legIndex`
    /// @param legIndex The leg index of this position (in {0,1,2,3})
    /// @return `self` with `_riskPartner` added to the RiskPartner slot for `legIndex`
    function addRiskPartner(
        tokenId self,
        uint256 _riskPartner,
        uint256 legIndex
    ) internal pure returns (tokenId) {
        unchecked {
            return
                tokenId.wrap(
                    tokenId.unwrap(self) + (uint256(_riskPartner % 4) <<< (64 + legIndex * 48 + 10))
                );
        }
    }

    /// @notice Add the strike price tick of the nth leg (index `legIndex`).
    /// @param self The tokenId to add `_strike` to
    /// @param _strike The strike price tick to add to the Strike slot in `self` for `legIndex`
    /// @param legIndex The leg index of this position (in {0,1,2,3})
    /// @return `self` with `_strike` added to the Strike slot for `legIndex`
    function addStrike(
        tokenId self,
        int24 _strike,
        uint256 legIndex
    ) internal pure returns (tokenId) {
        unchecked {
            return
                tokenId.wrap(
                    tokenId.unwrap(self) +
                    uint256((int256(_strike) & BITMASK_INT24) <<< (64 + legIndex * 48 + 12))
                );
        }
    }

    /// @notice Add the width of the nth leg (index `legIndex`).
    /// @param self The tokenId to add `_width` to
    /// @param _width The width to add to the Width slot in `self` for `legIndex`
    /// @param legIndex The leg index of this position (in {0,1,2,3})
    /// @return `self` with `_width` added to the Width slot for `legIndex`
    function addWidth(
        tokenId self,
        int24 _width,
        uint256 legIndex
    ) internal pure returns (tokenId) {
        // % 4096 -> take 12 bits from the incoming 24 bits (there's no uint12)
        unchecked {
            return
                tokenId.wrap(
                    tokenId.unwrap(self) +
                    (uint256(uint24(_width) % 4096) <<< (64 + legIndex * 48 + 36))
                );
        }
    }

    /// @notice Add a leg to a tokenId.
    /// @param self The tokenId in the SFPM representing an option position
    /// @param legIndex The leg index of this position (in {0,1,2,3}) to add
    /// @param _optionRatio The relative size of the leg
    /// @param _asset The asset of the leg
    /// @param _isLong Whether the leg is long
    /// @param _tokenType The type of token moved for the leg
    /// @param _riskPartner The associated risk partner of the leg
    /// @param _strike The strike price tick of the leg
    /// @param _width The width of the leg
    /// @return tokenId The tokenId with the leg added
    function addLeg(
        tokenId self,
        uint256 legIndex,
        uint256 _optionRatio,
        uint256 _asset,
        uint256 _isLong,
        uint256 _tokenType,
        uint256 _riskPartner,

```

```

    int24 _strike,
    int24 _width
) internal pure returns (TokenId tokenId) {
    tokenId = addOptionRatio(self, _optionRatio, legIndex);
    tokenId = addAsset(tokenId, _asset, legIndex);
    tokenId = addIsLong(tokenId, _isLong, legIndex);
    tokenId = addTokenType(tokenId, _tokenType, legIndex);
    tokenId = addRiskPartner(tokenId, _riskPartner, legIndex);
    tokenId = addStrike(tokenId, _strike, legIndex);
    tokenId = addWidth(tokenId, _width, legIndex);
}
/*//////////////////////////////////////////////////////////////
                        HELPERS
//////////////////////////////////////////////////////////////
/// @notice Flip all the `isLong` positions in the legs in the `tokenId` option position.
/// @param self The TokenId to flip isLong for on all active legs
/// @return tokenId `self` with all `isLong` bits flipped
function flipToBurnToken(TokenId self) internal pure returns (TokenId) {
    unchecked {
        // NOTE: This is a hack to avoid blowing up the contract size.
        // We need to ensure that only active legs are flipped
        // In order to achieve this, we shift our long bit mask to the right by (4-# active legs)
        // i.e the whole mask is used to flip all legs with 4 legs, but only the first leg is flipped with 1 leg so ...
        // We also clear the poolId area of the mask to ensure the bits that are shifted right into the area don't f...
        return
            TokenId.wrap(
                TokenId.unwrap(self) ^
                    ((LONG_MASK >>> (48 * (4 - self.countLegs())))) & CLEAR_POOLID_MASK
            );
    }
}
/// @notice Count the number of legs (out of a maximum of 4) that are long positions.
/// @param self The TokenId to count longs for
/// @return The number of long positions in `self` (in the range {0,...,4})
function countLongs(TokenId self) internal pure returns (uint256) {
    unchecked {
        return self.isLong(0) + self.isLong(1) + self.isLong(2) + self.isLong(3);
    }
}
/// @notice Get the option position's nth leg's (index `legIndex`) tick ranges (lower, upper).
/// @param self The TokenId to extract the tick range from
/// @param legIndex The leg index of the position (in {0,1,2,3})
/// @return legLowerTick The lower tick of the leg/liquidity chunk
/// @return legUpperTick The upper tick of the leg/liquidity chunk
function asTicks(
    TokenId self,
    uint256 legIndex
) internal pure returns (int24 legLowerTick, int24 legUpperTick) {
    (legLowerTick, legUpperTick) = PanopticMath.getTicks(
        self.strike(legIndex),
        self.width(legIndex),
        self.tickSpacing()
    );
}
/// @notice Return the number of active legs in the option position.
/// @dev ASSUMPTION: For any leg, the option ratio is always > 0 (the leg always has a number of contracts associ...)
/// @param self The TokenId to count active legs for
/// @return numLegs The number of active legs in `self` (in the range {0,...,4})
function countLegs(TokenId self) internal pure returns (uint256 numLegs) {
    // Strip all bits except for the option ratios
    uint256 optionRatios = (TokenId.unwrap(self) & OPTION_RATIO_MASK) >>> 64;
    unchecked {
        // forge-lint: disable-next-line(incorrect-shift)
        while (optionRatios >= (1 <<< (48 * numLegs))) {
            ++numLegs;
        }
    }
}
/// @notice Clear a leg in an option position at `legIndex`.
/// @dev NOTE: it's important that the caller fills in the leg details after.
// - optionRatio is zeroed
// - asset is zeroed
// - width is zeroed
// - strike is zeroed
// - tokenType is zeroed
// - isLong is zeroed
// - riskPartner is zeroed
/// @param self The TokenId to clear the leg from
/// @param legIndex The leg index to reset, in {0,1,2,3}
/// @return `self` with the `legIndex`th leg zeroed
function clearLeg(TokenId self, uint256 legIndex) internal pure returns (TokenId) {
    if (legIndex == 0)
        return
            TokenId.wrap(

```



```

        tokenId.unwrap(self) &= 0xFFFFFFFFFFFFFFFFFFFFFFFF_000000000000_FFFFFFFFFFFFFFFFFF
    );
    if (legIndex == 1)
        return
            tokenId.wrap(
                tokenId.unwrap(self) &= 0xFFFFFFFFFFFFFFFF_000000000000_FFFFFFFFFFFFFFFFFF
            );
    if (legIndex == 2)
        return
            tokenId.wrap(
                tokenId.unwrap(self) &= 0xFFFFFFFFFFFFFFFF_000000000000_FFFFFFFFFFFFFFFFFF_FFFFFFFFFFFFFFFFFF
            );
    if (legIndex == 3)
        return
            tokenId.wrap(
                tokenId.unwrap(self) &= 0x000000000000_FFFFFFFFFFFFFFFFFF_FFFFFFFFFFFFFFFFFF_FFFFFFFFFFFFFFFFFF
            );
    return self;
}
// ***** VALIDATION *****
// *****
/// @notice Checks if a tokenId is valid and reverts with an error reflecting the incorrect parameter for invalid po...
/// @param self The tokenId to validate
function validate(tokenId self) internal pure {
    if (self.optionRatio(0) == 0) revert Errors.InvalidTokenIdParameter(1);
    // loop through the 4 (possible) legs in the tokenId `self`
    unchecked {
        // extract strike, width, and tokenType
        uint256 chunkData = (tokenId.unwrap(self) &= CHUNK_MASK) >> 64;
        for (uint256 i = 0; i < 4; ++i) {
            if (self.optionRatio(i) == 0) {
                // final leg in this position identified;
                // make sure any leg above this are zero as well
                // (we don't allow gaps eg having legs 1 and 4 active without 2 and 3 is not allowed)
                if ((tokenId.unwrap(self) >> (64 + 48 * i)) != 0)
                    revert Errors.InvalidTokenIdParameter(1);
                break; // we are done iterating over potential legs
            }
            // prevent legs touching the same chunks - all chunks in the position must be discrete
            uint256 numLegs = self.countLegs();
            for (uint256 j = i + 1; j < numLegs; ++j) {
                if (uint48(chunkData >> (48 * i)) == uint48(chunkData >> (48 * j))) {
                    revert Errors.InvalidTokenIdParameter(6);
                }
            }
            // Strike cannot be MIN_TICK or MAX_TICK
            if (
                (self.strike(i) == Constants.MIN_POOL_TICK) ||
                (self.strike(i) == Constants.MAX_POOL_TICK)
            ) revert Errors.InvalidTokenIdParameter(4);
            // In the following, we check whether the risk partner of this leg is itself
            // or another leg in this position.
            uint256 riskPartnerIndex = self.riskPartner(i);
            if (riskPartnerIndex != i) {
                // Ensures that risk partners are mutual
                if (self.riskPartner(riskPartnerIndex) != i)
                    revert Errors.InvalidTokenIdParameter(3);
            }
        }
    }
}

/// @notice Check whether a position `self` contains at least one exercisable long leg.
/// @dev A leg is considered exercisable if it is:
/// - long (isLong == 1), and
/// - not a loan/credit leg (width != 0).
/// @dev This function does NOT check moneyness or price ranges.
/// @return hasExercisableLong Returns 1 if such a leg exists, 0 otherwise.
function validateIsExercisable(tokenId self) internal pure returns (uint256) {
    unchecked {
        uint256 numLegs = self.countLegs();
        for (uint256 i = 0; i < numLegs; ++i) {
            if (self.isLong(i) == 1 && self.width(i) != 0) return 1; // validated
        }
    }
    // Fail if position has no legs that is far-out-of-the-money
    return 0;
}
}

```

